



新一代智慧終端嵌入式系統開發教學手冊

NuMaker-M55M1 開發板為例

Part. A M55M1 系統開發說明

壹、CPU GPU NPU 及邊緣 AI 介紹.....	5 -
一、CPU GPU NPU 介紹.....	5 -
二、邊緣 AI 介紹.....	10 -
三、邊緣 AI 應用領域.....	10 -
貳、NuMaker-M55M1 簡介.....	12 -
一、NuMaker-M55M1 開發板規格.....	12 -
二、Hardware Configuration	15 -
(一) 正面視圖	17 -
三、開發板擴充.....	20 -
(一) Arduino UNO 相容擴展介面	21 -
(二) mikroBUS 相容擴展介面	23 -
(三) M55M1 擴展介面 (Extension Connectors).....	24 -
參、開發應用整體部署流程說明	25 -
一、整體部署流程圖	25 -
二、TinyML 模型訓練.....	28 -
(一) TinyML 模型介紹	34 -
(二) YOLOX-Nano 訓練流程	39 -
(三) 機器學習框架(PyTorch 或 Tensorflow).....	41 -
(四) 模型轉檔.....	41 -
三、模型輕量化.....	42 -
(一)TensorFlow Lite Converter.....	42 -
(二) Model INT8 Quantization.....	43 -
四、Vela 編譯器 (NPU 運算加速).....	46 -
(一) Vela 功能簡介.....	47 -
(二) Vela 環境與擴展.....	48 -
(三) Vela 指令簡介.....	48 -
(四) Vela 編譯結果.....	49 -
(五) YOLOX-Nano 模型片段可視化：	51 -
五、使用 C 語言撰寫軟硬體設備配置	53 -
(二)Display 內容介紹:.....	55 -

(三) HyperRAM 內容介紹:.....	- 55 -
(四) ImageSensor 內容介紹:.....	- 55 -
(五) KEIL 內容解析:.....	- 55 -
(六) MODEL 資料夾概述與解析:.....	- 57 -
(七) NPU 資料夾概述:.....	- 57 -
六、開發板程式的開發及燒錄.....	- 59 -
(一) 集成開發環境 – Arm® Keil MDK μ Vision 5.....	- 59 -
(二) 程式開發流程.....	- 60 -
Part.B NuMaker-M55M1 開發板實作案例.....	- 72 -
壹、智慧工廠 1 – 安全帽偵測.....	- 72 -
一、案例介紹 – 安全帽穿戴偵測.....	- 72 -
二、資料集與 AI 模型訓練.....	- 75 -
三、PC 端 Anaconda 環境進行 PyTorch 框架模型訓練.....	- 81 -
四、新唐 M55M1 開發板上的推論程式系統流程步驟.....	- 88 -
貳、智慧工廠 2 – 火災偵測.....	- 98 -
一、案例介紹 – 火災偵測.....	- 98 -
二、資料集蒐集.....	- 99 -
三、在 PC 上使用 Anaconda 環境進行模型訓練.....	- 104 -
四、開發板上的推論程式系統流程圖.....	- 113 -
五、結論與未來發展.....	- 125 -
參、智慧工廠 3 – 降噪與關鍵字偵測.....	- 127 -
一、案例介紹 – 降噪與關鍵字偵測.....	- 127 -
二、開發流程概述.....	- 132 -
三、頻譜減法的理論基礎.....	- 137 -
四、Python 降噪模擬.....	- 142 -
五、C 語言在 M55M1 開發板上的實作.....	- 148 -
七、程式架構與最佳化策略.....	- 158 -
肆、智慧醫療 1 – 藥物辨識.....	- 165 -
一、案例介紹 – 藥物辨識.....	- 165 -
二、使用資料集與模型介紹.....	- 169 -
三、訓練環境與硬體介紹.....	- 170 -

四、 模型訓練.....	- 171 -
五、 C++ 軟體實現.....	- 178 -
六、 C++ 軟體設計介紹 - 模型推理.....	- 186 -
伍、智慧醫療 2 - 跌倒偵測.....	- 192 -
一、 案例介紹 - 跌倒偵測.....	- 192 -
二、 跌倒偵測技術種類介紹.....	- 194 -
三、 使用 Roboflow 的開源資料集.....	- 195 -
四、 在 PC 上使用 Anaconda 環境進行模型訓練.....	- 197 -
五、 開發板端的推論程式系統流程步驟.....	- 200 -
六、 結論與未來發展.....	- 208 -
陸、智慧家居 1 - 垃圾分類.....	- 209 -
一、 案例介紹 - 垃圾分類.....	- 209 -
二、 資料集與模型訓練.....	- 211 -
三、 Evaluate TFLite int8/float Model (概述).....	- 219 -
四、 C++ 軟體流程.....	- 222 -
五、 系統程式解析.....	- 224 -
六、 未來展望:.....	- 242 -
柒、智慧家居 2 - 人員追蹤.....	- 243 -
一、 案例介紹 - 人員追蹤.....	- 243 -
二、 模型比較.....	- 244 -
三、 LabelImg.....	- 247 -
四、 在 PC 上使用 Anaconda 環境進行模型訓練.....	- 248 -
五、 開發版上的推論程式系統流程圖.....	- 259 -
捌、智慧農業 1 - 蘋果品質辨識.....	- 268 -
一、 案例介紹 - 蘋果品質辨識.....	- 268 -
二、 模型比較分成以下三點介紹.....	- 270 -
三、 LabelImg.....	- 272 -
四、 在 PC 上使用 Anaconda 環境進行模型訓練.....	- 273 -
五、 未來展望.....	- 284 -
玖、智慧農業 2 - 魚苗計數.....	- 285 -
一、 案例介紹 - 魚苗計數.....	- 285 -

二、系統流程與程式模組說明 - 程式模組	- 286 -
三、資料集與模型訓練	- 287 -
四、C++ 設計、部署與燒錄	- 291 -
五、結論與未來展望	- 304 -
拾、智慧農業 3 - 魚種辨識	- 307 -
一、案例介紹 - 魚種辨識	- 307 -
二、系統流程與程式模組說明 - 程式模組	- 308 -
三、資料集與模型訓練	- 309 -
四、C++ 設計、部署與燒錄	- 313 -
五、結論與未來展望	- 326 -

壹、 CPU GPU NPU 及邊緣 AI 介紹

一、 CPU GPU NPU 介紹

隨著人工智慧技術的迅速發展，我們正處於一個全新的智慧時代。邊緣 AI（Edge AI）技術，作為智慧時代的核心驅動力之一，正以前所未有的速度改變我們的生活和工作方式。邊緣 AI 的核心優勢在於其將數據處理推向設備端，減少對雲端計算的依賴，實現了數據的即時處理與反應，這對於要求高效能與低延遲的應用場景至關重要。

與傳統的雲端計算相比，邊緣 AI 能夠顯著提升效率，並將處理過程中的延遲時間降至最低，這使得許多依賴即時響應的應用成為可能。例如，自動駕駛車輛依賴即時的路況分析來做出快速反應，智慧醫療設備需要即時分析患者的生理數據來進行決策，智慧製造系統則能夠即時監控生產線狀況並做出調整。這些應用都在邊緣 AI 的支持下得以實現。

隨著技術的進一步發展，邊緣 AI 將進一步普及，從傳統的數據中心延伸至各種嵌入式設備中，成為日常生活的一部分。從智慧家居到工業自動化，邊緣 AI 將不僅是技術革命的一部分，更是未來生活和工作的基礎。

(一) CPU、GPU 及 NPU 介紹

1. CPU

- (1) **設計目的：**CPU（中央處理器）是電腦的核心處理部件，主要負責執行計算機的指令以及邏輯運算。它是電腦的「大腦」，負責解讀並執行軟體中的指令。
- (2) **核心數量：**現代 CPU 通常具有多個核心，範圍為 1 到 16 個核心不等（甚至更高）。
 - a、 單核心：適合較基礎的運算需求。
 - b、 多核心：能夠同時執行多個程序，適用於高效能應用（如影像處理、遊戲）。
- (3) **效能優勢：** CPU 通常強調單個核心的運算速度，以應對複雜的控制邏輯、分支判斷和多程序處理。

- a、 控制邏輯：處理分支條件和順序控制（如流水線技術）。
- b、 分支預測：降低分支指令對效能的影響。
- c、 多程序處理：多核心的架構下，可以同時處理多個應用程序提升整體效能。

(4) **應用場景**：CPU 的主要應用包括

- a、 運行作業系統（如 Windows®、Linux®等）。
- b、 辦公軟件（如 Word®、Excel®）。
- c、 瀏覽網頁等日常使用。

(5) **製造商**

- a、 **Intel®**：全球最大的 CPU 生產商之一，提供多種性能卓越的處理器，如 Core i®系列。
- b、 **AMD®**：另一主要 CPU 製造商，以性價比和多核心處理能力見長，如 Ryzen®系列。

2. GPU

(1) **設計目的**：GPU（圖形處理單元）最初是為了加速圖形處理而設計的，主要應用於繪圖和渲染。隨著計算需求的發展，GPU 被廣泛應用於以下領域：

- a、 數據處理：適用於處理大量數據的場景。
- b、 科學計算：如模擬運算、天氣預測等需高性能計算的應用。

(2) **核心數量**：GPU 具有大量的運算核心，數量可達數百至數千核心，遠超 CPU 的核心數量。高核心數量使 GPU 非常適合並行計算工作，例如圖形渲染和矩陣運算。

(3) **效能優勢**：GPU 在以下方面擁有優勢

- a、 高度並行性：能處理大量相似運算，特別適合以下場景
 - 矩陣運算：如線性代數計算和神經網路訓練。
 - 深度學習：支援大規模數據計算（如 Tensor 運算）。
- b、 高效能：專為處理大量資料而最佳化，比 CPU 在特定應用中表現更佳。

(4) **應用場景**：GPU 的主要應用包括

- a、 圖形渲染：為遊戲、動畫及設計軟件提供高效渲染能力。

- b、 視頻處理：用於視頻編碼、解碼和後期處理。
- c、 大數據分析：加速數據運算及分析過程。
- d、 機器學習：如訓練深度學習模型和 AI 應用。

(5) **製造商**

- a、 **NVIDIA®**：全球領先的 GPU 製造商，其產品線（如 GeForce®、Tesla®）廣泛應用於遊戲、專業設計和高性能計算。

3. NPU

(1) **設計目的**：NPU（神經網路處理器）是一種專門為人工智慧（AI）與深度學習應用設計的處理器。其設計目的是加速神經網路模型的運算，尤其是涉及矩陣乘法、卷積運算等高計算量工作的處理。

- a、 AI 推論：實時處理神經網路推論，例如影像識別、語音識別。
- b、 AI 訓練：部分 NPU 也支援訓練過程，但主要針對推論最佳化。

(2) **核心數量**：NPU 的架構通常基於大量的小型專用核心，這些核心為高效處理神經網路運算而設計。

- a、 可能包含數百到數千個核心。
- b、 提供高度並行的計算能力，專注於加速特定的神經網路操作。

(3) **效能優勢**：NPU 相較於傳統處理器（CPU、GPU）在 AI 應用中有以下優勢：

- a、 能效比高：專門針對 AI 算法進行硬體最佳化，消耗更少的功耗完成更高效率的運算。
- b、 低延遲：實現實時 AI 推論，例如自駕車中的物體檢測。
- c、 定制化功能：內置 AI 加速指令集，針對特定模型（如 CNN、RNN）進行最佳化。

(4) **應用場景**：NPU 被廣泛應用於以下領域

- a、 自駕車：進行物體檢測與路徑規劃。
- b、 智慧手機：加速影像處理、語音助理等 AI 功能。
- c、 邊緣設備：物聯網（IoT）設備中的 AI 推論。
- d、 雲端計算：高效處理深度學習模型推論任務。

(5) 製造商

- a、 **Google®**：推出的 TPU（Tensor Processing Unit）是專用的 AI 加速器，屬於 NPU 的範疇。
- b、 **Apple®**：其 A 系列晶片中包含的「神經引擎」即為 NPU，用於影像處理和語音識別等功能。

表 1、CPU、GPU、NPU 比較表格

	CPU	GPU	NPU
設計目的	通用計算，執行指令和邏輯運算。	大規模並行計算，主要用於圖形處理。	專門為神經網路加速和深度學習設計。
核心數量	少量核心（通常 1-16）。	大量核心（數百到數千個核心）。	設計上可能有數百個運算單元，專為矩陣運算和卷積最佳化。
效能優勢	通用性強，但並行處理能力有限。	高並行處理能力，適合大規模數據處理。	專為神經網路計算和推理最佳化。
主要應用領域	操作系統、辦公軟體、簡單計算任務。	圖形渲染、視頻處理、大數據分析、機器學習。	深度學習推理、神經網路計算、嵌入式設備。
功耗	相對較低。	較高（尤其在高負載時）。	相對較低，尤其在移動設備中。

(二) 核心設備：新唐科技 NuMaker-M55M1 開發板

在這場邊緣 AI 的革命中，硬體設備的選擇至關重要。新唐科技 M55M1 是一款高性能的微控制器，專為邊緣 AI 應用設計，提供了強大的處理能力、低功耗和完整的開發支持。NuMaker-M55M1 開發板是實現高效邊緣計算的理想平台，具有以下顯著特點：

1. **高效能微控制器架構：**NuMaker-M55M1 開發板基於 ARM® Cortex®-M55 處理器，擁有高效的運算能力，支持多種主流 AI 訓練框架，並能進行即時推論運算。這使得開發者能夠在邊緣設備上直接運行機器學習模型，並將其應用於各種實際場景中。
2. **低功耗設計：**M55M1 的低功耗設計非常適合在能源受限的嵌入式環境中使用。無論是穿戴設備還是智慧感測器，這款開發板都能提供長時間的運行而不會消耗過多電力。
3. **完整的軟硬體開發支持：**新唐科技為 NuMaker-M55M1 開發板提供了完善的軟體開發工具鏈，涵蓋從驅動程式、開發套件到機器學習模型訓練的全方位支持。無論是初學者還是專業開發者，都能夠快速上手並實現自己的邊緣 AI 應用。

(三) 深入學習與應用

本書不僅是一本技術手冊，更是一份實踐指南，幫助讀者從硬體到軟體、從基礎到高階掌握邊緣 AI 技術的應用。本書的內容將涵蓋以下幾個關鍵領域：

1. **硬體配置與架構：**深入了解 NuMaker-M55M1 開發板的內部結構，從微處理器、記憶體到各種外部介面，幫助讀者理解如何利用開發板進行邊緣計算的設計與實現。
2. **模型訓練與最佳化：**邊緣 AI 的應用不僅限於硬體開發，還包括機器學習模型的選擇與最佳化。本書將介紹如何選擇適合的 AI 模型，並根據特定應用需求進行模型訓練和效能最佳化，從而提升邊緣設備的計算能力。
3. **應用案例展示：**通過智慧工廠、智慧醫療和智慧農業等具體案例，展示邊緣 AI 在實際場景中的應用。這些案例將展示如何將邊緣 AI 與實際需求相結合，從

而改變傳統行業的運作方式，提升生產力、降低成本並促進可持續發展。

(四) 適用對象

本書適合各類讀者，無論您是邊緣 AI 技術的初學者，還是希望拓展技術能力的專業工程師，都能從本書中獲益。對於初學者，書中的基礎概念與實例將幫助您快速入門，掌握核心技能；對於專業工程師，本書則提供了深入的技術細節與應用案例，幫助您在實際項目中應用邊緣 AI 技術，提升工作效率和創新能力。

隨著邊緣 AI 技術的發展，這一領域將持續創新並改變我們的生活。本書將幫助您踏出邊緣 AI 開發的第一步，並引領您進入這個充滿挑戰和機遇的領域。

二、邊緣 AI 介紹

1. Edge AI

邊緣人工智慧 (Edge AI) 是指將 AI 運算能力部署在靠近數據來源的設備上，如智慧感測器、工業控制器和監控攝像機，而非依賴傳統的雲端伺服器或數據中心。這種分散式架構使得邊緣設備能即時處理和分析數據，從而實現低延遲、高隱私性與更高的經濟效益。經過訓練的 AI 模型被部署到邊緣設備，使其能夠在本地快速反應，尤其適用於對時效性要求高的應用場景。

2. 邊緣 AI 的優勢

低延遲、高數據隱私性、可離線運行、節能。

三、邊緣 AI 應用領域

1. 智慧工廠：即時監控與預防性維護

智慧工廠是邊緣 AI 技術的一個典型應用領域。在工廠中，AI 能夠即時監控生產線上的設備運行狀況，發現異常並做出快速反應。例如，AI 系統能夠利用傳感器數據進行預測性維護，提前識別設備故障的潛在風險，從而減少停機時間和維修成本。透過影像辨識技術，邊緣 AI 還可以進行員工安全檢測，確保作業環境符合安全標準，及時發現不安全行為並做出警告，保障工廠內部的安全。

2. 智慧家居：提升生活品質

智慧家居是邊緣 AI 應用的另一個快速發展領域。通過邊緣 AI，家庭設備可以變得更加智慧化。比如，使用影像辨識技術來識別家庭成員的動作或臉部，從而自動調整家居環境。邊緣 AI 可以控制燈光、空調、家電等設備的運行狀況，根據環境變化或用戶的需求進行即時調整，提供更便捷、舒適的居住體驗。智慧門鎖、家庭監控系統和語音助手等設備，也利用邊緣 AI 技術提高了家庭的安全性和便捷性，讓生活更加智慧。

3. 智慧醫療：實時監控與健康管理

在智慧醫療領域，邊緣 AI 的應用改變了醫療服務的提供方式。AI 技術可以實時監控患者的健康狀況，並即時反應。舉例來說，穿戴設備可以隨時檢測用戶的心率、血壓、體溫等生理數據，並通過邊緣 AI 進行數據分析，當數據異常時，系統能夠立即發出警報，提醒醫療人員進行干預。此外，邊緣 AI 還可以用於藥物辨識、疾病預測等任務，對患者的健康管理起到輔助作用。它還可以協助進行跌倒偵測，尤其對於老年人群體，當偵測到跌倒事件時，系統會即時通知家人或醫療機構，減少危險並提高治療效率。

4. 智慧農業：提升農業管理效率

邊緣 AI 在農業領域的應用也日益增長。利用圖像分析技術，邊緣 AI 可以對農作物的生長狀況進行監測，檢測作物的健康狀態，及早發現病蟲害或營養不良等問題，並根據數據給出最佳的處理建議。這樣不僅可以提高農業生產效率，還能減少化學農藥的使用，實現更可持續的農業管理。此外，邊緣 AI 還能夠對農田環境進行即時監控，通過分析土壤濕度、氣候變化等因素，為農業決策提供有力支持，從而提高作物的產量和質量。

貳、NuMaker-M55M1 簡介

一、NuMaker-M55M1 開發板規格

NuMaker-M55M1 是新唐科技推出的微控制器系列，專為終端人工智慧應用設計，具有卓越的運算能力和低功耗特性，能夠應對邊緣計算所需的高效能需求。這款微控制器的核心架構和運算能力具有以下技術特點：

1. 處理器核心

- (1) **Arm® Cortex®-M55 核心**：M55M1 微控制器搭載了 Arm® Cortex®-M55 處理器，運行頻率高達 220 MHz，具備強大的數位訊號處理能力，適用於各種高效能的邊緣計算任務。**Cortex®-M55** 處理器專為低功耗和高效能設計，能夠滿足嵌入式系統對於計算與能效的雙重需求。
- (2) **高效數位訊號處理**：此核心可以高效處理各種數位訊號處理（DSP）任務，支持高性能數據處理，對於音頻處理、圖像處理以及其他 AI 應用都能提供穩定的性能支持。

2. 神經網路處理單元（NPU）

- (1) **Arm® Ethos™-U55 NPU**：M55M1 配備了專門為人工智慧設計的 Arm® Ethos™-U55 神經網路處理單元（NPU），運行頻率為 220 MHz。這款 NPU 能夠高效加速神經網路的運算，特別適合在資源受限的邊緣設備中運行 AI 模型，顯著提升推論效能。
- (2) **支援深度學習推論**：Ethos™-U55 NPU 支援多種神經網路架構，尤其是卷積神經網路（CNN）、遞歸神經網路（RNN）以及各類深度學習模型，提供強大的推論能力，尤其在物體識別、語音識別、圖像分類等應用中發揮出色的效果。
- (3) **支援 YOLO 模型**：此 NPU 還特別支援 YOLO（You Only Look Once）模型的推論，這是一種常見的實時物體檢測模型，廣泛應用於自動駕駛、智慧監控、安防等領域。

3. 記憶體

- (1) **1.5 MB SRAM**：M55M1 配備了 1.5 MB 的靜態隨機存取記憶體（SRAM），提供高速數據存取能力。這一容量適合進行大規模的運算與數據處理，尤其對於需要高效能計算的人工智慧（AI）應用非常合適。
- (2) **2 MB 快閃記憶體**：這款微控制器內建 2 MB 的快閃記憶體（Flash），可用於儲存程序程式碼、配置數據或其他非易失性資料。Flash 記憶體的容量使得系統能夠支援較大的程序運行，滿足各種複雜應用的需求。

4. 擴充性

- (1) **HyperBus 介面**：M55M1 支援 HyperBus 介面，可用於擴展外部 RAM 和 ROM。此介面適合對大量數據運算或是儲存有需求的嵌入式應用，如圖像處理、深度學習推論或大型數據儲存等場景。HyperBus 的支援使得這款微控制器具有極強的擴展能力，可以應對更高效的運算需求。

5. 安全性

- (1) **安全啟動（Secure Boot）**：Secure Boot 技術可保證在啟動過程中，只有合法且未經篡改的程式碼才能被執行，從而防止不當程式碼或惡意軟體對系統的干擾，保護系統啟動階段的安全性。
- (2) **Arm® TrustZone**：Arm® TrustZone 提供硬體級的安全隔離，將系統劃分為「安全區域」和「非安全區域」。這一技術能夠確保敏感數據的處理和儲存只在安全區域內進行，有效防止非授權訪問，從而加強整體系統的安全性。
- (3) **隨機數生成器（TRNG）**：M55M1 配備了硬體隨機數生成器（TRNG），能夠生成高品質的隨機數字，廣泛應用於加密和安全通訊中，提高數據安全性。
- (4) **加密模組（AES、ECC、RSA 等）**：該微控制器內建的加密模組包括 AES（Advanced Encryption Standard）、ECC（Elliptic Curve Cryptography）和 RSA（Rivest-Shamir-Adleman）等常見的加密算法，用於保障資料傳輸與儲存的安

全性，確保敏感資訊不會在傳輸過程中遭到竄改或竊取。

6. 支援 Arm® Helium 技術

Arm® Helium 技術是一種專為數位訊號處理（DSP）和機器學習（ML）應用設計的硬體加速技術，能顯著提升數據處理能力，尤其在低功耗嵌入式系統中具有重要意義。M55M1 微控制器內建 Arm® Helium 技術，提供以下優勢：

- (1) **效能提升：** Helium 技術可以為機器學習和數位訊號處理應用提供效能提升，顯著加快數據處理速度，並最佳化計算效能，特別適合用於實時 AI 推論和數據處理等場景。
- (2) **軟體工具與函式庫支持：** CMSIS-DSP 和 CMSIS-NN：這些是 Arm® 提供的高效能數位訊號處理（DSP）和神經網路（NN）函式庫。這些工具能充分發揮 Helium 技術的優勢，簡化應用開發過程，並提供現成的高效算法，讓開發者能更輕鬆地在 M55M1 上實現深度學習、音頻處理、圖像處理等功能。

7. 通訊介面

- (1) **UART（Universal Asynchronous Receiver/Transmitter）：**用於與其他設備進行序列通訊，支持簡單、低成本的連接。
- (2) **SPI（Serial Peripheral Interface）：**高速數據傳輸介面，適用於與外部儀器、感測器設備的交互。
- (3) **I2C（Inter-Integrated Circuit）：**低功耗且可靠的通訊協定，適用於連接多個外部設備，如感測器或顯示模組。
- (4) **USB（Universal Serial Bus）：**支持與各類外部裝置進行高速數據交換，增強了設備擴展性。

8. 周邊功能

- (1) **24 通道 PWM（Pulse Width Modulation）：**可用於控制電機、LED、音頻等設備，支持多通道的同步控制，實現精細的輸出控制。

- (2) **2 組 SAR ADC (Successive Approximation Register Analog-to-Digital Converter)**：用於將訊號類比訊號轉換為訊號數位訊號，適合處理各類感測器數據。
- (3) **1 組溫度感測器**：內建溫度感測功能，可用於實時監控系統溫度，適合嵌入式環境監控和保護功能。

9. 低功耗設計

- (1) M55M1 微控制器支援多種低功耗模式，使其在空閒或無需持續運算時能有效降低能耗，延長設備的使用壽命，特別適合於便攜設備和需要長時間運行的應用。
- (2) M55M1 配備了多個電壓監測模組，能夠監控系統的電壓狀況，防止因電壓異常導致系統錯誤或崩潰。

二、Hardware Configuration

M55M1 微控制器方案基於 Arm® Cortex®-M55 (CPU) 和 Ethos™-U55 (NPU) 的架構，主要特性如下：

1. CPU - Cortex®-M55

- (1) 主頻範圍：200 ~ 240 MHz

2. NPU - Ethos™-U55

- (1) 每個時鐘週期最高支持 256 MAC (8-bit) 運算效能。
- (2) 支援神經網路推論，如 CNN (卷積神經網路) 和 YOLO 模型。
- (3) 提供專門針對 AI 工作負載的運算加速。

3. 記憶體：

- (1) **1.5 MB SRAM**：適合高效能計算的即時數據存取需求。
- (2) **2 MB Flash**：內嵌快閃記憶體，用於儲存程式和非易失性數據。

4. HyperBus 擴充 - 支援 1 組 HyperBus

- (1) 頻率：系統頻率除以二（舉例：主頻為 220MHz, HyperBus 頻率則為 110 MHz)
- (2) 最大擴展容量：32 MB
- (3) 適合高效能嵌入式儲存的擴展需求。

5. 外部介面支持

- (1) 相機模組支持：1 組介面，可直接連接相機模組，應用於影像處理或視覺識別場景。
- (2) PDM 數位麥克風支持：多達 4 組 PDM 數位麥克風介面，用於語音處理和聲音識別。

表 2、硬體規格

	M55M1
CPU	Cortex®-M55
NPU	Ethos™-U55
Clock	220MHz
MAC/cc(8-bit)	256
Memory	1.5MB SRAM
HyperRAM(on-board)	8 MB
Flash	2MB Embedded
HyperBus	1(110MHz, 32MB)
Camera	1
PDM Dig. Mic	4

(一) 正面視圖

1. 處理器核心

- (1) 核心：M55M1 是基於 Arm® Cortex®-M55 與 Ethos™-U55 的處理器核心。

2. 記憶體儲存

- (1) HyperRAM: 支援高效能的外部 RAM 擴展，用於儲存更大數據集或進行更快的數據交換。

3. 通訊介面

- (1) FS-USB 連接器：全速 USB 介面，用於與外部設備進行數據傳輸。
- (2) HS-USB 連接器：高速 USB 介面，適合需要快速數據傳輸的應用場景。
- (3) RJ-45 介面：支持以太網連接，用於實現網路通訊。

4. Wi-Fi 通訊

- (1) ESP-12F 模塊：內建 Wi-Fi 功能，支持無線連接，適合 IoT 應用。

5. 音頻與感測器

- (1) MEMS 麥克風：支持多個數位麥克風輸入，適用於語音處理和音頻分析。
- (2) Phone-jack（耳機介面）：提供音頻輸出，適合多媒體或音頻處理的應用。
- (3) MPU6500：內建 6 軸加速度計與陀螺儀，用於運動檢測或姿態感測。

6. 內嵌調試與燒錄工具

- (1) Nu-Link2-Me：內建調試工具，支持快速開發與程序燒錄。
- (2) ICE USB 連接器：用於連接開發工具，進行系統調試。
- (3) VCOM Switch：虛擬 COM 序列埠，便於開發者與開發板進行序列埠通訊。
- (4) 狀態 LED：顯示調試器或系統的運行狀態。
- (5) 離線燒錄按鈕：支持脫離電腦進程序燒錄。

7. 用戶互動與擴展介面

- (1) 用戶按鍵：可用於調試或應用交互操作。
- (2) 用戶 LED：提供指示功能，用於顯示系統狀態或測試結果。
- (3) mikroBUS 介面：支持連接多種擴展模組，增加系統功能。
- (4) M55M1 擴展介面：與 Arduino UNO 兼容的擴展介面，適合快速原型開發。

8. 電源與系統控制

- (1) Reset 按鈕：重置系統，便於開發調試。
- (2) 電源 LED：顯示板卡供電狀態，便於確認系統是否正常通電。

9. 其他功能

- (1) CAN FD Transceiver：支持 CAN FD 通訊協議，適合工業自動化和車載應用。

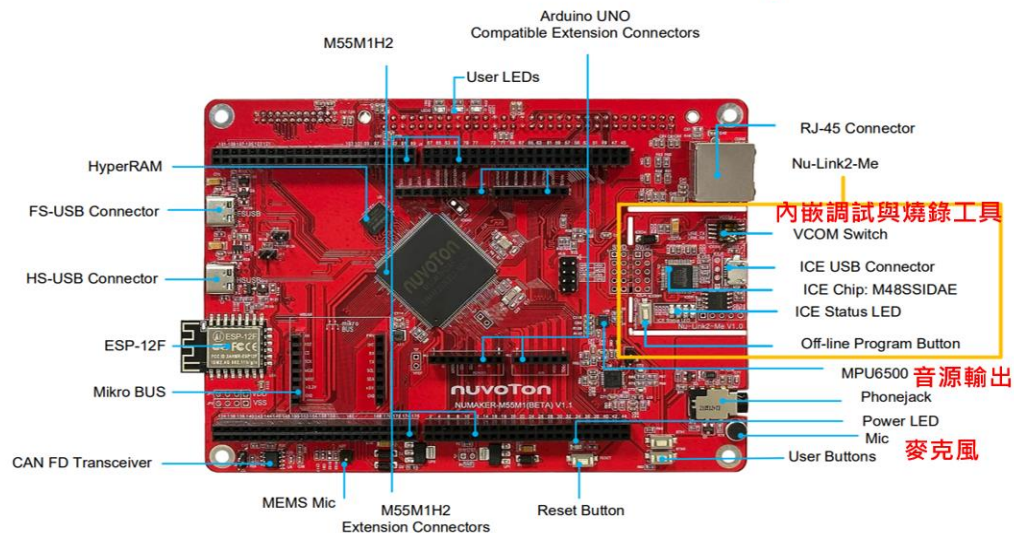


圖 1、開發板正面視圖

(二) 反面視圖

1. 顯示介面

- (1) TFT LCD 顯示器介面 (LCD I/F Connector)：支援 TFT LCD 顯示模組的連接，適合進行視覺化操作或顯示應用數據。提供可靠的介面，方便開發者在開發過程中進行顯示相關功能的調試與應用。

2. 相機模組介面

- (1) CMOS I/F Connector：支援 CMOS 相機模組的連接，適合用於影像處理、視覺辨識或實時監控等應用場景。

3. 儲存擴展

- (1) SD 卡槽 (SD Card Slot)：支援外部儲存卡，用於數據儲存或程序升級。適合需要大容量儲存的應用，例如錄影或圖像數據的保存。

4. 電源管理

- (1) ICEVCC Power Switch：用於調節調試器（ICE）的供電狀態，確保在進行調試時提供穩定電源。
- (2) MCUVCC Power Switch：用於調節 MCU 的主電源供應，支持開發者根據實際需求啟用或禁用主控電源。



圖 2、開發板反面視圖

(三) 電源供應介面

1. FS-USB 連接器

- (1) 功能：全速 USB 介面（Full Speed USB），可作為開發板的供電來源。
- (2) 應用場景：適合標準 USB 供電的應用，便於從電腦或其他 USB 電源輸出設備獲取穩定電源。

2. HS-USB 連接器

- (1) 功能：高速 USB 介面（High Speed USB），同樣支持供電功能。
- (2) 應用場景：當需要高速數據傳輸與供電功能結合使用時，非常適合嵌入式開發。

3. NU1 Pin8 (VIN)

- (1) 功能：提供一個外部直流電壓輸入介面，用於開發板的供電。
- (2) 電壓範圍：需要根據電路設計提供適配的直流電壓輸入。

4. 外部 VDD 和 VSS 連接器

- (1) External VDD Connector：支持外部電源（正極）供應給開發板。
- (2) External VSS Connector：接地（負極）介面，與 VDD 配合提供穩定電壓。
- (3) 應用場景：適合需要自定義電源解決方案或特殊供電需求的開發場景。

5. ICE USB Connector

- (1) 功能：用於通過 USB 進行調試與編程，同時提供供電功能。
- (2) 應用場景：當使用 Nu-Link 調試器進行開發時，可直接透過該介面供電並進行數據傳輸。



圖 3、開發版電源供應

三、開發板擴充

NuMaker-M55M1 開發板支持多種擴展介面，包含 M55M1 Extension Connectors、Arduino UNO、mikroBUS，為開發者提供了更多的應用開發選項。

(一) Arduino UNO 相容擴展介面

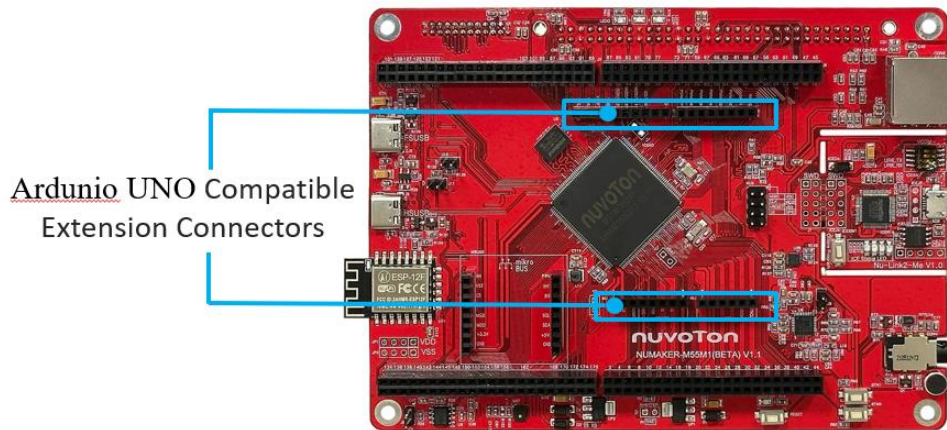


圖 4、Arduino UNO 相容擴展介面

1. 硬體相容性

- (1) 使用者可以通過 Arduino 擴充模組（如感測器、顯示器、無線通訊模組等）輕鬆擴展開發板的功能。

2. 接腳分佈與用途

(1) 數位訊號接腳（Digital Pins）

- a、提供多達 14 個數位 I/O 接腳（D0-D13），用於輸出或輸入數位訊號。
- b、支援多種協議（如 UART、SPI），適合用於控制模組、通訊以及數據交換。

(2) 類比訊號接腳（Analog Pins）

- a、提供 6 個類比接腳（A0-A5），可用於讀取類比感測器的輸入訊號，例如溫度、光強度或距離感測器。

(3) 電源接腳

- a、包括 3.3V、5V、VIN（外部供電）和接地（GND）介面，提供模組所需的多種電壓供應。

(二) mikroBUS 相容擴展介面

1. 標準化設計

- (1) mikroBUS 是由 MikroElektronika 設計的一種標準化腳位介面，旨在使用最少的引腳數量實現最大的擴展能力。
- (2) 其引腳布局簡單、明確，支持多種通訊協議（I2C、SPI 和 UART），適合快速模組化開發。

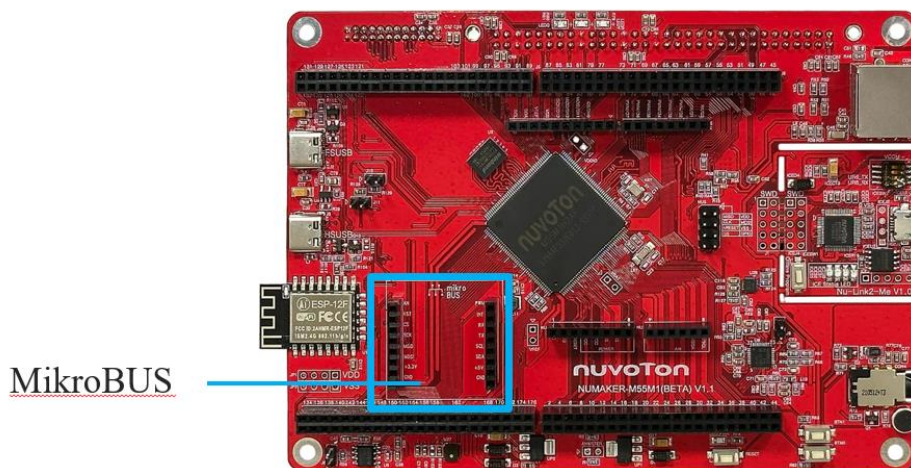


圖 7、mikroBUS 相容擴展介面

2. 兼容 Click Boards 系列模組

- (1) mikroBUS 主要用於連接 Click Boards 系列擴展模組。
- (2) Click Boards 提供超過 1645 種不同功能的擴充模組，涵蓋感測器、通訊、顯示、控制和電源管理等應用領域。



圖 8、Click Boards 擴充模組

(三) M55M1 擴展介面 (Extension Connectors)

1. 標準化介面設計

- (1) 介面分佈在開發板的上下兩側，提供多個引腳以支援外部擴展設備。
- (2) 可以根據需求進行自定義配置，用於連接各種感測器或其他專用模組。

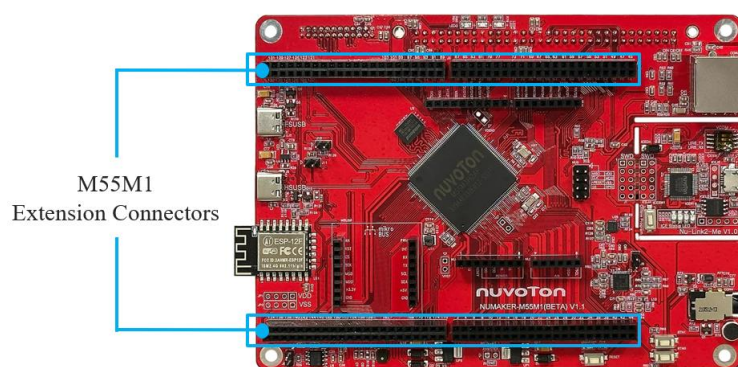


圖 9、M55M1 擴展介面

2. 高兼容性

- (1) 支援多種外圍介面，包括 GPIO、I2C、UART、SPI 等通訊協議。
- (2) 可與多樣的外部設備無縫對接，例如顯示器、儲存設備、數據通訊模組等。

參、開發應用整體部署流程說明

一、整體部署流程圖

M55M1 開發板的整體部署流程共分為五大階段(如下圖)，分別為 TinyML 訓練、輕量化模型與框架轉換、模型 Vela 編譯與配置、C 語言程式撰寫以及部署與燒錄至開發板。以下針對每個階段進行詳細介紹。

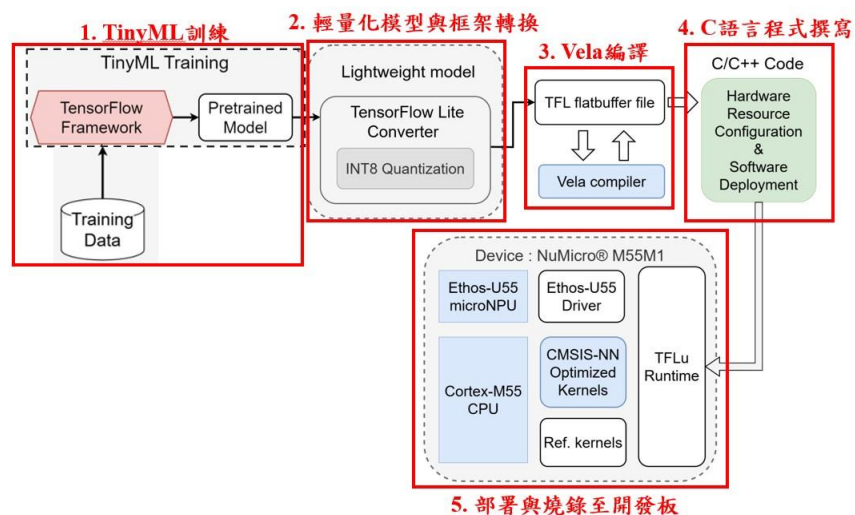


圖 10、使用 TensorFlow 框架

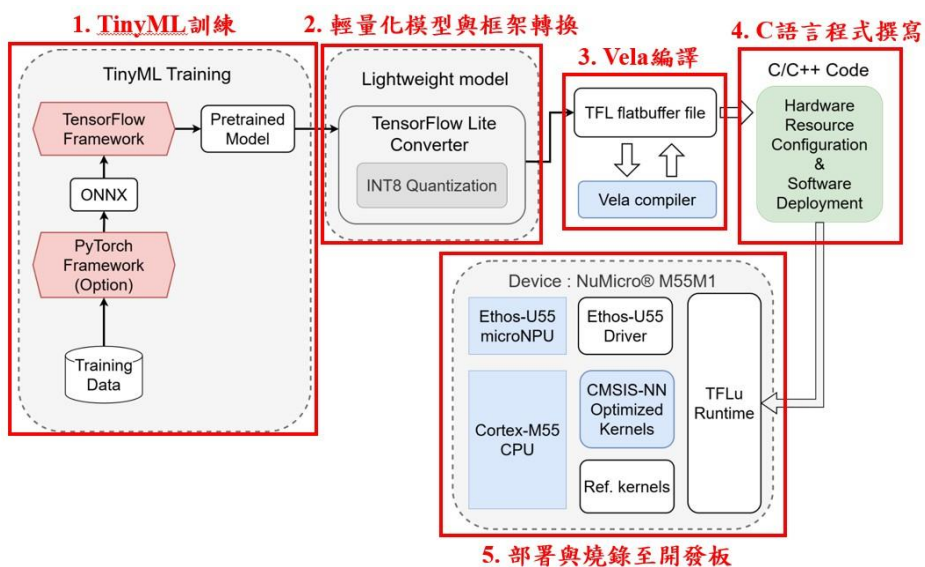


圖 11、使用 PyTorch 框架

(一) TinyML 訓練

TinyML 訓練階段是整個流程的基礎，其目的是生成預訓練模型，步驟如下：

1. 框架選擇：支援 TensorFlow 和 PyTorch 兩種深度學習框架，若使用 PyTorch 進行訓練，需將模型透過 ONNX 格式轉換至 TensorFlow 以適配開發板環境。
2. 資料集準備：使用者需準備高質量的訓練資料集（如 Kaggle、Roboflow 提供的開源數據），資料需分為訓練集、驗證集與測試集。
3. 模型選擇：使用 YOLOX-Nano 等輕量化模型，該模型具有小尺寸、高效能的特性，適合在資源有限的嵌入式設備上運行。
4. 輸出：生成預訓練模型，作為後續量化和部署的基礎。

(二) 輕量化模型與轉換

為了滿足 M55M1 開發板的性能需求，需對模型進行量化與轉換：

1. TensorFlow Lite Converter：將 TensorFlow 的模型轉換為 TensorFlow Lite 格式（TFLite），此格式專為嵌入式系統設計，具輕量化的特點。
2. 量化：採用 INT8 全整數量化技術，能有效縮小模型大小，降低記憶體佔用與計算需求，同時最大化利用開發板中的 Ethos™-U55 微型神經處理單元（NPU）。

(三) 模型 Vela 編譯與配置

Vela 編譯器是針對 Arm® Ethos™-U NPU 設計的專用工具，用以最佳化 TFLite 模型並提升運算性能：

1. 編譯器功能：將適合 NPU 運算的神經網路運算子最佳化或合併分配至 Ethos™-U55 NPU，加速推理過程。無法被 NPU 支援的神經網路運算子部分則由 Cortex®-M55 CPU 處理。

2. 配置選項：開發者可選擇記憶體使用最佳化（最小化 SRAM 占用）或性能最佳化（最大化推理速度）的模式。
3. 輸出：最佳化後的 TFLite 模型，準備進入硬體部署階段。

(四) C 語言程式撰寫

模型部署至開發板需要撰寫硬體資源配置及推理程式碼：

1. 硬體資源配置：使用 C/C++ 語言控制 M55M1 開發板的周邊設備（如 NPU、PWM、UART 介面等）。
2. 推理邏輯：撰寫 AI 模型的推理程式，實現輸入資料處理、模型運算與輸出結果解析。
3. 模型後處理：根據具體應用需求，實現結果的後續處理與應用功能（如物件偵測標記）。

(五) 部署與燒錄至開發板

最終，將配置好的程式與模型燒錄至 M55M1 開發板：

1. 燒錄工具：使用 Arm® Keil MDK μ Vision 5 或其他支援 M55M1 的開發工具。
2. 燒錄流程：包括專案建立、硬體配置、模型與程式碼導入，並通過 micro USB 進行燒錄。
3. 驗證與調試：完成燒錄後，調試系統以達到穩定運行的狀態。

(六) 總結

此完整部署流程將 TinyML 技術與嵌入式開發結合，通過 M55M1 開發板在邊緣設備上實現高效的 AI 應用。整體流程不僅適合初學者快速上手，也能幫助專業開發者構建精密的嵌入式智慧系統。

二、 TinyML 模型訓練

(一) 應用場景概述

本次教學案例的應用場景涵蓋四大智慧領域：

1. 智慧工廠：包括人員安全監測、設備運行狀態檢測及預防性維護。
2. 智慧家居：應用於智慧家電控制、異常檢測，如火災與入侵警報。
3. 智慧醫療：實現跌倒偵測、藥物辨識等。
4. 智慧農業：用於水果品質分析與魚苗數量計算等。

這些應用場景對即時性與準確性的要求極高，因此模型選擇上需要兼顧性能與資源消耗。

(二) 使用的框架與模型

1. 框架選擇：PyTorch

- (1) PyTorch 是目前深度學習領域中最受歡迎的框架之一，選擇它的原因在於其多方面的優勢：

2. 動態計算圖：

- (1) PyTorch 的計算圖是動態的，這意味著每次執行操作時都會生成新的計算圖，能夠靈活處理不同的數據格式和模型結構。

3. 直觀的 API 設計：

- (1) PyTorch 提供了類似於 Python 的操作風格，開發者可以以更直觀的方式構建和調試模型。
- (2) 其基於 Tensor 的操作與 NumPy 十分相似，適合從數據科學過渡到深度學習的用戶。

4. 社群與生態系統：

- (1) 廣泛的開發者社群，提供開源模型和工具，降低學習曲線。
- (2) 支援多種預訓練模型（如 ResNet、YOLO、Transformers），加快模型開發流程。

5. 支持 GPU 加速：

- (1) PyTorch 能夠無縫支持 GPU 計算，大大提高深度學習訓練效率。
- (2) 針對多 GPU 訓練提供內建分布式計算功能。

6. 支持 ONNX 格式：

- (1) PyTorch 訓練完成的模型可以轉換為 ONNX 格式，便於將模型移植到其他平台（如 TensorFlow 或邊緣設備）。

(三) 模型選擇：YOLOX-Nano

1. YOLOX-Nano 的介紹與特點

- (1) YOLOX-Nano 是 YOLOX 系列中專為資源受限場景設計的輕量化版本。目標是兼顧嵌入式系統的計算資源限制和實時物件檢測的需求。

(四) 資料集準備

1. 模型訓練過程中，資料集需分為三大類型，確保模型在不同階段的最佳化：

(1) Training Set（訓練資料集）：

- a. 用於模型特徵學習和參數調整，提高對訓練數據的擬合度。

(2) Validation Set（驗證資料集）：

- a. 用於訓練期間評估模型性能，及時檢測過擬合問題。

(3) Test Set（測試資料集）：

- a. 在未知數據上評估模型的真實泛化能力與實際應用性能。

(五) 模型訓練平台選擇：

在進行深度學習模型訓練時，選擇適合的開發環境至關重要。常見的選擇包括雲端平台如 Google Colab，以及本地端訓練環境。每種環境各有其優缺點，適合不同的使用情境。以下將詳細比較這兩種環境的特性，協助您根據需求做出最佳選擇。

1. Google Colab

Google Colab 是由 Google 提供的免費雲端平台，允許使用者在瀏覽器中撰寫和執行 Python 程式碼，特別適合深度學習模型的開發與訓練。

2. 優點：

免費提供 GPU 資源：Google Colab 提供免費的 NVIDIA Tesla K80、T4、P100 或 V100 GPU，適合進行深度學習模型的訓練。

(1) 即時環境配置：

預先安裝了常用的深度學習框架（如 TensorFlow、PyTorch），使用者無需自行配置環境，開箱即用。

(2) 雲端儲存整合：

可直接與 Google Drive 整合，方便存取和保存訓練資料及模型。

(3) 跨平台使用：

只需瀏覽器即可操作，無論在 Windows、macOS 還是 Linux 系統上均可使用。

3. 缺點：

(1) 資源限制：

免費版 GPU 使用時間有限制，長時間訓練可能會中斷，需重新連接。

(2) 性能波動：

由於資源共享，GPU 性能可能因使用者數量而波動，影響訓練穩定性。

(3) 網路依賴：

需要穩定的網路連接，網速或連接問題會影響訓練過程。資料可能會有隱私疑慮，資料儲存於雲端，可能存在隱私和安全性風險，需注意資料保護。

4. 本地端訓練

本地端訓練指在自己的電腦或伺服器上進行模型訓練，適合需要高性能和完全控制的使用者。

5. 優點：

(1) 高性能硬體：

可使用高效能 GPU（如 NVIDIA RTX 系列），提供更快的訓練速度和穩定性。

(2) 完全控制：

對硬體和軟體環境擁有控制權，根據需求最佳化和調整。

(3) 資料安全：

所有資料儲存於本地，降低資料外洩風險。

(4) 無資源限制：

無需擔心雲端平台的資源限制，可進行長時間或大規模的模型訓練。

6. 缺點：

(1) 高成本投入：

需要購買高性能硬體，投入成本較高，且需考慮硬體維護和升級。

7. 環境配置複雜：

需要自行安裝和配置深度學習框架、驅動程式等。

8. 限制：

受限於本地硬體資源，無法像雲端平台靈活擴展計算資源。

9. 維護成本：

定期維護硬體，處理故障和更新，增加時間和人力成本。

環境架設以 NVIDIA GPU 為例

TinyML 訓練之前，硬體與軟體環境的正確配置是模型訓練成功的基石。針對使用 NVIDIA GPU 進行深度學習開發的需求，推薦以下兩個步驟進行環境搭建。

10. CUDA 與 cuDNN 的安裝與配置

- (1) CUDA和cuDNN是 NVIDIA 提供的 GPU 加速工具，用於高效處理深度學習。
- (2) CUDA：CUDA 是一個平行計算平台，通過提供開發者 API（應用程式介面），使深度學習框架如 TensorFlow 或 PyTorch 能利用 NVIDIA GPU 進行加速運算。
- (3) cuDNN：cuDNN 是 NVIDIA 專為深度學習設計的 GPU 加速庫，提供高度最佳化的卷積神經網路計算功能，能顯著提升訓練與推理速度。
- (4) 安裝流程：
 - a. 瀏覽NVIDIA官方網站，選定你的作業系統以及想要下載的版本，須注意顯卡所支援的版本。(https://developer.nvidia.com/cuda-toolkit-archive?source=post_page-----ab5425746bd0)

- b. 根據 GPU 型號和作業系統，選擇適合的 CUDA 版本下載並安裝。
- c. 在cuDNN官方下載頁面選擇對應的 cuDNN 版本，下載後將文件解壓並安裝至 CUDA 路徑中
(https://developer.nvidia.com/cudnn?source=post_page-----ab5425746bd0)。

(5) 注意事項：

- a. 確保 CUDA 和 cuDNN 的版本與所使用的深度學習框架（如 PyTorch）兼容。
- b. 安裝完成後，請透過指令 `nvcc --version` 檢查 CUDA 是否正確配置。

(六) 使用 Anaconda 建立虛擬環境

瀏覽 Anaconda 官方網站 (<https://www.anaconda.com/>) 下載適合作業系統的版本。Anaconda 是一個開源平台，用於簡化 Python 環境管理和資料科學開發，特別適合處理多版本 Python 和依賴的問題，有以下優點：

- 1. 可以在同一台電腦上安裝多個獨立的 Python 虛擬環境，不會因框架衝突而影響其他項目。
- 2. 提供管理工具 Conda，便於安裝 TensorFlow、PyTorch 等深度學習框架。
- 3. 簡化環境切換與維護，對於需要頻繁切換開發環境的使用者友善。

(一) TinyML 模型介紹

1. 早期基礎 - CNN

CNN 的歷史可以追溯到 1980 年代，Yann LeCun 提出了最早的卷積神經網路架構，並且在 1998 年推出了 LeNet-5 模型，這是一個成功應用於手寫數字辨識的模型。LeNet-5 開創了卷積神經網路在計算機視覺中的應用，並且促使研究者們開始嘗試將深度學習技術應用到更為複雜的影像識別任務中。

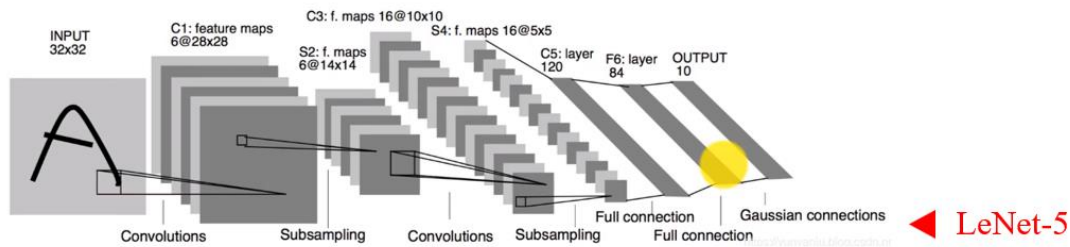


圖 12、LeNet-5 架構圖

進一步的突破出現在 2012 年，由 Alex Krizhevsky 提出的 AlexNet 模型通過使用更深的網路結構以及 ReLU 激活函數等技術，使得圖像分類的錯誤率從 26.2% 大幅下降到 15.3%，並且在 ImageNet 比賽中一舉奪冠，揭開了深度學習在計算機視覺中迅速普及的序幕。AlexNet 的成功展示了深度神經網路處理大規模圖像數據的潛力，並激發了大量後續模型的誕生。

R-CNN 系列模型是另一個重大突破，首次將區域建議結合卷積神經網路，將影像中的物體進行定位和分類。R-CNN 的核心流程包括透過邊界框 (Bounding Box) 在影像中定位多個物體位置，然後利用深度學習模型進行特徵提取與分類，從而準確地辨識物體種類。這樣的多階段處理使得 R-CNN 能夠提供較高的準確性，但由於流程的複雜性，也存在運算速度較慢、硬體需求較高的缺點。

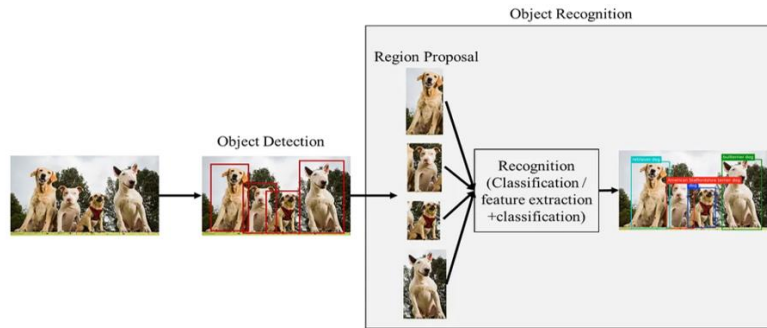


圖 13、RCNN 示意圖

YOLO（You Only Look Once）系列的誕生解決了 R-CNN 的一些局限。YOLO 由 Joseph Redmon 等人在 2016 年提出，提出了一種全新的單階段物體檢測方式。YOLO 的核心理念在於一次性地完成物體的檢測與分類，這大幅提高了物體檢測的速度和效率。相比於 R-CNN，YOLO 的主要優勢在於速度的提升，使其非常適合即時應用場景。同時，YOLO 的計算資源需求較低，這也使其非常適合在資源受限的嵌入式系統中部署，例如智慧家庭設備、無人機等。

這些技術進展構成了 YOLO 模型的前生，也展示了深度學習技術在物體檢測領域的逐步演變，最終促使我們得到了像 YoloX Nano 這樣高效、輕量的模型，能夠在低功耗的邊緣設備上實現優異的性能表現。

2. YOLO 系列的核心特點

YOLO 系列的物件檢測模型以其高效的單階段檢測方式而著稱。該模型將影像劃分為固定的網格，每個網格負責偵測多個邊界框的座標、信心度以及物體類別的辨識機率。這種單階段的方式（One-Stage）使得 YOLO 能夠在短時間內完成物體定位與分類，適合需要即時回應的應用場景。相比於傳統的物件檢測方法，YOLO 採用了完全卷積的神經網路架構，能有效地簡化步驟，將物體檢測、邊界框生成和分類任務整合到一個模型中，從而顯著提高處理速度。

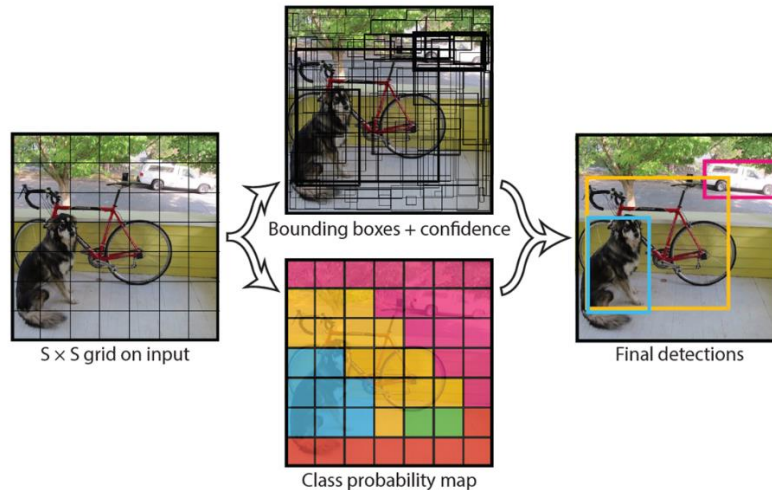


圖 14、YOLO 辨識示意圖 ¹

YOLO 的主要優勢在於速度快和運算負荷低。由於 YOLO 的架構能夠同時進行物體定位與分類，它非常適合用於對計算資源有較高要求的嵌入式設備。此外，由於它的計算步驟被簡化，因此需要的硬體資源較少，這意味著在運算能力有限的邊緣設備上依然可以實現高效的物體檢測能力。

3. RCNN 與 YOLO 的比較

在物件檢測的發展歷程中，RCNN (Region-based Convolutional Neural Network) 是一種較早的深度學習模型，最初通過將影像中的候選區域提取出來，再進行特徵提取和分類來完成物體的辨識。然而，RCNN 的多階段處理方式，如區域提議、特徵提取和分類，使其在運算速度上較為緩慢，並且對硬體資源的要求非常高。

YOLO 的出現改變了這一點。相比於 RCNN，YOLO 將所有步驟整合在一個模型中，採用單階段的方法完成檢測，這顯著提高了檢測速度，且有效降低了系統的計算負載。YOLO 還採用了 Anchor-Free 的機制，使得模型不必再依賴複雜的先驗框設置，這在一定程度上進一步簡化了計算過程並提高了效率，適合用於即時應用

¹ "Methods for object recognition and classification for tele-controlled service robots," IOP Conference Series: Materials Science and Engineering, vol. 878, no. 1, 2020, doi: 10.1088/1757-899X/878/1/012005.

場景。

4. YOLO 系列進展

(1) 各版本特點

YOLO 系列模型經歷了多次演進，每一版本都有顯著的改進。YOLOv5 是基於 PyTorch 開發的，具有易於使用和部署的特點，並引入了自適應錨點生成（AutoAnchor）技術，能夠根據輸入數據自動調整模型結構，進一步提升了檢測的準確性和部署靈活性。YOLOX-Nano 則是 YOLO 系列中專門為邊緣運算設計的版本，其極度輕量化的特點使其能夠在低資源設備上實現高效的物體檢測。該模型採用 EfficientNet-Lite 作為 backbone，通過減少參數量來提升運算效率，並保持了模型的高性能。

YOLOv7 在模型結構設計上進行了重新設計，以進一步提升推理速度和準確率，特別適合實時應用場景。YOLOv8 則採用了模塊化設計，支持多任務應用，例如物體檢測、圖像分割和姿態估計，使得該模型具有很高的靈活性和擴展性，能夠根據應用需求進行靈活配置。

5. YOLOX-Nano 模型詳解

(1) 參數與性能

YOLOX-Nano 是 YOLO 系列中最輕量級的版本之一，參數量僅約 1.0M，模型大小約為 2MB。儘管其體積小巧，YOLOX-Nano 在輕量化模型中依然能達到良好的準確性和效率平衡，這使其成為資源受限設備上的理想選擇。該模型在設計時強調高效能與低運算負荷的結合，尤其適用於嵌入式系統和物聯網設備。

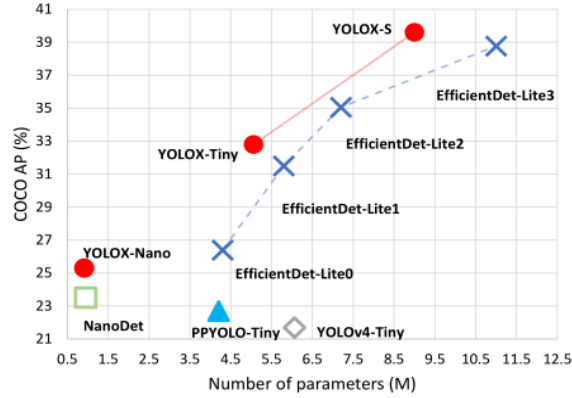


圖 15、YOLO 官方 GithubX-Nano 官方 Github 性能比較

6. 模型架構

YOLOX-Nano 的架構包括三個主要部分。首先是 **Backbone**，它採用 **EfficientNet-Lite**，這是一種經過最佳化的輕量級網路，能夠在保持高效能的同時減少計算量。其次是 **Detection Head**，YOLOX Nano 採用了 **YOLO-style** 檢測頭，能夠快速處理多物體的檢測任務。最後，YOLOX-Nano 採用了 **Anchor-Free** 的檢測方法，這種設計不需要預設錨框，使得檢測過程更加靈活和高效。

7. 優勢與應用場景

YOLOX-Nano 的主要優勢在於其高速檢測能力和精簡的模型結構，特別適合資源受限的設備，例如行動裝置、嵌入式系統和物聯網應用。由於其參數量小且計算需求低，YOLOX-Nano 易於部署並能夠在多種不同的硬體平台上運行。這使得它在需要高效率與低功耗的應用中脫穎而出。

YOLOX-Nano 通過其高效的設計和性能最佳化，實現了在低資源設備上的高效物體檢測。該模型的輕量化和高準確性使其成為邊緣設備應用的理想選擇，尤其在需要低延遲和低功耗的場景中具有巨大的潛力。未來，YOLOX-Nano 將進一步推動 TinyML 在多領域的應用發展，為智慧城市、物聯網和其他資源受限的應用提供可靠支持。

隨著邊緣運算技術的不斷發展，YOLOX-Nano 這類輕量級物體檢測模型將變得愈加重要。它能夠適應各種不同的應用場景，並且能夠在硬體限制的情況下提供高效能的解決方案，這對於未來的智慧化社會建設有著重要的意義。透過不斷的最佳化和改進，YOLOX-Nano 及其後續版本有望進一步提升性能，拓展更多應用可能性，為更多資源受限的場景提供智慧解決方案。

(二) YOLOX-Nano 訓練流程



圖 16、YoloX 官方圖

1. 參考資料：

- YOLOX-Nano 的官方 GitHub 資源可供使用者參考：
<https://github.com/Megvii-BaseDetection/YOLOX>
- 在該網址中，您可以找到 詳細說明與範例程式碼。

2. 資料準備

(1) 目的與重點：

- a. 資料集的準備是 YOLOX-Nano 訓練流程的關鍵第一步，目的是為模型提供豐富且多樣化的影像數據，從而提升模型的物件偵測準確性。

(2) 資料來源：

a. 提供了三個常見的開源資料集平臺，這些平臺適合進行影像處理與標註：

- (a) **Kaggle**：全球最大的數據競賽平臺，包含大量已標註的影像資料集，例如 COCO 資料集。
- (b) **Roboflow**：專為機器學習而設計的影像數據平台，提供標註、增強以及管理功能，適合快速構建訓練數據集。
- (c) **Open Images Dataset**：Google 提供的大型開源影像數據集，涵蓋多樣化的場景與標註類別，適合物件偵測與分類任務。



圖 17、Roboflow



圖 18、Open Images Dataset



圖 19、Kaggle

當以上資料準備完成後，即可進入模型的 TinyML 訓練階段。

(三) 機器學習框架(PyTorch 或 Tensorflow)

訓練框架可使用 PyTorch 或 Tensorflow，因本開發板需在 Tensorflow 框架下實現，故若使用 PyTorch 開發需增加利用 ONNX 執行轉換為 Tensorflow 的框架。

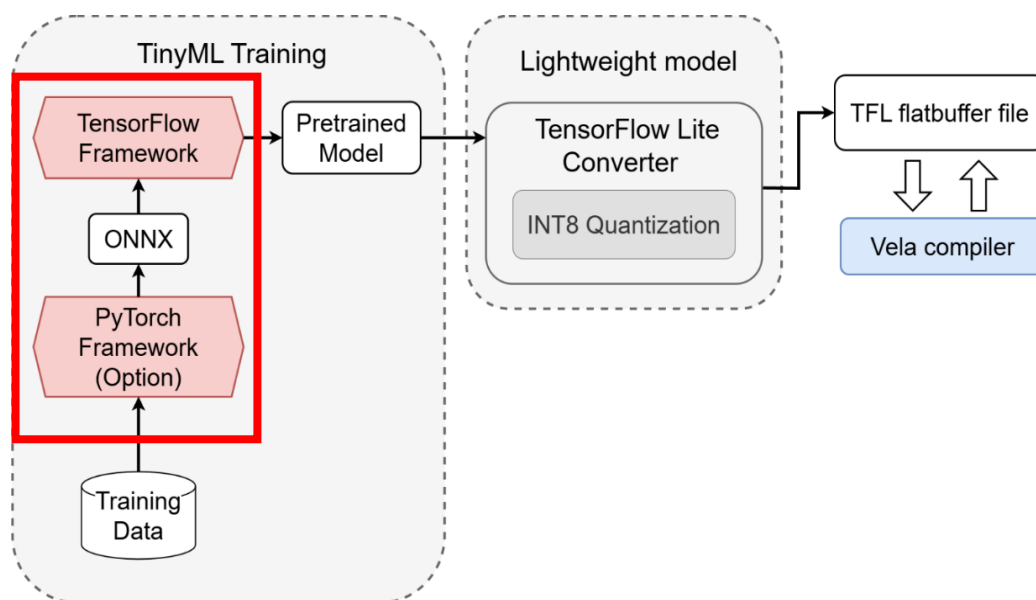


圖 20、訓練流程圖

(四) 模型轉檔

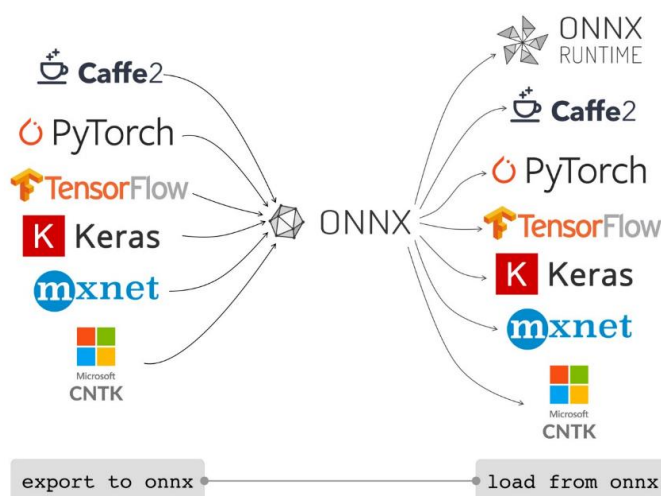


圖 21、ONNX 模型轉檔

三、模型輕量化

概述：模型輕量化是 TinyML 領域中的一個重要步驟，目的是將預訓練的深度學習模型轉化為適合嵌入式設備的輕量化模型，從而在資源有限的硬體上實現高效推理。此過程通常包括模型壓縮和量化技術，以降低計算和儲存需求，提升模型性能。

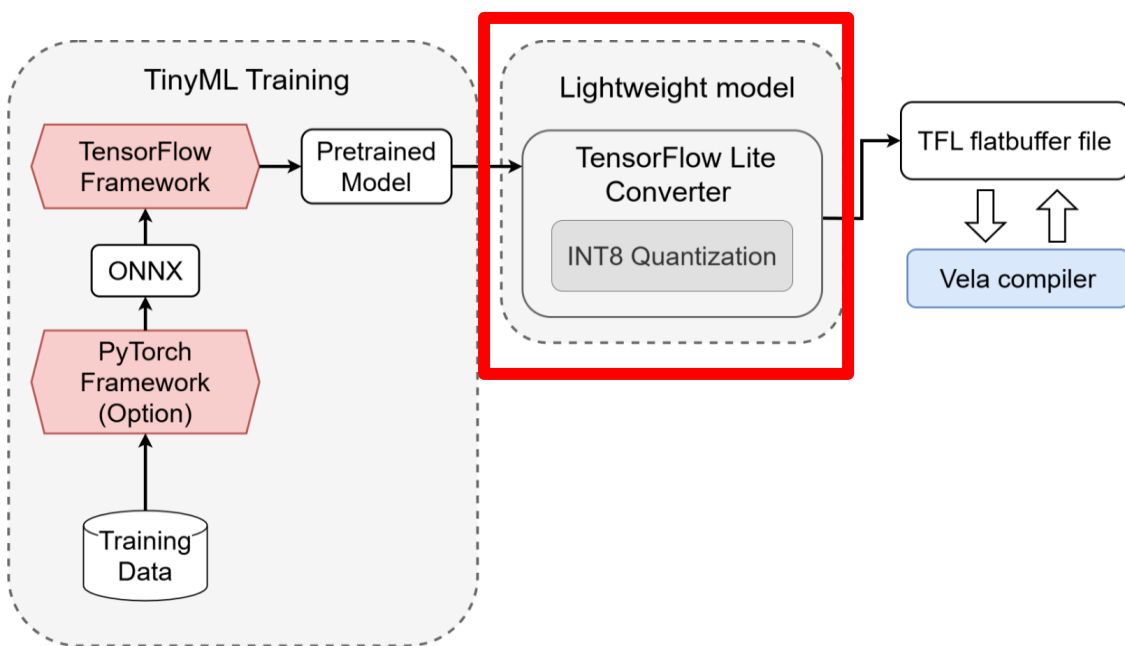


圖 22、訓練流程圖

(一)TensorFlow Lite Converter

1. TensorFlow Lite 的應用與功能

- (1) TensorFlow Lite 是一種行動裝置程式庫，專為在行動裝置、微控制器及其他邊緣設備上高效部署深度學習模型設計。
- (2) 其核心目的是讓預訓練模型能夠在資源有限的環境中，保留原始推理能力的同時顯著減少記憶體佔用和運算資源需求。

2. TensorFlow Lite Converter 的角色

- (1) TensorFlow Lite Converter 是 TensorFlow Lite 的重要組件，用於將 TensorFlow 模型（通常是 .pb 或 SavedModel 格式）轉換為輕量化的 .tflite 格式。
- (2) 經過轉換的模型具有更小的儲存空間需求，並可直接在資源受限的嵌入式設備上進行高效推理運算。

3. 量化技術的優勢

- (1) 在轉換過程中，可應用 **Quantization 技術**（例如 INT8 量化）來進一步最佳化模型大小和性能。
- (2) 該技術不僅顯著降低了模型的儲存和計算成本，還在大多數應用場景中保持推理精度，對於邊緣設備的部署尤為重要。

(二) Model INT8 Quantization

1. 量化技術的應用背景

- (1) 量化技術是深度學習模型最佳化的重要工具，特別是在資源受限的環境中（例如手持設備、物聯網裝置等），可以顯著減少模型大小，提升推理速度。
- (2) 儘管量化過程中可能導致部分精度損失，但在大多數應用中，這種損失可以接受，且對性能提升的幫助是顯著的。

2. INT8 量化的特點與優勢

- (1) INT8 量化可將模型中使用的權重和激勵函數，從高精度的浮點數（FP32 或 FP16）轉換為 8 位整數表示。
- (2) 這種轉換不僅有效地縮小了模型的記憶體佔用，還顯著降低了運算資源的消耗，特別是在嵌入式或邊緣設備中部署模型時，這是一項非常關鍵的最佳化手段。

3.硬體支持與應用

- (1) 許多專門針對邊緣運算最佳化的處理器（例如 Arm® 的 Ethos™ NPU）是專門設計用來支持 全整數 INT8 格式的推理運算。
- (2) 在這些處理器上部署 INT8 量化模型，可以充分發揮硬體的性能優勢，實現高效且資源友善的深度學習應用。

4.量化方式分類與詳細說明

在模型量化的過程中，根據應用場景和性能需求，量化技術可分為以下三種主要類型：

(1) 全整數量化 (Full Integer Quantization)

- a **描述：**將模型中的權重與激勵函數完全量化為整數表示，這包括所有的模型運算也在整數域內完成。
- b **特點：**
 - (a) 浮點數轉換為整數後，模型佔用的記憶體空間顯著減少。
 - (b) 適合資源受限的嵌入式設備，能有效提升運算速度和效率。
- c **應用案例：**本次實作案例皆採用全整數量化，充分發揮其在嵌入式硬體上的優勢。

(2) 動態範圍量化 (Dynamic Range Quantization)

- a **描述：**僅在推理時將浮點數值轉換為整數表示，但部分運算（如權重初始化）仍然保留浮點表示。
- b **特點：**
 - (a) 是一種快速量化方法，適合於測試需求或對性能提升需求不高的場景。

- (b) 相較於全整數量化，精度可能略低，但計算效率仍有一定程度的提升。

(3) 混合量化 (Hybrid Quantization)

- a **描述：**部分模型層保留浮點表示，其餘層進行量化處理。
- b **特點：**
 - (a) 適合對精度要求較高的模型，例如需要保留部分浮點運算的層來提升模型性能。
 - (b) 在模型大小和運算效率上，介於全整數量化和完全浮點模型之間，是一種折衷方案。

5. 量化流程

量化是一個多步驟的過程，需確保模型在記憶體和運算效率上得到最佳化，同時保持推理性能的穩定性與準確性。以下為具體流程說明：

(1) 準備浮點模型

- a 在模型訓練完成後，將 FP32 模型導出為標準格式（如 TensorFlow SavedModel）。
- b 這些高精度的浮點模型為量化提供了基礎，並確保模型初始狀態的準確性。

(2) 校準數據集

- a 提供真實且具代表性的數據，用於模擬模型在實際應用場景中的運行情況。
- b 校準數據的選擇和準備對於量化精度至關重要，確保模型在降低數值精度時仍能保持推理性能與準確性。

(3) 量化過程

- a 使用專用工具（如 TensorFlow Lite Converter）進行全整數量化處理。
- b 工具會自動將模型中的權重、偏置及激勵函數轉換為 INT8 格式，從而顯著減少模型的儲存需求與計算成本。

(4) 輸出為 TFLite 格式

- a 完成量化的模型會被導出為 TensorFlow Lite (.tflite) 格式。
- b 該格式的模型高度最佳化，能方便地部署於資源受限的設備（如邊緣裝置或嵌入式系統）。

四、 Vela 編譯器 (NPU 運算加速)

在 TinyML Training 階段，模型可以通過下圖步驟進行。

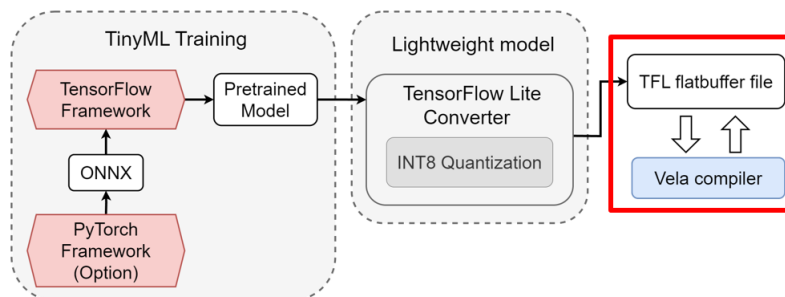


圖 23、訓練流程圖

(一) Vela 功能簡介

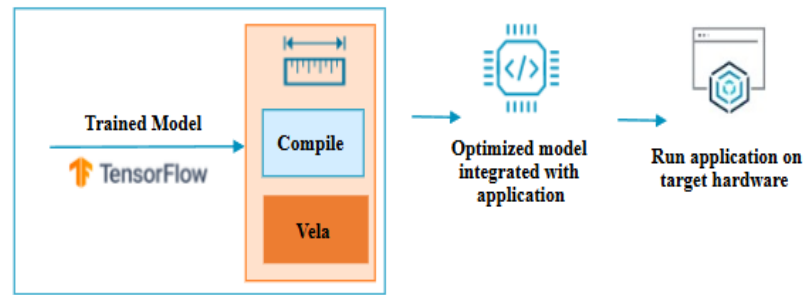


圖 24、Vela 編譯

1.Vela 功能簡介

- (1) Vela 是專為 Arm® Ethos™-U 神經處理單元 (NPU) 設計的神經網路模型編譯工具。該工具可將 TensorFlow Lite 模型編譯為最佳化版本，以便在包含 Ethos™-U NPU 的嵌入式系統上運行。

2.主要功能：

- (1) 最佳化 TensorFlow Lite 模型，生成經過最佳化的 TFLite 文件。
 - a Vela 可以針對嵌入式環境，對模型結構進行裁剪和最佳化，以最大程度地減少計算需求與記憶體占用，從而更適合在資源受限的設備上運行。
- (2) 將模型中適合的運算操作分配到 Ethos™-U NPU，藉此加速模型推理。
 - a Vela 分析模型中的運算操作，並自動將符合條件的部分分配至 Ethos™-U NPU 上執行。這可以顯著提高模型的運行速度。
- (3) 無法加速的部分則在 Cortex®-M 系列 CPU 上運行，並使用 Arm® 最佳化的 CMSIS-NN 核心執行：
 - a 針對無法由 NPU 加速的部分，將改由 CPU 執行。為了提高效率，這些操作將利用 Arm® 提供的 CMSIS-NN 庫進行最佳化。
- (4) 記憶體訪問模式最佳化、和指令流最佳化：
 - a Vela 對模型中的記憶體訪問方式進行最佳化，減少訪問延遲。此外，

對指令流也進行最佳化，確保運行過程中能夠更快地處理指令。

(二) Vela 環境與擴展

ethos-u-vela 對應支持的 TensorFlow 以及 Python 版本詳細說明如下網址：
<https://pypi.org/project/ethos-u-vela/>

1. **運行環境：**ethos-u-vela 可以在 Linux 和 Microsoft Windows 10 操作系統上運行，這意味著無論使用的是哪一種主要桌面操作系統，都可以輕鬆地部署此工具來最佳化模型。
2. **詳細擴展：**ethos-u-vela 提供了嵌入式環境下模型編譯的重要功能，使得開發者能更好地利用嵌入式硬體加速深度學習模型的推理過程。這工具允許開發者將符合條件的深度學習模型以更高效的方式部署到特定的硬體平臺，例如 Arm® 的 Ethos™-U NPU，以達到顯著的性能提升。

(三) Vela 指令簡介

1. 指令：

- (1) 針對特定 Ethos™-U NPU 的編譯：

```
$ vela --accelerator-config ethos-u55-256 helmet_quantized_.tflite
```

說明：這條指令將模型 helmet_quantized_.tflite 配置到指定的 ethos-u55-256 NPU 上進行最佳化編譯，確保能夠在該硬體上高效運行。

- (2) 最佳化以最小化峰值 SRAM 使用量：

```
$ vela --optimise Size my_model.tflite
```

說明：這條指令將模型 my_model.tflite 最佳化為最低峰值的記憶體需求，適合在具有非常有限記憶體的嵌入式設備上運行。

最佳化以最大化性能：

```
$ vela --optimise Performance my_model.tflite
```

說明：這條指令著重於性能的最佳化，適合應用在有足夠硬體資源且追求運行速度的場景中。

(3) 在記憶體使用限制內最佳化性能：

```
$ vela --optimise Performance --arena-cache-size 300000 my_model.tflite
```

說明：這條指令以 300,000 bytes 的記憶體容量為限制，進行模型 my_model.tflite 的性能最佳化，確保模型能在記憶體資源受限的環境中依然發揮出色性能。

(四) Vela 編譯結果

1.指令：

```
$ vela --accelerator-config ethos-u55-256 helmet_quantized_.tflite
```

(1) 說明：在這條指令中，ethos-u55-256 是指定的 NPU 型號及 MAC 數，而 helmet_quantized_.tflite 是要編譯的模型，且使用預設最佳性能最佳化選項。這樣能確保模型在目標 NPU 上執行時可以達到最佳性能。

```
Accelerator configuration      Ethos_U55_256
System configuration          internal-default
Memory mode                   internal-default
Accelerator clock              500 MHz
Design peak SRAM bandwidth    4.00 GB/s
Design peak Off-chip Flash bandwidth 0.50 GB/s

Total SRAM used                2028.00 KiB
Total Off-chip Flash used      1168.20 KiB

CPU operators = 161 (28.8%)
NPU operators = 398 (71.2%)
```

圖 25、輸出結果

2. 說明：

- (1) 模型中有(71.2%)的運算操作由 NPU 加速執行，表明 NPU 負責大部分的計算負荷。
- (2) (28.8%)的運算操作則由 CPU 處理，這些操作可能是由於無法通過 NPU 加速的特定運算，或者是 NPU 不支持的操作類型。

3.總結：

- (1) 經過 Vela 編譯後，大部分運算負荷成功由 NPU 加速處理（71.2%），這樣可以顯著提高模型的執行效率並減少推理時間。剩餘的 28.8% 由 CPU 處理，適合不支持 NPU 加速的情形，這樣可以達到兩者之間的平衡來獲得最佳性能。

(五) YOLOX-Nano 模型片段可視化：

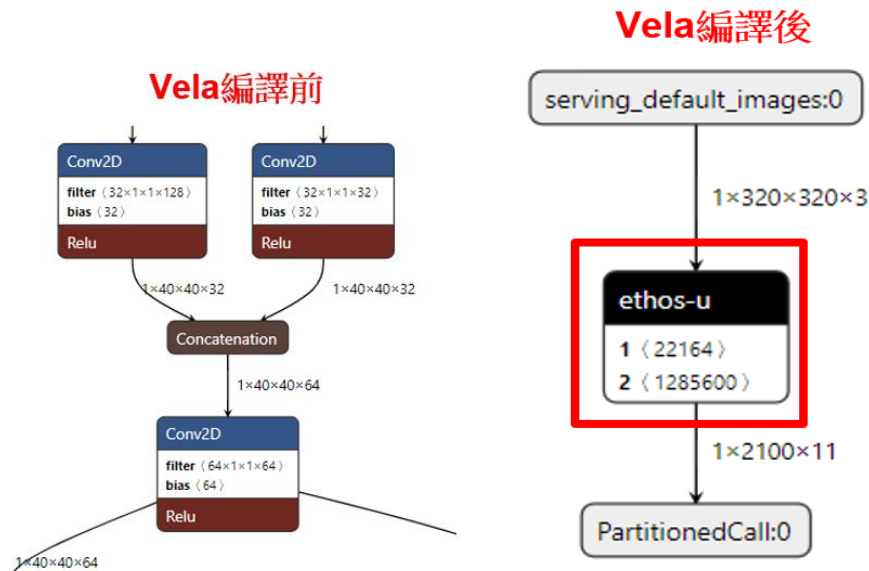


圖 26、模型片段可視化

在這兩張圖中，我們可以看到 YOLOX-Nano 模型在 Vela 編譯前後的計算圖形變化情況。

1. 編譯前：

模型最初在 CPU 上進行計算時，有一個 Concatenation（串接）操作，這需要額外的記憶體讀寫操作，導致模型運行的整體效率降低。在 CPU 上進行串接操作會導致 SRAM（靜態隨機存取記憶體）的額外負擔，這會拖慢整體推理速度，特別是在運行大模型或資源受限的嵌入式設備上。

2. Vela 編譯後：

使用 Ethos™-U NPU 進行編譯後，編譯器將模型的結構進行了最佳化，將串接操作變得不再必要。Ethos™-U NPU 更適合處理切片後的小型張量，從而減少了數據搬運的工作量，顯著提升了性能。NPU 負責處理更多的計算負荷，這樣的最佳化使得模型在嵌入式硬體上的執行更加高效。在圖中可以看到，編譯後 NPU 能直接

處理張量，而不需要進行串接，從而節省了記憶體拷貝所帶來的性能損失。

3. 總結：

透過使用 Vela 將 YOLOX-Nano 模型編譯後，將原先需要在 CPU 上進行的串接操作移除，交由 NPU 更高效地處理。這樣的變化意味著模型在嵌入式設備上可以達到更好的性能，減少記憶體帶寬的使用，並使得運行時間縮短。Vela 編譯的優勢體現在使得 NPU 可以承擔更多的計算，減少記憶體搬運，從而加速推理過程，這對於資源受限的嵌入式系統非常重要。

五、使用 C 語言撰寫軟硬體設備配置

M55M1 BSP 資源載點 <https://github.com/OpenNuvoton/M55M1-eBook-Sample-Code>，物件辨識案例範例程式 ObjectDetection_FreeRTOS_yoloxn 可從該載點取得，往後官方 BSP 會持續進版更新，並提供下載，載點如下，<https://github.com/OpenNuvoton/M55M1BSP>，各資料夾與程式碼總覽，本章節會分別介紹驅動設置位置以及物件辨識案例各檔案(圖 29)所負責之功能。

(一)、Document (M55M1BSP/Document) (圖 27)

1. CMSIS.html:CMSIS 資料存放位置
2. NuMicro M55M1 Series Driver Reference Guide.chm:解釋 M55M1 系列 BSP 驅動功能
3. Revision History.pdf:介紹 M55M1 系列 BSP 修正歷程

 CMSIS.html	2024/5/24 13:43	Chrome HTML D...	1 KB
 NuMicro M55M1 Series Driver Referen...	2024/5/24 13:43	編譯的 HTML 說明...	89,120 KB
 Revision History.pdf	2024/5/24 13:43	PDF Document	114 KB

圖 27、Document (M55M1BSP/Document)

(二)、Library (M55M1BSP/ Library) (圖 28)

1. CMSIS:Cortex® Microcontroller Software Interface Standard 相關資料
2. Commu:通訊協議的輔助功能，例如 XMODEM
3. CryptoAccelerator: MbedTLS 函式庫的加密加速器原始碼
4. Device: 符合 CMSIS 標準的設備標頭文件
5. JpegAcceleratorLib: 用於 libjpeg 的 SIMD 加速器和標頭檔
6. PowerDeliveryLib: 支持雙角色(源和接收器)的電力傳輸函式庫和標頭檔
7. SmartcardLib: 智能卡函式庫和標頭檔
8. StdDriver: 所有周邊驅動的標頭檔和原始碼
9. UsbHostLib: USB Host 函式庫的程式碼原始碼

■ CMSIS	2024/8/1 13:09	檔案資料夾
■ Commu	2024/8/1 13:09	檔案資料夾
■ CryptoAccelerator	2024/8/1 13:09	檔案資料夾
■ Device	2024/8/1 13:09	檔案資料夾
■ JpegAcceleratorLib	2024/8/1 13:09	檔案資料夾
■ PowerDeliveryLib	2024/8/1 13:09	檔案資料夾
■ SmartcardLib	2024/8/1 13:09	檔案資料夾
■ StdDriver	2024/8/1 13:09	檔案資料夾
■ UsbHostLib	2024/8/1 13:09	檔案資料夾

圖 28、Library (M55M1BSP/Library)

■ Device	2024/11/22 上午 12:18	檔案資料夾	
■ GCC	2024/11/22 上午 12:18	檔案資料夾	
■ IAR	2024/11/22 上午 12:18	檔案資料夾	
■ KEIL	2024/11/22 上午 12:18	檔案資料夾	
■ Model	2024/11/22 上午 12:18	檔案資料夾	
■ NPU	2024/11/22 上午 12:18	檔案資料夾	
■ Pattern	2024/11/22 上午 12:18	檔案資料夾	
■ ProfilerCounter	2024/11/22 上午 12:18	檔案資料夾	
📄 BoardInit.cpp	2024/11/23 下午 07:41	C++ 來源檔案	4 KB
📄 BoardInit.hpp	2024/9/3 上午 12:16	C++ Header 來...	1 KB
📄 DetectionResult.hpp	2024/9/3 上午 12:16	C++ Header 來...	2 KB
📄 DetectorPostProcessing.cpp	2024/11/23 下午 07:41	C++ 來源檔案	10 KB
📄 DetectorPostProcessing.hpp	2024/9/3 上午 12:16	C++ Header 來...	5 KB
📄 InferenceTask.cpp	2024/11/23 下午 07:41	C++ 來源檔案	4 KB
📄 InferenceTask.hpp	2024/9/3 上午 12:16	C++ Header 來...	2 KB
📄 main.cpp	2024/9/3 上午 12:16	C++ 來源檔案	21 KB
📄 mpu_config_M55M1.h	2024/9/3 上午 12:16	C Header 來源檔案	10 KB

圖 29、物件辨識案例 範例程式各資料夾與程式碼總覽

(二)Display 內容介紹:

Display.c 負責核心顯示功能，主要作用為初始化 LCD 硬體(包括配置 EBI 介面和多功能管腳)，提供矩形區域填充功能，用於圖像處理結果的渲染，以及支援顯示字元與延遲功能，應用情境如渲染物件檢測結果的邊界框，實現系統狀態顯示（如 FPS 或模型名稱）。

Font8_16.c 負責字型定義，能定義 8x16 點陣字型數據，用於 LCD 字符顯示，並且能將字元表通過數組形式提供，與 Display_PutText() 協同工作。

LCD_FSA506.c 負責 LCD 驅動，功能為通過 EBI 介面向 LCD 發送命令與數據，以及實現 LCD 的列與頁設置（例如 fsa506_set_column()）。初始化流程為硬體重置後，設置面板大小、時鐘格式及 TFT 輸出時序。

LCD.h 負責驅動介面定義，用於 LCD 功能結構與相關配置，和提供結構 S_LCD_INFO，包括初始化函數與顯示操作的函數指針。

(三) HyperRAM 內容介紹:

hyperram_code.c 負責實現 HyperRAM 功能，主要能實現 HyperRAM 擦除、數據寫入、延遲校正等功能，另外也能提供 PLL 校正功能，用於調整讀取延遲。接著介紹上述功能的細節運作，擦除邏輯是通過 HyperRAM_Erase() 遍歷擦除區塊，確保數據為 0。PLL 校正則是實現延遲步驟校正，確保可靠的數據傳輸。默認配置為配置 Chip Select 和讀寫訪問時間。

(四) ImageSensor 內容介紹:

ImageSensor.c 為影像傳感器主模組，核心功能為初始化影像傳感器和 CCAP 模組，配置影像輸出格式（如 YUV422、RGB565 等），實現圖像擷取與裁剪功能，以及從傳感器獲取推論輸入圖像。

(五) KEIL 內容解析:

M55M1.scatter 為記憶體配置文件，功能為定義程式的記憶體配置，包括程式碼段、數據段及堆疊區域的地址範圍，且定義了 Flash 和 RAM 的分區(如圖

30)，應用場景在指導 Linker 如何將程序分配到目標硬體的記憶體。

```
131 APP_IMAGE FLASH_START FLASH_SIZE
132 {
133     ; Flash 2 MB
134     rom_exec FLASH_START FLASH_SIZE
135     {
136         *.o (RESET, +First)
137         * (InRoot$$Sections)
138         ; Make sure reset_handler ends up in root segment
139         startup_M55M1.o
140         .ANY (+RO)
141         ..\..\..\..\ThirdParty\tflite_micro\Lib\tflu.lib (.init_array)
142     }
143     rom_netwrok_executor +0
144     {
145         ..\..\..\..\ThirdParty\tflite_micro\Lib\tflu.lib (+RO)
146         ethosu_*.o
147     }
```

圖 30、M55M1.scatter 定義 Flash 和 RAM 的分區相關程式碼

HyperRAM 是一種外部的高速 RAM，適用於儲存中間計算結果或臨時數據，常用於神經網路計算或圖形渲染，優點為容量大(32MB 以上)、且通過 SPI 介面連接，成本較低，缺點為讀取速度較內部 SRAM 慢。定義 HyperRAM 地址相關程式碼如(圖 31)。

```
187 #if 0
188     HYPERRAM SPIM0_START UNINIT SPIM0_SIZE
189     {
190         ; Place tensor arena in SRAM if we do not have a fast memory area
191         .ANY (.bss.NoInit.activation_buf_sram)
192     }
193
194
195     HYPERRAM SPIM1_START UNINIT SPIM1_SIZE
196     {
197         ; Place tensor arena in SRAM if we do not have a fast memory area
198         .ANY (.bss.NoInit.activation_buf_sram)
199     }
200 #endif
```

圖 31、M55M1.scatter 定義 HyperRAM 的相關程式碼

HyperFlash 是一種高速的外部 Flash 儲存，用於儲存程式碼和 Read Only 數據，常用於存放 Firmware 或靜態資料(如圖像、音檔等)，優點為容量大(64MB 以上)，且在 SPI 介面連接下能支援快速加載資料至記憶體，缺點為寫入速度慢。定義 HyperFlash 地址相關程式碼如(圖 32)。

```

94     #define FLASH_START      0x00100000
95     #define FLASH_SIZE      0x00200000

```

圖 32、M55M1.scatter 定義 HyperFlash 的相關程式碼

SRAM 是發揮性記憶體，一旦切斷電源記憶體內部的資料會完全消失，兩者最大差異是 SRAM 的存取（access）速度是所有記憶體中最快的 IC，而且不需作類似 DRAM IC 的更新（refresh）動作，SRAM 唯一缺點是資料儲存容量偏低。。

定義 SRAM 地址相關程式碼如圖 33

```

180     SRAM012 SRAM012_START SRAM012_SIZE
181     {
182         .ANY1 (.bss.sram.data)
183         ; Place tensor arena in SRAM if we do not have a fast memory area
184         .ANY (.bss.NoInit.activation_buf_sram)
185     }

```

圖 33、M55M1.scatter 定義 SRAM 的相關程式碼

(六) MODEL 資料夾概述與解析：

標籤模組展示標籤如何從靜態數組載入到動態容器中。圖示標籤結構及其在推論中的應用。Labels.cpp 提供目標檢測模型的標籤列表，用於對推論結果進行標籤化。

YOLO 模型管理說明模型數據如何與推論過程整合。繪製模型初始化與運算支持的流程圖。YoloXnanoNu.cpp 用於封裝 YOLOX -Nano 推論模型。模型數據結構強調量化模型在 NPU 環境中的應用，yolox_nano_*.tflite.cc 為模型數據文件，包含量化後的 YOLOX Nano 模型數據。

標籤與推論的整合說明標籤模組如何與 YOLO 模型結合，產生人類可讀的推論結果。Labels.hpp 為標籤管理介面，可提供標籤模組的外部介面。

(七) NPU 資料夾概述：

1. CPU 快取管理模組描述 ethsou_cpu_cache 的功能，強調數據在 NPU 與 CPU 間一致性的管理。
2. NPU 初始化模組說明 ethosu_npu_init 的初始化步驟，繪製中斷和記憶體配

置流程。

3. 性能分析模組使用圖表展示性能計數器（如空閒週期、數據流量）如何捕獲 NPU 運行狀態。說明 `ethosu_profiler` 如何支持性能最佳化。
4. 記憶體配置模組圖解記憶體分區結構，展示快取區域和緩衝區的配置。強調對性能和數據存取效率的影響。

六、開發板程式的開發及燒錄

(一) 集成開發環境 – Arm® Keil MDK µVision 5

Arm® Keil MDK µVision 5 是一款專為嵌入式開發設計的集成開發環境 (IDE)，主要用於基於 Arm® Cortex®-M 微控制器的應用程序開發。

1. 軟體特點

- (1) **整合的開發環境：** µVision 提供從編輯、編譯到調試的全流程支持，允許開發者在單一界面中進行所有操作。
- (2) **C/C++ 編譯器：** 使用高效能的 Arm® Compiler，可以產生最佳化的程式碼，減少二進制大小並提升執行效能。
- (3) **豐富的調試工具：**

支持實時追蹤、執行分析與硬體調試。提供模擬器，允許在沒有實體硬體的情況下進程式碼測試。

2. 多樣化的版本

- (1) **Community Edition** 提供給開發者免費使用，適合學習或小型項目開發。
 - (1) 新唐科技免費提供給開發者 Keil MDK Nuvoton Edition 可用於商業開發。
(進一步了解 [Keil MDK Nuvoton Edition – Full Cortex®-M – Nuvoton](#))。

3. 主要應用領域

- (1) **物聯網 (IoT)：**開發低功耗、高性能的 IoT 設備。
- (2) **消費電子：**如家電控制器和手持設備。
- (3) **工業控制：**嵌入式系統應用如機器控制與監控。
- (4) **醫療電子：**如可穿戴裝置與監控設備。

4. 功能簡介

- (1) **設備樹支持 (Device Tree)：**簡化了晶片和周邊配置，快速生成配置程式碼。
- (2) **Pack Management：**允許開發者下載和管理不同微控制器的設備支持包

(Device Family Pack, DFP)。

- a. **RTOS 支持**：提供 Keil RTX Real-Time Operating System 的集成。允許開發者高效開發多任務應用程序。

(二) 程式開發流程

1. 建立新專案

在進行燒錄前，需要先在 Keil μ Vision 開發環境中建立一個新專案。以下是詳細步驟：

(1) 啟動 Keil μ Vision 軟體：

打開您的 Keil μ Vision 開發工具，確保您已正確安裝該軟體，並檢查是否為正確的版本。

(2) 開啟「Project」選單：

在軟體上方功能列中，找到「Project」功能表，並點擊進入。

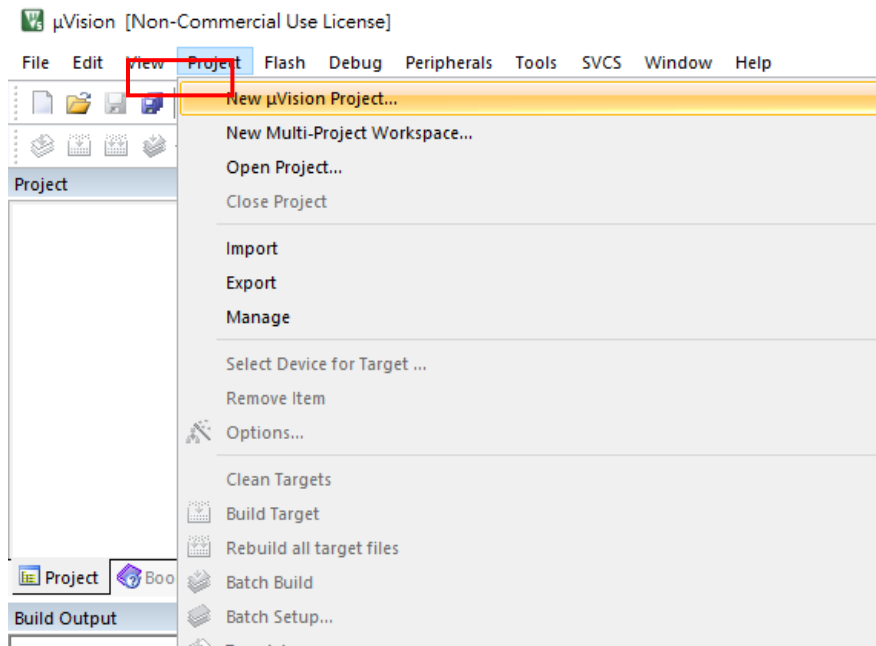


圖 34、開啟「Project」選單

(3) 選擇「New μ Vision Project」：

在下拉選單中，點擊「New μ Vision Project」選項，啟動新專案的建立過程。

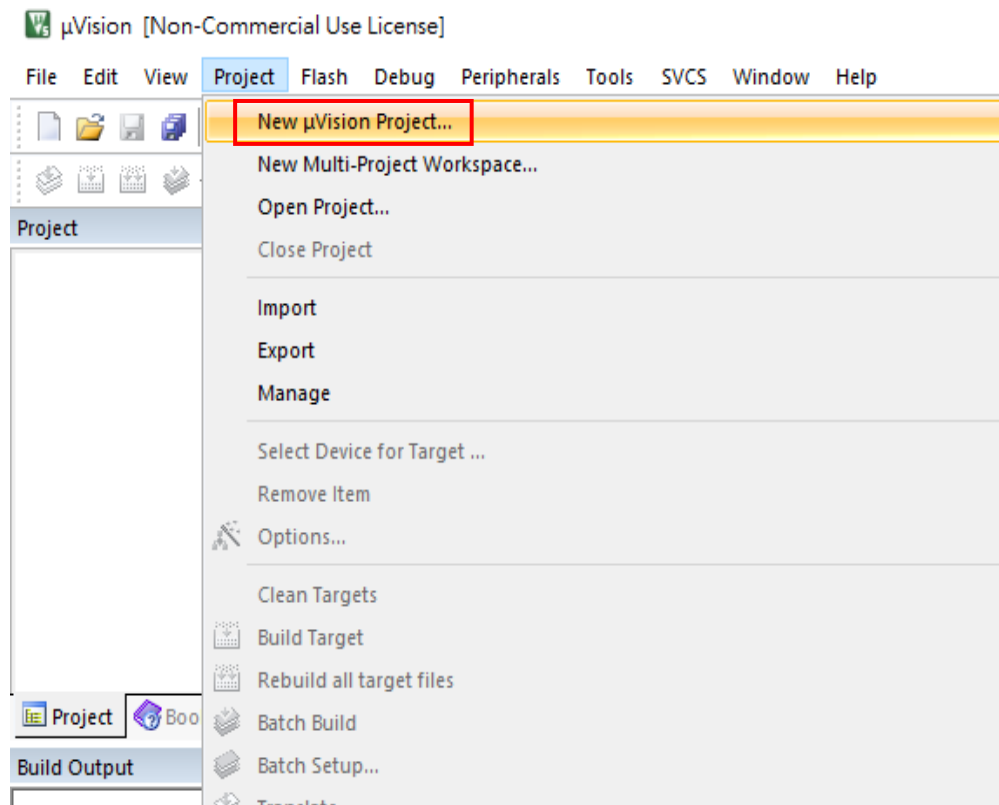


圖 35、選擇「New μ Vision Project」

(4) 命名您的專案並選擇儲存路徑：

在彈出的對話框中，為您的專案命名，並選擇一個合適的路徑用於存放專案檔案。建議使用有意義的命名方式，例如以專案名稱或日期命名，以便日後管理。

(5) 選擇目標硬體設備：

接著，系統會要求您選擇目標硬體的設備型號。從提供的設備清單中，選擇您將使用的微控制器或處理器型號（例如 Arm® Cortex®-M55）。

2. 建立專案名稱

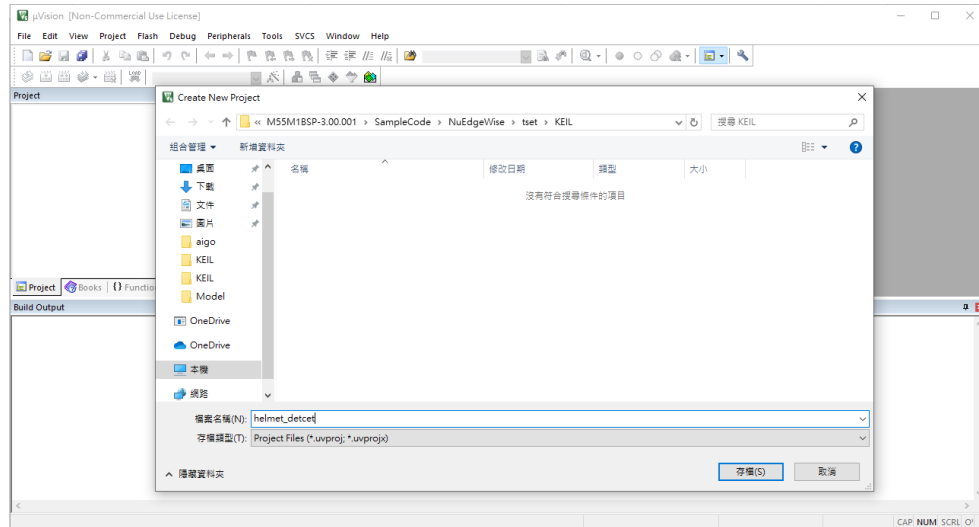


圖 36、建立專案名稱

完成新專案的建立步驟後，需要為專案命名並選擇存放路徑。以下是詳細的步驟說明：

(1) 輸入專案名稱：

在彈出的「Create New Project」對話框中，找到「檔案名稱 (N)」欄位，輸入您為此專案設定的名稱。

(2) 選擇存放路徑：

在對話框的路徑選擇區域，導航到您希望保存專案檔案的目錄。

(3) 確認並保存：

確保所有輸入內容無誤後，點擊右下角的「存檔 (Save)」按鈕，完成專案檔案的命名與保存。

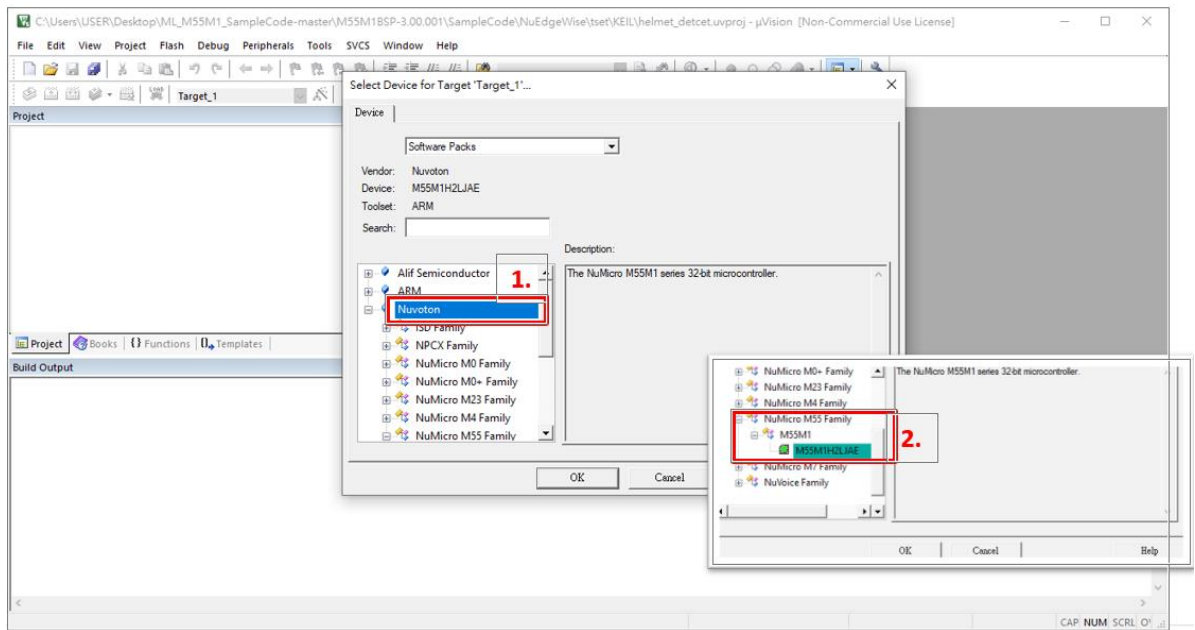


圖 37、選擇微控制器型號（選擇 M55M1 開發板型號）

在新專案建立後，需要指定目標微控制器型號。以下是選擇目標硬體的詳細步驟：

(4) 開啟「Select Device for Target」視窗：

當完成新專案保存後，Keil μ Vision 會自動彈出一個「Select Device for Target」對話框，提示您選擇目標設備。

(5) 選擇微控制器廠商：

在左側的「Device」列表中，找到並展開「**Nuvoton**」分類，以下提示。

- 如果您使用的是新唐科技的開發板，請直接搜尋或展開對應廠商的目錄。
- 您也可以在上方的搜尋欄輸入部分型號關鍵字來快速篩選。

(6) 選擇目標型號：

在展開的 Nuvoton 目錄下，依次展開「**NuMicro M55 Family**」，然後點擊「**M55M1H2LJAE**」型號

(7) 確認選擇：

點擊對話框右下角的「**OK**」按鈕，完成目標設備的設定。系統將根據所選型號自動加載相關的開發配置。

3. 選擇和配置專案中需要使用的軟體組件

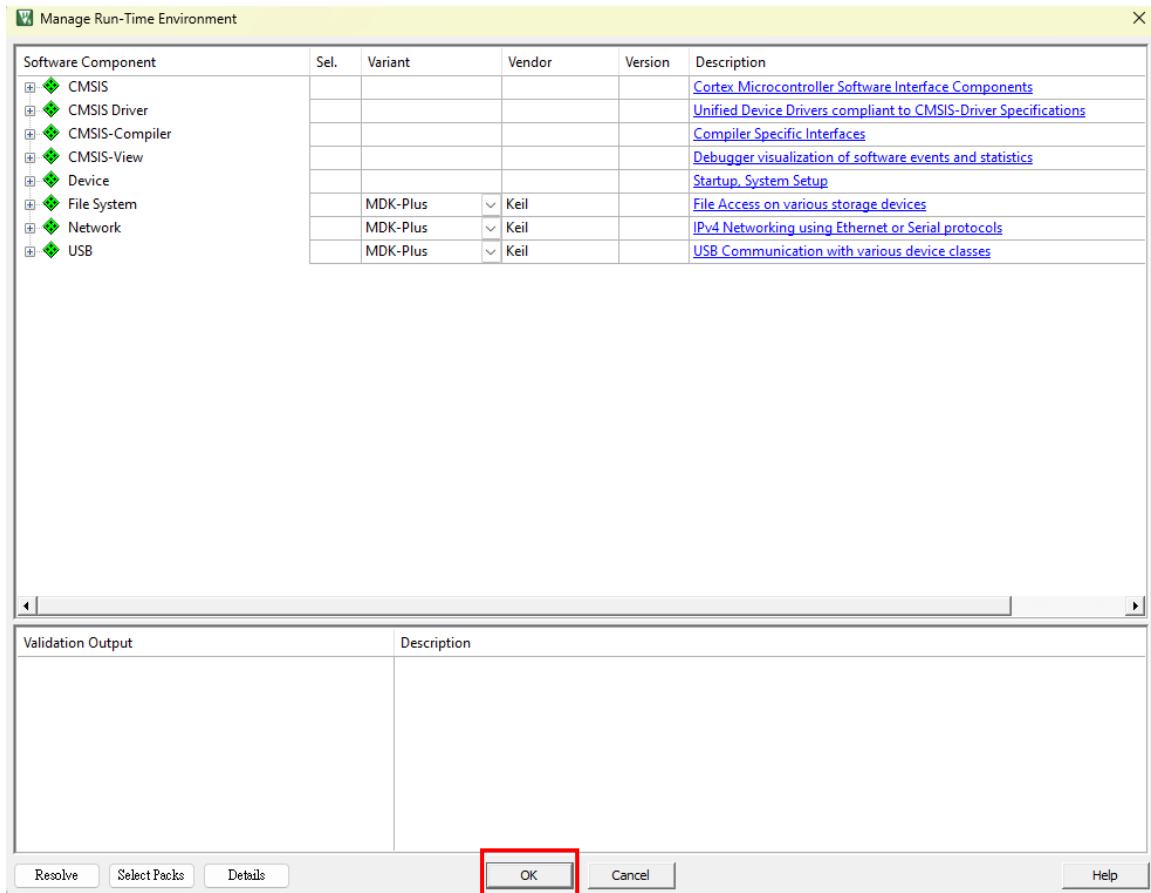


圖 38、選擇和配置專案中需要使用的軟體組件

在新專案中正確選擇和配置所需的軟體組件是開發工作的基礎步驟。以下為具體操作說明：

(1) 開啟「Manage Run-Time Environment」視窗：

在 Keil μVision 中，點擊功能列上的「Project」功能表，然後選擇「Manage Run-Time Environment」，進入組件配置界面。

(2) 選擇必要的軟體組件：

在彈出的視窗中，左側顯示了各種組件類別，例如 CMSIS、Compiler、RTOS 等，您需要根據專案需求進行選擇：

- a. **CMSIS:** Cortex® 微控制器軟體介面標準，提供基本的微控制器支持和調試工具。

- b. **Compiler:** 使用 Keil 提供的 ARM® 編譯器來支持程式編譯。
- c. **RTOS (FreeRTOS):** 如需實時作業系統支持，可選擇 FreeRTOS。
- d. **USB 和 Network:** 若專案涉及 USB 通訊或網路協議，勾選相關組件（例如 MDK-Plus 提供的 USB 支援）。

(3) 檢查右側的組件詳情：

選中每個組件後，右側會顯示其詳細資訊，包括版本號、供應商以及功能描述。請確認是否符合專案需求：

- a. **版本匹配:** 確保所選版本與開發板或硬體相容。
- b. **功能對應:** 仔細檢查組件是否支持您專案中的特定功能。

(4) 確認並點擊 OK：

配置完成後，點擊右下角的「OK」按鈕，保存所選組件。系統將自動下載和加載相關組件，並更新專案環境。

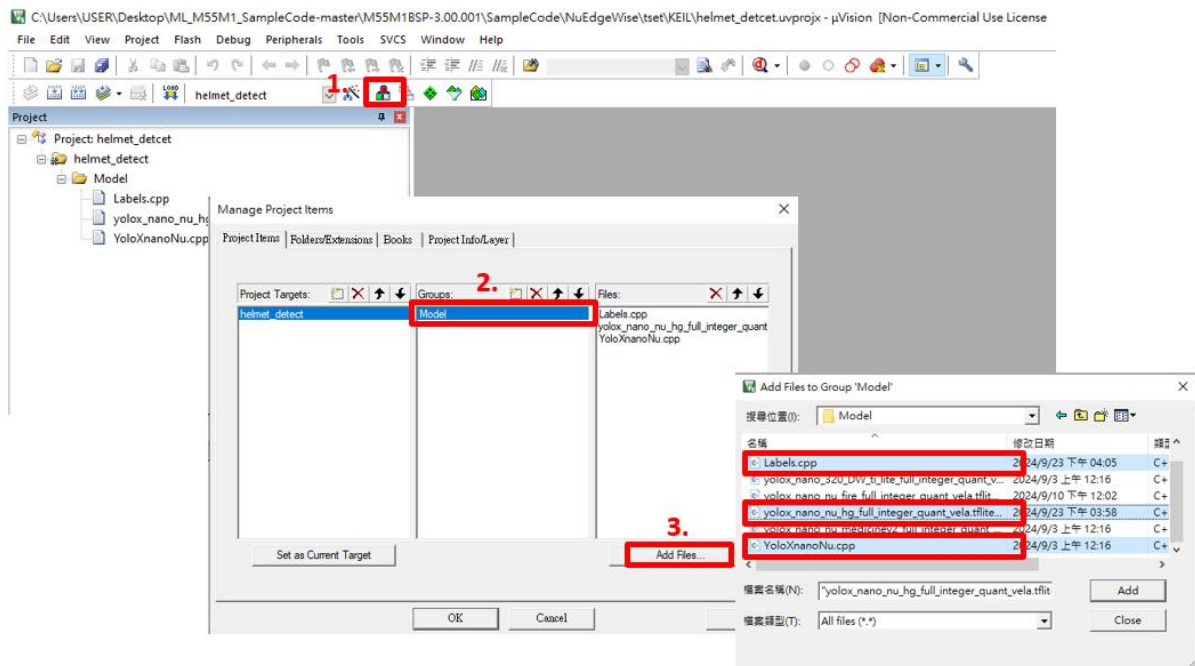


圖 39、將模型、硬體配置等檔案新增到專案

完成專案的基本配置後，需要將相關檔案（如模型檔案、硬體配置檔案等）新增到專案中，以下為詳細步驟：

(1) 開啟「**Manage Project Items**」視窗：

在 Keil μ Vision 的專案視窗中，點擊功能列中的「**新增檔案 (Add Files)**」按鈕，進入檔案管理介面。

(2) 選擇檔案群組：

在「**Manage Project Items**」視窗中，找到「**Groups**」區域，並點擊需要新增檔案的群組。若尚未建立群組，可以先新增一個群組以便於管理。

(3) 新增檔案到群組：

- a. 點擊視窗右下角的「**Add Files...**」按鈕，開啟檔案選擇器。
- b. 瀏覽到目標檔案所在的目錄，選擇需要新增的檔案。舉例來說，可選擇以下檔案：

(a) **Labels.cpp**（標籤定義檔案，用於物件識別）

(b) **YOLO 模型檔案**（如 yolox_nano 等量化模型檔案，用於推論）

(c) 其他相關程式檔案（如 YoloXNanoNu.cpp）

(4) 確認新增的檔案：

選擇完檔案後，點擊對話框中的「**Add**」按鈕，將檔案新增到指定的群組。

確認新增的檔案出現在群組中，然後點擊「**OK**」保存更改。

(5) 檢查專案結構：

回到主專案視窗，檢查新增的檔案是否已正確顯示在專案樹狀結構中，並確認檔案內容可以正常編譯。

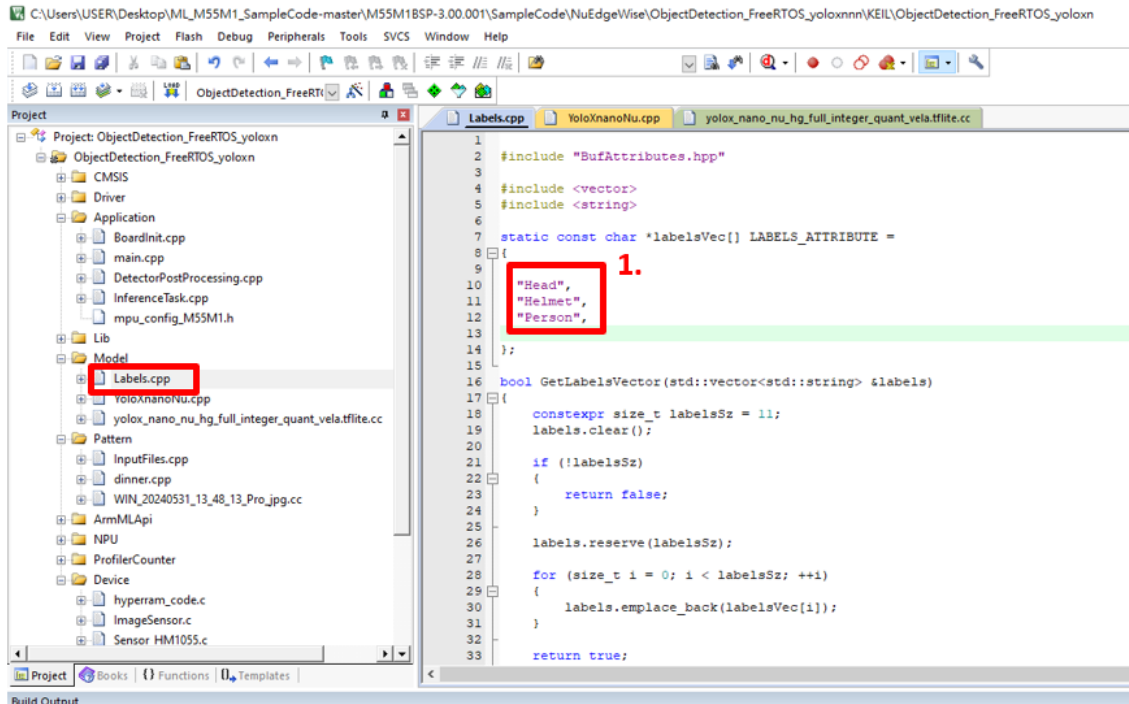


圖 40、若有已建立過之專案，只需更改 Labels 類別名稱

在已建立的專案中，若需要更改標籤類別名稱 (Labels)，可直接編輯專案中的相關檔案，以下為詳細操作步驟：

(6) 打開專案並定位到 Labels.cpp 檔案：

- 在 Keil μ Vision 的專案樹狀視圖中，找到目標專案（例如 **ObjectDetection_FreeRTOS_yoloxn**）。
- 展開 **Lib > Model** 資料夾，並點擊 **Labels.cpp** 檔案，以編輯標籤類別。

(7) 編輯標籤內容：

- 在右側編輯器視窗中，找到定義標籤類別的陣列 **labelsVec[]**。該陣列用於儲存標籤名稱，如下範例所示：


```

1 static const char *labelsVec[] LABELS_ATTRIBUTE = {
2     "Head",
3     "Helmet",
4     "Person"
5 };

```

圖 41、定義標籤類別的陣列

根據需求修改陣列中的標籤內容。例如，若需新增或調整標籤，可直接替換或增加新項目。

(8) 保存檔案並重新編譯專案：

- a. 編輯完成後，點擊「保存」按鈕以保存修改。
- b. 返回主專案視窗，點擊「Build」按鈕重新編譯專案，確保更改生效且編譯無錯誤。

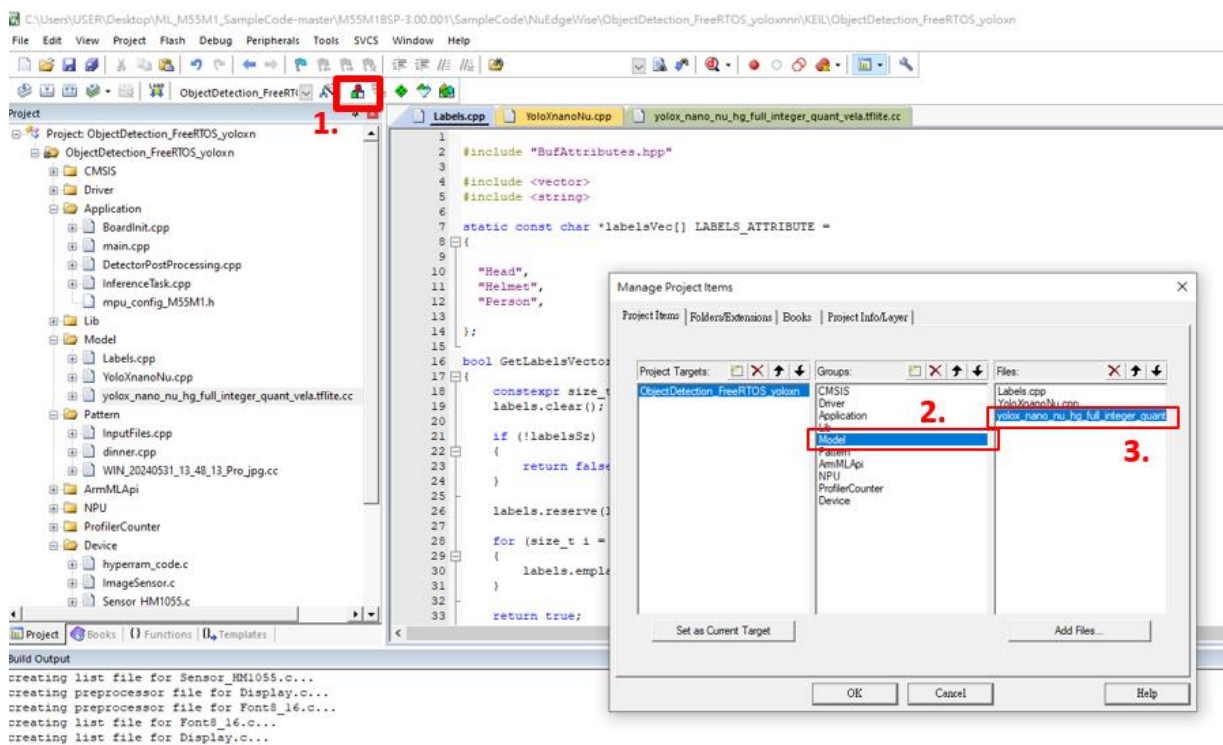


圖 42、新增自行訓練之權重檔至專案，並根據使用者需求，新增額外的硬體配置檔（如音訊裝置等）

在專案開發過程中，若需要加入自行訓練的模型權重檔案或其他硬體配置檔案，可以依照以下步驟進行操作：

1. 開啟專案檔案管理視窗

- (1) 在 Keil μ Vision 的專案視窗中，點擊工具列中的「**Manage Project Items**」按鈕，進入檔案管理介面。

2. 新增檔案群組

- (1) 在「**Manage Project Items**」視窗中，定位至左側的「**Groups**」區域，選擇您需要新增檔案的群組。
- (2) 若無相應的群組，可點擊「**Add Group**」按鈕新增一個新的檔案群組，以便管理新增檔案。

3. 更新模型檔與其他配置檔案

- (1) 點擊右下角的「**Add Files**」按鈕，進入檔案選擇視窗。
- (2) 瀏覽至您的模型檔案或硬體配置檔所在的目錄，選擇需要新增的檔案。
例如：
 - a. 自行訓練的模型檔案(如 `yolox_nano_nu_hg_full_integer_quant`)。
 - b. 音訊裝置驅動檔案、感測器配置檔案等硬體相關檔案。
- (3) 選中檔案後，點擊「**Add**」按鈕，將檔案新增到指定群組中。

4. 檢查專案結構

- (1) 返回專案視窗，確認新增的檔案已正確顯示在專案樹狀結構中。
- (2) 若有其他硬體需求，請重複步驟 3，將所需的配置檔案一併加入專案。

(三) 進行編譯並將程式燒錄至開發板

完成專案配置與檔案新增後，可以進行專案編譯，並將生成的程式燒錄到開發板中運行。以下是詳細步驟：

1. 編譯專案

- (1) 在 Keil μ Vision 的功能列上，點擊「**Build Target**」按鈕，開始編譯整個專案。
- (2) 編譯過程中，右下角的「**Build Output**」視窗將顯示詳細的編譯進度和狀態。

若編譯成功，系統會顯示「**0 Error(s), 0 Warning(s)**」，表示編譯無錯誤與警告。

2. 生成燒錄檔案

- (1) 編譯完成後，系統會自動生成二進位檔案（通常為 .bin 或 .hex 格式），用於燒錄到開發板中。
- (2) 您可以在專案的輸出資料夾中找到生成的檔案，例如：
.\release\ObjectDetection_FreeRTOS_yoloxn.bin。

3. 燒錄至開發板

- (1) 點擊功能列上的「**Flash Download**」按鈕，開啟燒錄工具。
- (2) 確保開發板已正確連接至電腦，並選擇對應的 Nu-Link 硬體燒錄設定。
- (3) 確認設定無誤後，點擊「**Program**」按鈕，開始燒錄程式至開發板。
- (4) **提示：** 燒錄過程中，請勿斷開開發板與電腦的連接，以免燒錄失敗。

4. 驗證執行結果

- (1) 燒錄完成後，開發板將自動執行燒錄的程式。
- (2) 若有配備序列埠工具，可透過開發環境中的「**Debug**」功能或其他序列埠工具檢查執行結果是否符合預期。

Part.B NuMaker-M55M1 開發板實作案例

壹、 智慧工廠 1 - 安全帽偵測

一、 案例介紹 - 安全帽穿戴偵測

本案例利使用 Roboflow 開源資料庫中的 safety-helmet-dataset 數據集，在 PyTorch 框架下利用 YOLOX Nano 模型進行模型訓練。預訓練後的模型使用 ONNX 進行模型框架轉換，轉換到開發板可運行的 TensorFlow Lite 框架，模型輕量化方面使用 Full-INT8 進行模型輕量化以符合開發板的部署模型大小限制。最後，通過 Vela 編譯器進一步最佳化，將原本分配給 CPU 執行的運算，全部轉移到 NPU (Neural Processing Unit) 上執行。最後使用 Keil 工具將最佳化後的系統燒錄至 新唐科技 M55M1 開發板，成功實現一套即時、高效且節能的安全帽穿戴偵測偵測系統。

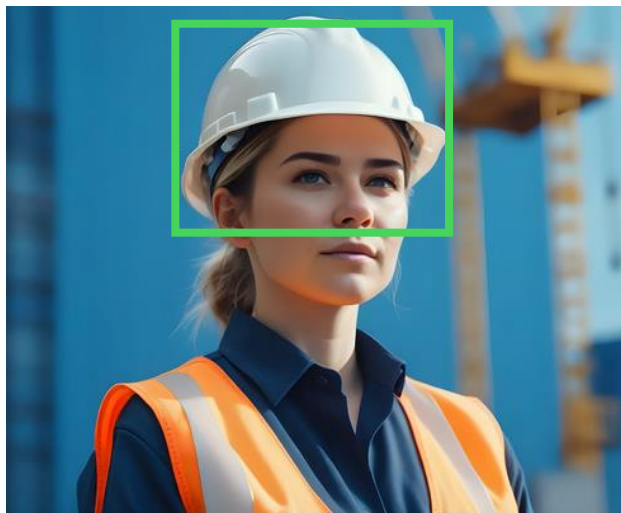


圖 43、安全帽穿戴偵測

(一)新唐開發板簡介與實際應用說明

1. 高效率的硬體效能

(1) Cortex®-M55 與 Ethos™-U55 微神經處理單元 (microNPU)

a、可實現高達 220MHz 的處理速度、可用於神經網路的推論。

b、支援 Full-INT8 量化

在模型量化後 (Full-INT8)，運算效率進一步提高，同時大幅降低記憶體和運算資源的使用，從而支援更有效率的即時預測。

2. 即時性、低延遲

(1) 透過 Ethos™ -U55 的硬體加速能力，以及 Cortex®-M55 的高吞吐量，在即時效能及偵測精度間取得了良好的平衡。

3. 豐富的擴充介面

(1) M55M1 提供了多種介面支持，如 CMOS 感測器介面 (CCAP) 和 TFT-LCD 顯示介面，可整合攝影機與顯示器來擷取畫面並現場進行推論。

4. 安全帽穿戴偵測系統中的實際應用

(1) 工廠端：工廠內有許多運行中的機械，穿戴安全帽可保護頭部避免遭墜下或飛散的物件衝擊，也可避免突出的物件或遭擺動的物件撞擊。因此進入廠區前必須有一道關卡檢測人員是否依規定配戴安全帽。



圖 44、未穿戴安全帽（Paula Bronstein/Getty Images）

5. 專案目標

本案例將在利用新唐 NuMaker-M55M1 開發板建構一套高效、即時的安全帽穿戴偵測系統，透過深度學習模型實現對人員安全帽穿戴狀態的準確檢測與警報。

(1) 快速檢測違規情境：

- a、利用機器學習對進入工廠區域的人員進行即時檢測，確保未穿戴安全帽的人員無法進入危險區域。

(2) 高準確率辨識：

- a、透過最佳化模型與硬體加速，提供高精度的安全帽穿戴狀態辨識結果，減少誤報與漏報的情況。

(3) 低功耗穩定性：

- a、利用 M55M1 開發板的低功耗特性，確保系統能長時間穩定運作，適用於工廠的全天候監控需求。

(4) 便於部署與擴展：

- a、支援工廠環境的實際應用，且可與其他的安全監控系統整合。

6. 專案成果：

透過新唐科技 M55M1 開發板搭載的安全帽穿戴偵測系統，我們取得了以下成果：

(1) 即時穿戴檢測：

- a、系統能有效辨識是否穿戴安全帽，並標註「未穿戴」警報，快速提醒現場管理者。

(2) 高偵測準確率：

- a、在目前測試中，系統使用 500 張測試圖片進行測試，整體準確率達 **95%**。

(3) 視覺化顯示：

- a、系統透過 新唐科技 M55M1 的顯示器即時呈現偵測結果，包括人員圖像與穿戴狀態。

(4) 最佳化模型效能：

- a、系統結合 TinyML 和 Full-INT8 壓縮技術，有效降低運算負擔，提升模型於終端裝置的運行效率。

二、資料集與 AI 模型訓練

(一) 資料集介紹：安全帽穿戴偵測資料集 (safety-helmet-dataset(Roboflow))

1. 資料集準備、

(1) 數據收集

- a、本資料集來自 Roboflow 平台的 safety-helmet-dataset 資料集，專為安全帽穿戴偵測任務設計，包含有穿戴安全帽、未穿戴安全帽等分類。
- b、資料來源可包括工廠內的監控錄影或模擬環境拍攝，確保涵蓋多樣化的場景與光線條件。

c、Roboflow 取得資料集步驟

Step1 : Step1 : 進入 Roboflow 官網 <https://roboflow.com>

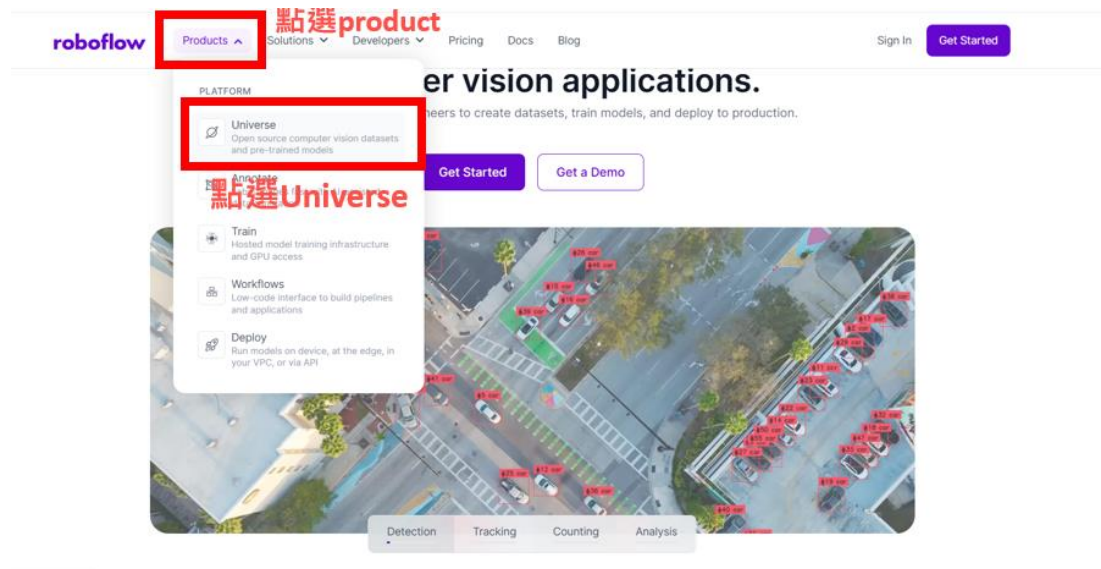
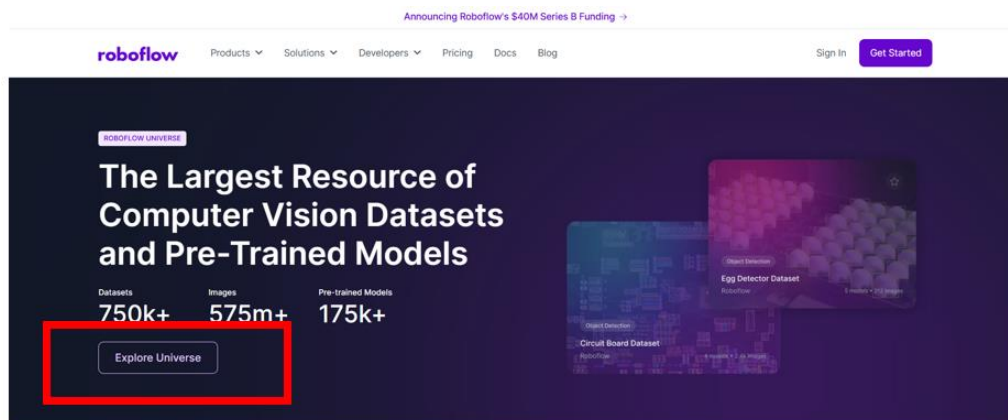


圖 45、Roboflow 官網

Step2 : 點選進入開源資料庫



點選Explore Universe即可開始瀏覽open source的datasets

圖 46、Roboflow 官網

Step3： 根據需求搜尋需要的 DATASET

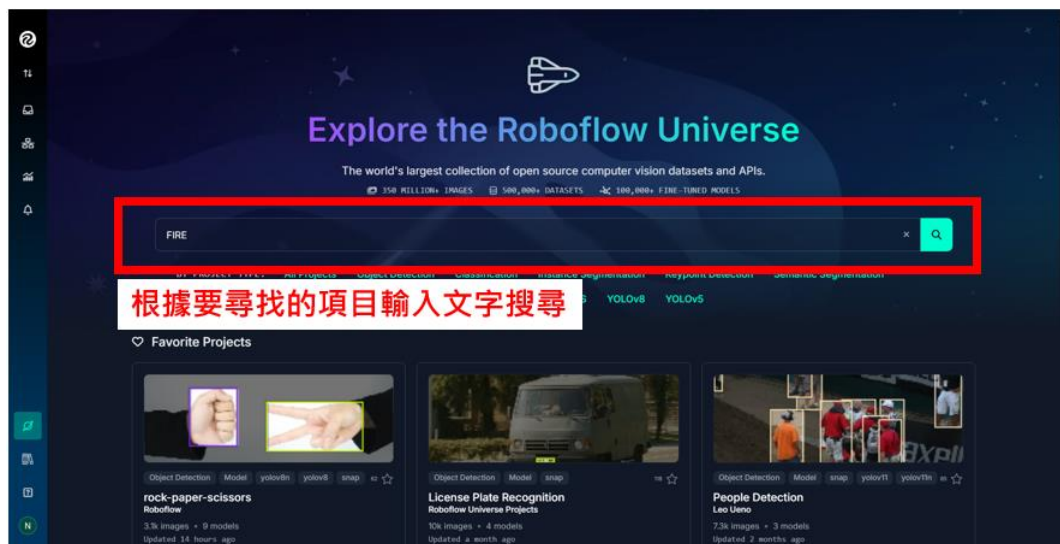


圖 47、Roboflow 官網

Step4： 選定項目後可 Download Project 之 dataset

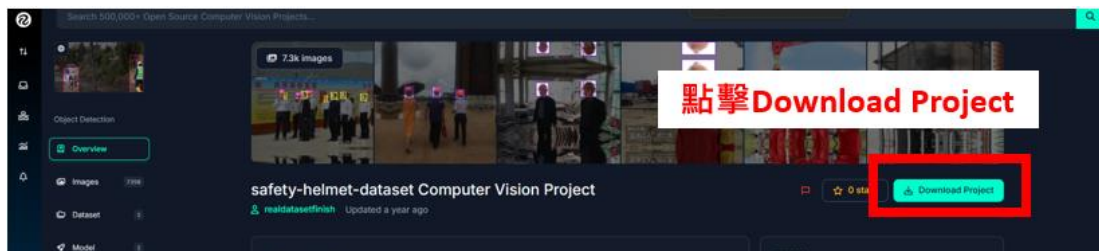


圖 48、Roboflow 官網

Step5： 選定格式即可下載 Dataset



圖 49、標註格式選擇

- d、**數據前處理**包含 BGR 轉換成 RGB、影像改變大小(Resize)和影像資料正規化至浮點數 0 與 1 區間。
- e、訓練 YOLOX 模型的訓練與驗證資料集須符合 **COCO JSON** 的格式。

(2) 資料格式

a、資料採用 COCO JSON 格式，COCO 格式具備以下特點：

- (a) **標註的標準化**：每張圖片的火焰位置均以邊界框形式進行標註，標註信息以 JSON 文件儲存，便於訓練框架讀取與加載。
- (b) **擴展性**：支援多類別標註和複雜場景描述。

b、資料集分為以下部分：

- (a) **訓練集 (Train Set)**：5145 張圖片
用於模型的主要訓練，提供足夠的樣本幫助模型學習與擬合。
- (b) **驗證集 (Valid Set)**：1470 張圖片
在訓練過程中用於評估模型效能，避免過擬合。
- (c) **測試集 (Test Set)**：735 張圖片
獨立於訓練與驗證集，用於最終的模型效能測試。
- (d) **資料集連結**：<https://universe.roboflow.com/realdatasetfinish/safety-helmet-dataset-ep6zc/dataset/1>

(3) 資料集與訓練環境設計

a、安裝與建立 Anaconda 環境

- (a) **Anaconda** 是一個廣泛使用的開源 Python 和 R 編程語言的數據科學平台。可為機器學習、數據分析以及應用部署提供完整的環境，常被稱為「資料科學的一站式解決方案」。
- (b) 開發者可以方便地在 **Anaconda** 中創建虛擬環境，確保不同專案的

依賴關係不互相衝突。

- (c) Anaconda 的安裝相當簡單快速。先開啟「Anaconda」的官網，並且點選上方的「Products」中的「Distribution」。

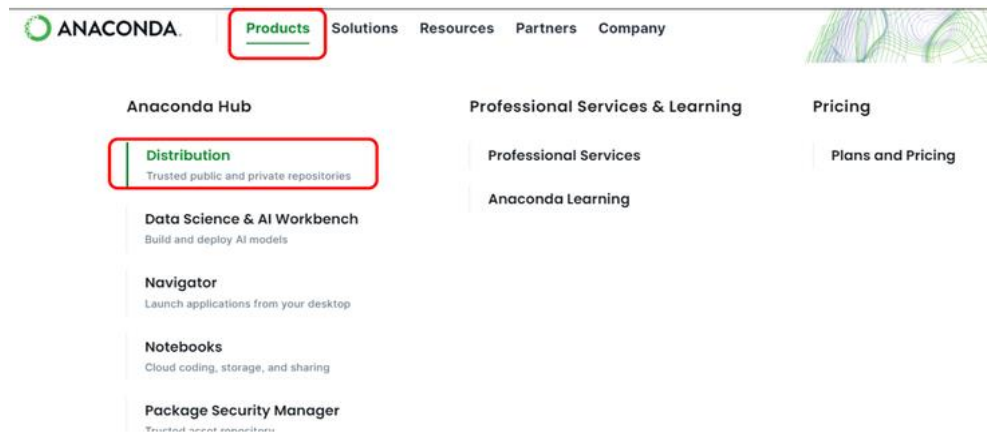


圖 50、Anaconda 官網

- (d) 接著點選「Skip registration」跳過註冊

Distribution

Free Download*

Register to get everything you need to get started on your workstation including Cloud Notebooks, Navigator, AI Assistant, Learning and more.

- ✓ Easily search and install thousands of data science, machine learning, and AI packages
- ✓ Manage packages and environments from a desktop application or work from the command line
- ✓ Deploy across hardware and software platforms
- ✓ Distribution installation on Windows, MacOS, or Linux

*Use of Anaconda's Offerings at an organization of more than 200 employees requires a Business or Enterprise License. [See Pricing](#)

Provide email to download Distribution

Email Address:

- ☐ Agree to receive communication from Anaconda regarding relevant content, products, and services. I understand that I can revoke this consent [here](#) at any time.

By continuing, I agree to Anaconda's [Privacy Policy](#) and [Terms of Service](#).

Submit >

Skip registration

圖 51、Anaconda 官網

- (e) 選擇電腦相對應的版本，本案例安裝 Windows 的版本。

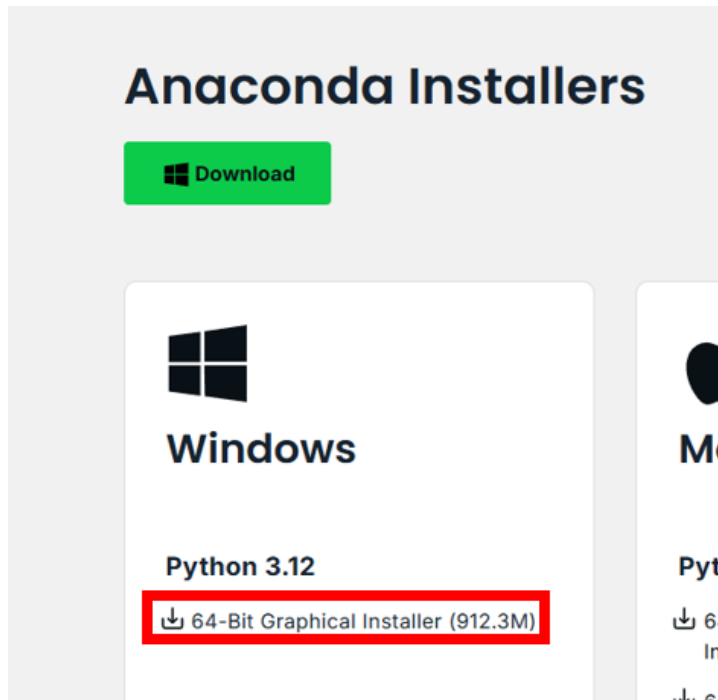


圖 52、Windows 版本

b、Anaconda 環境建立步驟

(a) 下載 YOLOX-TFLITE 專案

- <https://github.com/OpenNuvoton/M55M1-eBook-Sample-Code>

(b) 建立 python 環境

- `conda create --name yolox_nu python=3.10`
- `conda activate yolox_nu`
- 透過上述指令在 Anaconda 上建立環境

(c) 升級 pip 套件管理器 upgrade pip

- `python -m pip install --upgrade pip setuptools`

(d) 安裝 CUDA,PyTorch,MMCV (本案使用 CUDA11.8,torch2.0)

- `python -m pip install torch torchvision torchaudio --index-url`

<https://download.pytorch.org/whl/cu118>

(e) 根據您的硬體配置安裝 **mmcv** (本案使用 2.0.1 版本)

- `python -m pip install mmcv==2.0.1 -f`

<https://download.openmmlab.com/mmcv/dist/cu118/torch2.0/index.html>

(f) 安裝其它需求套件:

- 開啟安裝好的檔案目錄內含 requirements.txt 並透過下面指令下載

- `python -m pip install --no-input -r requirements.txt`

- requirements.txt 文件包含所有需要安裝的套件列表，確保環境中安裝完整。

(g) **Installing the YOLOX**

- 完成需求套件安裝後，運行以下命令來安裝 YOLOX 套件：

`python setup.py develop`

三、 PC 端 Anaconda 環境進行 PyTorch 框架模型訓練

(一) 選擇模型

1. 根據硬體資源選擇適合的輕量化模型。此專案選擇 YOLOX Nano，因其具備較小的模型尺寸與優異的效能。
2. 模型框架使用 PyTorch，支持靈活調整網路結構與參數。

(二) 訓練過程

1. 將資料分為訓練集與驗證集，使用 80% 的數據進行訓練，20% 用於模型驗證。
2. 使用自適應學習率 (Adaptive Learning Rate)，減少過擬合現象。

3. 數據增強策略，進行圖像縮放、裁剪及增強（如旋轉、調整亮度等），提升模型的泛化能力。
4. 利用硬體（如 GPU）加速訓練過程，提升效率。

(三) 訓練設備：

1. 硬體設備：NVIDIA GeForce RTX3070TI
2. CUDA 版本：11.8
3. Pytorch 版本：2.0.0
4. 訓練數據：5145 張圖像
5. 訓練參數：Epoch = 100, Batch size = 32
6. 訓練時長：約 1.5hr

(四) 使用預訓練好的模型訓練

1. exps/default/yolox_nano_ti_lite_nu.py
2. 準備好自己訓練資料並整理成下列形式並放在 dataset 目錄中

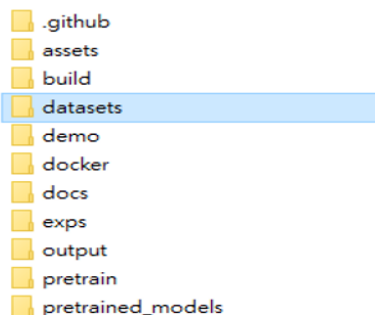


圖 53、dataset 目錄

3. 數據集要求格式：

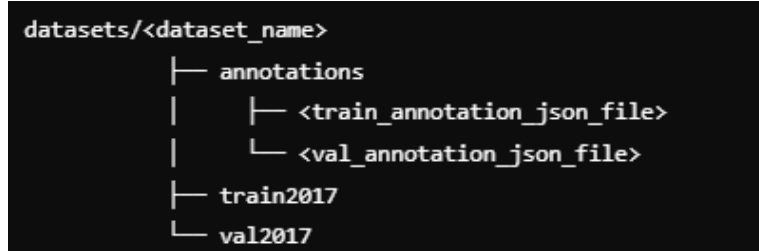


圖 54、數據集格式

4. 配置數據集路徑：

```
# Define yourself dataset path
self.data_dir = "datasets/COCO"
self.train_ann = "train_annotation_json_file.json"
self.val_ann = "val_annotation_json_file.json"
```

圖 55、數據集路徑

5. 更改 yolox_nano_ti_lite_nu.py 之參數

根據自己的
資料集種類設定class數

```
# ----- model config ----- #
self.num_classes = 4
self.depth = 0.33
self.width = 0.25
self.input_size = (320, 320)
self.random_size = (10, 20)
self.mosaic_scale = (0.5, 1.5)
self.mosaic_prob = 0.5
self.enable_mixup = False
self.exp_name = os.path.split(os.path.realpath(__file__))[1].split(".")[0]
self.act = "relu"
```

```
# ----- training config ----- #
self.warmup_epochs = 3
self.max_epoch = 100
# ----- testing config ----- #
self.test_size = (320, 320)
```

設定epoch為100

圖 56、參數設定

6. 訓練指令: python tools/train.py -f <MODEL_CONFIG_FILE> -d 1 -b
<BATCH_SIZE> --fp16 -o -c <PRETRAIN MODEL PATH>

(1) 本案例模型訓練設定檔 MODEL_CONFIG_FILE：

exps/default/yolox_nano_ti_lite_nu.py

(2) 本案例預訓練模型路徑 MODEL_CONFIG_FILE：

pretrain/tflite_yolox_nano_ti/320_DW/yolox_nano_320_DW_ti_lite.pth

(3) 參數解析：

- a、 -f <MODEL_CONFIG_FILE>：指定模型的配置檔案。
- b、 -d 1：設備數量（如 GPU）。
- c、 -b <BATCH_SIZE>：批次大小。
- d、 --fp16：啟用半精度浮點數進行訓練（加速）。
- e、 -o：使用自動混合精度訓練。
- f、 -c <PRETRAIN_MODEL_PATH>：提供預訓練模型路徑。

(五) 轉換工具

1. 使用 ONNX（Open Neural Network Exchange）格式進行框架轉換，讓模型可以在多種運行環境部署。

2. 執行以下命令將 PyTorch 模型轉換為 ONNX 格式：

(1) `python tools/export_onnx.py -f <MODEL_CONFIG_FILE> -c
<TRAINED_PYTORCH_MODEL> --output-name <ONNX_MODEL_PATH>`

(2) 本案例：`python tools/export_onnx.py -f
exps/default/yolox_nano_ti_lite_nu.py -c
YOLOX_outputs/yolox_nano_ti_lite_nu/best_ckpt.pth --output-name
YOLOX_outputs/ONNX/yolox_nano_nu_helmet.onnx`

(3) 參數解釋：

- a. <MODEL_CONFIG_FILE>：配置文件，定義模型結構和參數。
- b. <TRAINED_PYTORCH_MODEL>：已訓練的 PyTorch 模型權重文件。
- c. <ONNX_MODEL_PATH>：輸出的 ONNX 模型路徑。

(六) 模型輕量化

1. 模型輕量化是一種通過壓縮模型參數精度（如從浮點數轉為整數）以提升運行效率的技術。這在資源受限的設備（如嵌入式系統）上更為重要，本案採輕量化至 FULL-INTEGER8。

2. 創建校準數據（Create Calibration Data）

- (1) 執行以下命令生成校準數據：

a. `$python demo/TFLite/generate_calib_data.py --img-size <IMG_SIZE> --n-img <NUMBER_IMG_FOR_CALI> -o <CALI_DATA_NPY_FILE> --img-dir <PATH_OF_TRAIN_IMAGE_DIR>`

b. 本案例：`$python demo/TFLite/generate_calib_data.py --img-size 320 320 --n-img 1400 -o YOLOX_outputs\yolox_nano_ti_lite_nu\calib_data_320x320_n200.npy --img-dir datasets\COCO\train2017`

c. 參數解釋：

- (a) `<IMG_SIZE>`：圖像的輸入大小，例如 320x320。
- (b) `<NUMBER_IMG_FOR_CALI>`：用於校準的圖像數量。
- (c) `<CALI_DATA_NPY_FILE>`：輸出的校準數據文件路徑。
- (d) `<PATH_OF_TRAIN_IMAGE_DIR>`：訓練圖像的目錄路徑。

3. 將 ONNX 模型轉換為 TFLite

- (1) 執行以下命令進行模型輕量化並轉換為 TFLite 格式：

a. `$onnx2tf -i <ONNX_MODEL_PATH> -oiqt -qcind images <CALI_DATA_NPY_FILE> "[[[[0,0,0]]]]" "[[[[1,1,1]]]]"`

b. 本案例：`$onnx2tf -i YOLOX_outputs/ONNX/yolox_nano_nu_helmet.onnx -oiqt -qcind images`

```
YOLOX_outputs\yolox_nano_ti_lite_nu\calib_data_320x320_n200.npy  
"[[[0,0,0]]]" "[[1,1,1]]]"
```

c. 參數解釋：

- (a) <ONNX_MODEL_PATH>：輸入的 ONNX 模型路徑。
- (b) <TFLITE_MODEL_PATH>：輸出的 TFLite 模型路徑。
- (c) -oiqt：啟用整數量化（Full Integer8 Quantization）。
- (d) -qcind images：指定校準數據類型為圖像。
- (e) <CALI_DATA_NPY_FILE>：校準數據文件用於參數調整。
- (f) [[[0,0,0]]] 指定了張量的最小值， [[[1,1,1]]] 指定了張量的最大值。

(七) Vela 編譯器流程說明

1. Vela 編譯流程

(1) Step1：更改 variables.bat 檔參數(位置：vela/variables.bat)

```
set IMAGE_SIZE_EXPR="extern const int originalImageSize = 320;"  
set CHANNELS_EXPR="extern const int channelsImageDisplayed = 3;"  
set CLASS_EXPR="extern const int numClasses = 6;"  
  
set IMAGE_SRC_DIR=..\samples  
set IMAGE_SRC_WIDTH=320  
set IMAGE_SRC_HEIGHT=320  
  
::set LABEL_SRC_FILE  
::set GEN_LABEL_FILE 將MODEL_SRC_FILE、MODEL_OPTIMISE更新為自己的tflite檔案名稱  
  
set MODEL_SRC_DIR=generated  
set MODEL_SRC_FILE=yolox_nano_nu_medicine_full_integer_quant.tflite  
set MODEL_OPTIMISE yolox_nano_nu_medicine_full_integer_quant.tflite vela.tflite  
::The vela OPTIMISE FILE should be SRC FILE NAME + vela
```

將VELA_ACCEL_CONFIG設定為ethos-u55-256

```
::accelerator config. ethos-u55-32, ethos-u55-64, ethos-u55-128, ethos-u55-256, ethos-u65-256, ethos-u65-512  
set VELA_ACCEL_CONFIG=ethos-u55-256
```

圖 57、更改 variables.bat 檔參數

- a. set MODEL_SRC_FILE=<your tflite model>
 - (a) 本案例：`$set MODEL_SRC_FILE = yolox_nano_nu_helmet_full_integer_quant.tflite`
 - b. set MODEL_OPTIMISE_FILE=<output vela model>
 - (b) 本案例：`$set MODEL_OPTIMISE_FILE=yolox_nano_nu_helmet_full_integer_quant_vela.tflite`
 - c. 在 Anaconda Terminal 中，你可以使用 `call` 命令來執行.bat 文件：`call variables.bat`
- (2) Step2：執行 `gen_model_cpp.bat` 檔案，即可得經 Vela 編譯後之 Tensorflow Lite 檔案
- a. 執行成功會產生模型檔 `yolox_nano_ti_lite_nu_full_integer_quant_vela.tflite.cc`，檔案會生成在 `vela\generated` 路徑下
 - b. 執行完 `gen_model_cpp.bat` 所產生的預訓練權重.C 檔：`yolox_nano_nu_helmet_full_integer_quant_vela.tflite.cpp`
 - c. 將權重檔放入 Sample Code 資料夾下的 Model 資料夾
 - d. 預訓練權重.CC 檔執行燒錄步驟時會一併燒錄到開發板

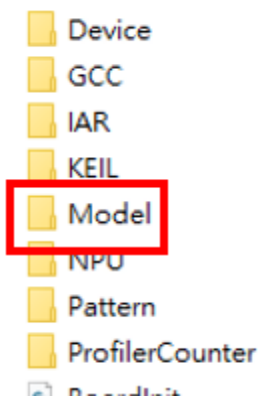


圖 58、權重檔擺放位置

```

{
static const uint8_t nn_model[] MODEL_TFLITE_ATTRIBUTE =
{
    0x24, 0x00, 0x00, 0x00, 0x54, 0x46, 0x4c, 0x33, 0x00, 0x00,
    0x00, 0x00, 0x04, 0x00, 0x12, 0x00, 0x00, 0x00, 0x18, 0x00,
    0x02, 0x00, 0x00, 0x00, 0x38, 0x00, 0x00, 0x00, 0x04, 0x00,
    0x69, 0x6e, 0x65, 0x4d, 0x65, 0x6d, 0x6f, 0x72, 0x79, 0x41,
    0x06, 0x00, 0x00, 0x00, 0x04, 0x00, 0x00, 0x00, 0x13, 0x00,
    0x08, 0x00, 0x00, 0x00, 0x00, 0x49, 0x08, 0x00, 0xf0, 0x48,
    0x04, 0x00, 0x00, 0x00, 0x26, 0xb7, 0xf7, 0xff, 0x04, 0x00,
    0x20, 0x38, 0x01, 0x00, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
    0xff, 0xff, 0xff, 0xff, 0x20, 0xaf, 0x02, 0x00, 0x00, 0x40,
    0x32, 0x2e, 0x33, 0x2e, 0x30, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x14, 0x00, 0x00, 0x00, 0x14, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x43, 0x4f, 0x50, 0x31, 0x01, 0x00, 0x10, 0x00, 0x08, 0x30,
    0x0f, 0x01, 0x01, 0x00, 0x00, 0x40, 0x00, 0x00, 0x20, 0xaf,
    0x00, 0x00, 0x00, 0x00, 0x0b, 0x01, 0x13, 0x00, 0x0c, 0x01,
    0x80, 0x07, 0x00, 0x00, 0x04, 0x40, 0x00, 0x00, 0x60, 0x00,
    0x03, 0x01, 0x00, 0x00, 0x02, 0x01, 0x00, 0x00, 0x1f, 0x01,

```

圖 59、權重檔內容

四、新唐 M55M1 開發板上的推論程式系統流程步驟

(一) 系統初始化

系統啟動時的硬體初始化流程，使用 BoardInit() 函式來執行各項初始化操作，包括時鐘設定、UART、HyperRAM 以及 NPU（可選）的初始化。

1. 函式功能

(1) BoardInit()

a. 目的：

- (a) 負責硬體相關的初始化。確保各模組（如時鐘、UART、記憶體、處理單元）已經準備好供後續流程使用。

b. 函式說明：

- (a) **SYS_Init()**：設定系統時鐘及相關參數，確保所有模組的時序一致。
- (b) **InitDebugUart()**：配置 UART（如 UART6），用於輸出調試資訊，允許程式進行 printf 調試。
- (c) **HyperRAM_Init()**：初始化 HyperRAM，設定其工作模式及介面。
- (d) **SPIM_HYPER_EnterDirectMapMode**：進入直接映射模式，允許應用程序更高效地訪問 HyperRAM。
- (e) **ethosu_npu_init()**：如果系統中包含 NPU（如 Arm® Ethos™-U），則

執行其初始化操作。若初始化失敗，函式返回錯誤狀態。

- (f) **鎖定/解鎖寄存器**：在修改系統保護寄存器時，需要使用 **SYS_UnlockReg()** 解鎖，修改完成後再用 **SYS_LockReg()** 鎖定，保護關鍵配置。

```
int BoardInit(void)
{
    /* Unlock protected registers */
    SYS_UnlockReg();

    SYS_Init();

    /* UART init - will enable valid use of printf (stdc
    * re-directed at this UART (UART6) */
    InitDebugUart();

    SYS_LockReg(); /* Unlock register

    HyperRAM_Init(HYPERRAM_SPIM_PORT);
    /* Enter direct-mapped mode to run new applications
    SPIM_HYPER_EnterDirectMapMode(HYPERRAM_SPIM_PORT);

    info("%s: complete\n", __FUNCTION__);

#ifdef ARM_NPU

    int state;

    /* If Arm Ethos-U NPU is to be used, we initialise :
    if (0 != (state = arm_ethosu_npu_init()))
    {
        return state;
    }

#endif /* ARM_NPU */
```

圖 60、BoardInit 函式

(二) 初始化 YOLO 及設置模型參數

1. 函式功能

(1) YoloXnanoNu::Init()

a. 目的：

- (a) 使用 YOLO 模型的 TensorFlow Lite 版本，通過 YoloXnanoNu::Init() 方法進行初始化。

b. 函式說明：

- (a) 使負責設置 YOLO 模型所需的運行環境，包括 TensorFlow Lite 的相關參數，包括模型資料與緩衝區的配置。

c. 圖、YoloXnanoNu::Init 函式

```
bool YoloXnanoNu::Init(uint8_t *tensorArena,
                        size_t tensorArenaSize,
                        uint8_t *modelPointer,
                        size_t modellen)
{
    info("Initializing YOLO model...\n");

    /* 設置 TensorFlow Lite 模型 */
    this->m_modelPointer = modelPointer;
    this->m_modellen = modellen;

    /* 初始化 TensorFlow Lite 解釋器 */
    this->m_interpreter = tflite::GetInterpreter(tensorArena, tensorArenaSize, modelPoint
    if (!this->m_interpreter)
    {
        printf_err("Failed to create TFLite interpreter.\n");
        return false;
    }
}
```

圖 61、YoloXnanoNu::Init 函式

(三) 系統擷取鏡頭影像輸入

1. 函式功能

(1) get_empty_framebuf()

a. 目的：

- (a) 尋找狀態為 eFRAMEBUF_EMPTY 的空緩衝區，作為存放新影像的目標。

b. 函式說明：

- (a) 依序檢查緩衝區陣列中的每一個緩衝區，若找到空的緩衝區 (eFRAMEBUF_EMPTY)，則返回其指標；否則返回 NULL。

```
//frame buffer managemnet function
static S_FRAMEBUF *get_empty_framebuf()
{
    int i;

    for (i = 0; i < NUM_FRAMEBUF; i++)
    {
        if (s_asFramebuf[i].eState == eFRAMEBUF_EMPTY)
            return &s_asFramebuf[i];
    }

    return NULL;
}
```

圖 62、get_empty_framebuf 函式

(2) ImageSensor_Capture()

a. 目的：

- (a) 負責控制影像感測器 (Image Sensor) 完成影像擷取。

b. 函式說明：

- (a) 設定緩衝區基址：調用 CCAP_SetPacketBuf()，指定數據存放緩衝區地址。
- (b) 啟動擷取：執行 CCAP_Start() 開始捕捉影像。
- (c) 停止擷取：執行 CCAP_Stop(TRUE) 停止擷取，並檢查執行結果。


```

int ImageSensor_Capture(uint32_t u32FrameBufAddr)
{
    int i32Ret = CCAP_OK;
    /* Set System Memory Packet Base Address Register */
    //printf("sensor capture address %x \n", u32FrameBufAddr);
    CCAP_SetPacketBuf((uint32_t)u32FrameBufAddr);

    /* Start image capture */
    CCAP_Start();

    /* Start image capture */
    i32Ret = CCAP_Stop(TRUE);

    if (i32Ret != CCAP_OK)
        return -1;

    return 0;
}

```

圖 63、ImageSensor_Capture 函式

(四) 安全帽檢測模型推論任務執行

根據鏡頭影像輸入資訊，進行模型推理，將每一個偵測到的物件標註其類別及在影像中的位置。整體分為以下幾個重點：

1. 函式功能

(1) get_full_framebuf()

a. 目的：

- (a) 該函式遍歷幀緩衝區陣列，尋找狀態標記為 eFRAMEBUF_FULL 的幀緩衝區，並將其返回以供後續推理任務使用。

b. 函式說明：

- (a) 使用迴圈檢查幀緩衝區陣列 (s_asFramebuf) 中的每個元素。
- (b) 如果某個幀緩衝區的狀態是 eFRAMEBUF_FULL，返回該緩衝區的指針。
- (c) 如果未找到符合條件的幀緩衝區，返回 NULL。

```

static S_FRAMEBUF *get_full_framebuf()
{
    int i;

    for (i = 0; i < NUM_FRAMEBUF; i++)
    {
        if (s_asFramebuf[i].eState == eFRAMEBUF_FULL)
            return &s_asFramebuf[i];
    }

    return NULL;
}

```

圖 64、get_full_framebuf 函式

(2) 推論任務 (inferenceJob)

a. 目的：

- (a) 這段程式碼的核心功能是構建一個推論任務 (inferenceJob)，並將其排入推論處理的佇列。推論任務負責將影像資料傳送給物件偵測模型，並在完成推論後回傳結果。

b. 函式說明：inferenceJob：表示一個推論任務的結構體實例，包含以下屬性：

- (a) responseQueue：指定一個佇列 (inferenceResponseQueue)，用來存放推論結果的回應。
- (b) pPostProc：指向後處理函數 (postProcess)，負責對推論結果進行後續處理，例如篩選信心度較低的結果或繪製偵測框。
- (c) modelCols、modelRows：指定輸入影像的寬度和高度 (inputImgCols 和 inputImgRows)，確保影像大小符合模型輸入規格。
- (d) srcImgWidth、srcImgHeight：設定來源影像 (fullFramebuf->frameImage) 的原始寬度與高度，用於調整邊界框相對於原影像的位置。
- (e) results：指向用於存放推論結果的結構體 (fullFramebuf->results)。

```

//trigger inference
inferenceJob->responseQueue = inferenceResponseQueue;
inferenceJob->pPostProc = &postProcess;
inferenceJob->modelCols = inputImgCols;
inferenceJob->modelRows = inputImgRows;
inferenceJob->srcImgWidth = fullFramebuf->frameImage.w;
inferenceJob->srcImgHeight = fullFramebuf->frameImage.h;
inferenceJob->results = &fullFramebuf->results;

xQueueSend(inferenceProcessQueue, &inferenceJob, portMAX_DELAY);
fullFramebuf->eState = eFRAMEBUF_INF;

```

圖 65、推論任務（inferenceJob）

(3) PresentInferenceResult()

a. 目的：

(a) PresentInferenceResult 函式的主要功能是展示從物件偵測模型得到的推論結果。它將模型偵測到的每個物件，標註其類別及其在影像中的位置。

b. 函式說明：

(a) results：這是 std::vector 型別，包含了一組物件偵測結果，每個結果由一個 DetectionResult 結構表示。每個 DetectionResult 包括以下資訊：

- m_cls：物件類別的索引。
- m_normalisedVal：信心度值（表示模型對該偵測結果的信心）。
- m_x0, m_y0, m_w, m_h：偵測框的位置與尺寸資訊，分別表示左上角的座標（x0, y0）以及寬度（w）和高度（h）。

(b) labels：這是 std::vector<std::string>，包含所有可能的物件類別名稱。透過這個參數，可以將類別索引轉換成可讀的類別名稱。

```

static bool PresentInferenceResult(const std::vector<arm::app::object_detection::DetectionResult> &results,
                                  std::vector<std::string> &labels)
{
    /* If profiling is enabled, and the time is valid. */
    info("Final results:\n");

    for (uint32_t i = 0; i < results.size(); ++i)
    {
        info("%" PRIu32 " " %s(%f) -> %s {x=%d,y=%d,w=%d,h=%d}\n", i,
            labels[results[i].m_cls].c_str(),
            results[i].m_normalisedVal, "Detection box:",
            results[i].m_x0, results[i].m_y0, results[i].m_w, results[i].m_h);
    }

    return true;
}

```

圖 66、PresentInferenceResult 函式

(五) 後處理模組

進行後處理 NMS (Non-Maximum Suppression)，其目的是提升物件偵測結果的準確性，通過去除多餘的重複檢測框來保留最有意義的檢測框。整體分為以下幾個重點：

1. NMS 的功能

(1) 目的：

- a. 在物件偵測任務中，可能會出現多個重疊的檢測框，NMS 的作用是去除冗餘檢測框，僅保留信心度較高且最具代表性的框。

2. 函式功能

(1) CalculateNMS 函式主要參數

a. 主要參數

- (a) **detections**：來自模型推論的輸出，包含所有的檢測框資訊（例如位置、信心度、類別等）。
- (b) **net.numClasses**：模型支持的類別總數。用於對不同類別分別執行 NMS，避免不同類別之間的檢測框互相影響。
- (c) **m_nms**：NMS 的參數，定義 IoU (Intersection over Union) 的閾值，控制去除重疊框的條件。較低的值會更嚴格地濾除重疊框，較高的值則會保留更多框。

(六)顯示結果

將物件偵測結果的視覺化實現，透過函式 `DrawImageDetectionBoxes`，將偵測結果以框和標籤的形式疊加到影像上，並顯示到畫面中。整體分為以下幾個重點：

1. 函式功能

(1) `DrawImageDetectionBoxes` 函式

a. 目的：

- (a) 將物件偵測的結果（`Bounding Box` 和分類標籤）繪製在影像上。
- (b) 讓使用者可以直觀地看到模型推論後的偵測效果。

b. 主要參數：

- (a) `results`：一個存放物件偵測結果的向量，每個結果包含框的位置與類別資訊。
- (b) `drawImg`：目標影像，將在此影像上繪製偵測結果。
- (c) `labels`：類別標籤的向量，用於顯示偵測物件的名稱，本案例為
"none","warning","helmet","person"。

```
static void DrawImageDetectionBoxes(  
    const std::vector<arm::app::object_detection::DetectionResult> &results,  
    image_t *drawImg,  
    std::vector<std::string> &labels)  
{  
    for (const auto &result : results)  
    {  
        imlib_draw_rectangle(drawImg, result.m_x0, result.m_y0, result.m_w, result.m_h, COLOR_B5_MAX, 1, false);  
        imlib_draw_string(drawImg, result.m_x0, result.m_y0 - 16, labels[result.m_cls].c_str(), COLOR_B5_MAX, 2, 0, 0, false,  
                           false, false, false, 0, false, false);  
    }  
}
```

圖 67、`DrawImageDetectionBoxes` 函式

(2) 函式內部運作

a. `for` 迴圈：遍歷 `results` 中的每個物件偵測結果，逐一進行以下操作：

(a) 繪製框（`Rectangle`）：

- 使用 `imlib_draw_rectangle` 函式，根據結果中的 `m_x0`、`m_y0`（左上角座標）、`m_w`（寬度）、`m_h`（高度）來繪製矩形框，框顏色和樣式可以自定義。

(b) 顯示標籤 (String)：

- 使用 `imlib_draw_string` 函式，在框的上方顯示標籤文字（由 `labels` 提供），並設定字型顏色、大小和樣式等。

貳、智慧工廠 2 - 火災偵測

一、案例介紹 - 火災偵測

本專案利用 Roboflow 開源資料庫的數據集，通過機器學習方式在 YOLOX Nano 上進行訓練。接著，模型經由 ONNX 進行模型框架轉換，轉換成 TensorFlow Lite 框架，並應用 Full-INT8 輕量化技術以實現輕量化來達到開發板模型大小限制。最後，通過 Vela 編譯器進一步最佳化，確保運算全部在 NPU (Neural Processing Unit) 上執行。最後，我們使用 Keil 工具將最佳化後的系統燒錄至 新唐科技 M55M1 開發板，成功實現一套即時、高效且節能的火災偵測系統。

(一) 新唐開發板簡介與實際應用說明

1. 高效率的硬體效能

Cortex®-M55 與 Ethos™-U55 微神經處理單元 (microNPU)，可實現高達 220MHz 的處理速度、可用於神經網路的推論。支援 Full-INT8 量化，在模型量化後 (Full-INT8)，運算效率進一步提高，同時大幅降低記憶體和運算資源的使用，從而支援更有效率的即時預測。

2. 即時性、低延遲

透過 Ethos™-U55 的硬體加速能力，以及 Cortex®-M55 的高吞吐量，在即時效能及偵測精度間取得了良好的平衡。

3. 豐富的擴充介面

M55M1 提供了多種介面支持，如 CMOS 感測器介面 (CCAP) 和 TFT-LCD 顯示介面，可整合攝影機與顯示器來擷取畫面並現場進行推論。

(二) 火災偵測系統中的實際應用

在工廠這種較高溫環境下易發生火災，且可能發生在不常注意的地方，當火災偵測系統偵測到火光時，發出警報，讓工廠人員能夠發現隱藏的危險。

基於上述優勢開發板的優勢，M55M1 開發板在火災偵測的優點包括：

1. 快速反應：火光偵測能力可配合感測器(如 arduino 蜂鳴器)，能夠在火災現場發出警報，減輕傷亡和財產損失。
2. 高效部署：透過低功耗與豐富的介面，可以部署到工廠、倉庫等複雜環境。

二、資料集蒐集

(一) 資料集介紹：火災偵測資料集 (Fire Detection Dataset - Roboflow)

1. 資料集來源

本資料集來自 Roboflow 平台的 Fire Detection 資料集，專為火災偵測任務設計，包含多種火焰場景的圖像。資料集覆蓋了多種環境（如室內外、白天與夜晚），以確保模型對不同場景的適應性和穩健性。

2. 資料格式

資料採用 COCO JSON 格式，COCO 格式具備以下特點：

- (1) 標註的標準化：每張圖片的火焰位置均以邊界框形式進行標註，標註信息以 JSON 文件儲存，便於訓練框架讀取與加載。
- (2) 擴展性：支援多類別標註和複雜場景描述。
- (3) 訓練集 (Training Set)：1210 張圖片，用於模型的主要訓練，提供足夠的樣本幫助模型學習與擬合。
- (4) 驗證集 (Valid Set)：347 張圖片，在訓練過程中用於評估模型效能，避免過擬合。
- (5) 測試集 (Test Set)：164 張，用於最終的模型效能測試。

3. 資料集特點

- (1) 多樣性：資料集包含多種場景（如工業區、家庭、野外等）和火焰形態（如明火、煙霧等），大幅提升模型的泛化能力。
- (2) 高品質標註：每張圖片中的火焰區域均經過精準標註，確保模型在邊界框定位上的準確度。
- (3) 開放性：資料集由 Roboflow 平台取得，可根據需求進行擴展或增強。

4. 模型訓練環境

為了處理本資料集，專案採用了以下環境：

- (1) 虛擬環境：Anaconda
- (2) 訓練框架：PyTorch
- (3) 訓練模型：YOLOX-Nano

5. 資料集用途

本資料集主要用於火災偵測任務，適用於以下場景：

- (1) 智慧安防：用於監控系統，即時偵測火災並觸發警報。
- (2) 工業安全：部署於工廠或危險區域，提升火災應急響應能力。

模型資源與資料集下載連結：

https://github.com/MaxCYCHEN/yolox-ti-lite_tflite_int8

(二) 資料集準備

本專案使用 Roboflow 之開源資料集 fire detection 做訓練。並且因選擇 yolox nano 作為訓練模型，資料集為 COCO 格式。

(三) Roboflow 簡介

Roboflow 是一個專門管理影像數據的平台，目標是幫助使用者更有效地管理和處理圖像數據。主要功能包括數據標註、數據清理、數據轉換和數據管理。平台內還提供了許多不同用戶所公開的資料集。使用者可以透過 Roboflow 平台輕鬆地瀏覽這些資料集，找到符合自己需求的資料集，具有以下三點優勢。

1. 多種格式：

可將資料集轉換為 COCO、YOLO、Pascal VOC 等格式，從而與不同的框架整合。

2. 資料增強：

提供旋轉、直線、翻轉、調整亮度等多種資料增強技術，增加模型的泛化能力。

3. 雲端管理：

資料集儲存在雲端，包括團隊協作和版本控制。

(四) 資料集與訓練環境設計：從資料收集到模型部署

1. Roboflow 取得資料集步驟

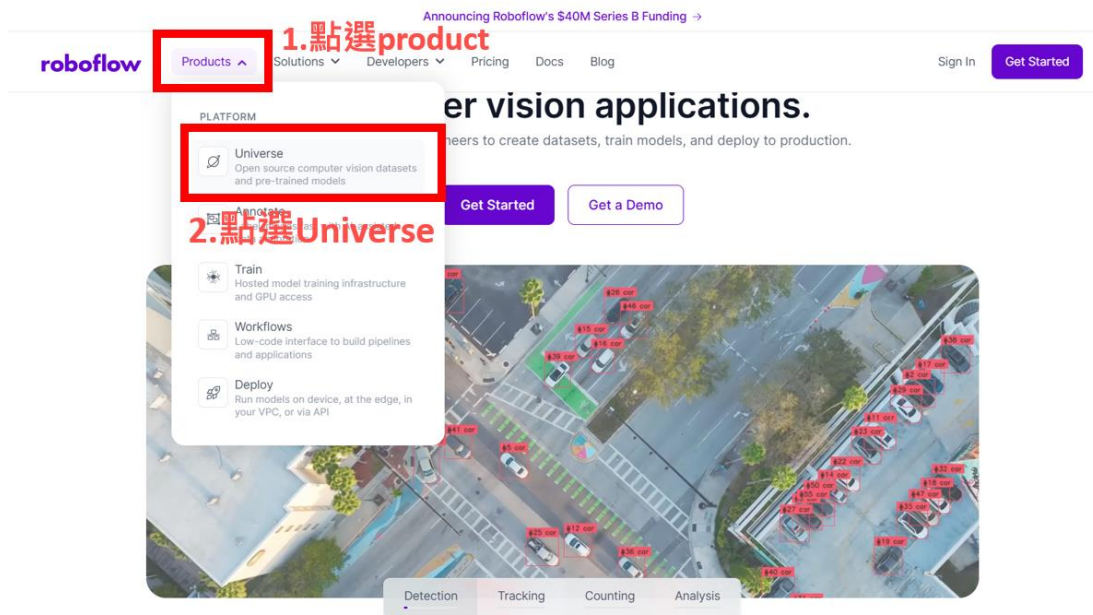
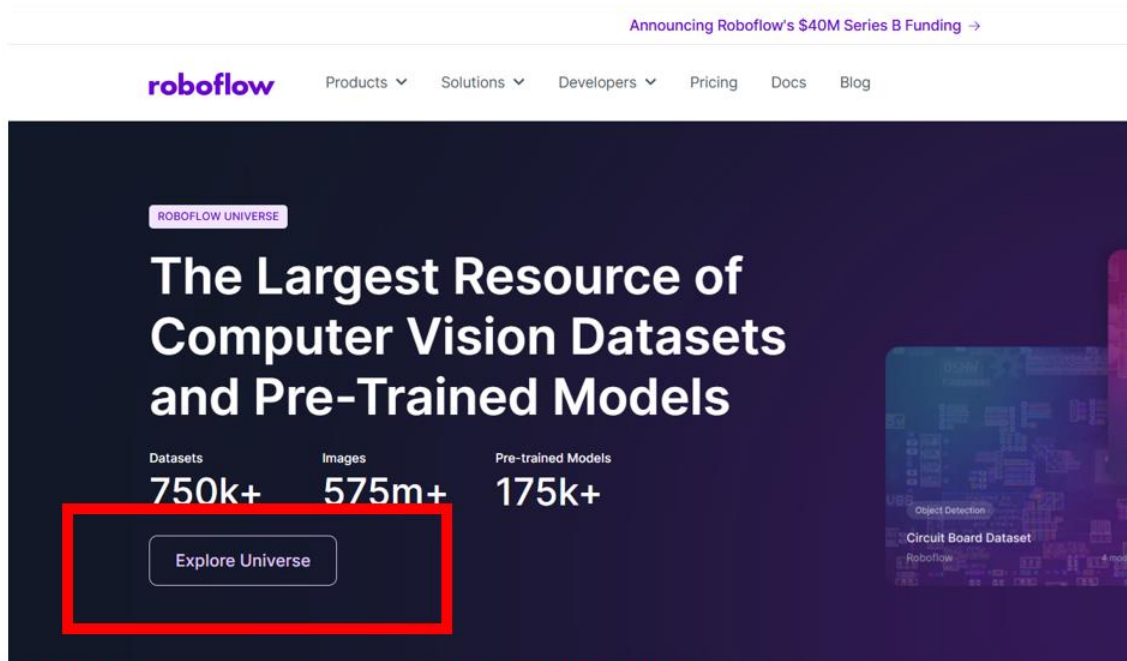


圖 68、Step1：進入 Roboflow 官網 <https://roboflow.com>



點選 Explore Universe 即可開始瀏覽 open source 的 datasets

圖 69、Step2：點選進入開源資料庫



圖 70、Step3：根據需求搜尋需要的 DATASET(本案關鍵字為 FIRE)

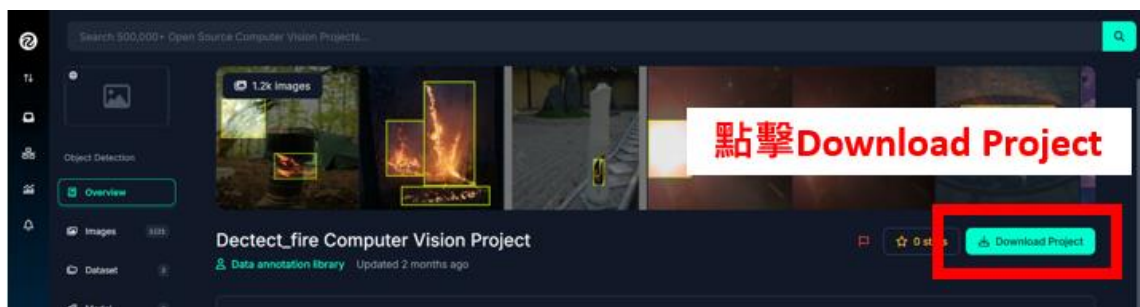


圖 71、Step4 :選定項目後可 Download Project 之 dataset



圖 72、Step5：選定格式即可下載 Dataset

三、在 PC 上使用 Anaconda 環境進行模型訓練

(一) 訓練設備

1. 硬體設備：NVIDIA GeForce RTX4060
2. CUDA 版本：11.8
3. Pytorch 版本：2.0.0
4. 訓練數據：1210 張圖像
5. 訓練參數：Epoch = 200, Batch size = 64
6. 訓練時長：約 1 hr
- 7.

(二) 火災偵測模型訓練完整流程

1. 環境建立步驟

(1) 建立 Python 環境

使用 conda 指令建立一個新的 Python 虛擬環境，指定名稱為 yolox_nu，並使用

Python 3.10 版本：`$conda create --name yolox_nu python=3.10`

激活剛建立的環境：`$conda activate yolox_nu`

升級 pip 套件管理器，使用以下命令升級 pip 和 setuptools 到最新版本：

`$ python -m pip install --upgrade pip setuptools` 安裝 CUDA、PyTorch (本案採用本案使用 CUDA11.8,torch2.0)

安裝與硬體相關的深度學習套件，包括 CUDA、PyTorch 及其相關工具，如 torchvision 和 torchaudio，指令：

```
$python -m pip install torch torchvision torchaudio --index-url  
https://download.pytorch.org/whl/cu118
```

(2) 根據硬體配置安裝 MMCV(本案使用 2.0.1 版本)

使用以下指令安裝 MMCV，這是一個基於 OpenMMLab 的深度學習工具庫，提供多種模型訓練的支持，安裝指令。

```
$python -m pip install mmcv==2.0.1 -f
```

<https://download.openmmlab.com/mmcv/dist/cu118/torch2.0/index.html>

(3) 安裝其他需求套件

將路徑切換至專案目錄後，使用以下命令安裝專案需要的套件，requirements.txt 文件包含所有需要安裝的套件列表，確保環境中安裝完整，--no-input 選項可避免交互式確認，適合自動化腳本。

```
$python -m pip install --no-input -r requirements.txt
```

(4) 安裝 YOLOX 模型

完成需求套件安裝後，運行以下命令來安裝 YOLOX 模型，setup.py 文件包含 YOLOX 的安裝配置，develop 模式允許在修改程式碼後無需重新安裝即可生效，方便開發者進行實時調試。完成這些步驟後，整個 YOLOX nano 訓練環境將準備就緒，接下來，會進一步設置數據集和參數進行模型訓練。

```
$python setup.py develop
```

2. 使用自訂訓練集的準備

以下是模型訓練前準備工作的主要步驟：

(1) 使用預訓練好的模型訓練

配置路徑為 exps/default/yolox_nano_ti_lite_nu.py，該檔案包含預設的 YOLOX Nano 配置。須確保該配置檔案已包含您所需的模型參數。

(2) 準備資料集

資料集需依照以下目錄結構進行組織，此步驟需要修改 YOLOX 的 Python 配置檔案內的 `self.data_dir`，指定資料集的實際路徑。

Datasets/<your_datasets_name>/

- annotations/
 - train_annotation_json_file
 - val_annotation_json_file
- train2017/
 - train_img
- val2017/
 - validation_img

圖 73、資料集目錄結構

```
# Define yourself dataset path
self.data_dir = "datasets/coco128"
self.train_ann = "train_annotations.coco.json"
self.val ann = "val annotations.coco.json"
```

設定自己的資料集路徑

圖 74、設定程式碼中的資料路徑

(3) 調整配置檔案參數

更改 yolox_nano_ti_lite_nu.py 之參數，self.num_classes = 6 表示模型的類別數量，需要根據資料集實際的類別數進行修改。例如，如果資料集中有 6 種分類，則設為 6。

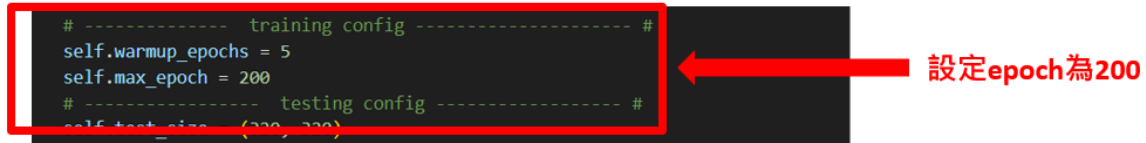


圖 75、設定 epoch

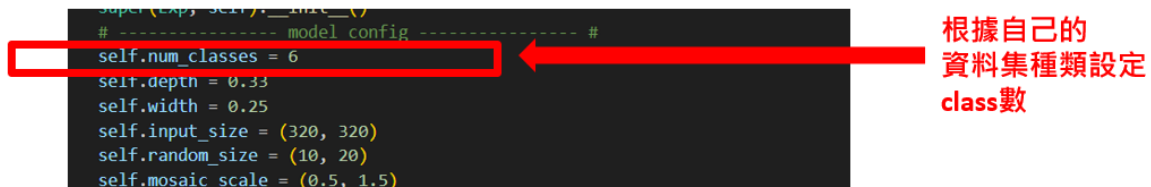


圖 76、設定 num_classes

(4) Epoch 介紹：

epoch 是指將整個訓練數據集完整地通過一次神經網路的過程。調整 epoch 的值可以影響模型的學習效果和性能。模型通過多次 epoch 使用完整數據集進行訓練，提取特徵並更新權重，提升準確性並追蹤在訓練集和驗證集上的學習進度。

a. 調整 Epoch 的效果

Epoch 的選擇需平衡訓練次數與模型性能，若 Epoch 過小，模型可能因訓練不足而無法充分學習數據特徵，導致欠擬合與低準確性；而若 Epoch 過多，則可能因過度訓練而記住數據噪聲，導致過擬合，同時浪費訓練時間且性能無法進一步改善。最佳 Epoch 通常透過驗證集的表現來判斷，並可使用 Early Stopping 技術，在驗證集性能停止提升時終止訓練，以確保效率與效果的平衡。

b. 如何選擇合適的 Epoch 數量

選擇合適的 Epoch 數量需考慮數據集大小、模型複雜度與驗證過程：小型數據集因樣本有限可能需要更多的 Epoch，而大型數據集則相對較少；複雜模型需要更多 Epoch 來充分學習，但過多可能導致過擬合；通過監控驗證集上的損失和準確率，選擇能平衡訓練與驗證性能的最佳 Epoch 數量。

c. 與其他參數的交互作用

Batch Size 和 Learning Rate 是影響訓練效率與模型收斂的重要參數：小的 Batch Size 更能捕捉細微特徵但需要更多的 Epoch，而大的 Batch Size 訓練穩定但需要較少的 Epoch；學習率（Learning Rate）過高可能導致錯誤更新需要更多 Epoch 修正，而學習率過低則需大量 Epoch 才能逐步收斂到最佳結果。

3. 開始訓練

在終端輸入下列指令即可訓練。

```
$python tools/train.py -f <MODEL_CONFIG_FILE> -d 1 -b <BATCH_SIZE> -fp16 -o  
-c <PRETRAIN_MODEL_PATH>
```

參數解析：

-f <MODEL_CONFIG_FILE>：指定模型的配置檔案。

-d 1：設備數量（如 GPU）。

-b <BATCH_SIZE>：批次大小。

--fp16：啟用半精度浮點數進行訓練（加速）。

-o：使用自動混合精度訓練。

-c <PRETRAIN_MODEL_PATH>：提供預訓練模型路徑。

(三) 機器學習模型框架轉換與輕量化詳細介紹

1. 機器學習模型框架轉換

PyTorch 是一個強大的深度學習框架，但在嵌入式設備或其他平台中可能需要更通用的模型格式。ONNX（Open Neural Network Exchange）提供了一種通用格式，方便將模型轉換為多種框架格式。執行以下命令將 PyTorch 模型轉換為 ONNX 格式，該步驟的目的是生成 ONNX 格式的模型，方便後續的最佳化或跨平台部署。

```
$python tools/export_onnx.py -f <MODEL_CONFIG_FILE> -c  
<TRAINED_PYTORCH_MODEL> --output-name <ONNX_MODEL_PATH>
```

參數解釋：

- (1) <MODEL_CONFIG_FILE>：配置文件，定義模型結構和參數。
- (2) <TRAINED_PYTORCH_MODEL>：已訓練的 PyTorch 模型權重文件。
- (3) <ONNX_MODEL_PATH>：輸出的 ONNX 模型路徑。

2. 模型輕量化

模型輕量化是一種通過壓縮模型參數精度（如從浮點數轉為整數）以提升運行效率的技術。這在資源受限的設備（如嵌入式系統）上更為重要，本案採輕量化至 FULL-INTEGER8，生成的校準數據將用於模型輕量化過程中的參數調整，確保輕量化模型的精度。。

- (1) 創建校準數據（Create Calibration Data）：執行以下命令生成校準數據：

```
$python demo/TFLite/generate_calib_data.py --img-size <IMG_SIZE> --n-img  
<NUMBER_IMG_FOR_CALI> -o <CALI_DATA_NPY_FILE> --img-dir  
<PATH_OF_TRAIN_IMAGE_DIR>
```

參數解釋：

- (a) <IMG_SIZE>：圖像的輸入大小，例如 320x320。
- (b) <NUMBER_IMG_FOR_CALI>：用於校準的圖像數量。
- (c) <CALI_DATA_NPY_FILE>：輸出的校準數據文件路徑。
- (d) <PATH_OF_TRAIN_IMAGE_DIR>：訓練圖像的目錄路徑。

將 ONNX 模型轉換為 TFLite，這一步將 ONNX 模型壓縮為 TFLite 格式，適用於資源有限的嵌入式設備，執行以下命令進行模型輕量化並轉換為 TFLite 格式。

```
$onnx2tf -i <ONNX_MODEL_PATH> -oiqt -qcind images <CALI_DATA_NPY_FILE>  
"[[[[0,0,0]]]]" "[[[[1,1,1]]]]"
```

參數解釋：

- (1) <ONNX_MODEL_PATH>：輸入的 ONNX 模型路徑。
- (2) <TFLITE_MODEL_PATH>：輸出的 TFLite 模型路徑。
- (3) -oiqt：啟用整數量化（Full Integer8 Quantization）。
- (4) -qcind images：指定校準數據類型為圖像。
- (5) <CALI_DATA_NPY_FILE>：校準數據文件，用於參數調整。
- (6) "[[[[1,1,1]]]]"：定義模型輸入張量的形狀

(四) Vela 編譯器簡介及流程說明

1. Vela 編譯器簡介

Vela 是一款專為 Arm® 微型神經網路處理單元 (microNPU) 設計的編譯器，其目的是最佳化 TensorFlow Lite 模型並生成適用於微型 NPU 的高效推理格式。

a. 主要功能

- (a) 最佳化模型：將 TensorFlow Lite (TFLite) 模型進行最佳化，生成使用 NPU 加速執行的版本。
- (b) 分配運算：將適合的運算分配至 NPU 執行，以減少 CPU 的負擔，藉此加速模型推理。
- (c) 最佳化訪存模式：針對記憶體訪問和指令流最佳化，提高推理效率。

b. 權重壓縮

支援無損壓縮技術，能在不影響準確度的前提下，壓縮權重文件的大小，減少 SRAM 和 Flash 的占用。

2. Vela 編譯流程

```
set IMAGE_SIZE_EXPR="extern const int originalImageSize = 520;"
set CHANNELS_EXPR="extern const int channelsImageDisplayed = 3;"
set CLASS_EXPR="extern const int numClasses = 6;"

set IMAGE_SRC_DIR=..\samples
set IMAGE_SRC_WIDTH=320
set IMAGE_SRC_HEIGHT=320

::set LABEL_SRC_FILE
::set GEN_LABEL_FILE 將MODEL_SRC_FILE、MODEL_OPTIMISE更新為自己的tflite檔案名稱

set MODEL_SRC_DIR
set MODEL_SRC_FILE=volox_nano_nu_medicine_full_integer_quant.tflite
set MODEL_OPTIMISE volox_nano_nu_medicine_full_integer_quant.tflite vela.tflite
::The vela OPTIMISE FILE should be SRC FILE NAME + vela
```

將VELA_ACCEL_CONFIG設定為ethos-u55-256

```
::accelerator config, ethos-u55-32, ethos-u55-64, ethos-u55-128, ethos-u55-256, ethos-u65-256, ethos-u65-512
set VELA_ACCEL_CONFIG=ethos-u55-256
```

圖 77、Step1：更改 variables.bat 檔參數

圖 78、Step2：執行 gen_model_cpp.bat 檔案

運行 gen_model_cpp.bat 批次檔案後，將會生成經過 Vela 編譯的 TFLite 檔案，該檔案經最佳化後可直接用於微型 NPU 的推理，可將檔案放入 Netron 來檢視模型結構。

Netron 是一款開源的視覺化工具，專門用於查看神經網路、卷積神經網路、循環神經網路和其他機器學習模型的結構和權重。可以看到在 Vela 編譯前(如下圖左)，在 CPU 上一些操作需要額外的記憶體讀寫，會拖慢整體推理速度，此 Tensorflow lite 模型經 Vela 編譯後全部運算由 Ethos™-u NPU 處理不需要進行拼接操作減少了額外的數據搬運工作(如下圖右)，提升推理的性能。

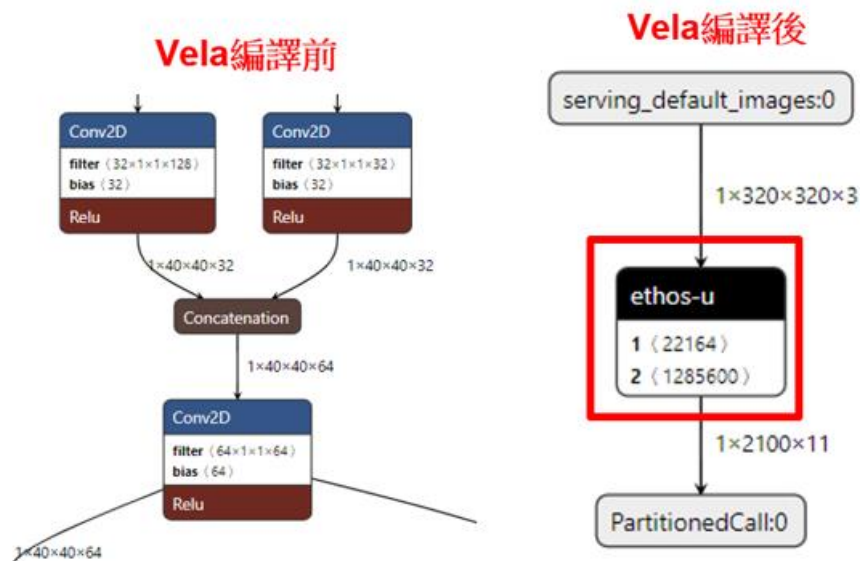


圖 79、Vela 編譯前後結構對比

四、開發板上的推論程式系統流程圖

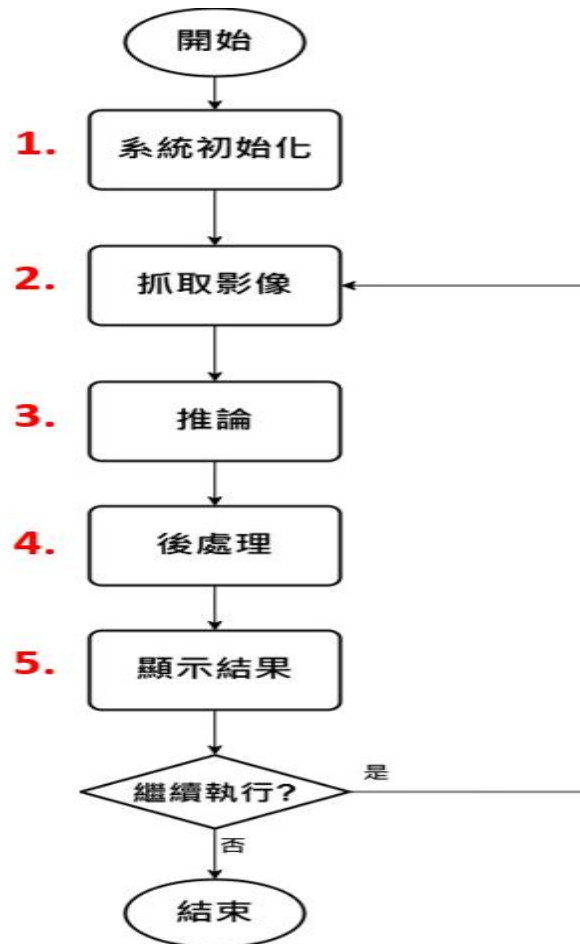


圖 80、程式流程圖

(一) 系統初始化 (System Initialization)

系統初始化 (System Initialization) 的主要目的是為系統運行建立穩定的基礎，避免運行過程中出現異常問題。此階段首先需要初始化硬體資源，確保設備（如傳感器、處理器等）能正常運作，並滿足系統運行的基本需求。接著，配置軟體環境，包括模型的加載與運行，以及相關參數的設置，以確保系統能以最佳狀態進行推論與處理。

在嵌入式系統開發中，硬體和時脈的初始化是確保系統穩定運行的關鍵步驟。以下是對硬體和時脈的初始化兩段程式碼中硬體和時鐘初始化過程的詳細分析。

1. 系統時鐘初始化 (SYS_Init 函數)

(1) 啟用內部 RC 12MHz 時鐘 (HIRC)：

- (a) 呼叫 CLK_EnableXtalRC(CLK_SRCCTL_HIRCEN_Msk)開啟內部時鐘。
- (b) 用 CLK_WaitClockReady(CLK_STATUS_HIRCSTB_Msk)確保時鐘穩定。

(2) 啟用高頻外部時鐘 (HXT)：

- (a) 啟用高頻外部時鐘 CLK_EnableXtalRC(CLK_SRCCTL_HXTEN_Msk)
- (b) 確保時鐘穩定 CLK_WaitClockReady(CLK_STATUS_HXTSTB_Msk)

(3) 設置系統時鐘：

- (a) 切換 clock 來源為 APLL0 並設為 180MHz，用 CLK_SetBusClock 完成。
- (b) 呼叫 SystemCoreClockUpdate() 更新系統核心時鐘值。

(4) 啟用模組時鐘：

- (a) 開啟 GPIO、NPU、FMC0 和 CCAP0 模組的時鐘。
- (b) 使用 SetDebugUartCLK() 設定 UART6 的時鐘來源為 HIRC。

(5) 多功能引腳配置：

- (a) 設置 UART 的 RXD 和 TXD 引腳，用於串列通訊。
- (b) 呼叫 HyperRAM_PinConfig 配置 HyperRAM 的 SPI 多功能引腳。

```

static void SYS_Init(void)
{
    /*-----
    /* Init System Clock
    /*-----
    CLK_EnableXtalRC(CLK_SRCCTL_HIRCEN_Msk);
    CLK_WaitClockReady(CLK_STATUS_HIRCSTB_Msk);
    CLK_EnableXtalRC(CLK_SRCCTL_HXTEN_Msk);
    CLK_WaitClockReady(CLK_STATUS_HXTSTB_Msk);
    CLK_SetBusClock(CLK_SCLKSEL_SCLKSEL_APLL0, CLK_APLLCTL_APLLSRC_HIRC, FREQ_180MHZ);
    SystemCoreClockUpdate();
    CLK_EnableModuleClock(GPIOA_MODULE);
    CLK_EnableModuleClock(GPIOB_MODULE);
    CLK_EnableModuleClock(GPIOC_MODULE);
    CLK_EnableModuleClock(GPIOD_MODULE);
    CLK_EnableModuleClock(GPIOE_MODULE);
    CLK_EnableModuleClock(GPIOF_MODULE);
    CLK_EnableModuleClock(GPIOG_MODULE);
    CLK_EnableModuleClock(GPIOH_MODULE);
    CLK_EnableModuleClock(GPIOI_MODULE);
    CLK_EnableModuleClock(GPIOJ_MODULE);
    CLK_EnableModuleClock(FMC0_MODULE);
    CLK_EnableModuleClock(NPU0_MODULE);
    CLK_EnableModuleClock(CCAP0_MODULE);
    SetDebugUartCLK();
    /*-----
    /* Init I/O Multi-function
    /*-----
    SetDebugUartMFP();
    HyperRAM_PinConfig(HYPERRAM_SPIM_PORT);
}

```

圖 81、系統時鐘初始化 (SYS_Init 函數)

2. 硬體初始化 (BoardInit 函數)

該函數負責呼叫 SYS_Init，並進一步完成其他硬體的初始化，以下為詳細步驟說明。

(1) 解鎖系統保護寄存器

使用 SYS_UnlockReg() 解鎖保護寄存器對時鐘和模組進行設置。

(2) 呼叫 SYS_Init 函數

完成系統時鐘及模組時鐘的初始化。

(3) 初始化 UART

使用 InitDebugUart() 初始化 UART6，確保能夠正確使用 printf 進行輸出。

(4) 重新鎖定寄存器

使用 SYS_LockReg() 鎖定寄存器，防止後續的非預期改動。

(5) 初始化 HyperRAM

呼叫 HyperRAM_Init 完成 HyperRAM 的初始化，切換 HyperRAM 至直接映射模式，使用 SPIM_HYPER_EnterDirectMapMode 實現。

(6) NPU 初始化（選用）

若啟用了 ARM® NPU，則使用 ethosu_npu_init 完成 Ethos™-U 的初始化。

(7) 輸出系統設計名稱

用 info("Target system: %s\n", DESIGN_NAME) 輸出目標系統設計名稱。

```
int BoardInit(void)
{
    SYS_UnlockReg();
    SYS_Init();
    InitDebugUart();
    SYS_LockReg();          /* Unlock register lock protect */
    HyperRAM_Init(HYPERRAM_SPIM_PORT);
    SPIM_HYPER_EnterDirectMapMode(HYPERRAM_SPIM_PORT);
    info("%s: complete\n", __FUNCTION__);
#ifdef ARM_NPU
    int state;
    /* If Arm Ethos-U NPU is to be used, we initialise it here */
    if (0 != (state = arm_ethosu_npu_init()))
    {
        return state;
    }
#endif /* ARM_NPU */
    /* Print target design info */
    info("Target system: %s\n", DESIGN_NAME);
    return 0;
}
```

圖 82、硬體初始化 (BoardInit 函數)

(二) 捕捉影像 (Capture Image)

捕捉影像 (Capture Image) 的主要目的是實時獲取待檢測的影像資料，作為模型推論的輸入，確保系統能快速回應並進行準確的判斷。此階段需要確保輸入數據的準確性和即時性，以提升整體檢測效率，避免因影像延遲或數據不準確而導致推論結果偏差。此外，捕捉影像功能還需支援動態場景中的監控需求，例如移動中的目標偵測或變化場景的即時更新，確保系統能適應多樣化的應用情境並保持高效運作。在 `main.cpp` 中，擷取影像主要由 `CCAP (Camera Capture)` 功能實現，其主要邏輯集中在 `main_task` 函數內。以下是詳細分析與步驟：

1. 初始化攝影機

- (1) 呼叫 `ImageSensor_Init`，初始化影像感測器的硬體設定。
- (2) 呼叫 `ImageSensor_Config`，設定影像格式為 `eIMAGE_FMT_RGB565` 並設定影像解析度，使用 `frameBuffer.w` 和 `frameBuffer.h`，這確保了影像感測器能夠捕捉所需格式和大小的影像。

```
#if defined (__USE_CCAP__)
//Setup image sensor
ImageSensor_Init();
ImageSensor_Config(eIMAGE_FMT_RGB565, frameBuffer.w, frameBuffer.h);
#endif
```

圖 83 初始化攝影機之程式碼

2. 擷取影像邏輯

使用 `get_empty_framebuf()` 檢查是否有空閒的影像緩衝區，如果存在空緩衝區，執行影像擷取。

```
static S_FRAMEBUF *get_empty_framebuf()
{
    int i;

    for (i = 0; i < NUM_FRAMEBUF; i++)
    {
        if (s_asFramebuf[i].eState == eFRAMEBUF_EMPTY)
            return &s_asFramebuf[i];
    }

    return NULL;
}
```

圖 84、確認可用的空影像緩衝區

3. 執行擷取操作

使用 `ImageSensor_Capture` 將攝影機影像捕捉到指定的緩衝區內，該函數將影像資料存入 `emptyFramebuf->frameImage.data`。

```
ImageSensor_Capture((uint32_t)(emptyFramebuf->frameImage.data));
```

圖 85、執行擷取操作

4. 更新影像緩衝區狀態

將緩衝區狀態設為 `eFRAMEBUF_FULL`，表示該緩衝區內已有完整影像。

```
emptyFramebuf->results.clear();
emptyFramebuf->eState = eFRAMEBUF_FULL;
```

圖 86、更新影像緩衝區狀態

5. 影像格式處理

在非 CCAP 模式下，影像擷取採用靜態影像來源，並進行格式轉換，將靜態影像的原始數據（pu8ImgSrc）複製到緩衝區，imlib_nvt_scale 函數進行影像縮放和格式轉換，確保模型輸入需求。

6. 影像緩衝區管理

影像緩衝區 (S_FRAMEBUF) 管理採用以下策略：

- (1) 空閒緩衝區：get_empty_framebuf 確保有可用的緩衝區供新影像擷取。
- (2) 滿緩衝區：get_full_framebuf 用於檢測已完成擷取但尚未處理的影像。
- (3) 推論緩衝區：get_inf_framebuf 確保推論後的影像可用於後續顯示。

7. 結果輸出與後續處理

擷取完成後，影像將進一步縮放、量化，並傳遞給推論模組進行物件檢測。結果再經過處理並顯示。

(三) 推論 (Inference)

推論 (Inference) 的主要目的是利用預訓練模型對捕捉到的輸入影像進行分析與處理，以完成目標檢測或分類等任務。在此階段，系統會提取影像中的關鍵特徵，並生成高效且準確的推論結果，以支持後續的決策或顯示。此外，為提升推論速度，系統會充分利用開發板上的硬體資源（如神經處理單元 NPU），通過硬體加速的方式顯著縮短處理時間，實現即時性的目標識別與判斷能力。推論過程的準確性和效率是系統整體性能的關鍵指標之一。

```
arm::app::YoloXnanoNu model;
```

圖 87、STEP1：創建模型實例

```

if (!model.Init(arm::app::tensorArena,
                sizeof(arm::app::tensorArena),
                arm::app::yoloxnanonu::GetModelPointer(),
                arm::app::yoloxnanonu::GetModelLen()))
{
    printf_err("Failed to initialise model\n");
    vTaskDelete(nullptr);
    return;
}

```

圖 88、STEP2：初始化模型

```

TfLiteIntArray *inputShape = model.GetInputShape(0);

const int inputImgCols = inputShape->data[arm::app::YoloXnanoNu::ms_inputColsIdx];
const int inputImgRows = inputShape->data[arm::app::YoloXnanoNu::ms_inputRowsIdx];

```

圖 89、STEP3：設定模型輸入尺寸

首先從鏡頭獲取完整的影像數據，並存儲於緩衝區中（例如使用 `S_FRAMEBUF *fullFramebuf`）。接著，透過函數 `imlib_nvt_scale` 將影像縮放至模型要求的輸入尺寸，同時定義 ROI（感興趣區域）進行裁剪與調整，以確保影像格式與尺寸符合模型的需求。完成縮放與格式轉換後，將處理後的影像數據載入模型的輸入張量區（例如使用 `TfLiteTensor *inputTensor = model.GetInputTensor(0)`）。最後，調用模型的推論函數（例如 `m_model->RunInference()`），執行影像數據的分析與處理，生成檢測結果。這一系列步驟確保了輸入數據的準確性與即時性，為後續的目標檢測和分類提供了可靠的基礎。

```

S_FRAMEBUF *fullFramebuf;

imlib_nvt_scale(&fullFramebuf->frameImage, &resizeImg, &roi);

TfLiteTensor *inputTensor = model.GetInputTensor(0);

bool runInf = m_model->RunInference();

```

圖 90、STEP4：影像預處理

(四) 後處理與繪製偵測結果 (Post-Processing & Draw Results)

後處理與繪製偵測結果 (Post-Processing & Draw Results) 的主要目的是對模型輸出的資訊進行進一步的篩選與優化，提升檢測結果的可用性與易讀性。系統會篩選輸出的資訊，例如濾除低信心度的檢測結果，確保只保留可靠性較高的數據。接著，透過繪製功能，將檢測到的目標以直觀的方式標示在影像上，例如添加邊界框、分類標籤或其他視覺化標記，清晰顯示目標的位置與屬性，後處理步驟詳細說明如下。

1. 將模型輸出的邊界框按照原始影像的比例進行縮放，確保框的位置和大小與實際影像對應，將模型輸出的 x, y, w, h 按比例映射到原始影像的大小。確保邊界框位於影像的合法範圍內。

```
for (auto &it : detections)
{
    // transfer to original img size (base on resize ratio)
    it.bbox.x = (it.bbox.x * originalImageWidth) / net.inputWidth;
    it.bbox.y = (it.bbox.y * originalImageHeight) / net.inputHeight;
    it.bbox.w = (it.bbox.w * originalImageWidth) / net.inputWidth;
    it.bbox.h = (it.bbox.h * originalImageHeight) / net.inputHeight;
```

圖 91、STEP1：縮放偵測框

2. 從縮放後的數據中，提取物體的類別、機率及位置，並以結構化形式保存，遍歷所有類別的偵測機率，若機率大於零，則將結果記錄下來。保存的結果包括類別、信心度及邊界框的座標資訊。。

```
for (int j = 0; j < net.numClasses; ++j)
{
    if (it.prob[j] > 0)
    {
        DetectionResult tmpResult = {};
        tmpResult.m_normalisedVal = it.prob[j];
        tmpResult.m_x0 = (int)boxX;
        tmpResult.m_y0 = (int)boxY;
        tmpResult.m_w = (int)boxWidth;
        tmpResult.m_h = (int)boxHeight;
        tmpResult.m_cls = j;

        resultsOut.push_back(tmpResult);
    }
}
```

圖 92、STEP2：提取偵測結果

3. NMS 用於移除重疊過多的邊界框，僅保留可能性最高的框，此函數基於指定的 m_nms 閾值執行 NMS，移除冗餘框。

```
/* Do nms */
CalculateNMS(detections, net.numClasses, m_nms);
```

圖 93、STEP3：執行 NMS (Non-Maximum Suppression)

4. 處理完畢的偵測結果被存入向量 resultsOut 中，供後續步驟使用。此後處理模組有效解析模型輸出的結果，進一步縮小到原始影像比例，並應用 NMS 移除冗餘框。偵測結果以結構化方式保存，供後續顯示或分析使用。

```
resultsOut.push_back(tmpResult);
```

圖 94、STEP4：儲存最終結果

(五) 顯示結果 (Display Results)

顯示結果 (Display Results) 的主要目的是將處理後的影像與檢測結果以直觀的方式通過螢幕或其他輸出設備進行展示，讓用戶能即時獲取監控區域的相關資訊。幫助快速了解環境狀況與目標動態，為後續行動（如觸發警報、數據記錄或決策執行）提供了重要依據。以下為顯示結果 (Draw Results) 的詳細解析

在 `main.cpp` 文件中的 `DrawImageDetectionBoxes` 函數負責將偵測結果以圖像方式視覺化，函數的主要流程如下。

1. 遍歷 `results` 中的每個物件偵測結果。
2. 將框的座標傳入 `imlib_draw_rectangle` 函數繪製矩形。
3. 將物件的類別標籤透過 `imlib_draw_string` 畫到框的上方，使用 `labels[result.m_cls]` 獲取分類名稱。

```
static void DrawImageDetectionBoxes(  
    const std::vector<arm::app::object_detection::DetectionResult> &results,  
    image_t *drawImg,  
    std::vector<std::string> &labels)  
{  
    for (const auto &result : results)  
    {  
        imlib_draw_rectangle(drawImg, result.m_x0, result.m_y0, result.m_w, result.m_h, COLOR_B5_MAX, 1, false);  
        imlib_draw_string(drawImg, result.m_x0, result.m_y0 - 16, labels[result.m_cls].c_str(), COLOR_B5_MAX, 2, 0, 0, false,  
            false, false, false, 0, false, false);  
    }  
}
```

圖 95、 `DrawImageDetectionBoxes` 函數

(六) 實際燒錄至開發板測試

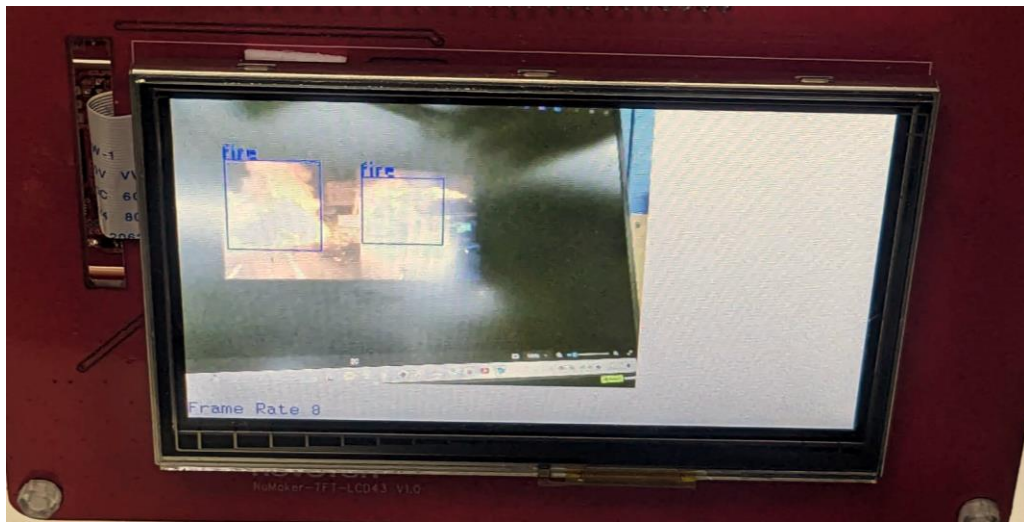


圖 96、測試圖片 1

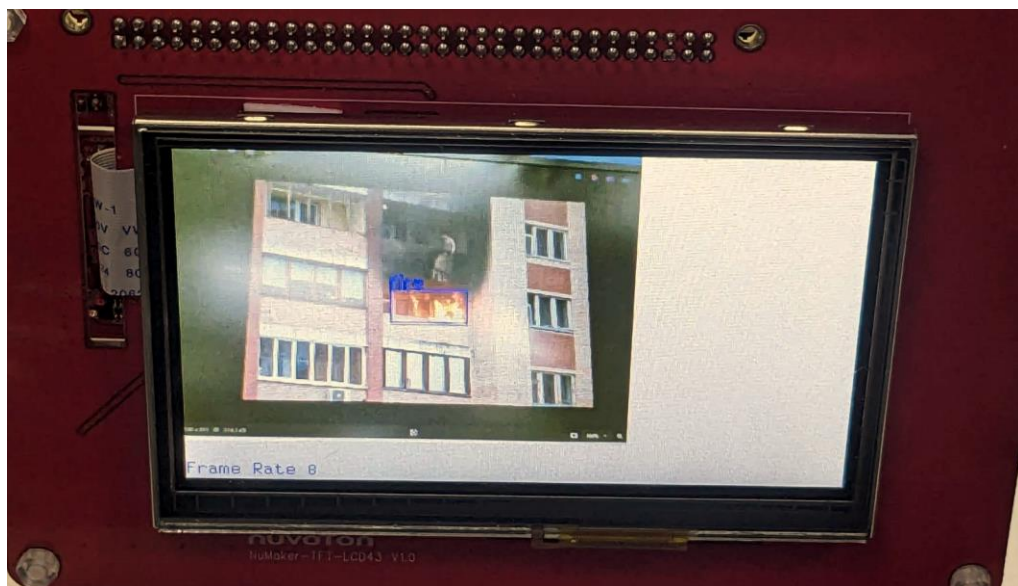


圖 97、測試圖片 2

五、結論與未來發展

在本次火災偵測系統的開發中，我們成功運用 Roboflow 資料集結合 YOLOX-Nano 模型，在新唐科技 M55M1 開發板上實現了一套高效、即時且節能的火災偵測解決方案。透過 Full-INT8 模型量化技術和 Vela 編譯器的最佳化，我們顯著提升了模型的推論速度和準確率，在測試中達到高達 98% 的偵測準確率。系統實現了以下目標：

- (一) 快速反應能力：即時捕捉並標註火光區域，為工廠和其他高風險環境提供高效的火災預警功能。
- (二) 高效能與低功耗：藉由 NPU 的硬體加速，實現了在嵌入式環境中的高效能運算，並降低了系統能耗。
- (三) 可擴展性：開發板的豐富介面支持使其適用於多種場景，包括工業區、家庭安全和公共設施的火災預防。
- (四) 多模態感測整合：結合其他感測技術（如溫度感測器或煙霧探測器），提升偵測的準確性和多樣性。
- (五) 提升模型的泛化能力：擴展資料集，增加不同火災環境和場景的樣本，強化模型在多樣化場景中的表現。
- (六) 邊緣運算最佳化：針對嵌入式系統，探索更高效的模型壓縮與最佳化方法，進一步降低運算資源需求。
- (七) 遠端監控與聯網功能：結合 IoT 技術，實現火災偵測系統的即時雲端數據上傳與遠端監控，提升應用的整體效能與便捷性。

這些發展方向將進一步強化火災偵測系統的功能，使其能更廣泛地應用於智慧工廠、智慧家庭與公共安全等領域，為減少火災帶來的生命與財產損失作出更大的貢獻。

參考資料：

<https://hackmd.io/@hohoho/rkdZNWaZh>

<https://medium.com/@andy6804tw/%E5%BF%AB%E9%80%9F%E4%B8%8A%E6%89%8Byolo-%E5%88%A9%E7%94%A8-roboflow-%E5%92%8C-ultralytics-hub-%E5%AE%8C%E6%88%90%E6%A8%A1%E5%9E%8B%E8%A8%93%E7%B7%B4%E8%88%87%E7%AE%A1%E7%90%86-%E4%B8%8A-37acd110a8a0>

https://hackmd.io/@nB1rzit6Toq8WdkZosRK_Q/r16-c7Ynj

<https://ithelp.ithome.com.tw/articles/10305905>

https://github.com/MaxCYCHEN/yolox-ti-lite_tflite_int8?tab=readme-ov-file

參、智慧工廠 3 – 降噪與關鍵字偵測

一、案例介紹 – 降噪與關鍵字偵測

(一)專案背景

在數位化與智慧化快速發展的時代，聲音處理技術已成為物聯網、智慧工廠以及嵌入式應用場景中的一項關鍵技術。隨著智慧設備的廣泛應用，用戶對語音交互的需求與日俱增。然而，在工業環境或家庭場景中，背景噪聲干擾往往使語音訊號的準確識別變得困難，這對智慧設備的使用體驗提出了更高的要求。

如何實現有效的噪聲抑制並準確識別語音指令，是當前語音處理技術研究的重要課題。本專案聚焦於基於新唐 M55M1 開發板的嵌入式語音處理解決方案，結合了頻譜減法降噪技術與關鍵字辨識模型，旨在解決噪聲環境下的語音指令準確性問題，並提升系統的實時性能。

本專案的具體目標包括：

1. **聲音降噪模組開發：** 利用頻譜減法技術，顯著降低背景噪聲，提升語音訊號的清晰度，為語音處理提供高質量的數據基礎。
2. **關鍵字辨識模組整合：** 通過結合關鍵字辨識技術，實現語音指令的快速識別，應用於語音控制場景。
3. **性能最佳化：** 採用 ARM® Helium 技術，提升運算效率，確保系統適配嵌入式設備的低功耗與實時處理需求。

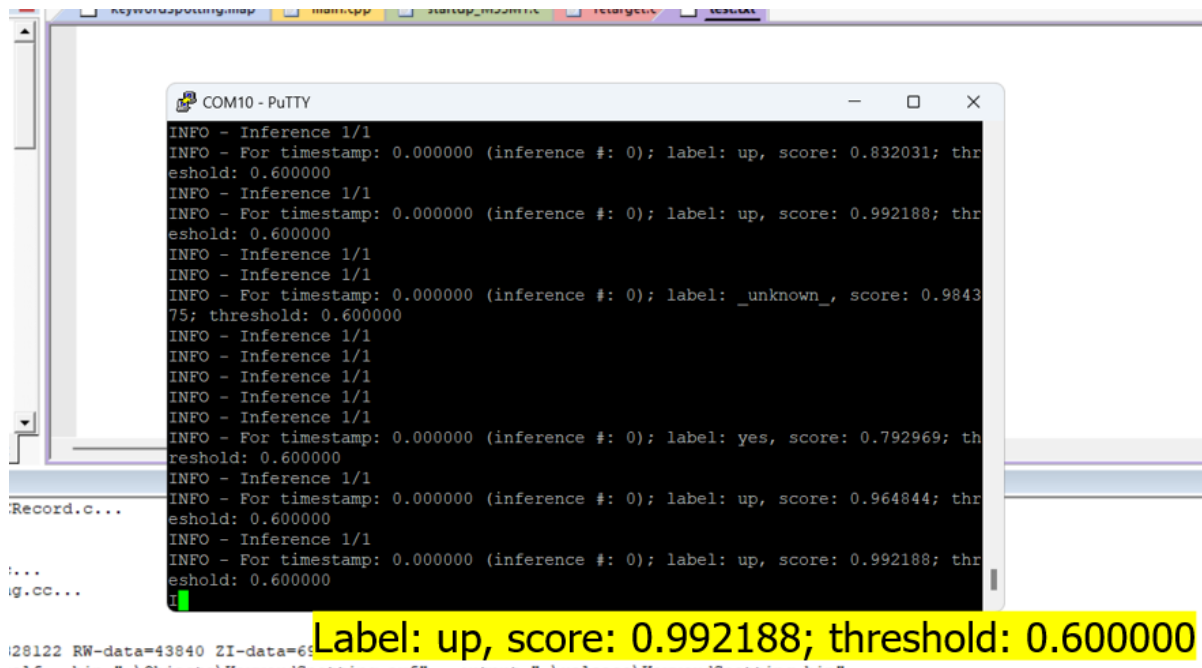


圖 98、專案預計實現流程

(二)預期目標

本專案基於新唐提供的關鍵字辨識模組，該模組能夠辨識十種常用關鍵字，包括："yes"、"no"、"up"、"down"、"left"、"right"、"on"、"off"、"stop"、"go"。在此基礎上，將引入透過 ARM® Helium 技術加速運算的頻譜減法降噪功能，進一步提升系統性能。同時，還將增強終端機的交互功能，提供更豐富的操作選項以便於使用者控制與調整。

原始的關鍵字辨識模組在噪聲干擾環境下測試的準確率為 69% (69/100)。透過加入頻譜減法的降噪功能後，辨識準確率顯著提升至 85% (85/100)，展現出此種降噪技術確實能增強語音識別穩定性與準確度。



The image shows a terminal window titled "COM10 - PuTTY" displaying the output of a voice recognition system. The output consists of multiple lines of log messages. Each line starts with "INFO - Inference 1/1" followed by "INFO - For timestamp: 0.000000 (inference #: 0); label: up, score: 0.832031; threshold: 0.600000". The score increases in subsequent lines, reaching 0.992188. A line with "label: _unknown_" is also visible. The final line, which is highlighted in yellow, shows "Label: up, score: 0.992188; threshold: 0.600000".

```
INFO - Inference 1/1
INFO - For timestamp: 0.000000 (inference #: 0); label: up, score: 0.832031; threshold: 0.600000
INFO - Inference 1/1
INFO - For timestamp: 0.000000 (inference #: 0); label: up, score: 0.992188; threshold: 0.600000
INFO - Inference 1/1
INFO - Inference 1/1
INFO - For timestamp: 0.000000 (inference #: 0); label: _unknown_, score: 0.984375; threshold: 0.600000
INFO - Inference 1/1
INFO - Inference 1/1
INFO - Inference 1/1
INFO - Inference 1/1
INFO - For timestamp: 0.000000 (inference #: 0); label: yes, score: 0.792969; threshold: 0.600000
INFO - Inference 1/1
INFO - For timestamp: 0.000000 (inference #: 0); label: up, score: 0.964844; threshold: 0.600000
INFO - Inference 1/1
INFO - For timestamp: 0.000000 (inference #: 0); label: up, score: 0.992188; threshold: 0.600000
Label: up, score: 0.992188; threshold: 0.600000
```

圖 99、語音輸入"up"於終端機顯示的辨識結果

(三) 新唐 M55M1 開發板簡介

新唐 M55M1 開發板是一款針對嵌入式應用場景設計的高性能硬體平台，其核心為基於 ARM® Cortex®-M55 處理器的架構設計，結合了多項技術優勢，使其在語音處理應用中具有突出的性能。

新唐 M55M1 的主要特性：

1. 高效能運算：

- (1) 支援 Helium 技術（M-Profile Vector Extension），顯著提升數位訊號處理（DSP）與機器學習（ML）任務的運算速度。
- (2) 能夠高效執行快速傅立葉變換（FFT）、矩陣運算等密集型處理。

2. 多樣化 I/O 介面：

- (1) 支援數位麥克風（DMIC）與微機電麥克風（MEMS MIC）等輸入設備，為語音數據的高精度捕捉提供硬體支持。
- (2) 豐富的 GPIO、UART 和 I2S 介面設計，滿足多種應用需求。

3. 開發便捷性：

- (1) 提供完備的 SDK 與開發工具，支援快速部署與測試。
- (2) 兼容 Keil、ARM CMSIS-DSP 等工具鏈，方便開發者快速上手。

應用價值：

新唐 M55M1 開發板在語音訊號處理中，能夠支持多項操作，包括訊號預處理、特徵提取和模型推理等任務。其出色的性能使其成為本專案的核心硬體平台，為降噪與關鍵字辨識功能的實現提供了堅實的基礎。

(四) 頻譜減法與關鍵字辨識技術概述

1. 頻譜減法 (Spectral Subtraction)

頻譜減法是一種經典的聲音降噪技術，其基本思想是通過估算背景噪聲的頻譜，從含噪語音的頻譜中扣除噪音成分，以達到降噪效果。

具體實現方式包括以下幾步：

- (1) **背景噪聲建模：** 將環境背景聲音的頻譜作為參考，生成噪聲模板。
- (2) **頻譜扣減：** 將背景噪聲的頻譜從語音頻譜中扣減，消除噪聲影響。
- (3) **訊號重建：** 通過逆傅立葉變換 (IFFT) 還原乾淨的時域訊號。

2. 關鍵字辨識

關鍵字辨識是語音處理中的一個重要應用，主要目的是在連續語音中檢測特定的語音指令（如「開啟燈光」）。其實現過程包括：

- (1) **語音特徵提取：** 使用梅爾頻率倒譜係數 (MFCC) 等方法由語音訊號中取得特徵。
- (2) **模型推理：** 將提取的特徵輸入預訓練的神經網路模型，進行識別。
- (3) **觸發應用：** 根據識別結果，觸發相應的指令或操作。

3. 技術結合的優勢：

將頻譜減法和關鍵字辨識技術結合，能有效消除工業或家庭場景中的噪聲干擾，提升語音指令的識別準確性，降低系統的誤觸發率。

(五) ARM® Helium 技術的特性與優勢

ARM® Helium 技術是一種專為 Cortex®-M 系列處理器設計的向量擴展技術（MVE），旨在顯著提升嵌入式設備的運算能力，尤其是在數位訊號處理（DSP）和機器學習（ML）領域的表現。

Helium 技術的主要特性

1. 高效運算：

- (1) 支持向量化操作，可一次處理多個數據，顯著加速 FFT、濾波等數位訊號處理任務。
- (2) 對降噪演算法（如頻譜減法）中的矩陣運算最佳化效果顯著。

2. 低功耗設計：

- (1) 為嵌入式設備量身打造，在提升運算性能的同時保持低功耗表現。
- (2) 適合於需長時間運行的智慧設備（如語音控制設備）。

3. 應用範圍廣：

- (1) 支援音訊處理、機器學習推理、影像處理等多種應用場景。
- (2) 能夠滿足智慧家居、工業控制、醫療設備等多領域需求。

在本專案中的應用

在本專案中，Helium 技術被用於最佳化頻譜減法與 MFCC 特徵提取的運算效率，顯著降低了處理延遲，並確保了系統的實時性能。這使得 M55M1 開發板能夠高效執行語音處理任務，成為嵌入式語音應用的理想選擇。

小結

第一章從專案背景、技術基礎到硬體平台的介紹，構建了一個全面的框架，為後續的開發流程與技術實現提供了清晰的目標與依據。

二、 開發流程概述

本專案的開發流程以實現高效降噪和準確關鍵字辨識為核心，通過清晰的硬體架構設計與軟體流程規劃，確保功能模組之間的緊密協作與穩定運行。

(一)系統架構設計

本案例的系統架構設計分為**硬體架構**與**軟體流程**兩大部分，各自負責聲音訊號的擷取、處理與輸出，實現從聲音數據收集到指令觸發的完整流程。

硬體架構設計

1. 麥克風輸入模組：

- (1) 作用：使用數位麥克風（DMIC）擷取語音訊號，保證輸入數據的高質量和穩定性。
- (2) 價值：DMIC 能有效減少類比麥克風常見的噪聲問題，提升語音數據的真實度與清晰度。

2. 核心處理模組：

- (1) 作用：以新唐 M55M1 開發板為主控單元，完成語音數據的收集、降噪處理與關鍵字識別。
- (2) 特點：M55M1 支援 ARM® Helium 技術，實現高效運算，能快速執行複雜的計算。

3. 輸出模組：

- (1) 作用：通過終端機輸出辨識結果，用戶可根據需求選擇驅動外部設備執行相應操作。
- (2) 應用場景：包括智慧家居命令觸發、工業設備控制等。

軟體流程設計

軟體流程分為以下三個主要步驟，確保語音數據能從原始輸入到指令執行：

1. 音頻預處理：

收集的語音訊號經過分段處理，並通過短時傅立葉變換（STFT）轉換至頻域，為降噪與特徵提取奠定基礎。

2. 降噪處理：

使用頻譜減法技術去除背景噪聲，保留語音訊號中的有效頻率成分，生成乾淨的語音數據。

3. 關鍵字辨識：

提取經降噪處理後的語音特徵（如 MFCC），並輸入預訓練的深度學習模型進行分類，實現語音指令的準確識別。

(二) 軟硬體需求清單

為確保專案的順利實施，需準備以下資源：

硬體需求

1. 新唐 M55M1 開發板：

作為主控單元，提供穩定的處理性能與豐富的 I/O 支援，是本專案的核心硬體平台。

2. 數位麥克風 (DMIC)：

負責擷取高品質的語音訊號，提供穩定輸入。

3. 終端機或顯示設備：

用於顯示降噪與辨識結果，方便用戶檢測系統的處理效果。

4. 供電模組：

為硬體系統提供穩定電源，確保設備長時間運行的可靠性。

提供 UART 界面，可與 PC 終端機通訊。

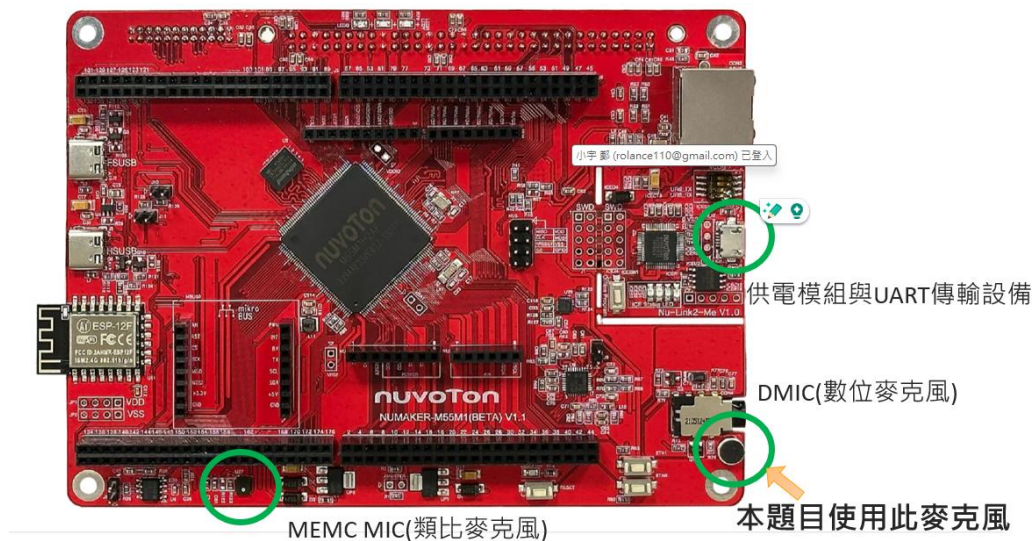


圖 100、專案相關 I/O 模組

軟體需求

1. Keil MDK：

作為主要的 C 語言開發工具，適合 M55M1 開發板的程式開發與編譯需求。

2. Python 環境：

用於模擬頻譜減法降噪，驗證演算法的可行性並進行參數調整。

3. ARM® DSP 函數庫：

提供高效的數位訊號處理工具，用於加速 STFT、頻譜減法與特徵提取的計算。

4. 終端機工具：

如 PuTTY 或 Tera Term，用於與嵌入式系統交互，進行調試與數據監控。

- **2.4 開發流程**

為了系統性完成本專案，開發步驟設計如下：

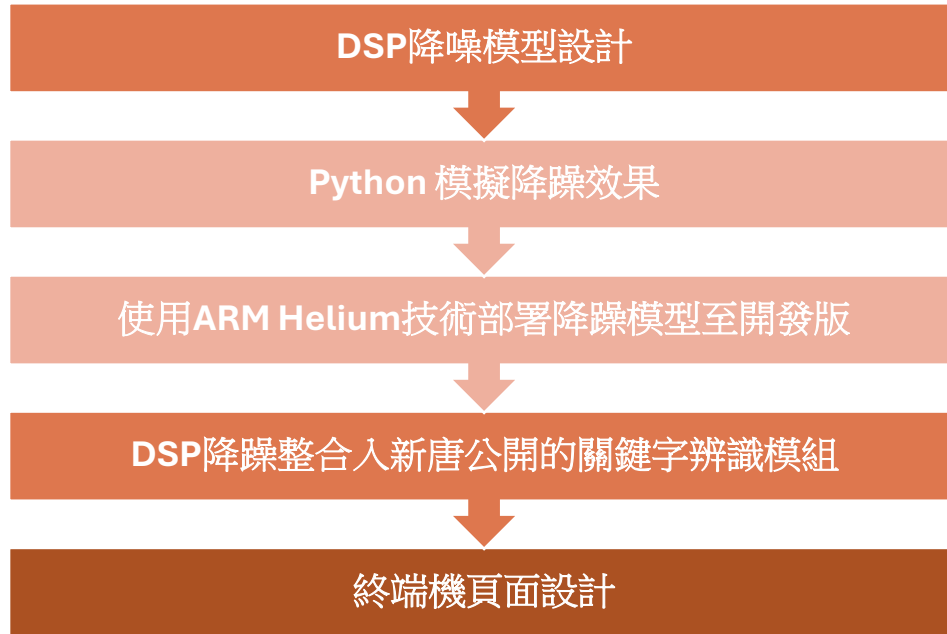


圖 101、系統開發流程

小結

第二章詳細說明了系統的設計與開發流程，從硬體架構到軟體邏輯均做了細緻規劃。模組化設計和清晰的步驟安排，為專案的實現提供了高效的開發路徑，也為後續章節的實作奠定了堅實的基礎。

三、 頻譜減法的理論基礎

頻譜減法作為一種經典的語音降噪技術，廣泛應用於噪聲環境下的語音訊號處理。本章將詳細介紹頻譜減法的訊號模型、核心算法原理以及其應用過程中的挑戰與解決方案。

(一)頻譜減法的訊號模型

頻譜減法的基本思想是通過建模背景噪聲的頻譜特性，從含噪語音的頻譜中扣除背景噪聲部分，以還原乾淨的語音訊號。

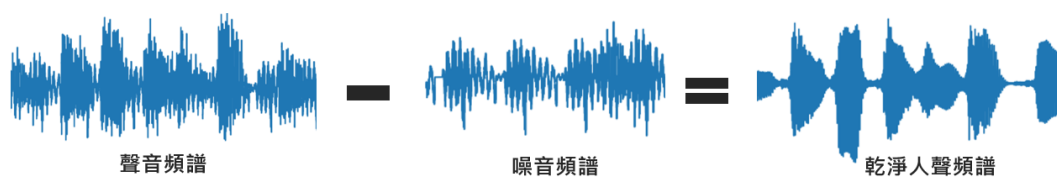


圖 102、頻譜減法示意圖

訊號模型數學表示

在時域中，含噪語音訊號 $y(t)$ 可以表示為：

$$y(t)=s(t)+n(t)$$

其中： $y(t)$ ：含噪語音訊號 $s(t)$ ：乾淨語音訊號 $n(t)$ ：背景噪聲訊號。

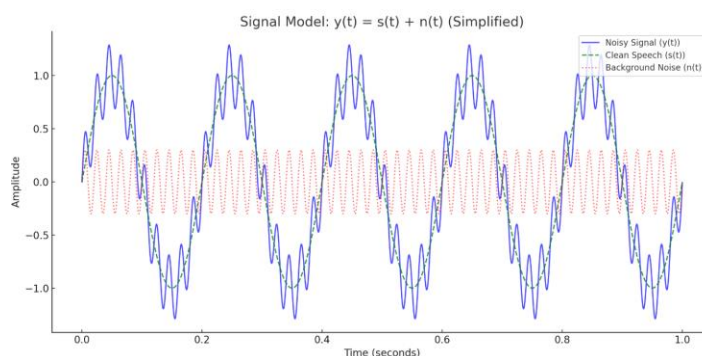


圖 103、 $y(t)=s(t)+n(t)$ 示意圖

通過傅立葉變換將訊號轉換到頻域後，模型可表示為：

$$Y(f)=S(f)+N(f)$$

其中：

- **Y(f)**：含噪語音的頻譜。
- **S(f)**：乾淨語音的頻譜。
- **N(f)**：背景噪聲的頻譜。

應用價值

此降噪算法的核心優勢在於簡單高效，特別適合嵌入式系統中的語音降噪應用。

(二)STFT（短時傅立葉變換）簡介

1. 語音訊號的特性

語音訊號是一種時變訊號，其頻率成分會隨時間變化。如果對整段語音進行傅立葉變換，只能獲得整段時間內的頻譜信息，無法捕捉頻率隨時間的變化。

2. STFT 的原理與優勢

短時傅立葉變換（Short-Time Fourier Transform, STFT）通過將訊號分段並加窗，對每段進行傅立葉變換，獲得頻率隨時間的變化。

數學公式：對於訊號 $x(t)$ ，其 STFT 表示為：

$$X(t, f) = \int_{-\infty}^{\infty} x(\tau) \cdot w(t - \tau) \cdot e^{-j2\pi f \tau} d\tau$$

圖 104、STFT 數學公式

其中：

- $w(t)$ ：窗口函數，用於取得短時訊號。
- $X(t, f)$ ：在時間 t 和頻率 f 上的頻譜。

3. STFT 的步驟

- (1) 分段：將訊號按固定長度（如 20ms）進行分段。
- (2) 加窗：對每段訊號乘以窗口函數（如 Hamming Window），減少失真。
- (3) 傅立葉變換：對每段訊號計算頻譜，獲得頻率隨時間的變化。

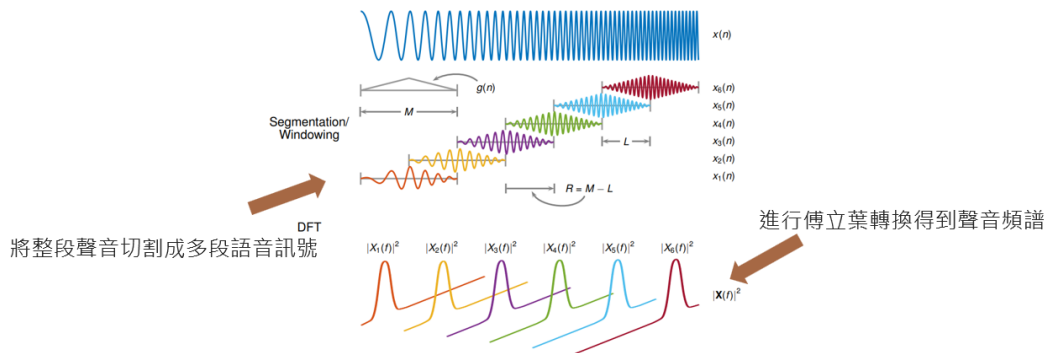


圖 105、STFT 圖解示意圖

4. 技術優勢

- (1) 計算效率高：簡單的頻譜減法運算適合實時處理。
- (2) 適用性強：可應用於多種噪聲環境，尤其是穩定背景噪聲場景。

(三)頻譜減法與實時處理的挑戰

儘管頻譜減法在理論上簡單有效，但在實際應用中仍需應對多種挑戰。

1. 噪聲頻譜估算的準確性

背景噪聲頻譜可能隨時間變化，導致固定模板的適用性下降。

解決方法：

使用動態更新機制，每隔一定時間重新錄製噪聲頻譜，並引入自適應算法根據環境變化調整模板。

2. 頻譜減法的音質損失

過度扣減可能導致語音頻譜中的有效成分受損，產生“機械聲”或“金屬聲”。

解決方法：

精確調整縮放因子 α ，平衡降噪效果與語音質量，同時採用頻譜平滑技術減少聲音的斷裂感。

3. 實時性需求

嵌入式系統中，STFT 和頻譜計算可能導致運行延遲，影響實時性。

解決方法：

使用 ARM® Helium 技術最佳化 FFT 和矩陣運算，顯著提高處理效率。

小結

頻譜減法作為一種簡單有效的降噪技術，憑藉其在頻域中的高效處理能力，在語音訊號處理中有著廣泛應用。本章不僅從數學模型到算法細節全面解讀了頻譜減法的原理，還剖析了實際應用中的挑戰與解決方案，為後續的系統實現奠定了理論基礎。

四、Python 降噪模擬

為了在嵌入式硬體實現之前驗證頻譜減法降噪技術的可行性，本章使用 Python 搭建模擬環境，完成從背景噪聲建模到降噪效果評估的全過程。Python 強大的音訊處理工具和豐富的數據集支持，使模擬過程高效且直觀。

(一)使用 Python 模擬的目的與價值

在語音訊號處理的實際應用中，嵌入式硬體開發需要大量的測試與調校，為了節省開發成本與時間，採用 Python 進行模擬具有以下優勢：

1. 快速驗證理論可行性：

頻譜減法的理論基於數學模型，但其在真實場景中的效果需要實際驗證。Python 提供快速原型開發環境，能夠便捷地測試不同參數組合下的降噪性能。

2. 簡化音訊處理：

利用 Python 的 Librosa 、 Scipy 等工具，可以輕鬆完成音訊的讀取、分析與處理，無需自行編寫複雜函數。

3. 便於視覺化與參數調整：

使用 Matplotlib 等可視化工具展示降噪前後的波形與頻譜圖，幫助設計者直觀比較效果並調整參數（如縮放因子 α ）。

(二) 相關函數介紹

在本章的模擬過程中，我們選用了以下 Python 工具完成關鍵任務：

1. Librosa：

- (1) 提供便捷的音訊讀取與短時傅立葉變換（STFT）功能。
- (2) 支持音訊頻譜的處理與重建，適合降噪與特徵分析。

2. Soundfile :

- (1) 支持高效讀取與儲存各類音訊格式（如 WAV、FLAC 等）。
- (2) 適合處理大型音訊數據，確保操作簡便且高效。

3. Scipy :

- (1) 提供傅立葉變換、濾波器設計等數學工具，用於數據預處理。
- (2) 可用於降噪過程中的頻譜分析與運算。

4. Matplotlib :

- (1) 繪製波形圖、頻譜圖與其他數據可視化圖表。
- (2) 幫助展示降噪效果，直觀比較音訊處理前後的差異。

(三)Python 實作

1. 實作流程概述

Step1: 收集背景噪聲作為基準

Step2: 將語音進行 STFT（短時傅立葉轉換），轉換到頻域。

Step3: 使用噪聲頻譜減去語音頻譜。

Step4: 應用 IFFT（逆傅立葉轉換）重建降噪後的聲音。



圖 106、Python 實作流程概述

2. 數據集的選擇與處理

為了模擬真實環境與環境的多樣性，可以使用以下公開數據集進行測試：

(1) Mozilla Common Voice (MCV)：

- 提供多樣化的語音樣本，適合語音處理測試。

(2) UrbanSound8K：

- 提供城市噪聲數據，如車輛聲、警笛聲等，用於模擬背景噪聲。

使用 UrbanSound8K 的噪聲樣本與 MCV 的語音樣本進行混合，生成用於測試的含噪語音，並將生成的音訊作為頻譜減法測試的輸入。

3. 收集背景噪聲

背景噪聲的頻譜建模是頻譜減法的關鍵基礎，其準確性直接影響降噪效果。

操作步驟：

Step1: 錄製一段包含背景噪聲的音訊文件，假設前 0.5 秒為純噪聲。

Step2: 使用 librosa.load 讀取音訊文件並提取噪聲片段。

Step3: 將噪聲片段進行頻譜分析，生成噪聲模板。

程式碼示例：

```
6 # 讀取音頻文件
7 input_file = 'mix_noisy.wav'
8 output_file = 'denoise_spectral.wav'
9
10 # 加載音頻
11 y, sr = librosa.load(input_file, sr=None)
```

圖 107、讀取音頻 WAV 文件

4. STFT 頻譜計算

短時傅立葉變換（STFT）是將時域訊號轉換到頻域的工具，用於分析語音與噪聲的頻率成分。

操作步驟：

Step1: 使用 Librosa 的 `stft` 函數計算語音與噪聲的頻譜。

Step2: 分解頻譜為振幅（幅值）與相位，為後續的頻譜操作做準備。

程式碼示例：

```
22 # compute STFT
23 y_stft = librosa.stft(y)
24 magnitude, phase = np.abs(y_stft), np.angle(y_stft)
```

圖 108、librosa.stft 計算 STFT

5. 頻譜減法應用

在頻譜域中，將背景噪聲頻譜從含噪語音頻譜中扣除以完成降噪。

操作步驟：

Step1: 計算背景噪聲頻譜的平均值。

Step2: 使用頻譜減法公式進行扣減，並設置縮放因子 α 調整降噪力度。

程式碼示例：

```
19 # adjust noise reduction factor
20 noise_reduction_factor = 2.5 # alpha
21 mean_noise_spectrum = np.mean(np.abs(noise_stft), axis=1) * noise_reduction_factor
```

圖 109、計算噪聲頻譜

6. IFFT 聲音重建

降噪完成後，通過逆短時傅立葉轉換（ISTFT）將頻譜還原為時域訊號。

操作步驟：

Step1: 將降噪後的振幅與原始相位結合。

Step2: 使用 `Librosa.istft` 函數將頻譜還原為波形訊號，並儲存為音訊文件。

程式碼示例：

```
26 # spectral subtraction
27 threshold_factor = 1.0 # spectral subtraction threshold
28 magnitude_denoised = np.maximum(magnitude - (mean_noise_spectrum[:, np.newaxis] * threshold_factor), 0)
29
30 # restore phase information
31 y_stft_denoised = magnitude_denoised * np.exp(1j * phase)
32
33 # ISTFT to restore audio
34 y_denoised = librosa.istft(y_stft_denoised)
35
36 # save denoised audio
37 sf.write(output_file, y_denoised, sr)
```

圖 110、librosa.istft 計算 ISTFT

(四)降噪前後效果比較與 STOI 評估

1. 可視化比較

使用 Matplotlib 繪製降噪前後的波形與頻譜圖，直觀比較音訊的改善效果。

2. STOI 評估

Short-Time Objective Intelligibility (STOI) 是衡量語音可懂度的重要指標，分數範圍為 0 至 1，分數越高表明可懂度越高。

程式碼示例：

```
23 # ensure both audio files have the same length
24 min_length = min(len(clean), len(denoised))
25 clean = clean[:min_length]
26 denoised = denoised[:min_length]
27
28 # compute STOI score
29 stoi_score = stoi(clean, denoised, sr_clean, extended=False)
30
31 print(f"STOI: {stoi_score:.2f}")
32
```

1] ✓ 5.3s

STOI: 0.99

圖 111、Short-Time Objective Intelligibility (STOI)

小結

本章從基礎開始詳細展示了 Python 實現頻譜減法降噪的完整流程，並通過可視化與 STOI 分數評估了降噪效果。這一過程為嵌入式硬體上的實現提供了理論驗證與參數最佳化依據，為後續實作奠定了堅實基礎。

五、C 語言在 M55M1 開發板上的實作

第五章詳細介紹了如何在 M55M1 開發板上實現頻譜減法降噪，從簡化架構設計到進階架構最佳化，逐步剖析其理論基礎與實現細節。本章強調了 Helium 技術在實現中的核心作用，以及程式模組化設計的優勢。

(一)Helium 技術在降噪中的應用

Helium 是 ARM® 提出的 M-Profile Vector Extension (MVE) 技術，專為加速 DSP 和機器學習設計，具有高效、低功耗的特性。Helium 技術在頻譜減法降噪中的應用主要體現在以下幾方面：

1. 高效傅立葉轉換 (FFT) 計算

- 使用 ARM® DSP 函數庫中的 `Arm®_rfft_fast_f32` 函數，快速完成 STFT 和 IFFT 計算。
- Helium 支援向量化操作，可以同時處理多個數據，大幅提升傅立葉轉換的計算速度。

2. 低功耗運算

- Helium 的架構針對嵌入式設備進行了能耗最佳化，適合在資源受限的 M55M1 開發板上長時間運行。

3. 向量操作的支持

- 頻譜減法中涉及的大量矩陣操作（如頻譜扣減）可以利用 Helium 的 SIMD (Single Instruction Multiple Data) 特性快速處理。

4. 數據存取最佳化

- Helium 提供高效的記憶體管理功能，減少頻繁的數據加載與存取操作，進一步提升處理效率。

(二)頻譜減法架構介紹

5.2.1 頻譜減法架構

在嵌入式系統中，計算資源與儲存空間有限，因此需要設計一個高效、易於實現的簡化版頻譜減法架構。簡化架構的設計目標是在確保降噪效果的同時，最大程度降低計算複雜度。

簡化架構的特點

1. 模組化處理：

- 整個流程被拆分為數個模組，包括音頻輸入、STFT（短時傅立葉轉換）、頻譜減法、IFFT（逆傅立葉轉換）以及音頻輸出，便於開發、測試與調試。

2. 固定參數設置：

- 採用固定的參數配置（如窗口大小、重疊率等），以減少計算資源的消耗，提升實現效率。

3. 專注核心功能：

- 去除過於複雜的降噪算法設計，專注於基於背景噪聲的頻譜減法，便於嵌入式實現。

處理流程

1. 音訊輸入：

- 收集含噪聲的語音數據，並存入緩衝區。

2. STFT 頻譜計算：

- 使用短時傅立葉轉換將時域訊號轉換為頻域訊號。

3. 頻譜減法：

- 以背景噪聲頻譜作為基準，從語音頻譜中扣除噪聲頻譜部分。

4. IFFT 頻譜還原：

- 使用逆傅立葉轉換將頻域訊號轉換回時域訊號。

5. 音訊輸出：

- 將處理後的語音訊號輸出，生成降噪後的語音數據。



圖 112、頻譜減法流程圖

(三)實作頻譜減法

1. 初始化與參數設定

初始化是頻譜減法實作的第一步，包括設置窗口大小、緩衝區與 FFT 處理器。此步驟確保所有資源已分配並準備就緒。

初始化步驟

1. 定義 FFT 窗口大小和重疊率。
2. 初始化 Helium FFT 函數庫。
3. 分配緩衝區，用於儲存語音數據、頻譜數據和結果。

示例程式碼：

```
55 // todo: STFT denoising function
56 void ApplySTFTDenoising(const std::vector<int16_t>& inputAudio, std::vector<int16_t>& outputAudio) {
57     size_t frameSize = 1024; // frame size
58     size_t hopSize = frameSize / 2; // overlap size
59     std::vector<float> frame(frameSize); // store time-domain frame data
60     std::vector<float> fftOutput(frameSize); // store FFT result
61
62     // initial FFT
63     arm_rfft_fast_instance_f32 fftInstance;
64     arm_rfft_fast_init_f32(&fftInstance, frameSize);
```

圖 113、STFT 初始化程式

2. STFT 與頻譜計算

STFT 是將語音訊號從時域轉換到頻域的核心步驟。Helium 技術的快速傅立葉轉換函數能顯著提升這一步驟的運算速度。

實作步驟

1. 將語音訊號分段，窗口大小為 frameSize，重疊率為 hopSize。
2. 對每段訊號加漢明窗(Hamming Window) 進行加權處理，減少頻譜旁瓣效應。
3. 使用 Arm®_rfft_fast_f32 計算頻譜。

示例程式碼：

```
69     for (size_t i = 0; i + frameSize <= inputAudio.size(); i += hopSize) {
70         // 1. convert the current frame of audio data to float
71         for (size_t j = 0; j < frameSize; ++j) {
72             frame[j] = static_cast<float>(inputAudio[i + j]);
73         }
74         // 2. apply Hamming window
75         for (size_t j = 0; j < frameSize; ++j) {
76             frame[j] *= 0.54f - 0.46f * cos(2 * PI_VALUE * j / (frameSize - 1)); // Hamming window
77         }
78         // 3. perform FFT
79         arm_rfft_fast_f32(&fftInstance, frame.data(), fftOutput.data(), 0);
```

圖 114、呼叫 arm_rfft_fast_f32 函式實作 FFT

3. 頻譜減法處理

頻譜減法的目的是從語音頻譜中扣除背景噪聲頻譜，Helium 技術可以顯著最佳化此步驟。

實作步驟

1. 計算背景噪聲的平均頻譜作為模板。
2. 將噪聲頻譜從語音頻譜中扣除，保留有效的語音頻譜。
3. 對扣減後的頻譜進行非負處理，防止數據失真。

示例程式碼：

```
80     // 4. spectral subtraction: subtract the current frame's spectrum from the background spectrum
81     float noiseThreshold = 0.1f; // set a noise threshold
82     for (size_t j = 0; j < fftOutput.size(); ++j) {
83         if (fftOutput[j] < noiseThreshold) {
84             fftOutput[j] = 0; // set the value to zero if it is below the threshold
85         }
86     }
```

圖 115、執行頻譜扣減

4. IFFT 與聲音重建

完成頻譜減法後，使用 IFFT 將頻域訊號轉回時域，並通過重疊相加（overlap-add）生成最終的降噪語音。

實作步驟

1. 使用 `Arm®_rfft_fast_f32` 將頻譜轉回時域訊號。
2. 將還原的訊號存入輸出緩衝區，並進行重疊相加。

示例程式碼：

```
88 // 5. perform inverse FFT (iFFT) to convert the spectrum data back to time domain
89 arm_rfft_fast_f32(&fftInstance, fftOutput.data(), frame.data(), 1);
90 // 6. copy the reconstructed frame data back to the output, perform overlap-add
91 for (size_t j = 0; j < frameSize; ++j) {
92     outputAudio[i + j] += static_cast<int16_t>(frame[j]);
93 }
```

圖 116、呼叫 `arm_rfft_fast_f32` 實作 IFFT

六、關鍵字辨識模組整合

本章描述了如何在 M55M1 平台上實現關鍵字辨識，並通過降噪模組的整合提升辨識準確率。模組化設計使系統能應對不同的環境挑戰，確保穩定的關鍵字檢測效果。

(一)關鍵字辨識模型簡介

通過語音訊號處理技術，實時檢測目標關鍵字並觸發相應操作。

模組流程

1. 實時輸入聲音：
 - 使用麥克風捕捉語音，轉換為訊號數位訊號供後續處理。
2. 存入緩衝區（Buffer）：
 - 將語音按固定長度切分，存入緩衝區以減少延遲。

3. 執行 MFCC（梅爾頻率倒譜係數）：

- 提取語音的關鍵頻率特徵，簡化模型訓練與推論。

4. 關鍵字辨識推論：

- 使用預訓練模型對特徵進行分類，檢測是否包含目標關鍵字。

5. 顯示推論結果：

- 即時將推論結果顯示於終端，並觸發後續應用。



圖 117、關鍵字辨識模型運作流程示意圖

(二) 與降噪模組的整合流程

將降噪模組與關鍵字辨識模組結合，有助於在噪聲環境中提高辨識準確率。

整合流程

1. **背景聲音錄製**：啟動時錄製背景噪聲，生成背景頻譜模板。
2. **降噪模組插入**：在 Buffer 與 MFCC 之間加入降噪模組，確保輸入 MFCC 的訊號已降噪。
3. **降噪後的處理**：對 Buffer 中的數據已經是降噪後的高質量語音訊號。
4. **關鍵字辨識流程**：使用降噪後的語音訊號進行特徵提取與模型推論，提

區塊一：特徵提取與推論：

對實時錄製的音訊分段處理，通過 STFT 降噪提升音訊品質。

步驟解析：

1. **音訊樣本獲取**：
 - (1) 使用 `DMICRecord_AvailSamples()` 確保樣本數足夠。
 - (2) 調用 `DMICRecord_ReadSamples()` 讀取音訊數據存入緩衝區。
2. **降噪處理**：
 - (1) 調用 `ApplySTFTDenoising()` 將數據進行 STFT 降噪。
3. **分段滑動窗處理**：
 - (1) 使用滑動窗對音訊分段，為後續特徵提取做準備。

區塊二：特徵提取與推論

1. 功能與目標：

從降噪後的音訊中提取特徵（如 MFCC），並輸入模型進行推論。

2. 步驟解析：

(1) MFCC 特徵提取：

- 使用 `preProcess` 模組對每段音訊計算 MFCC 特徵，生成模型可用的數據格式。

(2) 推論執行：

- 將提取的特徵輸入模型進行推論，檢測是否包含目標關鍵字。

(3) 結果處理與輸出：

- 如果推論結果顯示為目標關鍵字，終端顯示檢測成功，並觸發相關應用功能。

3. 程式碼示例：

```
584         info("Inference %zu/%zu\n", audioDataSlider.Index() + 1,
585              audioDataSlider.TotalStrides() + 1);
586
587     #if defined(__PROFILE__)
588         u64StartCycle = pmu_get_systick_Count();
589     #endif
590
591     /* Run the pre-processing, inference and post-processing. */
592     if (!preProcess.DoPreProcess(inferenceWindow, arm::app::audio::KwsMFCC::ms_defaultSamplingFreq))
593     {
594         printf_err("Pre-processing failed.");
595         break;
596     }
```

圖 118、呼叫 MFCC 函式

關鍵點說明：

- (1) **MFCC 提取**：MFCC 是關鍵字辨識的重要特徵，通過模擬人類耳朵的聽覺響應來捕捉語音的關鍵信息。

(2) **推論模型**：推論使用的模型為預訓練的小型 CNN 模型。

6.5 整體執行流程總結

流程分步描述

1. **關鍵字偵測啟動**：用戶確認後，系統開始實時音訊錄製。
2. **實時音訊降噪**：將錄製的音訊進行分段，通過 STFT 和頻譜減法去除背景噪聲。
3. **MFCC 特徵提取**：對降噪後的音訊分段提取 MFCC 特徵，作為模型輸入。
4. **關鍵字推論與結果處理**：使用模型檢測特定關鍵字，顯示推論結果並觸發應用功能。

6.6 模組設計總結

關鍵字辨識模組通過結合降噪技術與語音特徵提取，顯著提升了系統的檢測準確率與穩定性。模組化的設計使得整個流程具備良好的可擴展性與適應性。

七、 程式架構與最佳化策略

本章系統性分析了程式的整體架構，從背景噪聲處理到關鍵字辨識的完整流程展開，並針對每個模組的角色與技術實現進行深入解說。最後，提出程式的最佳化與調試策略，旨在進一步提升系統效能和穩定性。

(一) 程式整體架構設計

1. 架構概述

本程式圍繞三大核心功能展開：背景噪聲處理、實時音訊降噪與關鍵字辨識。系統通過模組化設計，實現了流程的高效協同。以下為整體架構的主要階段：

1. **背景噪聲處理**：錄製並分析背景噪聲，生成頻譜模板。
2. **實時音訊降噪**：對實時錄製的音訊進行降噪處理，提升語音品質。
3. **關鍵字辨識**：通過提取語音特徵並輸入深度學習模型，檢測關鍵字。

2. 架構優勢

- **模組化設計**：功能拆分為獨立模組，便於開發與調試。
- **實時性保障**：基於 ARM® Helium 技術的加速處理，實現低延遲音訊處理。
- **高適應性**：能適應多種噪聲環境與語音場景。

(二)各模組的角色與實現細節

1. 實時音訊處理與降噪模組

- (1) 角色：處理實時音訊並消除背景噪聲。
- (2) 技術實現：
 - a. 應用滑動窗技術分段處理音訊，確保數據連續性。
 - b. 使用頻譜減法算法，結合 Helium 技術最佳化 FFT 運算。
- (3) 優勢：提升語音信噪比，降低噪聲對關鍵字辨識的干擾。

2. 關鍵字辨識模組

- (1) 角色：提取語音特徵並進行推論，檢測目標關鍵字。
- (2) 技術實現：
 - a. 使用 MFCC 提取語音的核心特徵。
 - b. 通過預訓練深度學習模型（如 DS-CNN）進行推論。
- (3) 優勢：準確率高，適合嵌入式環境的實時語音識別。

3. UART 交互模組

(1) **角色**：與用戶進行交互，控制背景噪聲錄製與關鍵字偵測的啟動。

(2) **技術實現**：

- 基於 UART 介面實現用戶命令接收與反饋。
- 提供輸入錯誤檢測，提升系統穩定性。

(3) **優勢**：操作簡單，適合嵌入式設備的人機交互需求。

(三)最佳化與調試策略

1. 降噪模組的最佳化

- (1) 動態更新背景頻譜：定期錄製背景噪聲，應對環境變化。
- (2) 頻譜平滑處理：使用平滑算法減少頻率突變對語音的影響。

7.3.2 提高關鍵字辨識準確率

- (1) 改進特徵提取：調整 MFCC 的參數，提取更精確的特徵。
- (2) 模型訓練數據增強：增加不同噪聲背景和語速樣本，提升模型泛化能力。

7.3.3 系統效能調整

- (1) 運算效率提升：
 - a. 使用 Helium 技術最佳化 FFT、矩陣計算等高頻操作。
 - b. 減少記憶體拷貝操作，更多使用指針與引用。
- (2) 時間與資源管理：記錄各模組的執行時間，定位性能瓶頸。

(四) 系統運行的邏輯流程

1. 初始化

系統啟動後，提示用戶錄製背景噪聲，計算並儲存頻譜模板。

2. 實時音訊處理

錄製實時音訊，通過滑動窗進行分段處理，並結合頻譜減法進行降噪。

3. 關鍵字辨識

將降噪後的音訊特徵輸入模型推論，檢測是否包含目標關鍵字。

4. 結果輸出

若檢測到目標關鍵字，終端顯示成功結果並觸發應用功能。

(五) 未來擴展方向

1. **進一步最佳化模型**：採用更高效的模型架構（如 MobileNet）以平衡性能與準確性。
2. **支持更多語音命令**：增加模型的識別範圍，支持多語言和多指令場景。
3. **模組功能擴展**：增加噪聲分類功能，進一步提高降噪模組的適應性。

結語

本系統結合了背景噪聲錄製、實時音訊處理、關鍵字偵測與降噪等核心技術，充分發揮了嵌入式系統在語音處理應用中的潛力。透過 Helium 指令集的最佳化，系統實現了高效的 STFT 和頻譜減法處理，並結合 MFCC 特徵提取和機器學習模型，成功實現了高準確度的關鍵字識別。

在設計過程中，我們強調了系統的模組化結構和邏輯清晰的流程，確保了程式的可讀性與維護性。同時，通過對降噪模組和推論準確性的調優，系統能夠適應各種不同的環境噪聲，提升了應用的實用性和穩定性。

展望未來，此架構可以進一步結合更先進的降噪演算法（如深度學習降噪技術）與更加高效的推論模型，拓展至更多語音相關應用場景，如語音助理、智慧家居控制或遠場語音識別等。最終，這樣的設計為嵌入式系統在語音智慧應用領域樹立了典範，也為後續研究提供了穩固的基礎。

參考資料

1. STFT MathWork

<https://ww2.mathworks.cn/help/signal/ref/stft.html>

肆、智慧醫療 1 – 藥物辨識

一、 案例介紹 – 藥物辨識

背景:在製藥工廠或醫療場景中，藥物辨識是一個關鍵需求。

(一) 準確性與效率：

1. 藥物辨識直接關係到患者的健康與安全，尤其是在大批量生產的情況下，傳統方法已無法滿足需求。
2. 利用智慧化工具進行自動辨識，可顯著降低人工錯誤，提升效率。

(二) 傳統方法的挑戰

1. 手工辨識不僅耗時，還容易受到疲勞或環境干擾的影響，導致出錯。
2. 傳統方法無法快速處理大規模藥物資料，亟需嵌入式智慧解決方案來應對。

(三) 使用場景舉例

1. 醫療應用:

- (1) 提供醫務人員和患者即時辨識藥物的功能，避免因錯誤用藥導致健康風險。
- (2) 支援患者自助查詢系統，通過掃描藥物即時顯示詳細信息，如劑量和注意事項。

2. 製藥工廠:

- (1) 在生產過程中，對不同藥物進行即時標識和分揀，避免混料。
- (2) 通過集成到生產線的機器學習系統，監控生產流程，減少人工干預和錯誤率。

(四) 進一步拓展

1. 智慧藥物辨識系統的潛力不限於辨識本身，還可以結合大數據分析，建立一個可追溯的藥物流通管理平台。

未來，可引入語音輔助功能，協助視障人群辨識藥物，提升藥物使用的普適性和便利性。

(1) 支持多樣化辨識：

系統能夠支持複數、多種與不同類型的藥物外觀辨識，包括膠囊、片劑和不同顏色、形狀的藥物。

(2) 高精度與高效率：

- a 辨識準確率達到 95%，滿足醫療與製藥行業的高標準要求。
- b 系統具備快速處理能力，能即時響應生產和醫療場景中的藥物辨識需求。

(3) 嵌入式硬體支持：

使用 NuMicro 微控制器進行處理，實現高效能和可靠性的藥物辨識。

- a 該硬體平台具備低功耗和高效能，適合醫療和工業領域的嵌入式應用。

(五) 示範與效果

1. 測試拍攝物件：

系統能夠準確識別並標註拍攝的膠囊與藥丸，展示其高準確率和可靠性。

2. 預期開發版 LCD 顯示結果：

在嵌入式平台上，通過 LCD 實時顯示藥物辨識結果，標註每個藥物名稱及類型，便於用戶理解和確認。

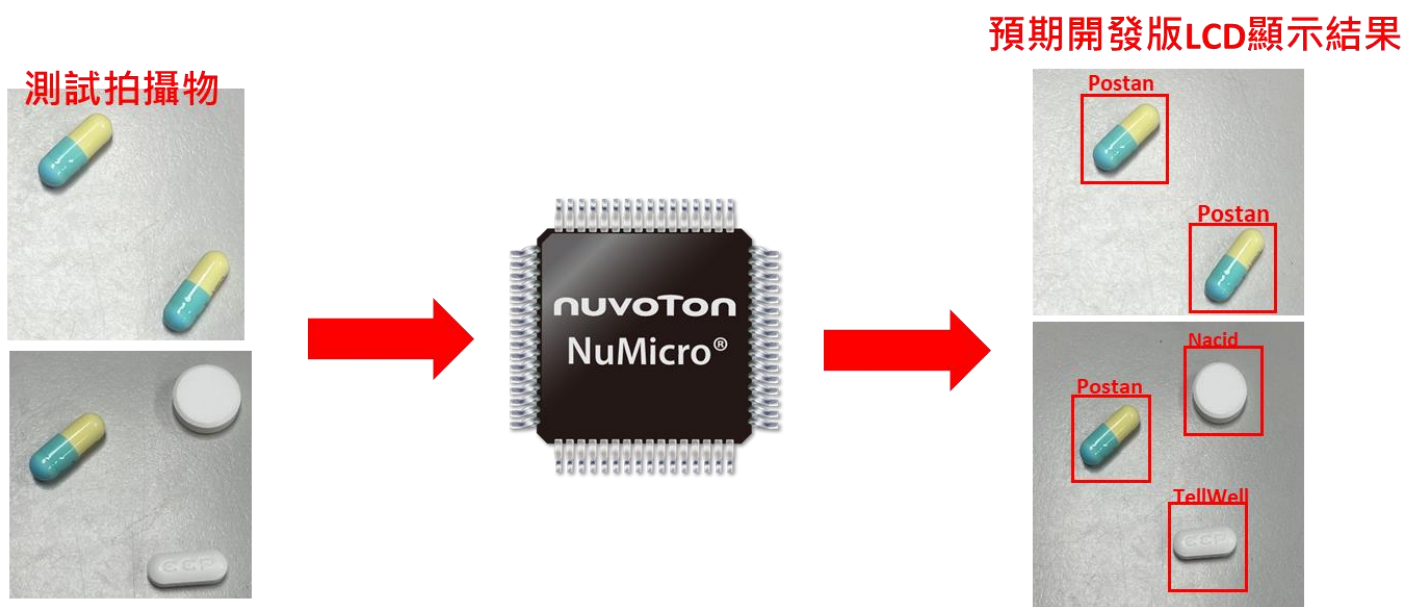


圖 119、藥物辨識顯示結果

(六) 資料集與模型訓練過程說明

1. 工具選擇：

- (1) LabelImg: 用於標註圖像數據的開源工具，可輕鬆繪製邊界框並生成標籤文件。
- (2) Roboflow: 提供在線標註和數據增強功能，支持多種格式的數據集導出，方便不同模型的訓練。

2. 數據增強：

- (3) 在數據集中加入旋轉、裁剪、顏色變換等技術，增加數據多樣性，提高模型的泛化能力。

3. 模型訓練

(1) 模型選擇：

- a 使用 YOLOX Nano 作為基礎模型，這是一款專為嵌入式應用最佳化的輕量級目標檢測模型。
- b 提供良好的速度與準確度平衡，適合運行於資源受限的嵌入式設備。

(2) 訓練過程：

- c 使用訓練數據進行模型微調（Fine-tuning），適配具體的藥物辨識場景。
 - d 在訓練中加入數據增強與正則化技術，以減少過擬合風險。
-

4. 模型框架轉換

(1) 從 Pytorch 到 TensorFlow Lite:

- a 使用 PyTorch 訓練模型後，通過 ONNX 格式進行中間轉換。
- b 將模型轉換為 TensorFlow Lite 格式，最佳化性能並適配嵌入式設備。

(2) 最佳化與測試:

- a 使用 TensorFlow Lite 的最佳化工具（如量化和權重修剪）進一步減少模型大小，提高運行效率。
- b 在嵌入式開發板上進行測試，驗證模型的準確性和執行速度。

二、 使用資料集與模型介紹

(一) 練框架與模型

1. 訓練框架：

使用 PyTorch 作為訓練框架，憑藉其靈活性和強大的模型開發能力，能夠高效進行深度學習模型的訓練和微調。

2. 訓練模型：

基於 YOLOX-Nano 模型進行訓練，這是一款專為嵌入式設備設計的輕量級目標檢測模型，具備高效能和低資源消耗的特點。

(二) 資料集準備與標註方式

1. 資料集來源：

成大醫院藥物資料集，包含多種膠囊與片劑的圖像，滿足多樣性需求。

2. 標註方式：

使用 LabelImg 工具進行標註，生成 COCO JSON 格式的標籤文件，便於模型訓練的數據讀取和處理。

3. 數據集劃分：

(1) 訓練集 (Train Set)：1500 張圖像

(2) 驗證集 (Valid Set)：300 張圖像

三、 訓練環境與硬體介紹

(一) 硬體設備：

1. 使用 NVIDIA RTX 3060 Ti 顯卡，提供高效的 GPU 運算能力。
2. 支援 CUDA 技術，能夠充分利用 GPU 加速深度學習模型的訓練。

(二) CUDA 和 PyTorch 版本：

1. CUDA 版本： 11.8
2. PyTorch 版本： 2.4.1

(三) 訓練數據：

1. 訓練數據總計包含 1800 張圖像，涵蓋多種藥物的外觀特徵。

(四) 訓練參數設置：

1. Epoch：200
2. Batch size：32

(五) 訓練時間：

1. 平均每次完整訓練耗時約 1.5 小時。

四、模型訓練

(一) 模型訓練流程

1. 環境建立：

準備必要的軟硬體環境，包括訓練框架（PyTorch、TensorFlow）和硬體（如 GPU）。

2. 資料集準備：

使用標註好的資料集，確保模型訓練的數據來源準確且多樣。

3. TinyML Training：

基於 YOLOX Nano 模型進行訓練，採用 PyTorch 框架作為基礎，並生成訓練模型。

4. 模型輕量化：

利用 TensorFlow Lite 進行模型轉換，包括 INT8 量化，生成輕量化模型，適配嵌入式環境的需求。

5. Vela 編譯：

使用 Vela 編譯器進一步最佳化輕量化模型，生成 TFL flatbuffer 文件，以提高模型執行效能。

(二) 模型訓練流程圖說明

1. Step 1：YOLOX Nano 訓練

(1) 資料集經過 PyTorch 框架以及預訓練模型訓練，生成目標模型。

(2) PyTorch 至 TensorFlow Lite 轉換

2. Step 2：輕量化

(1) 使用 TensorFlow Lite 進行 INT8 量化轉換，生成輕量化模型。

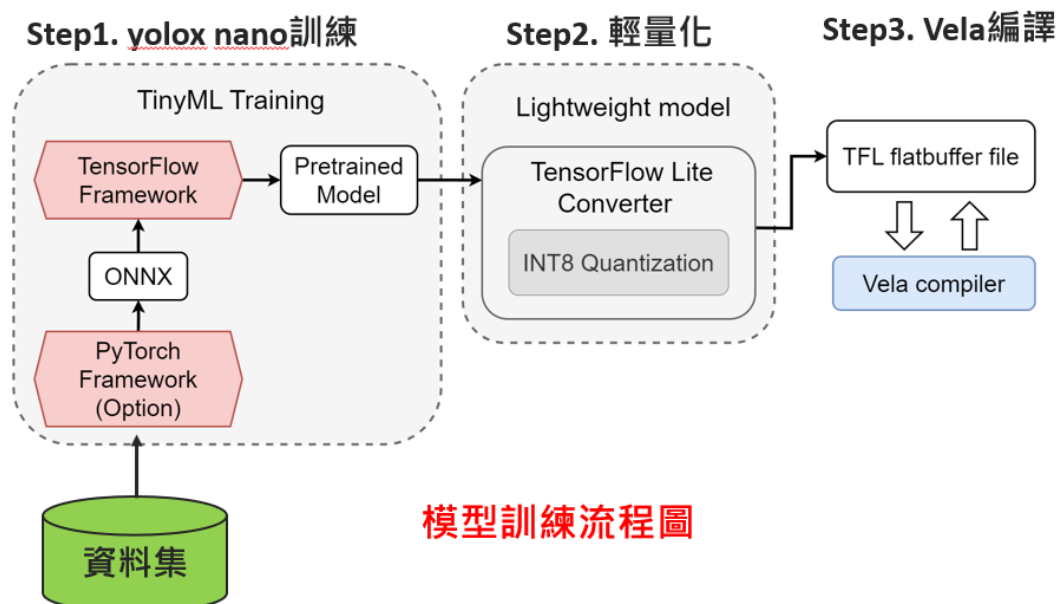


圖 120、模型訓練流程圖

3. Step 3：Vela 編譯

- (1) 將模型進行 Vela 編譯，生成適合嵌入式設備的 TFL flatbuffer 文件。

(三) 開始訓練詳細步驟:

1. 建立 Python 環境：

- (1) 執行以下指令以建立名為 yolox_nu 且使用 Python 3.10 的環境：

```
$conda create --name yolox_nu python=3.10
```

- (2) 激活建立的環境：

```
$conda activate yolox_nu
```

- (3) 此步驟會在 Anaconda 上建立專案所需的隔離環境。

2. 升級 pip：

執行以下指令升級 pip 和相關套件：

```
$python -m pip install --upgrade pip setuptools
```

3. 安裝 PyTorch：

- (1) 根據系統類型選擇適合的 CUDA 版本與 PyTorch 版本安裝，以下範例為 CUDA 11.8：

```
$python -m pip install torch torchvision torchaudio --index-url  
https://download.pytorch.org/whl/cu118
```

- (2) 請確認所使用的 CUDA 版本（如 cu118）與您的硬體相符。

4. 安裝 MMCV：

根據您的硬體配置選擇適當的版本，以下為 CUDA 11.8 的安裝指令：

```
$python -m pip install mmcv==2.0.1 -f  
https://download.openmmlab.com/mmcv/dist/cu118/torch2.0/index.html
```

5. 安裝其他需求套件：

開啟已安裝好的檔案目錄，並執行以下指令以安裝需求套件：

```
$python -m pip install --no-input -r requirements.txt
```

6. 安裝 YOLOX：

在 YOLOX 的專案目錄下執行以下指令以完成安裝：

```
python setup.py develop
```

(四) Yoloxnano 模型的資料集整理

1. 使用 COCO JSON 格式：

Yoloxnano 資料集須符合 COCO 格式，包含訓練和驗證的標註文件。

2. 準備好訓練資料並整理成以下目錄結構：

(1) 確保資料按如下格式存放於 Datasets 目錄中：

```
Datasets/<your_datasets_name>/
├── annotations/
│   ├── train_annotation_json_file
│   └── val_annotation_json_file
├── train2017/
│   └── train_img
├── val2017/
│   └── validation_img
```

(2) annotations/ 目錄存放訓練和驗證的 JSON 標註文件。

(3) train2017/ 目錄存放訓練圖片。

(4) val2017/ 目錄存放驗證圖片。

(五) Yoloxnano 模型訓練步驟

1. 設定模型訓練檔路徑：

編輯 `exps/default/yolox_nano_ti_lite_nu.py` 設定檔。

2. 更新設定檔內部檔案路徑：

指定資料集的相關路徑：

```
self.data_dir = "datasets/your_coco"
```

```
self.train_ann = "your_train.json"
```

```
self.val_ann = "your_val.json"
```

3. 預訓練模型路徑

使用下列預訓練模型路徑作為基礎：

```
$retrain/tflite_yolox_nano_ti/320_DW/yolox_nano_320_DW_ti_lite.pth
```

4. 開始訓練

執行以下指令以開始訓練：

```
$ python tools/train.py -f <MODEL_CONFIG_FILE> -d 1 -b <BATCH_SIZE> --fp16 -o  
-c <PRETRAIN_MODEL_PATH>
```

(1) **<MODEL_CONFIG_FILE>**：指定模型設定檔路徑，例如
`yolox_nano_ti_lite_nu.py`。

(2) **<BATCH_SIZE>**：設定每批訓練數據的大小，例如 32。

(3) **<PRETRAIN_MODEL_PATH>**：指定預訓練模型的完整路徑。

(六) 模型轉換

1. Pytorch to ONNX：

(1) 使用以下指令將 PyTorch 模型轉換為 ONNX 格式：

```
$ python tools/export_onnx.py -f <MODEL_CONFIG_FILE> -c  
<TRAINED_PYTORCH_MODEL> --output-name  
<ONNX_MODEL_PATH>
```

(2) 參數說明：

- a <MODEL_CONFIG_FILE>：模型的設定檔路徑，例如 yolox_nano_ti_lite_nu.py。
- b <TRAINED_PYTORCH_MODEL>：訓練完成的 PyTorch 模型路徑。
- c <ONNX_MODEL_PATH>：輸出的 ONNX 模型存放路徑。

2. Create Calibration Data：

(1) 創建校準數據以支持量化：

```
$ python demo/TFLite/generate_calib_data.py --img-size <IMG_SIZE> --  
n-img <NUMBER_IMG_FOR_CALI> -o <CALI_DATA_NPY_FILE> --  
img-dir <PATH_OF_TRAIN_IMAGE_DIR>
```

(2) 參數說明：

- a <IMG_SIZE>：輸入圖像的大小（例如 320）。
- b <NUMBER_IMG_FOR_CALI>：用於校準的圖像數量。
- c <CALI_DATA_NPY_FILE>：校準數據的輸出文件路徑。
- d <PATH_OF_TRAIN_IMAGE_DIR>：用於生成校準數據的圖像目錄。

3. Convert ONNX to Tflite：

(3) 使用以下指令將 ONNX 模型轉換為 TensorFlow Lite 格式：

```
& nx2tf -i <ONNX_MODEL_PATH> -oiqt -qcind images<CALI_DATA_NPY_FILE>  
"[[[[0,0,0]]]]"
```

(4) 參數說明：

- a <ONNX_MODEL_PATH>：ONNX 模型的存放路徑。
- b <CALI_DATA_NPY_FILE>：校準數據文件的路徑。

(七) Vela 編譯

1. 準備輕量化模型：

- (1) 將已經完成量化的模型放置到目錄 vela\generated\ 中。

2. 修改變量文件：

- (1) 打開 vela 目錄下的 variables.bat 文件，並進行如下設置：

```
set MODEL_SRC_FILE=<your tflite model>  
set MODEL_OPTIMISE_FILE=<output vela model>
```

3. 執行編譯命令：

- (1) 執行 gen_modle_cpp 檔案進行模型編譯。
- (2) 編譯結果將輸出到以下路徑：

```
vela\generated\yolox_nano_ti_lite_nu_full_integer_quant_vela.tflite.cc
```

五、 C++ 軟體實現

(一) C++ 軟體功能設計

1. C++ Sample Code 簡介：

提供基於硬體平台的範例程式碼，展示模型推理流程。

2. C++ 軟體流程圖：

描述從數據加載到模型推理的完整流程。

3. C++ 軟體設計介紹：

詳述程序模組分工，包括數據處理模組、模型加載模組和推理模組。

(二) 系統流程圖

1. 模型訓練（藍色部分）

(1) TinyML Training

- a 使用 PyTorch 框架進行訓練，生成初始模型。
- b 模型轉換為 ONNX 格式後，使用 TensorFlow Lite 進行量化（如 INT8 量化），生成輕量級模型。

(2) Vela 編譯

- a 通過 Vela 工具對 TFLite 模型進行最佳化，生成適用於嵌入式設備的編譯文件（TFL flatbuffer file）。
-

2. 硬體與軟體實現（紅色部分）

(1) C/C++ 軟體實現

- a 開發基於硬體平台的 C/C++ 軟體，用於資源配置與模型部署。

(2) 硬體開發板燒錄

- a 使用硬體開發板（如 NuMicro® M55M1），整合硬體加速器（如 Ethos™-U55 microNPU）和 CMSIS-NN 最佳化內核，執行 TFLite Runtime。

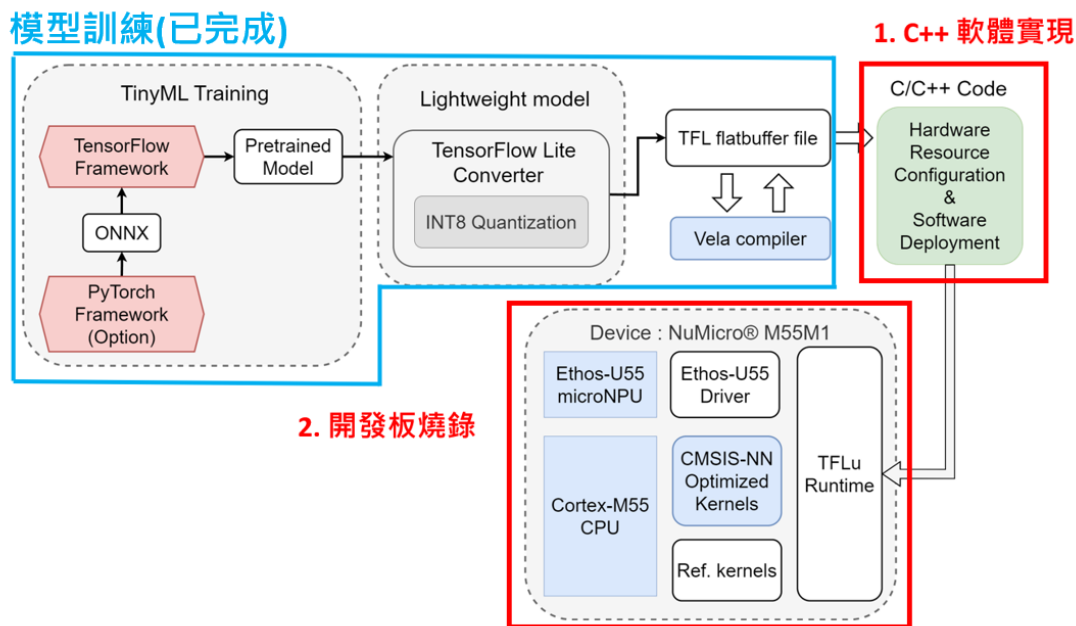


圖 121、系統整體流程圖

(三) 確認開發版硬體需求

1. LCD 模組：

- (1) 用於顯示使用者介面，提供藥物辨識結果的可視化。
- (2) 配置與 EBI（External Bus Interface）的通訊以實現數據傳輸。
- (3) 支援多種解析度設定以適配不同的應用場景。

2. 影像感測器（Image Sensor）：

- (1) 負責捕捉藥物的外觀圖像，用於辨識與分類。
- (2) 設置解析度以保證辨識準確性。
- (3) 配置與 I2C（Inter-Integrated Circuit）的通訊，用於圖像數據傳輸與模組配置。

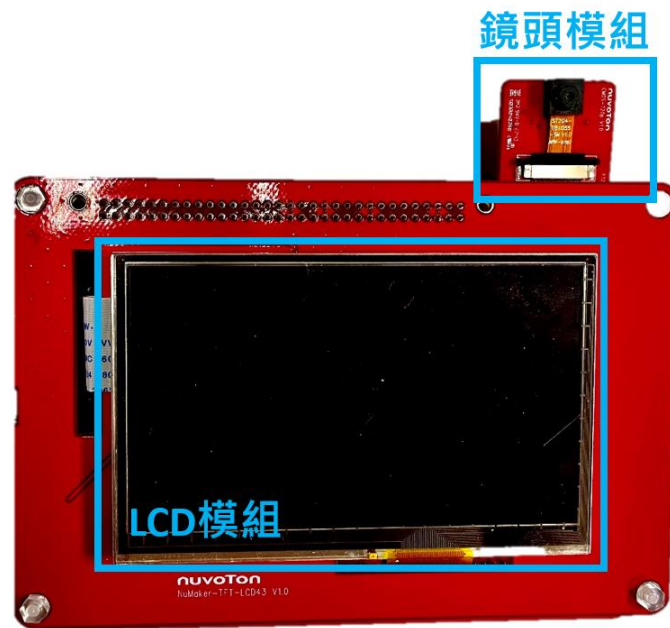


圖 122、開發板使用模組

(四) C++ 軟體執行流程

1. 硬體、模型與 Framebuffer 初始化設置：

初始化硬體資源以確保執行環境準備就緒。

- (1) 配置模型加載和推理所需的資源。
- (2) 初始化 Framebuffer 以便管理影像數據流。

2. 設計 Framebuffer 狀態對應的三種功能：

- (1) 捕捉影像：使用鏡頭模組捕捉藥物影像，並存入 Framebuffer。

- (2) 影像前處理與推理： 將影像數據送入 YOLOXnano 模型進行推理，獲取辨識結果。
 - (3) 模型結果後處理： 將推理結果進行格式化，並在開發板 LCD 上顯示辨識結果。
-

(五) 開發板內部軟體流程

1. 初始化：

- (1) 硬體模組初始化
- (2) 模型初始化
- (3) Framebuffer 初始化

2. 主要執行過程：

- (1) 影像捕捉：若 Framebuffer 為空，捕捉影像進行儲存。
- (2) 模型推理：當 Framebuffer 為 FULL 時將影像傳入 YOLOXnano 進行處理，得到推理結果。
- (3) 後處理與顯示：格式化結果，並在 LCD 上顯示。

3. 重複循環：

在主程序中，重置 Framebuffer 並重複執行流程。

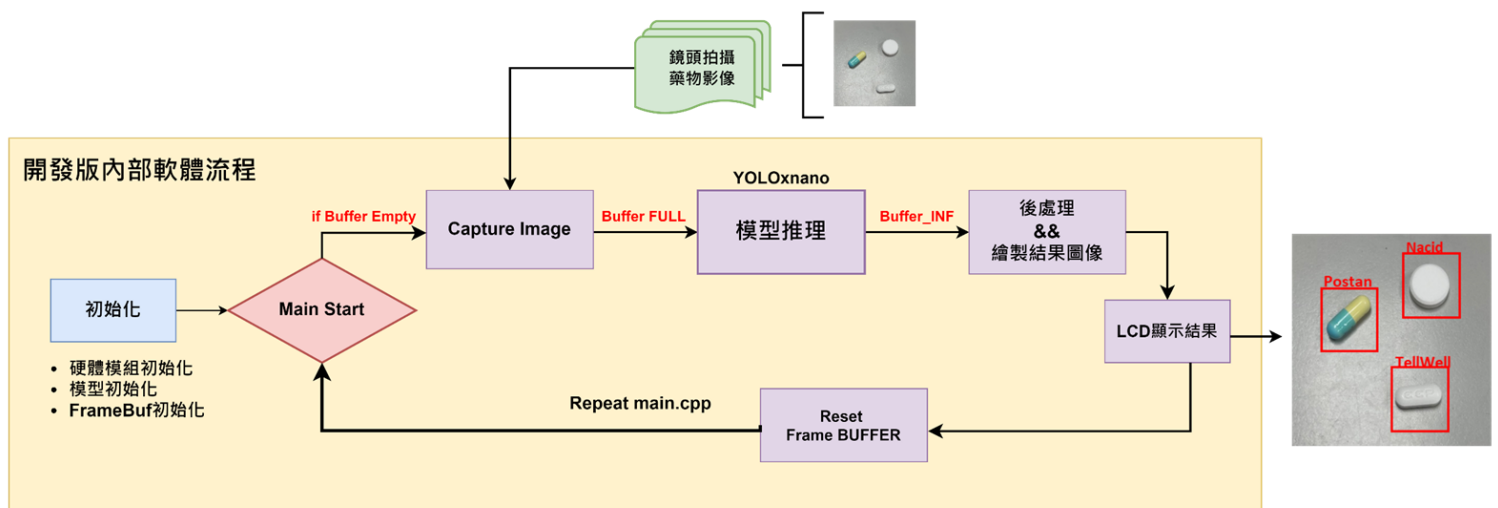


圖 123、開發板內部軟體流程圖

(六) 影像處理 function 介紹

1. 鏡頭模組設置 (Image Sensor)

(1) 設置步驟：

在 main.cpp 中呼叫 ImageSensor.h，可以使用該頭文件內的功能函數。

2. ImageSensor 功能介紹：

函數：int ImageSensor_Init()

ImageSensor_Init()函數：初始化圖像感測器，設置感測器的頻率和時序。

3. ImageSensor.h 函數定義

(1) ImageSensor_Init(void)：初始化影像感測器。

(2) ImageSensor_Capture()：啟動影像捕捉並將數據保存到指定的緩衝區。

(3) ImageSensor_Config()：配置影像的格式、寬度和高度。

(4) ImageSensor_TriggerCapture()：觸發捕捉影像操作。

(5) ImageSensor_WaitCaptureDone()：等待捕捉完成。

4. ImageSensor - 捕捉影像 (Capture Image)

(1) ImageSensor_Capture()

啟動影像捕捉，將捕捉到的影像存入指定的記憶體地址。

(2) ImageSensor_TriggerCapture()

用於觸發圖像捕捉，並設置快門訊號。

(3) ImageSensor_WaitCaptureDone()

等待影像捕捉完成，並處理超時情況。

(七) 輸入影像處理

1. Main.cpp - framebuffer (暫緩區)

- (1) `main_task()` 函數的主循環中，程式不斷地從相機獲取新幀。
- (2) 使用 `Framebuffer` 暫存相機捕捉的影像幀，然後將其送入 `YOLOX_nano` 模型進行推理。
- (3) `S_FRAMEBUF` 用於管理影像數據與檢測結果。

2. **Framebuffer** 功能介紹

(1) `eFRAMEBUF_EMPTY` (空緩衝區):

- a 當 `Framebuffer` 處於空狀態時，主循環會尋找空緩衝區來填充新的影像數據。
- b 如程式中所示，`get_empty_framebuf()` 函數會返回一個空的緩衝區，隨後進行以下操作：
 - (a) 如果使用影像感測器 (CCAP)，會觸發 `ImageSensor_Capture()` 捕獲新的影像數據，並存入緩衝區。
 - (b) 如果是靜態影像，則從記憶體中複製影像數據到緩衝區。
- c 最後，緩衝區狀態會被更新為 `eFRAMEBUF_FULL`，以準備進入後續處理。

(2) `eFRAMEBUF_FULL` (已填充的緩衝區):

- a 當 `Framebuffer` 處於已填充狀態時，主循環會檢查並處理這些緩衝區。
- b 如程式中所示，`get_full_framebuf()` 函數會返回一個已填充的緩衝區，隨後進行以下操作：
 - (a) 影像會被縮放到模型輸入所需的大小 (例如使用 `imlib_nvt_scale()` 函數)。

- (b) 如果模型需要進行量化處理，則將影像數據量化為整數格式。
- (c) 將數據送入推論隊列 (Inference Process Queue)，並將緩衝區狀態更新為 `eFRAMEBUF_INF`。

(3) `eFRAMEBUF_INF` (推論中緩衝區):

- a 當 `Framebuf` 處於推論狀態時，主循環會處理推論完成的結果。
- b 如程式中所示，`get_inf_framebuf()` 函數會返回一個推論中的緩衝區，隨後進行以下操作：
 - (a) 將推論結果進行後處理 (例如使用 `DetectorPostprocessing`)，將檢測框繪製在影像上。
 - (b) 如果有顯示設備 (`DISPLAY`)，會將結果顯示在螢幕上。
 - (c) 最後，推論完成的緩衝區狀態會被重置為 `eFRAMEBUF_EMPTY`，以準備下一輪處理。

六、 C++ 軟體設計介紹 - 模型推理

(一) 推理任務創建：

1. 使用 FreeRTOS 的 xTaskCreate 函數

- (1) 創建推理任務，負責處理推理所需的影像數據。
- (2) 任務會根據狀態 (eFRAMEBUF_FULL) 從 Framebuf 中獲取完整影像數據，並傳遞至推理流程。

(二) 模型推理流程：

1. 執行條件：

- (1) 當 Framebuf 狀態為 eFRAMEBUF_FULL 時，表示影像數據已準備完成，可以開始推理。

2. 推理步驟：

(1) 設置推理任務的參數：

- a 設置推理的輸出緩衝區 (responseQueue) 和後處理函數 (PostProcess)。
- b 傳遞影像的行數與列數 (inputImgCols 和 inputImgRows)。
- c 將來源影像數據 (srcImgWidth 和 srcImgHeight) 與完整影像緩衝區綁定。

(2) 將推理任務添加到隊列：

- a 通過 xQueueSend 將設置完成的 inferenceJob 傳遞給推理處理隊列。

- b 更新 Framebuf 狀態為 eFRAMEBUF_INF，表示該幀影像已進入推理階段。

3. 推理完成後：

- (1) 緩衝區的狀態會被更新為新的值（例如處理完成後會進一步重置或進行下一階段處理）

4. 推理完成標記：

- (1) 緩衝區更新狀態為 eFRAMEBUF_INF，標記推理結果已就緒。

5. 數據結構傳遞：

- (1) 推理模塊的輸出結果被存入緩衝區的 results 區域，等待後處理模塊調用。

6. 後處理觸發：

- (1) 後處理模塊根據 eFRAMEBUF_INF 狀態，自動調用相應的處理函數，執行如偵測框縮放和篩選操作。

(三) 後處理與繪製流程

在完成模型推理後，需要對輸出的檢測框進行處理，轉換回原圖大小，並篩選出有效的物體檢測結果。主要步驟如下：

1. 縮放偵測框

(1) 功能：

- (a) 將推理網路輸出的檢測框尺寸（imgNetRows, imgNetCols）按比例放大，回轉至原始影像的大小（imgSrcRows, imgSrcCols）。

(b) 確保偵測框正確對應到原圖像的位置。

2. 提取偵測結果

(1) 功能：

從模型的輸出張量中解析出每個物體的檢測框（Bounding Box）、物體類別（Class ID）以及對應的信心度分數（Confidence Score）。

(2) 注意點：

排除低於設定閾值的檢測結果，例如僅保留信心度高於 0.5 的框：

3. 執行 NMS（Non-Maximum Suppression）

(1) 功能：

- a 應用非極大值抑制（NMS），移除高度重疊的檢測框，僅保留具有最高信心度的框。
- b 此步驟有效避免多次檢測同一目標，提升檢測結果的準確性。

(2) 算法步驟：

- a 將檢測框按信心度降序排列。
- b 比較重疊率（IOU, Intersection Over Union），若超過指定閾值（如 0.5），移除重疊的框。
- c 保留其餘檢測框作為最終結果。

4. 結果輸出

(1) 功能：

- a 將最終的檢測結果（包括檢測框位置和物體類別）儲存到輸出向量（resultsOut）中，以便後續繪製和展示。

(2) 應用場景：

- a 將結果作為後處理顯示的一部分，顯示於開發板 LCD 或進一步傳遞至其他處理模塊。

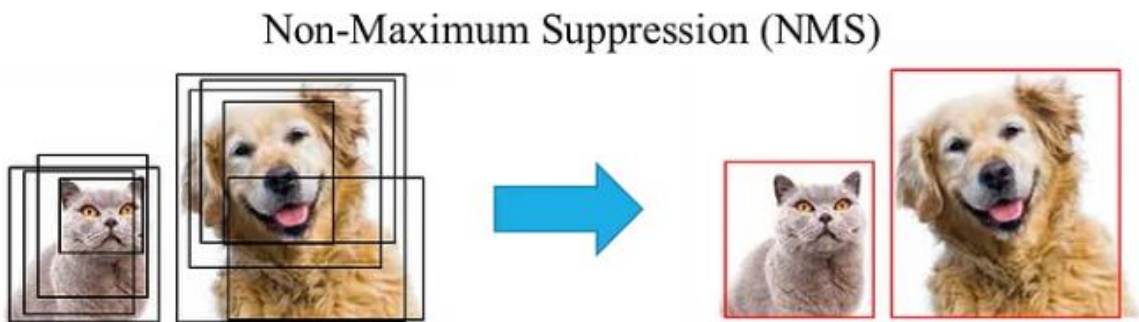


圖 124、NMS 結果

(四) 設定 Label.cpp 檔

甲、新增標籤數據：

- (1) 在 Label.cpp 中，透過 labelVec[] 這個陣列來定義模型中需要辨識的所有類別名稱。
- (2) 根據模型所訓練的資料集，需將所有類別名稱（例如：TellWell、Postan 和 Nacid）依次列入陣列內，確保模型推理結果能正確對應到實際的物體名稱。

2. 標籤命名規則：

- (1) 每個類別名稱應清晰且準確，以對應到訓練數據的實際標籤。
- (2) 類別名稱應避免重複或有歧義，因為這可能會導致模型結果混淆。

3. 功能描述：

- (1) labelVec[] 陣列中的值，將在模型推理完成後用於結果對應。每個檢測到的物件，其類別編號會與陣列中的名稱相匹配，這樣才能在輸出結果中顯示正確的物件標籤。

(五) 實現範例：

```
static const char *labelsVec[] LABELS_ATTRIBUTE = {  
    "TellWell",  
    "Postan",  
    "Nacid",  
};
```

1. 這段程式碼定義了一個靜態常數字符陣列，包含了模型可能檢測到的所有標籤名稱。
2. LABELS_ATTRIBUTE 是一個屬性標記，可能用於編譯器最佳化或特殊記憶體對齊。

(六) 注意事項：

1. 若新增或修改了模型的訓練數據集，記得同步更新這個陣列，否則模型輸出的結果可能無法正確對應到類別名稱。
2. 更新後的 Label.cpp 檔案需重新編譯，確保新添加的標籤在程式運行時生效。

完成上述步驟就能根據前面的燒錄教學完成後續步驟

參考資料:

Yoloxnano 訓練: https://github.com/MaxCYCHEN/yolox-tf-lite_tflite_int8

COCO 資料集: <https://cocodataset.org/#home>

CUDA TOOLKIT : <https://developer.nvidia.com/cuda-toolkit>

OPENCV : <https://opencv.org/>

NMS : <https://ieeexplore.ieee.org/document/8100168>

伍、智慧醫療 2 - 跌倒偵測

一、 案例介紹 - 跌倒偵測

(一) 專案摘要

本系統由新唐 M55M1 開發板判斷人的 3 種姿勢 Fall-down(圖 113)、sitting(圖 114)、standing(圖 115)，資料集來源為 Roboflow 取得，在 Pytorch 框架訓練 YoloX-nano 模型，後續經過模型框架轉換和輕量化後轉成 Tensorflow-lite 框架，再經過 Vela 編譯處理即可部署至開發板上，最後將 C 語言軟體和模型經由 KEIL 燒錄至開發板。使用機器學習的方式判斷偵測範圍內是否有人跌倒，不僅能夠減少醫療人力需求且能更快偵測到緊急情況。



圖 125、開發板成功偵測 Fall-down 狀態成果圖



圖 126、開發板成功偵測 sitting 狀態成果圖

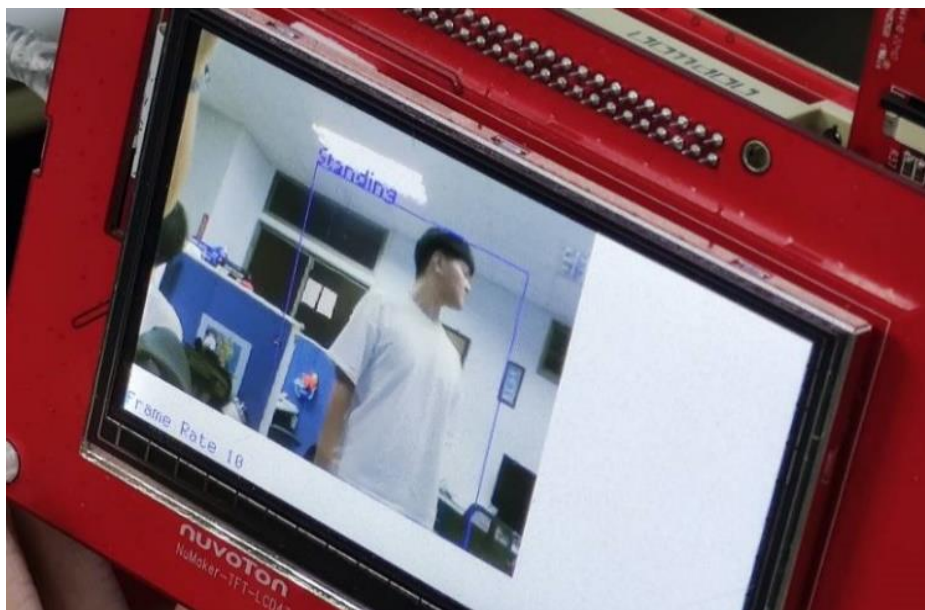


圖 127、開發板成功偵測 standing 狀態成果圖

二、 跌倒偵測技術種類介紹

機器學習跌倒偵測模型的方法分為 2 種 Keypoint-detection 和 Object-detection，本次報告會分別介紹兩者的做法以及解釋為何採用 Object-detection 的方式。

Keypoint-detection 的作法是將圖片中人的軀幹和肢體狀態做標註(如圖 116)，對姿勢的辨識準確率更高，但標註過程更為複雜且需要更大量的資料集才能準確判斷，適用於辨識姿勢種類變化差異小的應用(如辨識坐姿好壞)。



圖 128、使用 Keypoint-detection 作法標註示意圖

Object-detection 則是和一般物件偵測原理相同，標記過程只需框出圖片中的目標並標記姿勢種類即可(如圖 117)，相對較簡單，且以本次應用的 3 種姿勢(Fall-detected、sitting、standing)變化大的情況下，準確率也相當良好。



圖 129、使用 Object-detection 作法標註示意圖

三、 使用 Roboflow 的開源資料集

資料集來源使用 Roboflow Universe 上找尋適用跌倒偵測訓練的資料，以 COCO JSON 格式下載即可提供給 Pytorch 做訓練。資料集:Fall Detection Dataset 將目標姿勢分為 Fall-down(圖 118)、sitting(圖 119)、standing(圖 120) 3 種狀態判斷。資料集數量分別為 Train set: 731 images、Valid set: 208 images、Test set: 104 images



圖 130、Fall Detection Dataset 中 Fall-detected 訓練用圖



圖 131、Fall Detection Dataset 中 sitting 訓練用圖



圖 132、Fall Detection Dataset 中 standing 訓練用圖

四、在 PC 上使用 Anaconda 環境進行模型訓練

(一) 訓練環境安裝方法:

根據以下步驟依序安裝

1. 建立 python 環境

```
$ conda create --name yolox_nano python=3.12
```

```
$ conda activate yolox_nano
```

透過上述指令在 Anaconda 上建立環境

2. upgrade pip

```
$ python -m pip install --upgrade pip setuptools
```

3. 至 Pytorch 官網安裝 Pytorch

```
$ python -m pip install torch torchvision torchaudio --index-url-  
https://download.pytorch.org/whl/cu118
```

4. 安裝 mmdcv

```
$ python -m pip install mmdcv==2.0.1 -f  
https://download.openmmlab.com/mmdcv/dist/cu118/torch2.0/index.html
```

5. 安裝其它需求套件:

開啟安裝好的檔案目錄並透過下面指令下載

```
$ python -m pip install --no-input -r requirements.txt
```

6. Installing the YOLOX

```
$ python setup.py develop
```

(二) 模型訓練設定:

將資料集整理成以下形式放在 Dataset 目錄中

Datasets/

<your_datasets_name>/

annotations/

<train_annotation_json_file>

<val_annotation_json_file>

train2017/

<train_img>

val2017/

<validation_img>

在訓練檔 `yolox_nano_ti_lite_nu.py` 中指定訓練和驗證資料集位置和訓練設定(如圖 121)設定適量迭代次數(500 次需 3 小時)，在類別檔 `coco_classes.py` 中定義資料集種類數(`none`, `fall-detected`, `sitting`, `standing`) (如圖 122)，最後輸入訓練指令 `$ python tools/train.py -f exps/default/yolox_nano_ti_lite_nu.py -d 1 -b 64 --fp16 -o -c pretrain/tflite_yolox_nano_ti/320_DW/yolox_nano_320_DW_ti_lite.pth`

```
27         # Define yourself dataset path
28         self.data_dir = "datasets/COCO"
29         self.train_ann = "train_annotations.coco.json"
30         self.val_ann = "val_annotations.coco.json"
31
32         # ----- training config ----- #
33         self.num_classes = 4
34         self.warmup_epochs = 5
35         self.max_epoch = 500
```

圖 133、訓練檔 `yolox_nano_ti_lite_nu.py` 中設定相關程式碼

```
5     COCO_CLASSES = (
6         "none",
7         "fall_detected",
8         "sitting",
9         "standing"
10    )
```

圖 134、類別檔 `coco_classes.py` 中設定相關程式碼

之後可以利用測試集中的圖片執行模型推論檢查準確率(如圖 123、圖 124)，本次範例中以 Pytorch 框架訓練 `yolox-nano` 模型結果，訓練測試集 104 張圖片僅 10 張圖片有誤(準確率 90.3%)

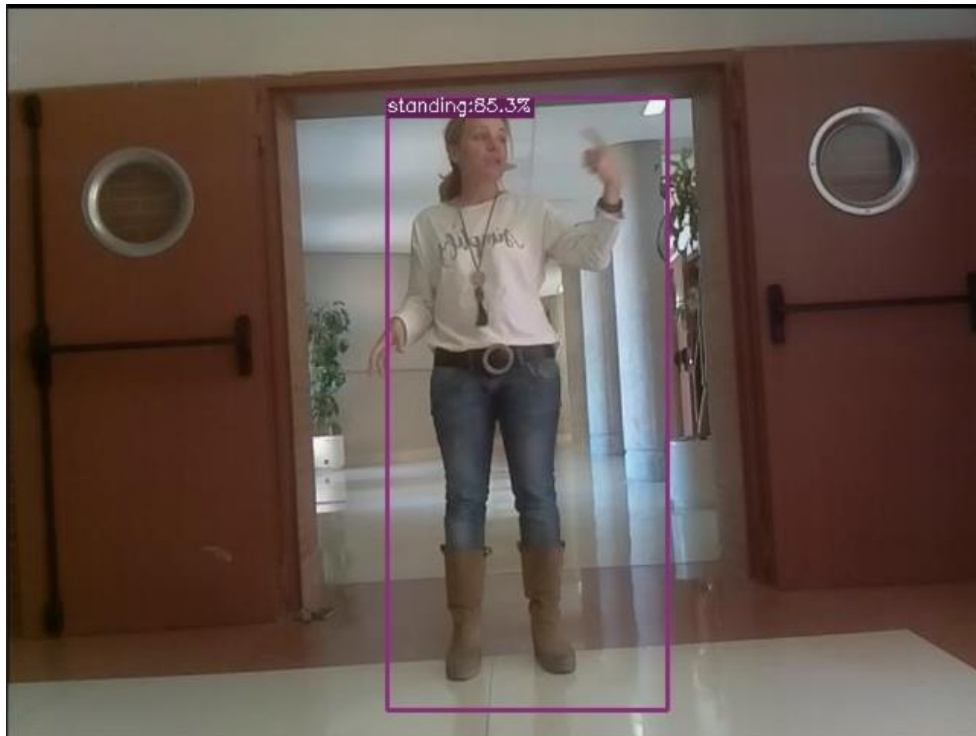


圖 135、使用測試集圖片 Standing 進行模型推論圖

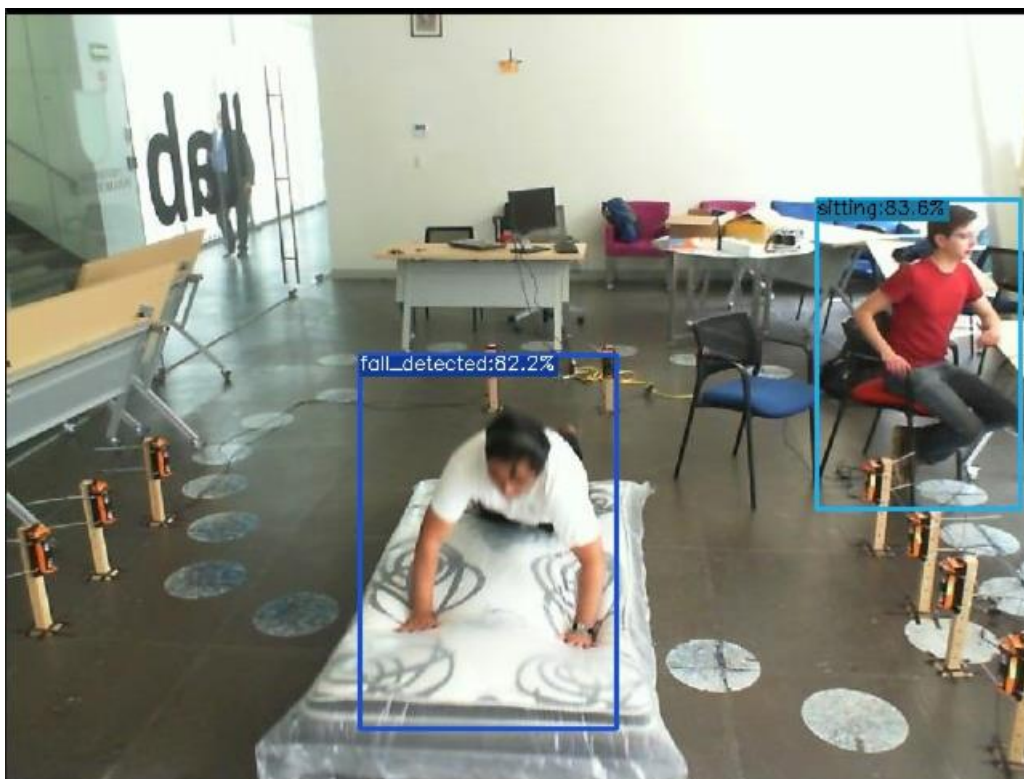


圖 136、使用測試集圖片 Sitting 和 Fall-down 進行模型推論圖

(三) 模型框架轉換:

1. 將訓練完成的 checkpoint 檔(.ckpt)轉成 onnx 檔(.onnx)，轉換指令

```
$ python tools/export_onnx.py -f exps/default/yolox_nano_ti_lite_nu.py -c  
YOLOX_outputs/yolox_nano_ti_lite_nu/latest_ckpt.pth --output-name  
YOLOX_outputs/yolox_nano_ti_lite_nu/yolox_nano_nu_fall.onnx
```

2. 將 onnx 檔轉成 tensorflow 框架並輕量化轉成 tflite 檔(.tflite)，轉換指令

```
$ python demo/TFLite/generate_calib_data.py --img-size 320 320 --n-img 200  
-o YOLOX_outputs\yolox_nano_ti_lite_nu\calib_data_320x320_n200.npy --  
img-dir datasets\COCO\train2017  
$ onnx2tf -i  
YOLOX_outputs/yolox_nano_ti_lite_nu/yolox_nano_nu_fall2.onnx -oiqt -  
qcind images  
YOLOX_outputs\yolox_nano_ti_lite_nu\calib_data_320x320_n200.npy  
"[[[[0,0,0]]]]" "[[[[1,1,1]]]]"
```

(四) Vela 編譯步驟:

1. 將輕量化完的模型放到 vela\generated\，並且修改 vela 目錄下的
variables.bat

```
$ set MODEL_SRC_FILE=yolox_nano_nu_fall_full_integer_quant.tflite  
$ set  
MODEL_OPTIMISE_FILE=yolox_nano_nu_fall_full_integer_quant_vela.tflit  
e
```

2. 執行 gen_model_cpp 檔案

執行結果會出現在

vela\generated\yolox_nano_ti_lite_nu_fall_full_integer_quant_vela.tflite.cc

五、開發板端的推論程式系統流程步驟

C 語言程式系統流程圖(圖 125)及說明，整體流程主要分為 3 個步驟，1.初始化硬體與模型，2.根據 Frame buffer 不同狀態進行不同作業，3.更新 Frame buffer 狀態之後重複執行 2~3 步驟，接下來會詳細說明相關程式碼

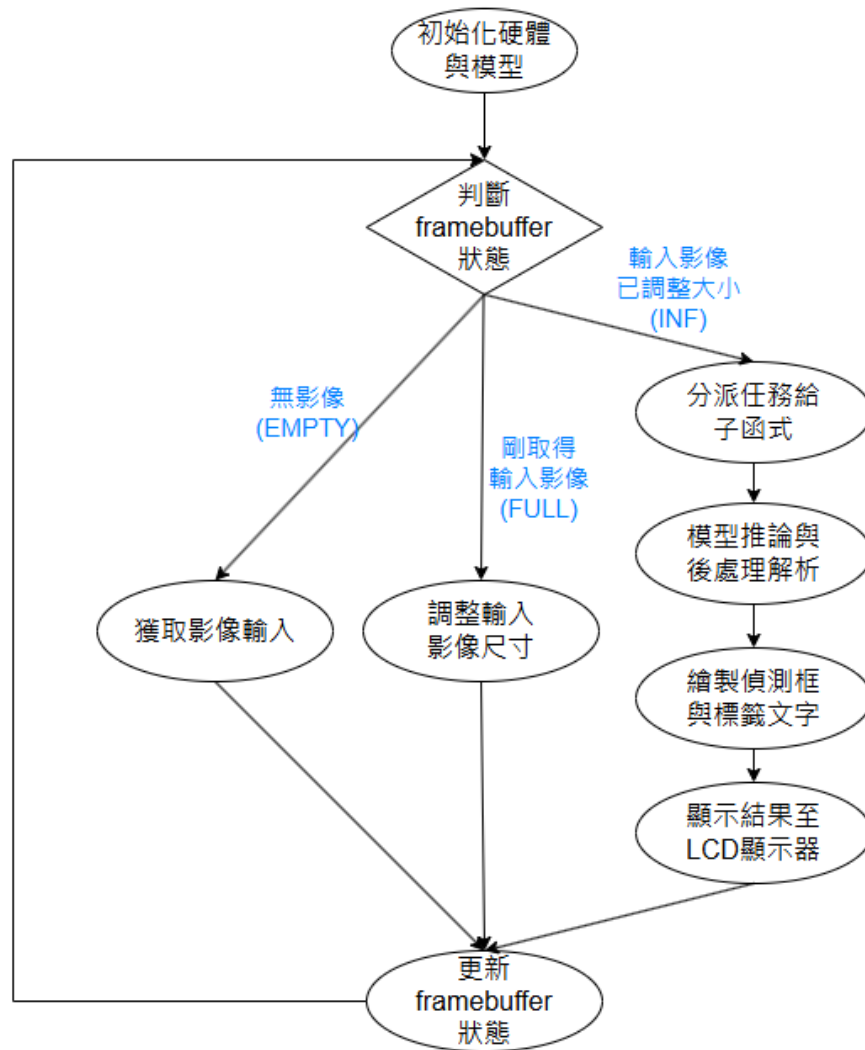


圖 137、C 語言程式系統流程圖

(一) 初始化硬體與模型(圖 126):

初始化硬體資源、模型，包括：設定 frame buffer 的尺寸與格式，並將 frame buffer 的狀態設為 EMPTY，表示可用於儲存新影像。

```

150 static void omv_init()
151 {
152     image_t frameBuffer;
153     int i;
154
155     frameBuffer.w = GLCD_WIDTH;
156     frameBuffer.h = GLCD_HEIGHT;
157     frameBuffer.size = GLCD_WIDTH * GLCD_HEIGHT * 2;
158     frameBuffer.pixfmt = PIXFORMAT_RGB565;
159
160     _fb_base = fb_array;
161     _fb_end = fb_array + OMV_FB_SIZE - 1;
162     _fballoc = _fb_base + OMV_FB_SIZE + OMV_FB_ALLOC_SIZE;
163     _jpeg_buf = jpeg_array;
164
165     fb_alloc_init0();
166
167     framebuffer_init0();
168     framebuffer_init_from_image(&frameBuffer);
169
170     for (i = 0 ; i < NUM_FRAMEBUF; i++)
171     {
172         s_asFramebuf[i].eState = eFRAMEBUF_EMPTY;
173     }
174
175     framebuffer_init_image(&s_asFramebuf[0].frameImage);

```

圖 138、初始化硬體與模型程式碼

(二) 獲取影像輸入(圖 127):

從相機捕捉影像或從內嵌影像載入圖片，並更新 Framebuffer 狀態至 FULL

```

545  #if defined (__USE_CCAP__)
546      //capture frame from CCAP
547  #if defined(__PROFILE__)
548      u64CCAPStartCycle = pmu_get_systick_Count();
549  #endif
550
551      ImageSensor_Capture((uint32_t)(emptyFramebuf->frameImage.data));
552
553  #if defined(__PROFILE__)
554      u64CCAPEndCycle = pmu_get_systick_Count();
555      info("ccap capture cycles %llu \n", (u64CCAPEndCycle - u64CCAPStartCycle));
556  #endif
557
558  #else
559      //copy source image to frame buffer
560      image_t srcImg;
561
562      srcImg.w = IMAGE_WIDTH;
563      srcImg.h = IMAGE_HEIGHT;
564      srcImg.data = (uint8_t *)pu8ImgSrc;
565      srcImg.pixfmt = PIXFORMAT_RGB888;
566
567      roi.x = 0;
568      roi.y = 0;
569      roi.w = IMAGE_WIDTH;
570      roi.h = IMAGE_HEIGHT;
571
572      imlib_nvt_scale(&srcImg, &emptyFramebuf->frameImage, &roi);
573  #endif

```

圖 139、獲取影像輸入程式碼

(三) 調整輸入影像尺寸(圖 128):

將輸入影像調整為模型尺寸大小，並更新 Framebuffer 狀態至 INF


```

395     if (fullFramebuf)
396     {
397         //resize full image to input tensor
398         image_t resizeImg;
399
400         roi.x = 0;
401         roi.y = 0;
402         roi.w = fullFramebuf->frameImage.w;
403         roi.h = fullFramebuf->frameImage.h;
404
405         resizeImg.w = inputImgCols;
406         resizeImg.h = inputImgRows;
407         resizeImg.data = (uint8_t *)inputTensor->data.data; //direct resize to input tensor buffer
408         resizeImg.pixfmt = PIXFORMAT_RGB888;
409
410 #if defined(__PROFILE__)
411         u64StartCycle = pmu_get_systick_Count();
412 #endif
413         imlib_nvt_scale(&fullFramebuf->frameImage, &resizeImg, &roi);

```

圖 140、調整輸入影像尺寸程式碼

(四) 處理待推論影像(圖 129、圖 130、圖 131):

分派任務給相關子函式，需要進行推論及後處理解析、繪製物件偵測框、顯示結果至 LCD 顯示器、更新 Framebuffer 狀態至 EMPTY

```

378     if (infFramebuf)
379     {
380         /* Detector post-processing*/
381
382         inferenceJob->responseQueue = inferenceResponseQueue;
383         inferenceJob->pPostProc = &postProcess;
384         inferenceJob->modelCols = inputImgCols;
385         inferenceJob->modelRows = inputImgRows;
386         inferenceJob->srcImgWidth = infFramebuf->frameImage.w;
387         inferenceJob->srcImgHeight = infFramebuf->frameImage.h;
388         inferenceJob->results = &infFramebuf->results; //&results;
389
390         xQueueReceive(inferenceResponseQueue, &inferenceJob, portMAX_DELAY);
391     }

```

圖 141、處理待推論影像程式碼

```

447     if (infFramebuf)
448     {
449         //draw bbox and render
450         /* Draw boxes. */
451         DrawImageDetectionBoxes(infFramebuf->results, &infFramebuf->frameImage, labels);
452
453         //display result image
454     #if defined (__USE_DISPLAY__)
455         //Display image on LCD
456         sDispRect.u32TopLeftX = 0;
457         sDispRect.u32TopLeftY = 0;
458         sDispRect.u32BottonRightX = (infFramebuf->frameImage.w - 1);
459         sDispRect.u32BottonRightY = (infFramebuf->frameImage.h - 1);
460
461     #if defined(__PROFILE__)
462         u64StartCycle = pmu_get_systick_Count();
463     #endif
464
465         Display_FillRect((uint16_t *)infFramebuf->frameImage.data, &sDispRect);
466
467     #if defined(__PROFILE__)
468         u64EndCycle = pmu_get_systick_Count();
469         info("display image cycles %llu \n", (u64EndCycle - u64StartCycle));
470     #endif
471
472     #endif
473
474     u64PerfFrames ++;

```

圖 142、處理待推論影像程式碼

```

476         if ((uint64_t) pmu_get_systick_Count() > u64PerfCycle)
477         {
478             info("Total inference rate: %llu\n", u64PerfFrames / EACH_PERF_SEC);
479 #if defined (__USE_DISPLAY__)
480             sprintf(szDisplayText, "Frame Rate %llu", u64PerfFrames / EACH_PERF_SEC);
481             //             sprintf(szDisplayText, "Time %llu", (uint64_t) pmu_get_systick_Count() / (uint64_t)SystemCoreClock);
482
483             sDispRect.u32TopLeftX = 0;
484             sDispRect.u32TopLeftY = framebuffer.h + FONT_HTIGHT;
485             sDispRect.u32BottomRightX = (framebuffer.w);
486             sDispRect.u32BottomRightY = (framebuffer.h + (2 * FONT_HTIGHT) - 1);
487
488             Display_ClearRect(C_WHITE, &sDispRect);
489             Display_PutText(
490                 szDisplayText,
491                 strlen(szDisplayText),
492                 0,
493                 framebuffer.h + FONT_HTIGHT,
494                 C_BLUE,
495                 C_WHITE,
496                 false
497             );
498 #endif
499             u64PerfCycle = (uint64_t)pmu_get_systick_Count() + (uint64_t)(SystemCoreClock * EACH_PERF_SEC);
500             u64PerfFrames = 0;
501         }
502
503         PresentInferenceResult(infFramebuf->results, labels);
504         infFramebuf->eState = eFRAMEBUF_EMPTY;
505     }

```

圖 143、處理待推論影像程式碼

(五) 模型推論及後處理解析(圖 132):

程式運作依序為獲取模型輸入張量、執行推論、獲取模型輸出張量、使用後處理解析推論結果

```

300     TfLiteTensor *inputTensor = model.GetInputTensor(0);
301     TfLiteTensor *outputTensor = model.GetOutputTensor(0);
302
303     if (!inputTensor->dims)
304     {
305         printf_err("Invalid input tensor dims\n");
306         vTaskDelete(nullptr);
307         return;
308     }
309     else if (inputTensor->dims->size < 3)
310     {
311         printf_err("Input tensor dimension should be >= 3\n");
312         vTaskDelete(nullptr);
313         return;
314     }
315
316     TfLiteIntArray *inputShape = model.GetInputShape(0);
317
318     const int inputImgCols = inputShape->data[arm::app::YoloXnanoNu::ms_inputColsIdx];
319     const int inputImgRows = inputShape->data[arm::app::YoloXnanoNu::ms_inputRowsIdx];
320
321     // postProcess
322     arm::app::object_detection::DetectorPostprocessing postProcess(0.6, 0.65, numClasses, 0);

```

圖 144、模型推論及後處理解析程式碼

(六) 繪製偵測框(圖 133)與標籤文字(圖 134):

從推論結果獲取相關資訊並繪製相關資訊到影像上

```
211 imlib_draw_rectangle(drawImg, result.m_x0, result.m_y0, result.m_w, result.m_h, COLOR_B5_MAX, 1, false);
```

圖 145、繪製偵測框程式碼

```
212 imlib_draw_string(drawImg, result.m_x0, result.m_y0 - 16, labels[result.m_cls].c_str(), COLOR_B5_MAX, 2, 0, 0, false,  
213 false, false, false, 0, false, false);
```

圖 146、繪製標籤文字程式碼

(七) 顯示結果至 LCD 顯示器(圖 135):

將繪製完成的影像輸出至 LCD 顯示器上

```
453 //display result image  
454 #if defined (__USE_DISPLAY__)  
455 //Display image on LCD  
456 sDispRect.u32TopLeftX = 0;  
457 sDispRect.u32TopLeftY = 0;  
458 sDispRect.u32BottonRightX = (infFramebuf->frameImage.w - 1);  
459 sDispRect.u32BottonRightY = (infFramebuf->frameImage.h - 1);  
460  
461 #if defined(__PROFILE__)  
462 u64StartCycle = pmu_get_systick_Count();  
463 #endif  
464  
465 Display_FillRect((uint16_t *)infFramebuf->frameImage.data, &sDispRect);  
466  
467 #if defined(__PROFILE__)  
468 u64EndCycle = pmu_get_systick_Count();  
469 info("display image cycles %llu \n", (u64EndCycle - u64StartCycle));  
470 #endif  
471  
472 #endif  
473  
474 u64PerfFrames ++;  
475  
476 if ((uint64_t) pmu_get_systick_Count() > u64PerfCycle)  
477 {  
478 info("Total inference rate: %llu\n", u64PerfFrames / EACH_PERF_SEC);
```

圖 147、顯示結果至 LCD 顯示器程式碼

六、 結論與未來發展

(一) 結論:

本次跌倒偵測系統能有效利用 Roboflow 資料集結合 YOLOX-Nano 模型，在新唐科技 M55M1 開發板上創造出更快速且節省人力資源的方式來偵測跌倒達到提高醫療品質的效果。透過 Full-INT8 模型量化技術和 Vela 編譯器的最佳化，來提升了模型的推論速度和準確率，在測試中達到高達 90%以上的偵測準確率。

(二) 未來發展:

本次的開發方式僅透過最簡單的攝像鏡頭來捕捉影像，未來可以配合更多功能的鏡頭(如 3D 攝影機)或使用 Keypoint-Detection 的做法來增加姿勢的種類與辨識率，做到更專業及精確的應用。

參考資料

Yoloxnano 訓練: https://github.com/MaxCYCHEN/yolox-ti-lite_tflite_int8

Roboflow 跌倒偵測資料集: <https://universe.roboflow.com/search?q=class:fall-detected>

Roboflow keypoint-detection 介紹: <https://blog.roboflow.com/pose-estimation-algorithms-history/>

陸、智慧家居 1 – 垃圾分類

一、 案例介紹 – 垃圾分類

(一) 簡介及應用

1. 技術基礎：

- (1) 通過新唐科技開發的 M55M1 開發板 實作影像辨識垃圾分類。此系統結合鏡頭模組，可有效辨識不同種類的垃圾，並進行快速判斷，透過即時顯示，提供垃圾種類的詳細資訊。
- (2) 使用 YOLOX_nano 訓練模型：運用深度學習技術訓練垃圾分類的模型，實現多類垃圾的準確辨識，並將已訓練好的模型燒錄至 M55M1 開發板中進行應用。

2. 實際應用情境：

- (1) 在家庭環境中，垃圾分類對小孩子而言通常具挑戰性，主要因為不同垃圾在顏色、材質及大小上存在明顯差異，這可能導致誤分或無法有效完成分類。
- (2) 使用該垃圾分類系統，孩子只需將垃圾放在鏡頭前，螢幕即可即時顯示垃圾種類，並通過視覺化提醒與引導，達到準確分類的目的。

(二) 優點與特色

1. 即時性： 實現快速的影像處理與結果顯示。
2. 教育意義： 提升兒童垃圾分類意識，從小培養環保行為。
3. 準確性： 憑藉深度學習模型的高精度，避免人工分類中的誤差

(三) 目標與成果

1. 準確率：

該系統針對四種垃圾類型（紙類、塑膠瓶、鋁罐、紙杯）進行辨識，達到了 90.1% 的準確率，展現了系統在實際應用中的高效表現。

2. 模型規模：

訓練後的模型大小控制在 2MB 以下，完全符合開發板燒錄標準，並在嵌入式環境中流暢運行。

3. 分類能力：

支援 同時辨識多種垃圾 的功能，提升垃圾分類的效率與實用性。

(四) 圖片說明

下圖展示了預計結果的實際應用場景，螢幕顯示了即時影像處理結果，系統成功將不同類型的垃圾（例如鋁罐、塑膠瓶等）準確地框選並標註。

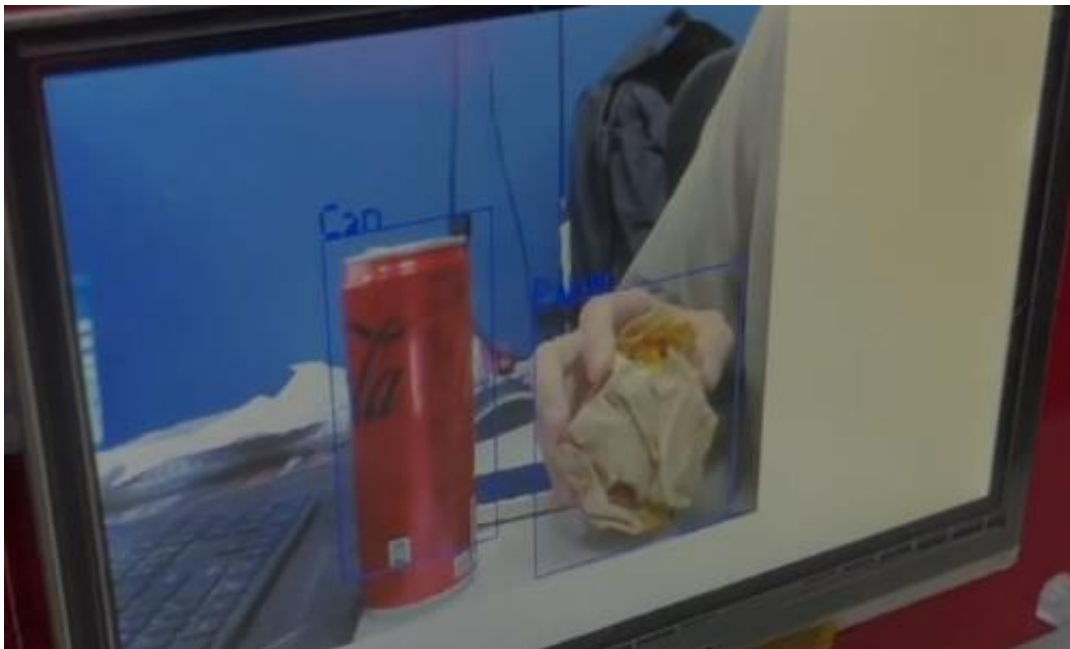


圖 148、影像辨識 demo 成果

二、 資料集與模型訓練

(一) 資料集

1. 工具： 使用 LabelImg 進行標註與標籤化。
2. 數據分割：
 - (1) 訓練集 (Train set)：共 378 張影像。
 - (2) 驗證集 (Valid set)：共 36 張影像。
 - (3) 測試集 (Test set)：共 19 張影像。

(二) 模型訓練

1. 模型： 採用 YOLOX-Nano (Light Model)，適用於輕量化的嵌入式系統。
2. 虛擬環境： 利用 Anaconda 建立虛擬環境，確保開發過程的模組化與隔離性。
3. 訓練框架： 使用 PyTorch 深度學習框架進行模型訓練，支援靈活的模型構建與訓練流程。
4. 資料集格式： 遵循 COCO JSON 格式進行數據管理，以保持與主流模型框架的兼容性

(三) 資料集準備流程

資料集主要透過自行蒐集的照片進行標註和訓練，以下為具體步驟：

1. 蒐集照片：
 - (1) 蒐集足夠數量的圖片作為訓練資料（本案例約使用 400 張照片）。
資料量越多，模型的訓練效果越佳。
2. 標註資料：
 - (1) 使用 LabelImg 工具進行影像標籤化，將照片中目標物進行標記。
 - (2) 此工具可透過網路下載，簡單易用，適合自定義資料集的標註。
3. 確認格式一致性：
 - (1) 確保標註完成的資料集與模型訓練需求的格式一致。本案例採用 COCO JSON 格式，適用於多數深度學習框架。
4. 格式轉換：
 - (1) 將原始的 YOLO 格式標註資料，透過格式轉換工具轉為 COCO JSON 格式，以供模型訓練使用。

(四) 下載與安裝 LabelImg

1. 下載鏈結：
 - (1) 前往以下 GitHub 頁面下載 LabelImg 工具：[LabelImg Releases](#)
2. 選擇版本：
 - (1) 在下載頁面中，點擊下載 Binary v1.8.1 對應的壓縮檔案 windows_v1.8.1.zip。
3. 安裝與執行：

- (1) 解壓縮下載的檔案，進入解壓後的目錄，找到執行檔 labelImg.exe 並雙擊運行，即可進入標註工具介面。

4. 工具特色

- (1) 跨平台支持：適用於 Windows、Linux 與 MacOS。
- (2) 簡單易用：支援快速圖像標註，適合初學者進行影像標籤化作業。
- (3) 靈活格式選擇：支援多種輸出格式（如 PASCAL VOC 和 COCO JSON 格式），方便整合於主流深度學習框架。

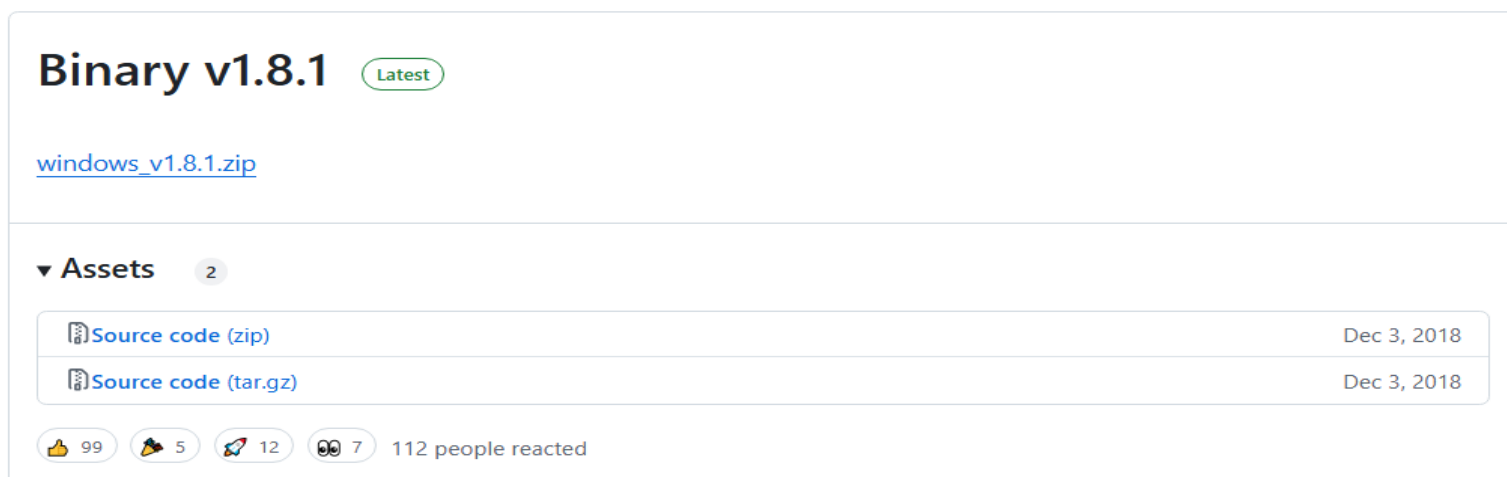


圖 149、label 軟體下載

5. 設定資料集路徑與儲存位置

(1) 選擇需要標註的資料集位置：

- 點擊 Open Dir 按鈕，選取資料集的來源資料夾，該資料夾內應包含所有需要進行標註的影像檔案。

(2) 選擇標註結果的儲存位置：

- 點擊 Change Save Dir 按鈕，設定標註完成後的標籤檔案儲存位置。

(3) 確認設定完成：

- 完成路徑設定後，右下角的檔案清單區域會顯示所有待標註的影像檔案。您可以檢查是否正確載入所有圖片。

6. 圖片說明

圖例中標注了操作按鈕的位置與功能，指引用戶快速完成資料集目錄與儲存目錄的設置。

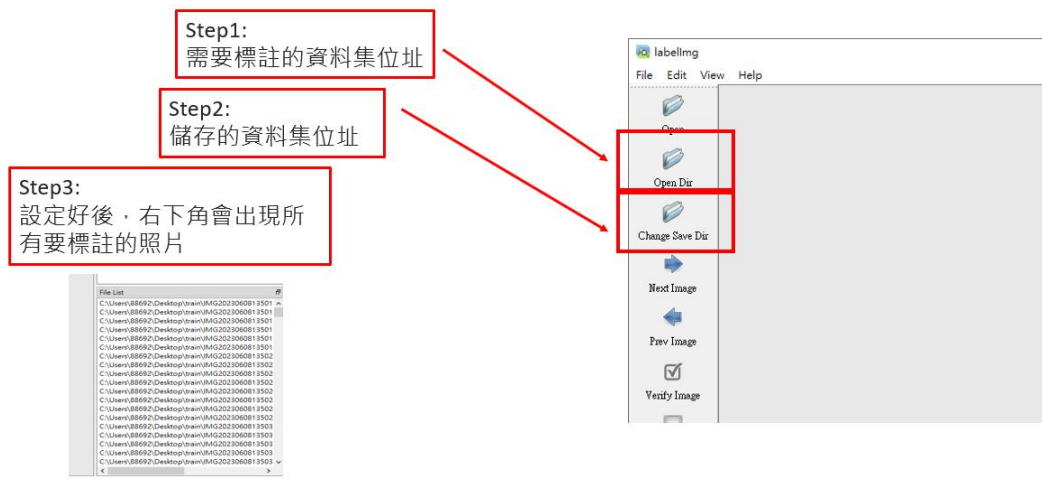


圖 150、label 工具教學

7. 選擇儲存的標註格式

根據訓練的模型需求，選擇適合的標註格式。常用格式包括 PascalVOC、YOLO 和 CreateML，以下為格式特性及用途：

(1) PascalVOC 格式：

- a. 儲存方式：以 XML 文件 儲存標註信息，包括圖片檔案名稱、尺寸（高度、寬度）以及每個標註框的類別和座標。
- b. 應用場景：適用於 TensorFlow 和 Keras 等框架的深度學習模型訓練。

(2) YOLO 格式（本次使用）：

- a. 儲存方式：純文字文件，簡潔高效，每行表示一個標註框，格式為，
<class_id> <x_center> <y_center> <width> <height>
 - (a) class_id：標註框的類別 ID。
 - (b) x_center, y_center：標註框中心的座標（相對於圖片比例）。
 - (c) width, height：標註框的寬度與高度（相對於圖片比例）。
- b. 應用場景：適用於 YOLO（You Only Look Once）系列模型，專為即時物件檢測設計。

(3) CreateML 格式：

- a. 儲存方式：使用 JSON 文件 儲存標註信息，描述每張圖片的標註框位置、類別以及相關數據。
- b. 應用場景：適用於 Apple 的 CreateML 平台，便於在 macOS 環境下進行訓練與部署

(4) 格式選擇建議：

- a. 若使用 TensorFlow 或 Keras：建議選擇 PascalVOC 格式。
- b. 若進行 YOLO 模型訓練：需選擇 YOLO 格式（本次選用）。

c. 若開發 iOS 應用：選擇 CreateML 格式。

8. 設定標註種類並進行標註

(1) 創建標註類別文件：

- a. 創建一個名為 label.txt 的文字檔案，並在檔案內填寫所有需要標註的類別名稱（例如：Aluminum Can, Paper, Paper Cup, Plastic Bottle）。
- b. 此檔案將作為標註工具的類別參考，確保標註時能正確選擇類別

(2) 開始標註：

- a. 點擊工具欄中的 Create RectBox 按鈕（快捷鍵為 W），使用滑鼠拖曳，框選出需要標註的物品。
- b. 框選完成後，系統將彈出輸入框，讓使用者選擇或輸入該標註框的類別名稱。

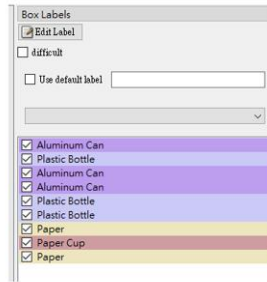
(3) 完成標註檢查：

- a. 標註完成一張照片後，右側的 Box Labels 區域將顯示該照片內所有標註的物品及其類別。
- b. 使用者可檢查所有標註框是否正確對應到指定的類別名稱，並進行必要的調整。

(4) 圖片說明

- a. 左圖：展示了工具的標註類別清單，包括 Aluminum Can、Paper、Paper Cup 等物品。
- b. 右圖：顯示了一個標註框及其對應的類別名稱輸入操作範例，框選的物品即標記為 Aluminum Can。

所有標註
的物品



標註框並
給class

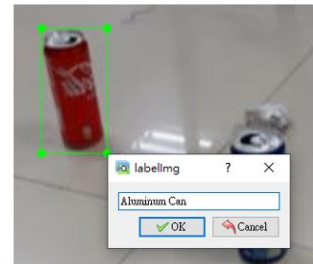


圖 151、標註方式以及 class 使用

9. 轉換 YOLO 格式為 COCO JSON 格式

(1) 標註完成後的格式轉換需求：

- a. 使用 YOLO 格式進行標註的檔案完成後，需透過一段 Python 程式將 YOLO 格式轉換為 COCO JSON 格式。
- b. 原因：COCO JSON 格式適用於多數深度學習框架，尤其在進行複雜物件檢測任務時具有更高的兼容性。

(2) 使用轉換程式：

- a. 下方提供的程式檔案 yolo2cocojson.py 可用於格式轉換：
https://drive.google.com/file/d/15cSq0MiBjgWUIwJn0Z_0t2c51vUY2jh6/view?usp=sharing
- b. 使用者只需依據自身需求進行少量修改，程式即能自動完成格式轉換。

10. 設定轉換程式

(1) 加入標註類別：

- a. 按照標註種類順序，更新 categories 區塊，範例如下：

```
categories = [  
    {"id": 0, "name": "class1"},  
    {"id": 1, "name": "class2"},  
    {"id": 2, "name": "class3"},  
    {"id": 3, "name": "class4"}  
]
```

圖 152、設定 class 以及編號

(2) 設定資料集路徑：

- a. 更新程式中圖片與標註檔的路徑，範例如下：

- b. 資料夾名稱必須為 Images 和 Labels，並確保圖片和標註檔案名稱一致。

```
images_dir = "/images" # 圖片資料夾  
labels_dir = "/labels" # YOLO 標註檔案資料夾
```

圖 153、設定照片以及標註檔路徑

三、 Evaluate TFLite int8/float Model (概述)

(一) 模型性能測試

1. 測試目的：

(1) 評估模型的準確性和效能，重點包括：

- a. 準確率 (AP, Average Precision)：測試每個類別的預測準確性。
- b. 交並比 (IOU, Intersection Over Union)：衡量模型預測框與真實框 (Ground Truth) 的重疊程度：

$$\text{IoU} = \frac{\text{預測框與真實框的交集面積}}{\text{預測框與真實框的聯集面積}}$$

圖 154、IOU 定義

2. 評估方法：

- (1) 使用 Python 腳本對模型進行推論，統計各類別的性能數據。
- (2) 支援整數量化的 TFLite 模型和浮點模型進行評估。

(二) 使用指令進行評估

1. 基本指令格式

```
$ python demo\TFLite\tflite_inference.py -m <FULL_INTEGER_QUANT_TFLITE> -s  
<SCORE_THR> -i <PATH_OF_IMAGE> -a <PATH_OF_VAL_ANNOTATION_FILE>
```

2. 實際範例指令：

```
python demo\TFLite\tflite_inference.py -m  
YOLOX_outputs/yolox_nano_ti_lite_nu/vox_nano_nu_hg_full_integer_quant.tflite -s 0.6 -i  
datasets/hagrid_coco/val2017/0001.jpg -a hagrid_coco/annotations/hagrid_val.json
```

(三) 指令參數說明

1. FULL_INTEGER_QUANT_TFLITE：

(1) 指向需要評估的整數量化 TFLite 模型（.tflite 格式）。

2. SCORE_THR：

(1) 設定分數閾值，檢測結果的可信度必須高於此閾值才會被保留。範例值為 0.6。

3. PATH_OF_IMAGE：

(1) 指定需要進行推論的圖片文件路徑。

4. PATH_OF_VAL_ANNOTATION_FILE：

(1) 驗證標註檔案路徑，包含所有圖片的真實標註數據（.json 格式）。

(四) 測試模型效能

1. 單張圖片測試指令：

基本指令格式：

```
$ python demo\TFLite\tflite_inference.py -m <FULL_INTEGER_QUANT_TFLITE> -s  
<SCORE_THR> -i <PATH_OF_IMAGE> -a <PATH_OF_VAL_ANNOTATION_FILE>
```

2. 範例指令：

使用整數量化的 TFLite 模型對單張圖片進行測試：

```
python demo\TFLite\tflite_inference.py -m  
YOLOX_outputs/yolox_nano_ti_lite_nu/vox_nano_ti_lite_nu_full_integer_quant.tflite -s  
0.6 -i datasets/hagrid_coco/val2017/0001.jpg -a hagrid_coco/annotations/hagrid_val.json
```

(五) 參數說明

1. FULL_INTEGER_QUANT_TFLITE：

(1) 模型檔案路徑，指定整數量化的 TFLite 模型（.tflite 格式）。

2. SCORE_THR：

(1) 物體檢測的信心度閾值，僅保留高於此閾值的檢測結果，範例中設為 0.6。

3. PATH_OF_IMAGE：

(1) 測試圖片的完整路徑，例

datasets/hagrid_coco/val2017/0001.jpg

4. PATH_OF_VAL_ANNOTATION_FILE：

(1) 驗證集的標註檔案，包含真實框資訊，範例中為：

hagrid_coco/annotations/hagrid_val.json

四、 C++ 軟體流程

(一) 系統初始化 (System Initialization)

1. 主要檔案位置: BoardInit.cpp 與 mpu_config_M55M1.h。
 2. 硬體資源設置:
 - (1) 設定系統的基礎硬體資源，例如時鐘、UART、記憶體與 NPU (Neural Processing Unit)。
 - (2) 配置記憶體保護單元 (MPU)，分配記憶體區域與訪問屬性，詳細設置定義於 mpu_config_M55M1 中。
 3. 目標: 確保硬體基礎環境已準備妥當，為後續的程式執行提供穩定支援。
-

(二) 資料準備 (Data Preparation)

1. 載入與處理資料:
 - (1) 載入圖片資料，進行必要的預處理，例如尺寸縮放或格式轉換。
 - (2) 確保輸入資料滿足模型推論的要求，進一步提升準確性與效率。
 2. 任務排程:
 - (1) 任務分派由 InferenceTask.cpp 負責，該模組處理資料的接收與初步排程，為推論模組提供準確的輸入。
-

(三) 模型推論 (Model Inference)

1. 主要檔案位置: InferenceTask.cpp
2. 執行內容:
 - (1) 使用模型核心方法執行推論 (m_model->RunInference())，並取得輸出張

量結果 (GetOutputTensor)。

(2) 推論過程中，計算各分類的機率與預測框，將模型的預測轉化為可讀結果。

3. 目標:

(1) 實現推論階段的高效執行，並確保結果準確反映輸入資料。

(四) 結果輸出及後處理 (Post-Processing)

1. 主要檔案位置: DetectorPostProcessing.cpp

2. 執行內容:

(1) 使用非極大值抑制 (Non-Maximum Suppression, NMS) 過濾低相關預測框，提升模型檢測的準確度。

(2) 將模型輸出的結果映射回原始圖片的座標，便於後續處理與可視化。

3. 目標:

(1) 調整輸出結果，使其能準確定位目標並完成分類，最終獲取精確的檢測輸出。

(五) 主迴圈 (Main Loop)

1. 功能:

(1) 包含循環控制邏輯，確保系統持續處於運行狀態，直到觸發退出條件 (如接收到退出訊號)。

2. 關鍵檢查:

(1) 確保所有資源在退出時正確釋放，避免資源泄漏，保證系統穩定性。

五、系統程式解析

(一) 主要檔案：BoardInit.cpp 與 mpu_config_M55M1.h

系統時鐘與模組時鐘初始化 (SYS_Init)：

確保系統時鐘與外部振盪器穩定運行，為所有模組提供時鐘源。

UART 初始化：

設置 UART 作為基本調試序列埠，允許使用 printf 進行標準輸出。

HyperRAM 初始化與模式配置：

配置 HyperRAM 的通訊端口，並進入直接映射模式以提升存取效能。

Arm® Ethos™-U NPU 初始化：

確保嵌入式 AI 模型能夠正常運行。若初始化失敗，需返回錯誤程式碼進行調試。

硬體保護設置完成：

在初始化結束前，解鎖寄存器進行配置，最後重新鎖定以確保系統安全。

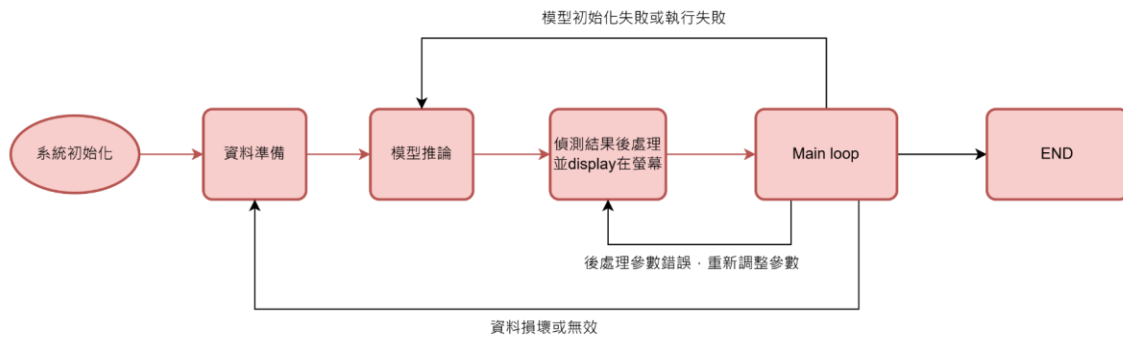


圖 155、系統流程圖

1. 系統時鐘與模組時鐘初始化

(1) 步驟:

- a. 開啟內部與外部振盪器，並等待其穩定。
- b. 設置 APLL 鎖相迴路為系統時鐘源，頻率為 180 MHz。
- c. 啟用所有需要的硬體模組時鐘。

(2) 程式範例:

```
/* 開啟內建 RC 振盪器與外部晶體振盪器 */  
CLK_EnableXtalRC(CLK_SRCTL_HIRCEN_Msk);  
CLK_WaitClockReady(CLK_STATUS_HIRCSTB_Msk);  
CLK_EnableXtalRC(CLK_SRCTL_HXTEN_Msk);  
CLK_WaitClockReady(CLK_STATUS_HXTSTB_Msk);  
  
/* 設定系統時鐘頻率與更新核心時鐘 */  
CLK_SetBusClock(CLK_SCLKSRC_APLL, FREQ_180MHZ);  
SystemCoreClockUpdate();
```

圖 156、系統初始化程式碼

2. UART 初始化

(1) 功能:

配置 UART6 作為標準輸出端口，並完成調試功能的設置。

(2) 步驟:

- a. 設置 UART 的時鐘來源與多功能引腳。
- b. 初始化 UART，支援標準輸出功能。

(3) 程式範例:

```
/* 設置 UART 時鐘為 HIRC 並啟用 */  
SetDebugUartCLK();  
SetDebugUartMFP();  
InitDebugUart(); // 初始化 UART，支援標準輸出
```

圖 157、UART 程式範例

3. HyperRAM 初始化與模式配置

(1) 功能:

配置 HyperRAM 的通訊端口並進入直接映射模式，提升存取效能。

(2) 步驟:

a. 初始化 HyperRAM 的引腳配置與基本初始化。

b. 切換至 Direct Map 模式以應用於高效存取。

(3) 程式範例:

```
HyperRAM_PinConfig(HYPERRAM_SPIM_PORT);  
HyperRAM_Init(HYPERRAM_SPIM_PORT);  
SPIM_HYPER_EnterDirectMapMode(HYPERRAM_SPIM_PORT);
```

圖 158、HyperRAM 程式範例

4. Arm® Ethos™-U NPU 初始化

(1) 功能:

初始化 Arm® NPU，並檢查狀態是否成功，以確保 AI 模型正常運行。

(2) 步驟:

1. 啟用 Arm® NPU。

2. 驗證初始化狀態，若失敗則返回錯誤程式碼進行調試。

(3) 程式範例:

```
#if defined(ARM_NPU)
int state;

/* 初始化 Arm Ethos-U NPU，若失敗則返回錯誤碼 */
if (0 != (state = arm_ethosu_npu_init())) {
    return state;
}
#endif
```

圖 159、NPU 初始化程式範例

5. 確保硬體保護設置完成

(1) 功能:

解鎖寄存器，進行保護配置，最後鎖定寄存器以確保系統安全。

(2) 步驟:

- a. 解鎖受保護的寄存器。
- b. 設置硬體保護。
- c. 鎖定寄存器，輸出初始化完成訊息。

(3) 程式範例:

```
SYS_UnlockReg(); // 解鎖受保護寄存器
SYS_LockReg();   // 鎖定寄存器保護
info("%s: complete\n", __FUNCTION__); // 初始化完成輸出
```

圖 160、硬體保護設置

(二) 模型推理程式解析

1. 重點功能概述 (inferenceTask.cpp)

(1) 推論執行 (Inference Execution)

a. 呼叫模型推論函式:

執行模型推理，處理輸入資料並生成結果。使用 `m_model->RunInference()` 方法執行核心計算。

b. 後處理功能 (Post-Processing):

使用 `DetectorPostProcessing` 將模型輸出的 Tensor 資料解析為物體偵測結果，包含目標座標框與標籤。

(2) 任務處理 (Task Handling)

a. 提供 FreeRTOS 的多任務執行架構，負責處理推論任務佇列 (Queue) 中的資料。

b. 在任務間，使用訊號量 (Semaphore) 與互斥鎖 (Mutex) 進行同步，確保執行安全且高效。

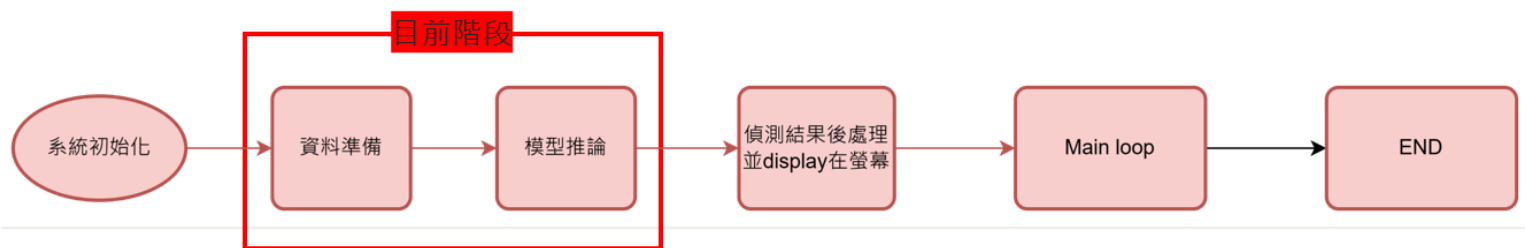


圖 161、模型推理程式解析流程圖

2. 推論以及執行 (RunInference)

(1) 功能描述:

- a. 利用 YOLO 模型進行推論，執行核心函式 `m_model->RunInference()`。
- b. 執行後處理函式 `pPostProc->RunPostProcessing()`，將模型輸出的 Tensor 資料轉化為可讀的物體偵測結果，包括座標框與標籤。

(2) 程式範例:

```
bool InferenceProcess::RunJob(  
    object_detection::DetectorPostprocessing *pPostProc,  
    int modelCols,  
    int modelRows,  
    int srcImgWidth,  
    int srcImgHeight,  
    std::vector<object_detection::DetectionResult> *results)  
{  
    bool runInf = m_model->RunInference(); // 執行模型推論  
    pPostProc->RunPostProcessing(  
        modelRows,  
        modelCols,  
        srcImgHeight,  
        srcImgWidth,  
        modelOutput0,  
        *results); // 執行後處理  
}
```

圖 162、推論以及執行程式範例

(3) 目標:

將模型輸出的結果結構化，便於後續顯示與使用。

3. FreeRTOS 的多任務架構

(1) 功能用途:

- a. 從任務佇列中接收推論請求 (xQueueReceive)，負責處理推論作業。
- b. 執行推論後處理，並將結果回傳給發送者 (xQueueSend)。

(2) 程式範例:

```
void inferenceProcessTask(void *pvParameters)
{
    struct ProcessTaskParams params = *reinterpret_cast<struct ProcessTaskParams *>(pvParameters);

    InferenceProcess::InferenceProcess inferenceProcess(params.model);

    for (;;)
    {
        xInferenceJob *xJob;

        xQueueReceive(params.queueHandle, &xJob, portMAX_DELAY); // 等待推論任務

        inferenceProcess.RunJob(
            xJob->pPostProc,
            xJob->modelCols,
            xJob->modelRows,
            xJob->srcImgWidth,
            xJob->srcImgHeight,
            xJob->results
        );

        xQueueSend(xJob->responseQueue, &xJob, portMAX_DELAY); // 回傳結果
    }
}
```

圖 163、多任務架構程式範例

(3) 架構優勢:

- a. 提高執行效率，利用任務佇列進行資料的有序處理。
- b. 確保任務執行過程中的資源安全，避免多任務間的衝突。

(三) 後處理程式解析 (DetectorProcessing.cpp)

1. 重點功能介紹

(1) 初始化後處理參數:

設置後處理所需的參數，例如偵測門檻值、非極大值抑制 (NMS) 的 IOU 閾值、分類數量與 Top-N 選項。

(2) 處理模型輸出:

使用 YOLO 模型輸出的 Tensor，解析偵測框的位置與大小，並篩選符合門檻值的物體。

(3) 非極大值抑制 (NMS):

移除重疊過高的偵測框，僅保留最相關的結果。

(4) 轉換至原始影像尺寸:

將模型輸出的座標框比例轉換至原始影像大小，得出真實的物體位置。



圖 164、後處理流程圖

2. 後處理初始化

(1) 參數定義與意義:

- a. threshold: 偵測門檻值，低於此值的偵測框會被忽略。
- b. nms: 非極大值抑制的 IOU 閾值，用於移除多餘的重複框。
- c. numClasses: 模型支援的分類數量。
- d. topN: 返回的最高信心度結果數量。

(2) 程式範例

```
DetectorPostprocessing::DetectorPostprocessing(  
    const float threshold,  
    const float nms,  
    int numClasses,  
    int topN)  
: m_threshold(threshold),  
  m_nms(nms),  
  m_numClasses(numClasses),  
  m_topN(topN) {}
```

圖 165、後處理初始化程式範例

(3) 功能:

初始化 YOLO 模型的後處理參數，為後續處理過程提供基礎配置。

3. 執行後處理

(1) 功能說明:

- a. 初始化 YOLO 模型輸出的結構化資料 (Network)。
- b. 使用 modelOutput0 提取 Tensor，包括邊框座標與分類資訊。

c. 使用 scale 與 zeroPoint 將量化模型輸出轉換為浮點數格式。

(2) 程式範例:

```
void DetectorPostprocessing::RunPostProcessing(  
    uint32_t imgNetRows,  
    uint32_t imgNetCols,  
    uint32_t imgSrcRows,  
    uint32_t imgSrcCols,  
    TfLiteTensor *modelOutput0,  
    std::vector<DetectionResult> &resultsOut    /* init postprocessing */  
)
```

```
Network net  
{  
    .inputWidth = static_cast<int>(imgNetCols),  
    .inputHeight = static_cast<int>(imgNetRows),  
    .numClasses = m_numClasses,  
    .branches = {  
        Branch {  
            .resolution = modelOutput0->dims->data[1],  
            .modelOutput = modelOutput0->data.int8,  
            .scale = ((TfLiteAffineQuantization *) (modelOutput0->quantization.params))->scale->data[0],  
            .zeroPoint = ((TfLiteAffineQuantization *) (modelOutput0->quantization.params))->zero_point->data[0],  
            .size = modelOutput0->bytes  
        }  
    },  
    .topN = m_topN  
}
```

圖 166、執行後處理程式範例

4. 偵測框解析與篩選

(1) 處理邏輯:

- a. 從模型輸出中解析每個偵測框的座標、大小和信心度。
- b. 篩選信心度高於 threshold 的框，並將座標框從模型輸入比例轉換至原始影像比例。

(2) 程式範例:

```
GetNetworkBoxes(  
    net,  
    originalImageWidth,  
    originalImageHeight,  
    m_threshold,  
    detections);
```

圖 167、偵測框解析程式範例

5. 非極大值抑制 (NMS)

(1) 目的:

- a. 移除重疊率 (IOU) 高於 nms 閾值的重複框，只保留最高信心度的框。
 - b. 確保每個物體僅對應一個框。
-

6. 轉換至原始影像尺寸

(1) 轉換邏輯:

根據模型輸出的框座標與模型輸入影像的比例，計算對應原始影像的位置與大小。

(2) 程式範例:

```
for (auto &it : detections)
{
    it.bbox.x = (it.bbox.x * originalImageWidth) / net.inputWidth;
    it.bbox.y = (it.bbox.y * originalImageHeight) / net.inputHeight;
    it.bbox.w = (it.bbox.w * originalImageWidth) / net.inputWidth;
    it.bbox.h = (it.bbox.h * originalImageHeight) / net.inputHeight;
}
```

圖 168、轉換影像尺寸程式範例

7. 最終結果生成

(1) 生成內容:

- 框的起點座標 (m_x0, m_y0) 和大小 (m_w, m_h)。
- 偵測信心度 (m_normalisedVal)。
- 物體分類 (m_cls)。

(2) 程式範例:


```
DetectionResult tmpResult = {};  
tmpResult.m_normalisedVal = it.prob[j];  
tmpResult.m_x0 = (int)boxX;  
tmpResult.m_y0 = (int)boxY;  
tmpResult.m_w = (int)boxWidth;  
tmpResult.m_h = (int)boxHeight;  
tmpResult.m_cls = j;  
  
resultsOut.push_back(tmpResult);
```

(四) 主迴圈 (Main Loop) 分析與實現

主迴圈是系統的核心模塊，負責調度資料流、執行推論任務及處理輸出結果。以下為其詳細分析與實現過程。

主迴圈結構與流程

1. 接收數據 (回到資料準備階段)

- (1) 功能: 主迴圈檢查是否有新的數據需要處理，並從緩衝區中獲取完整的幀圖像。
 - (2) 操作步驟:
 1. 獲取空間緩衝區 (函數: `get_empty_framebuf()`)。
 2. 獲取完整緩衝區 (函數: `get_full_framebuf()`)。
-

2. 推論執行 (回到模型推論階段)

- (1) 功能: 當緩衝區的數據準備就緒後，將數據發送至推論任務，執行模型推論。
 - (2) 實現方式: 使用 TensorFlow Lite 模型執行推論 (`RunInference()` 方法)。
 - (3) 主迴圈目標: 等待推論結果完成並進行後續處理。
-

3. 結果後處理 (回到後處理階段)

- (1) 功能:
 - a. 對輸出數據進行後處理，移除冗餘框 (Non-Maximum Suppression, NMS)。

- b. 將坐標從模型輸入空間轉換到原始影像空間。
- c. 最佳化有效的檢測結果，便於顯示。

4. 顯示與結果輸出 (回到結果處理與輸出階段)

(1) 功能:

- a. 將處理完成的檢測結果顯示在輸出設備 (如 LCD 屏幕或序列埠)。
- b. 更新結果緩衝區狀態 (設置為 eFRAMEBUF_EMPTY 表示處理完成)。

5. Main Loop 問題及總結

(1) 主迴圈運行錯誤處理:

- a. 若數據損壞或無效，需重試以獲取有效數據。
- b. 若模型初始化或執行失敗，需重設模型並重新啟動推論任務。
- c. 若後處理參數錯誤，需調整輸入參數或模型輸出配置。

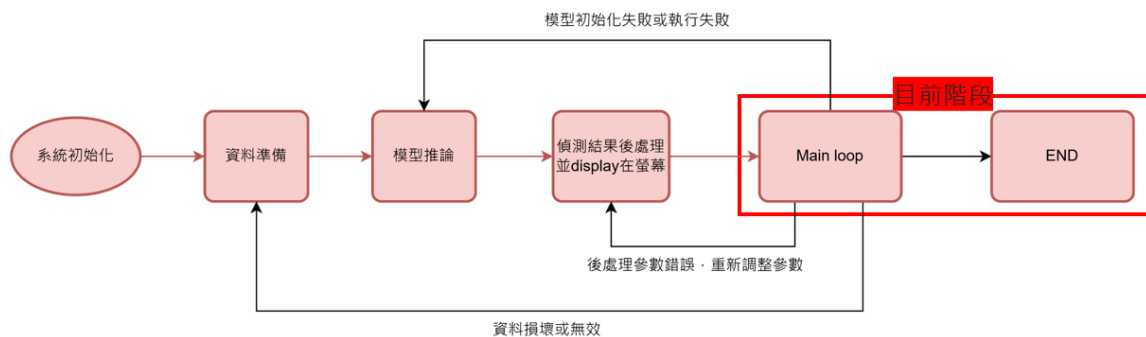


圖 169、Main Loop 流程圖

(五) 關鍵程式解析 (Main.cpp)

1. 影像緩衝區管理

- (1) 功能: 管理不同狀態的緩衝區以便影像填充、推論或結果處理。

```
static S_FRAMEBUF *get_empty_framebuf();  
static S_FRAMEBUF *get_full_framebuf();  
static S_FRAMEBUF *get_inf_framebuf();
```

圖 170、影像緩衝程式解析

- (2) 函數:

- a. get_empty_framebuf(): 獲取空緩衝區。
- b. get_full_framebuf(): 獲取已填充的緩衝區。
- c. get_inf_framebuf(): 獲取推論中的緩衝區。

2. 模型初始化

- (1) 操作步驟:

- a. 初始化 YOLO 模型並分配 Tensor 緩衝區。
- b. 載入模型標籤與長度資訊。

```
arm::app::YoloXnanoNu model;  
model.Init(arm::app::tensorArena, sizeof(arm::app::tensorArena),  
           arm::app::yoloxnanonu::GetModelPointer(), arm::app::yoloxnanonu::GetModelLen());
```

圖 171、模型初始化程式解釋

3. 輸入影像處理

(1) 功能: 將影像縮放到模型要求的尺寸，並進行格式轉換 (如轉換為 int8 格式)。

(2) 程式範例:

```
imlib_nvt_scale(&fullFramebuf->frameImage, &resizeImg, &roi);  
if (model.IsDataSigned()) {  
    arm::app::image::ConvertImgToInt8(inputTensor->data.data, inputTensor->bytes);  
}
```

圖 172、影像處理程式範例

4. 推論觸發

(1) 功能: 利用 FreeRTOS 隊列 (Queue) 傳遞推論任務，根據當前影像進行物體檢測。

5. 後處理與結果呈現

(1) 功能:

- a. 繪製檢測結果框與標籤 (函數: DrawImageDetectionBoxes())。
- b. 顯示結果資訊 (函數: PresentInferenceResult())。

```
DrawImageDetectionBoxes(infFramebuf->results, &infFramebuf->frameImage, labels);  
PresentInferenceResult(infFramebuf->results, labels);
```

圖 173、後處理與結果顯示

6. 主迴圈邏輯

(1) 運行邏輯:

- a. 不斷檢查緩衝區狀態並執行相應處理。
- b. 為避免占用過多 CPU 資源，每次迴圈間加入延遲 (vTaskDelay(1))。

```
while (1) {  
    infFramebuf = get_inf_framebuf();  
    fullFramebuf = get_full_framebuf();  
    emptyFramebuf = get_empty_framebuf();  
    // 推論、後處理與顯示邏輯...  
    vTaskDelay(1);  
}
```

圖 174、Main Loop 邏輯

六、 未來展望：

可透過 HyperRam 來訓練一個大的 model，可以對於更多的 class 進行辨識，並且提升準確率，最後可以安裝在垃圾輸送帶上，搭配自動化機械來實現，快速準確的全自動垃圾分類。

參考資料：

[1]labelimg 教學: <https://hackmd.io/@osense-rd-public/H1ekDPqBt>

[2]anaconda 環境建立: https://hackmd.io/@nB1rzit6Toq8WdkZosRK_Q/r16-c7Ynj

[3]CUDA Toolkit: <https://developer.nvidia.com/cuda-toolkit>

[4]Netron: <https://netron.app/>

柒、智慧家居 2 - 人員追蹤

一、案例介紹 - 人員追蹤

本案例的核心目標是基於新唐 M55M1 的硬體平台，結合機器學習模型，實現智慧居家人員檢測。該技術的特點包括：

高精度的目標檢測能力，能準確識別居家環境中的人員。

實時推論，適合邊緣運算需求。

低功耗設計，適合長時間運行。

本平台應用場景廣泛，例如安全監控、老人護理及資源管理等。

右圖展示了新唐 M55M1 平台的人員檢測實際應用畫面，顯示其可以即時處理並顯示檢測結果。該平台的設計符合嵌入式系統應用需求，並具備出色的計算效能與硬體兼容性。

需求

人員檢測系統的主要需求在於提高安全性，尤其是在工廠、施工現場等具有潛在危險的環境中。該系統需能準確監控人員進入或離開特定區域的情況，確保符合安全規範。以下是其核心需求：

安全性：在危險區域實施監控，防止未授權人員進入。

實時性：提供即時檢測與警報功能。

易用性：系統需易於安裝與操作，適用於多種環境。

使用場景舉例

以下為人員檢測系統在實際應用中的案例：

1. 工業生產線：禁止無關人員進入作業區域。在作業區域安裝檢測設備，用於監測是否有未授權人員進入，並在檢測到異常時即時觸發警報。
2. 施工現場：監控人員進入高危區域，防止意外發生，並記錄相關活動數據以

供後續分析。

案例介紹 - 案例預期結果

本案例基於 TinyML 技術進行模型訓練與部署，並分別在 PC 端和燒錄至開發板後進行測試，取得以下結果：模型於 PC 端驗證數據集的測試準確率達到 97%（143/148），顯示模型在理想環境下具有高可靠性。燒錄至開發板後，測試準確率達到 92.5%（136/148），表明該模型在實際應用環境中仍能保持良好的性能。

1.技術流程

為實現模型在開發板上的高效運行，該案例採用了以下技術流程：

- (1) TinyML 訓練：基於邊緣運算需求設計輕量化模型，並進行訓練與測試。
- (2) 輕量化模型與轉換：對模型進行壓縮與最佳化，確保其適配於低算力設備。
- (3) 模型 Vela 編譯與配置：利用 Arm®的 Vela 工具進行模型編譯，為開發板提供最佳化支持。
- (4) C 語言程式撰寫：編寫相應的推論程式碼，整合模型與硬體介面。
部署至開發板：將最終模型與程式燒錄至開發板，並進行實地測試與調試。

二、模型比較

(一) 模型大小與推論速度

在本案例中，模型選擇的關鍵在於兼顧性能與資源需求，特別是針對低算力設備的最佳化能力。以下是模型的詳細比較：

1. YOLOX Nano (本專案使用)

YOLOX Nano 為本專案選用的模型，其特點如下：

- (1) 極小的模型大小：僅約 0.91M 參數量，適合部署在資源有限的硬體上。
高效能與低功耗：針對低算力設備進行最佳化，適用於即時應用，如智慧攝影機、無人機等場景。
- (2) 推論速度快：在內建 NPU 算力平台（如 M55M1）上表現出色，能夠快速處理任務並提供即時結果。

2. 訓練效率

在模型的訓練過程中，訓練效率是評估其實用性的重要指標。以下是本專案選用的 YOLOX Nano 與其他 YOLO 模型在訓練效率上的比較：

(1) YOLOX Nano (本專案使用)

YOLOX Nano 的訓練效率具有以下特點：

- a. Anchor-free 結構：簡化了訓練過程，減少了對 anchor 的依賴，從而簡化了超參數的調整。
- b. 資料集適配性強：相較於其他 YOLO 模型，YOLOX Nano 更容易適配新的資料集，特別是在應對動態變化的數據時。

(2) 其他 YOLO 模型

其他 YOLO 模型的訓練效率特性：

- a. YOLOv4 / YOLOv5：採用 Anchor-based 方法，對於資料標註格式和 anchor 設置有較高的要求，因此可能需要更多的超參數調整才能達到最佳效果。
- b. YOLOX 標準版本：同樣採用 Anchor-free 方法，與 YOLOX Nano 的訓練流程一致，但由於模型更大，需要更高的計算力來支持其訓練過程，適合於高性能硬體環境。

3. 應用場景

不同 YOLO 模型的應用場景各有側重，根據其設計特性與效能需求，以下是常見的應用場景介紹：

(1) YOLOX Nano (本專案使用)

YOLOX Nano 適用於低算力或即時性要求高的場景，以下是一些具體應用：

- a. 智慧家庭：如智慧門鎖和人員偵測，能快速響應環境變化。
- b. 移動設備：在手機 App 中進行物體偵測，結合即時運算功能。
- c. 工業邊緣計算：適用於小型攝影機等邊緣設備的即時偵測需求，如在工業生產線上進行目標物件的快速檢測。

(2) 其他 YOLO 模型

- a. YOLOX 標準版：適合於專業應用，如自駕車感測、商業監控系統等，對精度要求較高的場合。

三、LabelImg

LabelImg 是一款開源的圖像標註工具，用於為物體檢測任務創建標註數據集。以下是安裝和使用 LabelImg 的步驟：

(一) 安裝 LabelImg

確保已安裝 Python 環境，並使用 pip 安裝 LabelImg 套件。

執行以下命令以安裝：

```
$ pip install labelImg
```

(二) 執行 LabelImg

安裝完成後，可通過以下命令啟動 LabelImg：

在終端中輸入：

```
$ labelImg
```

LabelImg 啟動後，您可以加載圖片並進行框選與標註，生成所需的標註文件（如 COCO JSON 或 PASCAL VOC XML 格式）。

(三) 參考教學

若需要進一步的操作指導，可以參考以下教學資源：

- [LabelImg 使用步驟]

<https://blog.csdn.net/knighthood2001/article/details/125883343>

四、在 PC 上使用 Anaconda 環境進行模型訓練

(一) 訓練硬體規格

1. GPU：Nvidia GTX 1080 Ti

- (1) 可透過以下方法確認 GPU 規格：
- (2) 方法一：在終端輸入 `nvidia-smi` 指令，即可顯示顯卡型號與規格。
- (3) 方法二：打開 **工作管理員**，切換至 **效能** 分頁，找到 GPU 資訊以確認型號與使用狀況。

2. 透過 GPU 找適合的 CUDA 與 Torch 版本：

- (1) 使用以下網站查詢顯卡所支持的 CUDA 版本：
PyTorch CUDA 支援查詢
-

(二) 軟體與版本需求

1. **CUDA**：11.8
 2. **Torch**：2.0.0
 3. **Python**：3.10
-

(三) 訓練設定與結果

1. **訓練時長**：
模型訓練所需時間約為 **1.5 小時**。
2. **訓練數據**：
使用約 **600 張圖片** 作為訓練資料。

(四) 訓練環境

步驟一：安裝 Anaconda3

前往 [Anaconda 官方網站](#) 下載並安裝：Anaconda 下載頁面。

安裝完成後，確保 Anaconda 的 conda 指令可正常運行。

步驟二：建立新環境

開啟 Anaconda Prompt（終端）。

建立專屬環境並指定 Python 版本為 3.10

```
$ conda create --name yolox_nu python = 3.10
```

啟動該環境

```
$ conda activate yolox_nu
```

步驟三：下載 YOLOX Nano 的 GitHub 資源

使用 GitHub 命令下載 YOLOX Nano 的專案 zip 檔案：

```
$ gh repo clone MaxCYCHEN/yolox-ti-lite_tflite_int8
```

步驟四：更新 Pip 與相關工具

確保 Pip 與 Setuptools 為最新版本，避免安裝過程中出現相依性問題

```
$ python -m pip install --upgrade pip setuptools
```

(五) 安裝環境

步驟一：安裝基礎框架

1. 安裝 PyTorch、CUDA 與 MMCV

(1) PyTorch、CUDA、MMCV 版本需相互匹配。若使用 **CPU** 訓練，可忽略 CUDA 的安裝。

(2) 使用 GPU 訓練時，需根據自身 GPU 規格選擇對應版本：

CUDA 版本：**11.8**

Torch 版本：**2.0.0**

MMCV 安裝指令如下

```
$ python -m pip install mmcv==2.0.1 -f
```

<https://download.openmmlab.com/mmcv/dist/cu118/torch2.0/index.html>

步驟二：安裝其他 Python 套件

透過 **requirements.txt** 安裝依賴項：

使用以下指令，自動安裝所有所需套件：

```
$ python -m pip install --no-input -r requirements.txt
```

步驟三：安裝 YOLOX

執行 YOLOX 安裝：

在 YOLOX 檔案目錄中運行以下指令完成安裝：

```
$ python setup.py develop
```

(六) YOLOX_Nano 模型訓練

1. 模型特性 – 限制條件

- (1) 由於模型大小限制，在無外接 Hyper RAM 的情況下，只能使用 **2MB** 以下的模型。
- (2) 本案例選用 **YOLOX_Nano_ti_lite_nu**，其大小僅 **0.91MB**，是最適合 **M55M1** 開發板的模型。

2. 訓練步驟

執行訓練指令：

此指令針對模型配置檔案與預訓練模型進行訓練。

```
$ python tools/train.py -f exps/default/yolox_nano_ti_lite_nu.py -d 1 -b 64 --fp16 -o -c  
pretrain/tflite_yolox_nano_ti/320_DW/yolox_nano_320_DW_ti_lite.pth
```

自定義訓練參數：

若需調整訓練參數，可使用以下格式指令：

```
$ python tools/train.py -f <MODEL_CONFIG_FILE> -d 1 -b <BATCH_SIZE> --fp16 -o  
-c <PRETRAIN_MODEL_PATH>
```

3. 設置參數解釋

MODEL_CONFIG_FILE：

指向模型配置檔案，需指定下載的 YOLOX Nano 中的 yolox-nano-ti-nu 模型配置檔。

BATCH_SIZE：

設置訓練批量大小，預設為 64。若系統資源不足，可調低至 32 或 16。

PRETRAIN_MODEL_PATH：

放置預訓練模型的權重檔案，此檔案可從 YOLOX Nano 預訓練模型目錄下載。

(七)Train Dataset Format

1. 資料集結構

(1) 資料集按照以下結構分類，主要包含以下目錄與檔案：

Annotations（標註檔案）：

包含 **coco.json** 格式的訓練與驗證標註檔案。

Train2017（訓練圖片）：

存放訓練圖片。

Val2017（驗證圖片）：

存放驗證圖片。

資料集目錄結構範例

```
datasets/<dataset_name>
  annotations
    train_annotation_json_file
    val_annotation_json_file
  train2017
    train img
  val2017
    validation img
```

(2) 設定 Dataset 路徑

在程式中設定資料集路徑，範例如下

```
self.data_dir = "datasets/hagrid_coco "
self.train_ann = "hagrid_train.json"
self.val_ann = "hagrid_val.json"
```

(八) 訓練的自定義參數 (train.py 檔)

1. 調整的主要參數

圖片大小 (Resolution) :

`self.input_size` 和 `self.test_size` : 定義訓練與測試圖片的解析度。範例設定為 **320×320**。

類別數量 (Classes) :

`self.num_classes` : 設定需要辨識的標註種類總數。
範例設定為 **11 個類別**。

Epoch 次數 :

`self.max_epoch` : 指定訓練資料集通過模型的完整次數，範例設定為 **150 次**。

注意 :

Epoch 次數不一定越多越好，建議在 **150-200** 範圍內進行預設。

若次數過多可能導致 **過擬合 (Overfitting)**，反而降低模型的準確率。

2. 參數程式碼範例

```
self.input_size = (320, 320)
self.test_size = (320, 320)
self.num_classes = 1
self.max_epoch = 150
```

以下為對應的程式碼設定

範例說明

類別數量： 設定為 11 個類別，表示需要辨識 11 種物體。

Epoch 次數： 訓練執行 150 個完整的 Epoch，適合避免過擬合的情況。

(九) PyTorch to ONNX

1. 模型轉換流程

從 PyTorch 模型轉換為 ONNX 格式

使用以下指令完成轉換：

```
$ python tools/export_onnx.py -f <MODEL_CONFIG_FILE> -c  
<TRAINED_PYTORCH_MODEL> --output-name <ONNX_MODEL_PATH>
```

MODEL_CONFIG_FILE：用於描述模型結構與參數，通常位於 YOLOX Nano 資料夾中。

TRAINED_PYTORCH_MODEL：已訓練完成的 PyTorch 模型檔案（格式為 .pth），作為轉換來源。

ONNX_MODEL_PATH：轉換後的 ONNX 模型儲存路徑與檔名。

2. 範例指令：

```
$ python tools/export_onnx.py -f exps/default/yolox_nano_ti_lite_nu.py -c  
YOLOX_outputs/yolox_nano_ti_lite_nu/latest_ckpt.pth --output-name  
YOLOX_outputs/yolox_nano_ti_lite_nu/vox_nano_nu_medicine.onnx
```

3. 範例解析：

exps/default/yolox_nano_ti_lite_nu.py：模型結構檔案的路徑。

YOLOX_outputs/yolox_nano_ti_lite_nu/latest_ckpt.pth：最新訓練完成的 PyTorch 模型權重檔。

YOLOX_outputs/yolox_nano_ti_lite_nu/vox_nano_nu_medicine.onnx：轉換後的 ONNX 模型儲存路徑與檔名。

(十) ONNX to TFLite

1. 模型轉換流程

將 ONNX 模型轉換為適用於 TFLite 框架的格式，執行以下指令進行轉換：

2. 指令範例

```
onnx2tf -i <ONNX_MODEL_PATH> -oiqt -qcind images <CALI_DATA_NPY_FILE>  
"[[[0,0,0]]]"
```

ONNX_MODEL_PATH：ONNX 模型檔案的路徑。

CALI_DATA_NPY_FILE：校準資料檔案的路徑，通常為 .npz 格式的 NumPy 檔案。

3. 實際範例指令

```
onnx2tf -i YOLOX_outputs/yolox_nano_ti_lite_nu/vox_nano_nu_medicine.onnx -oiqt -  
qcind images YOLOX_outputs/yolox_nano_ti_lite_nu/calib_data_320x320_n200.npz  
"[[[0,0,0]]]"
```

ONNX_MODEL_PATH：

將 ONNX 模型的儲存路徑替換為您的實際檔案路徑，例如：

YOLOX_outputs/yolox_nano_ti_lite_nu/vox_nano_nu_medicine.onnx。

CALI_DATA_NPY_FILE：

校準資料的 NumPy 檔案路徑，範例中為：

YOLOX_outputs/yolox_nano_ti_lite_nu/calib_data_320x320_n200.npz。

校準參數：

[0,0,0] 為校準值，可根據具體需求進行調整。

(十一) Vela Compiler and Convert to Deployment Format

使用 **Vela Compiler** 進行模型轉換

1. 設定模型路徑與輸出路徑：

使用以下指令進行設定

```
$ set MODEL_SRC_FILE = <your tflite model>
```

```
$ set MODEL_OPTIMISE_FILE = <output vela model>
```

<your tflite model>：指向原始 TFLite 模型檔案 (.tflite 格式)。

<output vela model>：設定 Vela 編譯器輸出的最佳化後模型檔案。

2. 輸出檔案 (Output File)

編譯完成後，輸出檔案會位於以下路徑

```
vela\generated\yolox_nano_ti_lite_nu_full_integer_quant_vela.tflite
```

此檔案將用於後續部署處理，方便於嵌入式平台運行。

五、 開發版上的推論程式系統流程圖

本節介紹開發版上推論程式系統的完整流程，包含從系統初始化到結果顯示的各個步驟。

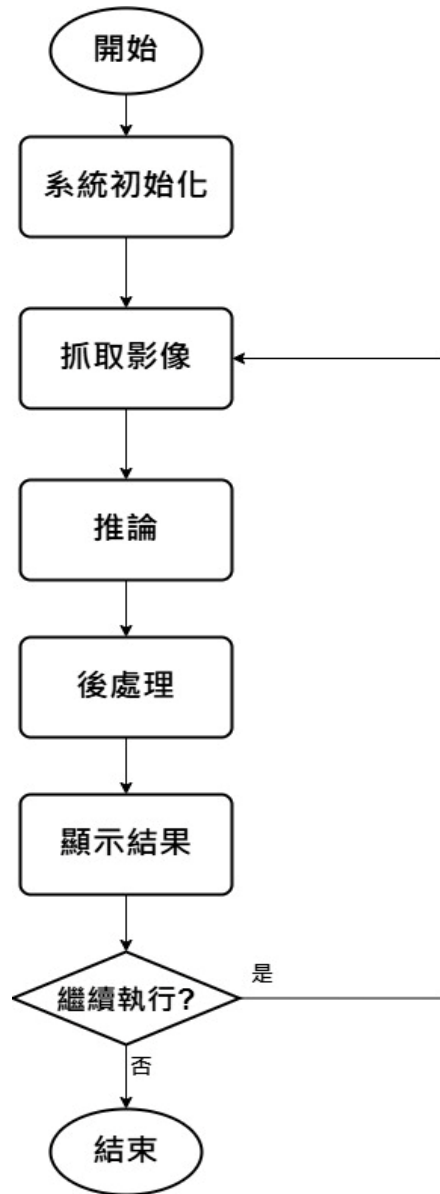


圖 175 、 系統流程圖

1.系統流程

(1) 系統初始化 (System Initialization)：設置系統硬體與軟體環境，載入模

型與配置文件，準備推論環境。

- (2) 捕捉影像 (Capture Image)：通過相機模組或其他影像輸入設備捕捉即時影像作為模型輸入數據。
- (3) 推論 (Inference)：使用已部署的模型對輸入影像進行推論，生成初步檢測結果。
- (4) 後處理與繪製檢測結果 (Post-Processing & Draw Results)：對模型輸出的數據進行後處理，例如解碼邊界框、分類標籤等，並將檢測結果繪製在影像上。
- (5) 顯示結果 (Display Results)：在顯示設備上輸出處理後的影像與檢測結果，供用戶觀察。

流程結束後，系統會根據用戶需求選擇是否繼續執行下一輪推論，或結束整個過程。

2. 流程圖說明

本節進一步詳細說明推論程式流程圖中的每個步驟及其對應的操作邏輯。

- (1) 開發板周邊初始化：

驗證周邊設備（如感測器、相機等）的初始化是否成功。

若初始化成功，系統進入影像偵測模式；若失敗，則返回檢查硬體或軟體配置。

- (2) 啟動影像捕捉：

通過相機模組啟動影像捕捉功能，準備即時輸入數據。

捕捉到的影像會立即傳遞給燒錄的深度學習模型進行處理。

- (3) 進行推論：

系統使用已部署的模型對輸入影像進行推論，識別並定位目標物體（如人物）。

模型輸出初步的檢測結果，包括物體位置與類別標籤。

(3) 進入後處理：

將推論結果進行後處理，對識別到的目標進行邊界框繪製與分類結果標註。

若檢測到人物，則將人物圖像框起來以標示其位置與範圍。

(4) 輸出結果：

將繪製好的檢測結果顯示在屏幕上，供用戶觀察。

(5) 流程控制：

根據用戶需求，選擇是否繼續執行後續影像偵測。若選擇繼續，則返回影像捕捉步驟；若選擇結束，則結束程式執行。

3. 系統初始化

系統初始化是推論系統運行的第一步，主要負責配置硬體資源，確保其他模組能夠穩定運行。以下詳細說明初始化流程及 `BoardInit` 函數的具體功能與實現。

(1) `BoardInit()` 函數

`BoardInit()` 是一個關鍵函數，負責執行硬體相關的初始化操作，確保基本硬體資源準備完成。執行的目的是：

確保整個系統其他模組（如影像處理、神經網路推理）的運行有穩定的

CLK 與可靠的通訊機制。提供基本的輸入/輸出功能支持。

(2) 核心程式碼解釋

`BoardInit` 函數的部分程式碼及其解釋：

```

static void SYS_Init(void)
{
    /*-----
    /* Init System Clock
    /*-----

    /* Enable Internal RC 12MHz clock */
    CLK_EnableXtalRC(CLK_SRCCTL_HIRCEN_Msk);

    /* Waiting for Internal RC clock ready */
    CLK_WaitClockReady(CLK_STATUS_HIRCSTB_Msk);

    /* Enable HXT clock */
    CLK_EnableXtalRC(CLK_SRCCTL_HXTEN_Msk);

    /* Waiting for HXT clock ready */
    CLK_WaitClockReady(CLK_STATUS_HXTSTB_Msk);

    /* Switch SCLK clock source to APLL0 and Enable APLL0 180MHz clock */
    CLK_SetBusClock(CLK_SCLKSEL_SCLKSEL_APLL0, CLK_APLLCTL_APLLSRC_HIRC, FREQ_180MHZ);

    /* Update System Core Clock */
    /* User can use SystemCoreClockUpdate() to calculate SystemCoreClock. */
    SystemCoreClockUpdate();
}

```

```

int BoardInit(void)
{
    /* Unlock protected registers */
    SYS_UnlockReg();

    SYS_Init();

    /* UART init - will enable valid use of printf (stdout
    | * re-directed at this UART (UART6) */
    InitDebugUart();

    SYS_LockReg();          /* Unlock register lock protect */

    HyperRAM_Init(HYPERRAM_SPIM_PORT);
    /* Enter direct-mapped mode to run new applications */
    SPIM_HYPER_EnterDirectMapMode(HYPERRAM_SPIM_PORT);

    info("%s: complete\n", __FUNCTION__);
}

```

圖 176、boardinit()函數

這些步驟保證系統硬體的穩定性與功能可用性。

4. 捕捉影像程式片段說明

影像捕捉是推論系統的重要步驟之一，通過管理緩衝區（frame buffer）來儲存捕捉到的影像數據。以下是影像捕捉程序中的核心邏輯與函數實現。

緩衝區（frame buffer）

緩衝區是用於暫存捕捉到的影像數據的記憶體區域。在系統運行過程中，需要根據緩衝區的狀態來查找符合條件的緩衝區以儲存新的影像數據。

(1) get_empty_framebuf 函數

該函數的主要作用是：

檢查緩衝區列表，尋找處於空閒狀態的緩衝區
（eFRAMEBUF_EMPTY）。

返回找到的空閒緩衝區的指標，用於儲存即將捕捉的影像數據。

以下是程式碼示例：

```
//frame buffer managemnet function
static S_FRAMEBUF *get_empty_framebuf()
{
    int i;

    for (i = 0; i < NUM_FRAMEBUF; i ++ )
    {
        if (s_asFramebuf[i].eState == eFRAMEBUF_EMPTY)
            return &s_asFramebuf[i];
    }

    return NULL;
}
```

圖 177、get_empty_framebuf()函數

該函數循環檢查所有緩衝區的狀態，返回第一個處於空閒狀態的緩衝區；如果找不到，則返回 NULL。

(2) ImageSensor_Capture 函數

該函數負責完成影像捕捉的核心操作，流程如下：

- a. 調用 `get_empty_framebuf` 函數獲取一個空閒緩衝區。
- b. 通過影像傳感器捕捉新影像，並將其儲存在選定的緩衝區中。
- c. 處理捕捉過程中的錯誤，例如未能獲取有效的影像數據時進行報錯。

以下是程式碼示例：

```
void ImageSensor_Capture(uint32_t* frameImageData) {
    S_FRAMEBUF* emptyFramebuf = get_empty_framebuf();
    if (emptyFramebuf) {
        const uint8_t* pu8ImgSrc = get_img_array(u8ImgIdx);
        if (pu8ImgSrc == NULL) {
            printf("Failed to get image index");
            return;
        }
        memcpy(emptyFramebuf->frameImage.data, pu8ImgSrc, IMAGE_SIZE);
    }
}
```

圖 178、ImageSensor_Capture()函數

通過該函數，捕捉的影像數據將被妥善儲存在緩衝區，供後續的推論模組使用。

5.推論

推論是整個系統的核心部分，負責對捕捉到的影像進行分析，輸出檢測結果。以下將詳細介紹推論過程中涉及的核心函數與相關邏輯。

(1) get_full_framebuf 函數

該函數用於從已填滿的緩衝區中獲取一個影像數據，用於後續的推論操作。

以下是該函數的主要邏輯：

```

static S_FRAMEBUF *get_full_framebuf()
{
    int i;

    for (i = 0; i < NUM_FRAMEBUF; i++)
    {
        if (s_asFramebuf[i].eState == eFRAMEBUF_FULL)
            return &s_asFramebuf[i];
    }

    return NULL;
}

```

圖 179、get_full_frameput()函數

該函數循環檢查所有緩衝區的狀態，返回第一個處於填滿狀態的緩衝區；如果找不到，則返回 NULL。

(2) 推論程式邏輯

推論過程包括以下步驟：

獲取一個已填滿的緩衝區作為輸入影像數據。

配置推論任務，包括輸入影像大小、模型行列數、後處理配置等。將配置完成的推論任務加入推論隊列，等待執行。

以下是推論的主要程式碼：

通過這些步驟，系統可以高效地完成推論任務，並為後處理提供基礎數據。

(3) PresentInferenceResult 函數

推論過程的最後一步是將檢測結果可視化或輸出，以便用戶理解。

PresentInferenceResult 函數的主要作用是格式化並輸出每一個檢測到的物件，包括其類別標籤和位置資訊。

以下是該函數的程式碼：

```
static bool PresentInferenceResult(const std::vector<arm::app::object_detection::DetectionResult> &results,
                                  std::vector<std::string> &labels)
{
    /* If profiling is enabled, and the time is valid. */
    info("Final results:\n");

    for (uint32_t i = 0; i < results.size(); ++i)
    {
        info("%" PRIu32 " ") %s(%f) -> %s {x=%d,y=%d,w=%d,h=%d}\n", i,
            labels[results[i].m_cls].c_str(),
            results[i].m_normalisedVal, "Detection box:",
            results[i].m_x0, results[i].m_y0, results[i].m_w, results[i].m_h);
    }

    return true;
}
```

圖 180、PresentInferenceResult() 函數

這一函數的主要功能是为推論結果生成易於理解的輸出形式，通常用於調試或為用戶提供檢測數據的摘要。

6. 後處理 - Non-Maximum Suppression (NMS)

後處理是目標檢測系統的重要組成部分，用於精煉推論結果。該過程主要包括以下幾步：

(1) 縮放偵測框 (Scaling Detection Boxes)：

將推理網路的輸出從模型輸出的尺度映射回原始影像大小。

確保輸出的偵測框與輸入影像的比例一致。

(2) 提取偵測結果 (Extracting Detection Results)：

從模型輸出的張量中提取出每個物件的邊界框座標與分類機率。

根據物件的分類分數進行排序。

(3) 執行 NMS (Non-Maximum Suppression)：

NMS 是目標檢測後處理的核心步驟，用於過濾重疊過多的框。

它會根據分類機率選擇最有可能的框，並排除多餘的重疊框。

此步驟可以有效提升偵測結果的準確性，減少重複檢測。

(4) 結果輸出 (Output Results)：

最後，將篩選後的偵測結果儲存到指定數據結構（如 `resultsOut` 向量）中，以便進行顯示或進一步分析。

(5) 以下為 Non-Maximum Suppression (NMS) 的基本邏輯：

按物件的分類分數對所有框排序。

選擇分數最高的框作為最終框，移除與其 IoU（交並比）大於閾值的其他框。

重複上步驟直到所有框處理完畢。

捌、智慧農業 1 – 蘋果品質辨識

一、 案例介紹 – 蘋果品質辨識

摘要

本案例的核心目標是基於新唐 M55M1 的硬體平台，結合機器學習模型，實現智慧農業蘋果品質辨識。該技術的特點包括：

1. 高精度的目標檢測能力，能準確識別蘋果品質。
2. 支持實時推論，適合邊緣運算需求。
3. 低功耗設計，適合長時間運行。

該平台的設計符合嵌入式系統應用需求，並具備出色的計算效能與硬體兼容性。

需求

1. 標準化

在蘋果品質辨識中，標準化是核心需求之一。傳統上，蘋果的分級往往依賴人工判斷，這容易受到個人主觀因素的影響，導致品質分級的誤差和不一致性。通過機器學習與電腦視覺技術，系統可以根據統一的標準進行品質分類，避免人工分級過程中的主觀誤差，保證每一顆蘋果的分級都符合相同的標準，從而提升品質控制的準確度與一致性。

2. 高效率

隨著現代化生產需求的提高，尤其是在大規模生產線上，蘋果品質辨識系統必須具備高效率的特性。系統應該能夠實現快速檢測，並在短時間內處理大量的蘋果影像數據。這不僅能提升生產效率，還能減少生產過程中的等待時間與浪費，為生產線帶來顯著的效益提升。

使用場景舉例

蘋果分揀與包裝生產線：

在蘋果的生產過程中，蘋果分揀和包裝是至關重要的步驟。傳統的蘋果分級往往由人工進行，這不僅耗時耗力，還容易受主觀因素影響，導致分級不一致，進而影響蘋果的市場價值。基於機器學習和電腦視覺技術的品質辨識系統，可以在蘋果分揀和包裝生產線上發揮重要作用。

具體來說，這一系統可以利用高解析度相機拍攝每顆蘋果的影像，並使用深度學習模型來分析蘋果的外觀特徵，例如顏色、形狀、瑕疵（如病斑、刮痕等）。系統根據這些特徵將蘋果自動分類為不同的品質等級，如高、中、低等級。這些分類結果將指導後續的包裝過程，保證不同品質的蘋果能夠進行合適的包裝和標籤。

此系統的優勢在於：

- 1.提高生產效率：系統可以實時對每顆蘋果進行檢測和分級，無需等待人工分級或人工干預，大大提高分揀速度和準確度。
- 2.減少人工成本：通過自動化分級與包裝，減少了人工操作的需求，降低了人工成本，同時避免了人工疲勞導致的分級誤差。
- 3.提高品質控制的標準化：系統基於統一標準進行分類，保證每顆蘋果的品質符合預設的標準，從而提升消費者的滿意度和產品的市場競爭力。

案例預期結果

本案例基於 TinyML 技術進行模型訓練與部署，並分別在 PC 端和燒錄至開發板後進行測試，取得以下結果：

燒錄至開發板後，測試準確率達到 92%（272/296），表明該模型在實際應用環境中仍能保持良好的性能。

二、 模型比較分成以下三點介紹

(一) 模型大小與推論速度

在本案例中，模型選擇的關鍵在於兼顧性能與資源需求，特別是針對低算力設備的最佳化能力。以下是模型的詳細比較：

1. YOLOX Nano (本專案使用)

YOLOX Nano 為本專案選用的模型，其特點如下：

- (1) 極小的模型大小：僅約 0.91M 參數量，適合部署在資源有限的硬體上。
- (2) 高效能與低功耗：針對低算力設備進行最佳化，適用於即時應用，如智慧攝影機、無人機等場景。
- (3) 推論速度快：在低算力平台（如 M55M1）上表現出色，能夠快速處理任務並提供即時結果。

2. 其他 YOLO 模型

雖然本專案選用 YOLOX Nano，但其他 YOLO 模型在特定需求下也具備其優勢：

YOLOX 標準版本：支持不同尺寸的模型（如 YOLOX-S、YOLOX-M），適用於更高準確率需求的場景，但推論速度相對較慢，特別是大型模型。

(二) 訓練效率

在模型的訓練過程中，訓練效率是評估其實用性的重要指標。以下是本專案選用的 YOLOX Nano 與其他 YOLO 模型在訓練效率上的比較：

1. YOLOX Nano (本專案使用)

YOLOX Nano 的訓練效率具有以下特點：

- (1) **Anchor-free 結構**：簡化了訓練過程，減少了對 **anchor** 的依賴，從而簡化了超參數的調整。
- (2) **資料集適配性強**：相較於其他 YOLO 模型，YOLOX Nano 更容易適配新的資料集，特別是在應對動態變化的數據時。

2. 其他 YOLO 模型

其他 YOLO 模型的訓練效率特性：

- (1) **YOLOv4 / YOLOv5**：採用 **Anchor-based** 方法，對於資料標註格式和 **anchor** 設置有較高的要求，因此可能需要更多的超參數調整才能達到最佳效果。
- (2) **YOLOX 標準版本**：同樣採用 **Anchor-free** 方法，與 YOLOX Nano 的訓練流程一致，但由於模型更大，需要更高的計算力來支持其訓練過程，適合於高性能硬體環境。

(三) 應用場景

不同 YOLO 模型的應用場景各有側重，根據其設計特性與效能需求，以下是常見的應用場景介紹：

1. YOLOX Nano (本專案使用)

YOLOX Nano 適用於低算力或即時性要求高的場景，以下是一些具體應用：

- (1) **智慧家庭**：如智慧門鎖和人員偵測，能快速響應環境變化。
- (2) **移動設備**：在手機 App 中進行物體偵測，結合即時運算功能。
- (3) **工業邊緣計算**：適用於小型攝影機等邊緣設備的即時偵測需求，如在工業生產線上進行目標物件的快速檢測。

三、 LabelImg

LabelImg 是一款開源的圖像標註工具，用於為物體檢測任務創建標註數據集。以下是安裝和使用 LabelImg 的步驟：

(一) 安裝 LabelImg

確保已安裝 Python 環境，並使用 pip 安裝 LabelImg 套件。

執行以下命令以安裝：

```
$ pip install labelImg
```

(二) 執行 LabelImg

安裝完成後，可通過以下命令啟動 LabelImg：

在終端中輸入：

```
$ labelImg
```

LabelImg 啟動後，您可以加載圖片並進行框選與標註，生成所需的標註文件（如 COCO JSON 或 PASCAL VOC XML 格式）。

(三) 參考教學

若需要進一步的操作指導，可以參考以下教學資源：

- [LabelImg 使用步驟]

<https://blog.csdn.net/knighthood2001/article/details/125883343>

四、在 PC 上使用 Anaconda 環境進行模型訓練

(一) 訓練硬體規格

1. GPU：Nvidia GTX 1080 Ti

- (1) 可透過以下方法確認 GPU 規格：
- (2) 方法一：在終端輸入 `nvidia-smi` 指令，即可顯示顯卡型號與規格。
- (3) 方法二：打開 **工作管理員**，切換至 **效能** 分頁，找到 GPU 資訊以確認型號與使用狀況。

2. 透過 GPU 找適合的 CUDA 與 Torch 版本：

- (1) 使用以下網站查詢顯卡所支持的 CUDA 版本：
PyTorch CUDA 支援查詢
-

(二) 軟體與版本需求

1. **CUDA**：11.8
 2. **Torch**：2.0.0
 3. **Python**：3.10
-

(三) 訓練設定與結果

1. 訓練時長：
 - (1) 模型訓練所需時間約為 **1.5 小時**。
2. 訓練數據：
 - (1) 使用約 **600 張圖片** 作為訓練資料。

(四) 訓練環境

步驟一：安裝 Anaconda3

前往 [Anaconda 官方網站](#) 下載並安裝：Anaconda 下載頁面。

安裝完成後，確保 Anaconda 的 conda 指令可正常運行。

步驟二：建立新環境

開啟 Anaconda Prompt（終端）。

建立專屬環境並指定 Python 版本為 3.10

```
$ conda create --name yolox_nu python = 3.10
```

啟動該環境

```
$ conda activate yolox_nu
```

步驟三：下載 YOLOX Nano 的 GitHub 資源

使用 GitHub 命令下載 YOLOX Nano 的專案 zip 檔案：

```
$ gh repo clone MaxCYCHEN/yolox-ti-lite_tflite_int8
```

步驟四：更新 Pip 與相關工具

確保 Pip 與 Setuptools 為最新版本，避免安裝過程中出現相依性問題

```
$ python -m pip install --upgrade pip setuptools
```

(五) 安裝環境

步驟一：安裝基礎框架

安裝 PyTorch、CUDA 與 MMCV

PyTorch、CUDA、MMCV 版本需相互匹配。若使用 **CPU** 訓練，可忽略 CUDA 的安裝。

使用 GPU 訓練時，需根據自身 GPU 規格選擇對應版本：

CUDA 版本：**11.8**

Torch 版本：**2.0.0**

MMCV 安裝指令如下

```
$ python -m pip install mmcv==2.0.1 -f
```

<https://download.openmmlab.com/mmcv/dist/cu118/torch2.0/index.html>

步驟二：安裝其他 Python 套件

透過 **requirements.txt** 安裝依賴項：

使用以下指令，自動安裝所有所需套件：

```
$ python -m pip install --no-input -r requirements.txt
```

步驟三：安裝 YOLOX

執行 YOLOX 安裝：

在 YOLOX 檔案目錄中運行以下指令完成安裝：

```
$ python setup.py develop
```


(六) YOLOX_Nano 模型訓練

1. 模型特性 – 限制條件

- (1) 由於模型大小限制，在無外接 Hyper RAM 的情況下，只能使用 **2MB** 以下的模型。
- (2) 本案例選用 **YOLOX_Nano_ti_lite_nu**，其大小僅 **0.91MB**，是最適合 **M55M1** 開發板的模型。

2. 訓練步驟

執行訓練指令：

此指令針對模型配置檔案與預訓練模型進行訓練。

```
$ python tools/train.py -f exps/default/yolox_nano_ti_lite_nu.py -d 1 -b 64 --fp16 -o -c  
pretrain/tflite_yolox_nano_ti/320_DW/yolox_nano_320_DW_ti_lite.pth
```

自定義訓練參數：

若需調整訓練參數，可使用以下格式指令：

```
$ python tools/train.py -f <MODEL_CONFIG_FILE> -d 1 -b <BATCH_SIZE> --fp16 -o  
-c <PRETRAIN_MODEL_PATH>
```

3. 設置參數解釋

MODEL_CONFIG_FILE：

指向模型配置檔案，需指定下載的 YOLOX Nano 中的 yolox-nano-ti-nu 模型配置檔。

BATCH_SIZE：

設置訓練批量大小，預設為 64。若系統資源不足，可調低至 32 或 16。

PRETRAIN_MODEL_PATH：

放置預訓練模型的權重檔案，此檔案可從 YOLOX Nano 預訓練模型目錄下載。

(七)Train Dataset Format

1. 資料集結構

(1) 資料集按照以下結構分類，主要包含以下目錄與檔案：

Annotations（標註檔案）：

包含 **coco.json** 格式的訓練與驗證標註檔案。

Train2017（訓練圖片）：

存放訓練圖片。

Val2017（驗證圖片）：

存放驗證圖片。

資料集目錄結構範例

```
datasets/<dataset_name>
  annotations
    train_annotation_json_file
    val_annotation_json_file
  train2017
    train img
  val2017
    validation img
```

(2) 設定 Dataset 路徑

在程式中設定資料集路徑，範例如下

```
self.data_dir = "datasets/hagrid_coco "
self.train_ann = "hagrid_train.json"
self.val_ann = "hagrid_val.json"
```

(八) 訓練的自定義參數 (train.py 檔)

1. 調整的主要參數

(1) 圖片大小 (Resolution) :

`self.input_size` 和 `self.test_size` : 定義訓練與測試圖片的解析度。
範例設定為 **320×320**。

(2) 類別數量 (Classes) :

`self.num_classes` : 設定需要辨識的標註種類總數。
範例設定為 **11** 個類別。

(3) Epoch 次數 :

`self.max_epoch` : 指定訓練資料集通過模型的完整次數，範例設定為 **150** 次。

注意：

Epoch 次數不一定越多越好，建議在 **150-200** 範圍內進行預設。

若次數過多可能導致 **過擬合 (Overfitting)**，反而降低模型的準確率。

2. 參數程式碼範例

```
self.input_size = (320, 320)
self.test_size = (320, 320)
self.num_classes = 1
self.max_epoch = 150
```

以下為對應的程式碼設定

範例說明

類別數量： 設定為 11 個類別，表示需要辨識 11 種物體。

Epoch 次數： 訓練執行 150 個完整的 Epoch，適合避免過擬合的情況。

(九) PyTorch to ONNX

模型轉換流程

1. 從 PyTorch 模型轉換為 ONNX 格式

使用以下指令完成轉換：

```
$ python tools/export_onnx.py -f <MODEL_CONFIG_FILE> -c  
<TRAINED_PYTORCH_MODEL> --output-name <ONNX_MODEL_PATH>
```

MODEL_CONFIG_FILE：用於描述模型結構與參數，通常位於 YOLOX Nano 資料夾中。

TRAINED_PYTORCH_MODEL：已訓練完成的 PyTorch 模型檔案（格式為 .pth），作為轉換來源。

ONNX_MODEL_PATH：轉換後的 ONNX 模型儲存路徑與檔名。

4. 範例指令：

```
$ python tools/export_onnx.py -f exps/default/yolox_nano_ti_lite_nu.py -c  
YOLOX_outputs/yolox_nano_ti_lite_nu/latest_ckpt.pth --output-name  
YOLOX_outputs/yolox_nano_ti_lite_nu/vox_nano_nu_medicine.onnx
```

5. 範例解析：

exps/default/yolox_nano_ti_lite_nu.py：模型結構檔案的路徑。

YOLOX_outputs/yolox_nano_ti_lite_nu/latest_ckpt.pth：最新訓練完成的 PyTorch 模型權重檔。

YOLOX_outputs/yolox_nano_ti_lite_nu/vox_nano_nu_medicine.onnx：轉換後的 ONNX 模型儲存路徑與檔名。

(十) ONNX to TFLite

1. 模型轉換流程

將 ONNX 模型轉換為適用於 TFLite 框架的格式，執行以下指令進行轉換：

2. 指令範例

```
onnx2tf -i <ONNX_MODEL_PATH> -oiqt -qcind images <CALI_DATA_NPY_FILE>  
"[[[0,0,0]]]"
```

ONNX_MODEL_PATH：ONNX 模型檔案的路徑。

CALI_DATA_NPY_FILE：校準資料檔案的路徑，通常為 .npz 格式的 NumPy 檔案。

3. 實際範例指令

```
onnx2tf -i YOLOX_outputs/yolox_nano_ti_lite_nu/vox_nano_nu_medicine.onnx -oiqt -  
qcind images YOLOX_outputs/yolox_nano_ti_lite_nu/calib_data_320x320_n200.npz  
"[[[0,0,0]]]"
```

ONNX_MODEL_PATH：

將 ONNX 模型的儲存路徑替換為您的實際檔案路徑，例如：

YOLOX_outputs/yolox_nano_ti_lite_nu/vox_nano_nu_medicine.onnx。

CALI_DATA_NPY_FILE：

校準資料的 NumPy 檔案路徑，範例中為：

YOLOX_outputs/yolox_nano_ti_lite_nu/calib_data_320x320_n200.npz。

校準參數：

[0,0,0] 為校準值，可根據具體需求進行調整。

(十一) Vela Compiler and Convert to Deployment Format

使用 **Vela Compiler** 進行模型轉換

1. 設定模型路徑與輸出路徑：

使用以下指令進行設定

```
$ set MODEL_SRC_FILE = <your tflite model>
```

```
$ set MODEL_OPTIMISE_FILE = <output vela model>
```

<your tflite model>：指向原始 TFLite 模型檔案 (.tflite 格式)。

<output vela model>：設定 Vela 編譯器輸出的最佳化後模型檔案。

2. 輸出檔案 (Output File)

編譯完成後，輸出檔案會位於以下路徑

```
vela\generated\yolox_nano_ti_lite_nu_full_integer_quant_vela.tflite
```

此檔案將用於後續部署處理，方便於嵌入式平台運行。

五、未來展望

隨著科技的發展，特別是在人工智慧（AI）、機器學習、物聯網（IoT）和嵌入式系統領域的持續創新，蘋果品質辨識技術的未來前景十分廣闊。從智慧農業到自動化生產，蘋果品質辨識系統將不僅僅提升農產品的市場競爭力，也將極大改善農業生產過程中的效率與質量控制。以下是蘋果品質辨識技術未來的一些發展方向：

（一）更高精度的辨識技術

未來，蘋果品質辨識將受益於更先進的深度學習模型與高解析度影像處理技術。隨著影像感測器技術的進步，未來的蘋果品質辨識系統能夠更加精確地識別蘋果的細微特徵，從顏色、形狀到表面瑕疵（如病斑、刮痕等），提供更高準確度的品質分類。新一代卷積神經網路（CNN）以及其變體（如自注意力機制、生成對抗網路 GAN 等）將進一步提升辨識精度與速度。

（五）跨設備與雲端整合

隨著物聯網的發展，蘋果品質辨識技術將不僅限於單一的嵌入式開發板或設備，未來可以集成到更為廣泛的智慧設備中，並實現跨設備協同工作。例如，傳感器、相機、無人機等設備可以共同協作進行農田監控與品質檢測。這些設備將生成大量數據並通過雲端進行分析，實現遠端監控與智慧決策支持，從而提高農業生產管理的自動化與智慧化水平。

（六）實時處理與邊緣運算

隨著邊緣計算（Edge Computing）的興起，蘋果品質辨識技術未來將更加注重實時處理能力，尤其是在農田現場等資源受限的環境中。新唐 M55M1 等嵌入式平台將繼續進行最佳化，支持更快的資料處理速度，從而能在無需依賴雲端的情況下完成即時辨識。這不僅能降低延遲，還能提高系統的可靠性與運行效率。

玖、智慧農業 2 - 魚苗計數

一、案例介紹 - 魚苗計數

(一) 應用情境

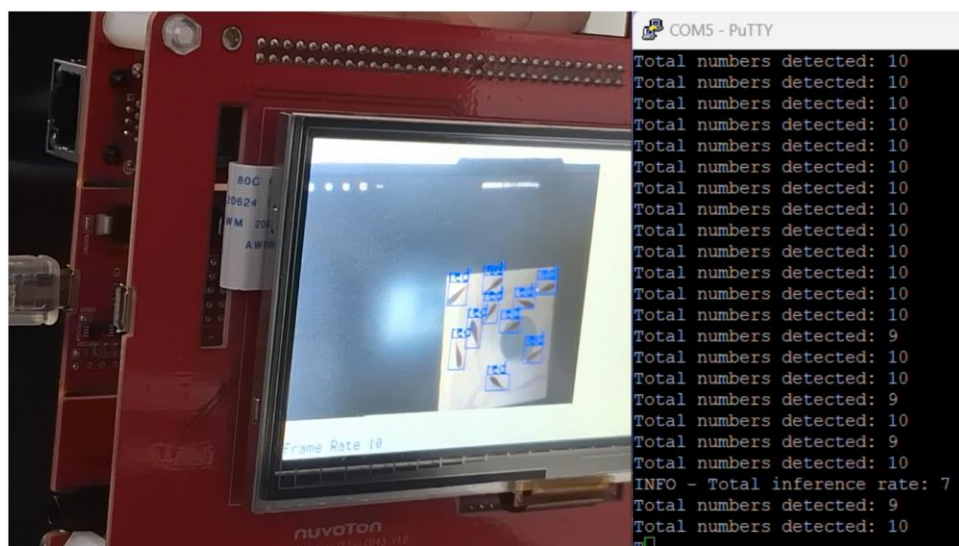


圖 181、魚苗計數結果

利用鏡頭模組拍攝水中的魚苗，經過新唐 M55M1 開發板運算後，可將魚苗名稱顯示於 LCD 顯示器如圖，並在終端機介面中顯示當下畫面中的魚苗數量。

(二) 專案摘要

利用 YOLOX-ti-lite_tflite_int8 模型訓練魚苗資料集，經過模型框架轉換：從 Pytorch 到 ONNX 再到 Tensorflow Lite，由量化技術轉化成 INT8 格式以減少模型的大小、提高推理速度，最佳化模型大小和性能，經過 Vela 編譯器將模型中適合的運算操作分配到 NPU，藉此加速模型推理，最後經由 Keil 編譯並將韌體下載至新唐 M55M1 開發板，計數功能可於開發板 LCD panel 及 PC 端終端機顯示結果。

(三) 數據處理與模型推理流程

1. 資料集準備 -> 2. 模型框架轉換和輕量化技術及 Vela 編譯 -> 3. 燒錄至開發板
->4.加入計數功能-> 5. 運算後顯示魚苗名稱，並在 PuTTY 顯示當下魚苗數量

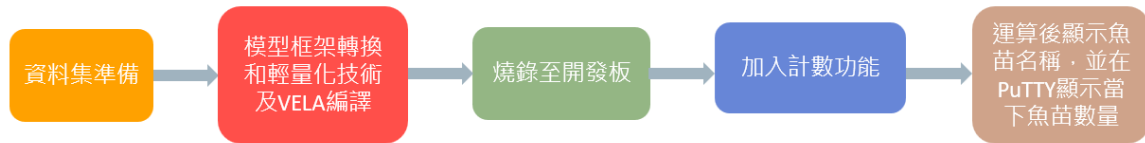


圖 182、數據處理與模型推理流程

(四) 結果總結

本案例顯示結果經由驗證集測試準確率達 95.7% (180/188)。

二、系統流程與程式模組說明 - 程式模組

(一) 核心檔案

1. main.cpp: 負責初始化系統與呼叫其他模組。
2. BoardInit.cpp/hpp: 用於初始化硬體板卡，如 GPIO、I2C、SPI 等。
3. InferenceTask.cpp/hpp: 負責模組的加載與執行模型推論。
4. DetectorPostProcessing.cpp/hpp: 處理模型輸出結果，將結果轉換為實際的 bounding box；因為模型輸出為座標數據 (例如 x, y, w, h 或直接的 bounding box)，需要在此檔案中完成數據解析、NMS (非極大值抑制)，以及座標轉換等邏輯。
5. mpu_config_M55M1.h: 定義了開發板上的硬體參數或處理器相關設定。

(二) 支援目錄

1. Device、GCC、Keil、IAR: 提供裝置驅動程式、編譯器專案設定檔。
2. Model: 存放機器學習模型與訓練結果檔案。
3. ProfilerCounter: 用於性能分析。

4. Pattern: 包含測試範例或模板數據。

三、 資料集與模型訓練

(一) 資料準備

1. 使用工具 LabelImg 進行標註。
2. 資料來源：自行拍攝花斑劍尾魚。
3. 資料標註格式：COCO JSON 格式，包含以下數據分布：
 - (1) Train Set: 752 images
 - (2) Valid Set: 188 images

(二) 模型訓練

1. 訓練框架：PyTorch。
2. 模型選擇：YOLOX-ti-lite_tflite_int8，具有高效能與輕量化特性，適合嵌入式設備。
3. 訓練環境：NVIDIA GeForce RTX 4060 GPU，單次訓練約需 60 分鐘。
4. 模型來源與參考鏈接：
 - (1) GitHub: https://github.com/MaxCYCHEN/yolox-ti-lite_tflite_int8

(三) 模型框架轉換

1. 模型框架從 PyTorch 轉換至 TensorFlow Lite，進一步進行 INT8 量化以最佳化模型大小與運算速度。
2. 量化技術應用：以兼顧準確度與效率的平衡為核心目標，適合嵌入式設備的部署需求。

(四) 環境建置步驟

1. 建立 Python 環境:

- (1) 使用 Anaconda 建立專用環境:

```
conda create --name yolox_nu python=3.10
```

```
conda activate yolox_nu
```

- (2) 通過上述指令在 Anaconda3 上建立即時環境。

2. 升級 pip:

- (1) 使用以下命令升級 pip 與 setuptools:

```
python -m pip install --upgrade pip setuptools
```

3. 安裝 CUDA、PyTorch 和 MMCV:

- (1) 根據電腦的顯卡配置選擇適合的 CUDA 版本，並依此選擇適配的 PyTorch 和 MMCV 版本。

本案例實際使用的版本: CUDA:11.8，PyTorch:2.0，MMCV:2.0.1

- (2) 安裝命令:

```
pip install mmcv==2.0.1 -f
```

<https://download.openmmlab.com/mmcv/dist/cu118/torch2.0/index.html>

4. 安裝其他需求套件:

- (1) 進入已下載的專案目錄，並使用以下命令安裝 requirements.txt 中的依賴:

```
python -m pip install --no-input -r requirements.txt
```

5. 安裝 YOLOX:

- (1) 使用以下命令完成 YOLOX 的安裝:

```
python setup.py develop
```

1. 模型訓練資料目錄結構

準備好自己的訓練資料，並整理成以下列式結構放在 dataset 目錄中:

Datasets/<your_datasets_name>/

(1) annotations/

train_annotation_json_file

val_annotation_json_file

(2) train2017/ train_img

(3) val2017/ validation_img

2. 訓練參數設置

在 yolox_nano_ti_lite_nu.py 中設置以下參數:

```
self.num_classes = 1
self.depth = 0.33
self.width = 0.25
self.input_size = (320, 320)
self.random_size = (10, 20)
self.mosaic_scale = (0.5, 1.5)
self.mosaic_prob = 0.5
self.enable_mixup = False
self.exp_name = os.path.split(os.path.realpath(__file__))[1].split(".")[0]
self.act = "relu"

# Define yourself dataset path
self.data_dir = "datasets/red_coco"
self.train_ann = "train2017_red.json"
self.val_ann = "val2017_red.json"

# ----- training config ----- #
self.warmup_epochs = 5
self.max_epoch = 150
# ----- testing config ----- #
self.test_size = (320, 320)
```

圖 183、訓練參數設置

(1) self.input_size = (320, 320)

(2) self.test_size = (320, 320)

(3) self.num_classes = 1

(4) self.max_epoch = 150

3. 使用預訓練模型進行訓練

- (1) 使用 `exps/default/yolox_nano_ti_lite_nu.py` 預設配置文件進行訓練。
- (2) 更新預訓練資料集檔案路徑:
 - a. `self.data_dir = "datasets/your_coco"`
 - b. `self.train_ann = "your_train.json"`
 - c. `self.val_ann = "your_val.json"`

```
# Define yourself dataset path
self.data_dir = "datasets/coco128"
self.train_ann = "train_annotations.coco.json"
self.val_ann = "val_annotations.coco.json"
```

圖 184、訓練路徑

4. 開始訓練:

- (1) 使用以下命令啟動訓練:

```
python tools/train.py -f <MODEL_CONFIG_FILE> -d 1 -b <BATCH_SIZE> --
fp16 -o -c <PRETRAIN_MODEL_PATH>
```

5. 機器學習模型框架轉換

- (1) PyTorch to ONNX:

- a. 使用以下命令將 PyTorch 模型轉換為 ONNX 模型:

```
python tools/export_onnx.py -f <MODEL_CONFIG_FILE> -c
<TRAINED_PYTORCH_MODEL> --output-name <ONNX_MODEL_PATH>
```

6. 輕量化

- (1) Create Calibration Data:

- a. 生成校準數據的命令:

```
python demo/TFLite/generate_calib_data.py --img-size <IMG_SIZE> --n-img
```

<NUMBER_IMG_FOR_CALI> -o <CALI_DATA_NPY_FILE> --img-dir
<PATH_OF_TRAIN_IMAGE_DIR>

(1) Convert ONNX to TFLite:

a. 使用以下命令進行模型轉換:

```
onnx2tf -i <ONNX_MODEL_PATH> -oiqt -qcind images  
<CALI_DATA_NPY_FILE> "[[[[1,1,1]]]]"
```

7. Vela 編譯

(1) 編譯步驟:

- 將輕量化完成的模型放到 vela/generated/ 目錄下。
- 修改 vela 目錄下的 variables.bat 文件，設置以下參數:
- set MODEL_SRC_FILE=<your tflite model>
- set MODEL_OPTIMISE_FILE=<output vela model>
- 執行 gen_modle_cpp 文件，生成編譯結果。
- 結果將出現在以下位置:

vela/generated/yolox_nano_ti_lite_nu_full_integer_quant_vela.tflite.cc

四、C++ 設計、部署與燒錄

(一) 程式流程簡介

1. 軟體功能設計:

(1) Main.cpp: 負責開發板模塊的調用，主要包含:

鏡頭模組、LCD 顯示模組、偵測框繪製功能

2. 燒錄部分:

- Keil 軟體介紹: 用於嵌入式開發與燒錄模組。
- Project 建立: 建立專案以支援軟硬體協同開發。
- 燒錄模型檔: 使用 keil 燒錄軟體將模型文件寫入目標硬體設備。

3. 程式流程簡介

(1) 系統啟動與初始化:

- a. 呼叫 `BoardInit.cpp` 初始化硬體資源。
- b. 初始化其他模組 (如記憶體分配、周邊啟動)。

(2) 模型載入與推論任務:

- a. 從 `Model` 資料夾載入機器學習模型。
- b. 呼叫 `InferenceTask.cpp` 執行模型推論:
 - (a) 將輸入資料 (如影像) 傳入模型。
 - (b) 接收輸出結果 (如目標檢測結果)。

(3) 推論結果後處理:

- a. 呼叫 `DetectorPostProcessing.cpp`:
 - (a) 對推論結果執行過濾與整理 (如非極大值抑制、邊界框處理)。
 - (b) 將處理後的結果儲存或傳回主程式。

(4) 輸出結果或回應:

- a. 回傳推論結果至主程式，供後續處理。
- b. 將結果顯示於螢幕或通過網路傳輸。

4. 硬體資源配置與軟體實現 - 主程式 `Main.cpp`

- (1) 執行開發板初始化: 包括時鐘、GPIO 和其他基本硬體模組的啟動。
- (2) 常數定義: 使用 `USE_CCAP & USE_DISPLAY` 確定是否啟用影像感測器 (CCAP) 與 LCD 顯示模組。
- (3) AI 模型推理初始化: 使用 `Arm®::app::YoloXnanoNu model` 建立 YOLO 模型實例，並初始化模型和 `Tensor Arena` 緩衝區。
- (4) 推理任務創建: 使用 `FreeRTOS` 的 `xTaskCreate` 函式創建推理任務，處理推理過程所需的影像資料。
- (5) 執行影像處理流程: 包括影像擷取、縮放、格式轉換、推理觸發以及結果的後處理，後處理包括偵測物件繪製框架及標籤。

5. 主程式流程

流程包括系統啟動、主循環執行影像處理與推理的多步驟邏輯，當完成所有影像處理時結束程式：

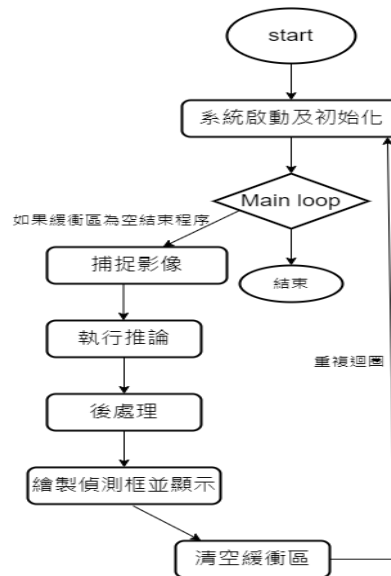


圖 185、主程式流程

- (1) 系統啟動與初始化。
- (2) 進入主循環 (Main loop)，逐步處理：
 - a. 擷取影像：從攝影機模組獲取資料。
 - b. 執行推論：將影像輸入至模型並獲得推論結果。
 - c. 後處理：包括邊界框繪製與結果顯示。
 - d. 清空緩衝區並重複。

6. 系統初始化

BoardInit() 函數：負責執行硬體相關的初始化操作，確保基礎硬體資源準備完成，並為後續影像處理與推理提供穩定環境。

- (1) 確保整個系統其他模組（如影像處理與神經網路推理）的運行有穩定的 CLK 與可靠的通訊機制。
- (2) 提供基本的輸入/輸出功能。
- (3) 初始化包括 HYPERRAM、UART 和核心時鐘 (CLK):
 - a. 開發板硬體初始化:
 - (a) 初始化系統時鐘 (SYS_Init)，啟用外部 RC 與 HXT 時鐘源。
 - (b) 設置時鐘路徑與更新系統核心頻率。

```
static void SYS_Init(void)
{
    /*-----
    /* Init System Clock
    /*-----

    /* Enable Internal RC 12MHz clock */
    CLK_EnableXtalRC(CLK_SRCCTL_HIRCEN_Msk);

    /* Waiting for Internal RC clock ready */
    CLK_WaitClockReady(CLK_STATUS_HIRCSTB_Msk);

    /* Enable HXT clock */
    CLK_EnableXtalRC(CLK_SRCCTL_HXTEN_Msk);

    /* Waiting for HXT clock ready */
    CLK_WaitClockReady(CLK_STATUS_HXTSTB_Msk);

    /* Switch SCLK clock source to APLL0 and Enable APLL0 180MHz clock */
    CLK_SetBusClock(CLK_SCLKSEL_SCLKSEL_APLL0, CLK_APLLCTL_APLLSRC_HIRC, FREQ_180MHZ);

    /* Update System Core Clock */
    /* User can use SystemCoreClockUpdate() to calculate SystemCoreClock. */
    SystemCoreClockUpdate();
}
```

圖 186、主程式 CLK 初始化

- b. 周邊裝置初始化:

```
int BoardInit(void)
{
    /* Unlock protected registers */
    SYS_UnlockReg();

    SYS_Init();

    /* UART init - will enable valid use of printf (stdout
     * re-directed at this UART (UART6) */
    InitDebugUart();

    SYS_LockReg();          /* Unlock register lock protect */

    HyperRAM_Init(HYPERRAM_SPIM_PORT);
    /* Enter direct-mapped mode to run new applications */
    SPIM_HYPER_EnterDirectMapMode(HYPERRAM_SPIM_PORT);

    info("%s: complete\n", __FUNCTION__);
}
```

圖 187、開發版周邊硬體初始化

- c. UART 配置: 用於通訊和除錯資訊的輸出。
- d. HYPERRAM 配置: 設定記憶體映射以支持神經網路運行所需的大量資料緩衝區。

7. 主程式-Main Loop

主程式中的主循環負責執行整個系統的核心處理邏輯，包括從影像擷取到結果顯示的所有步驟。

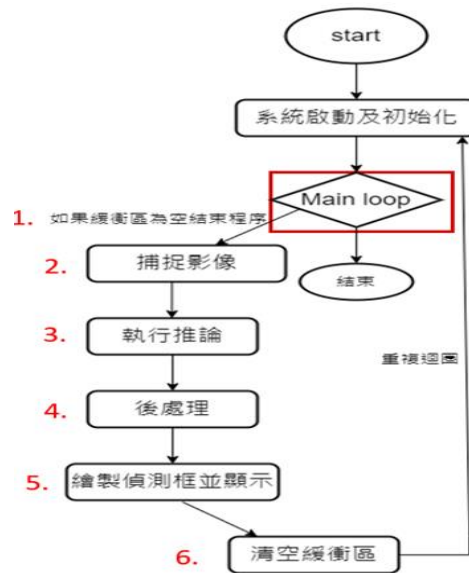


圖 188、主程式-Main Loop

(1) 檢查緩衝區是否為空:

- a. 每次循環開始時，檢查緩衝區是否有可用的影像數據。
- b. 如果緩衝區為空，則退出循環，結束程序。

(2) 擷取圖像:

- a. 使用相機模組或感測器擷取新的圖像，並將其放入緩衝區中。

(3) 執行推理:

- a. 使用預處理後的圖像進行魚種模型推理，檢測目標物件。

(4) 後處理:

- a. 將模型的輸出結果進行處理，例如篩選檢測框、解碼坐標，生成最終的檢測結果。

(5) 繪製檢測結果並顯示:

- a. 將檢測結果（目標物件的邊界框）繪製到圖像上，以便直觀顯示。
- b. 通過顯示模組將包含檢測結果的圖像顯示出來。

(6) 回收緩衝區:

- a. 將當前緩衝區清空，為下一次圖像捕捉做好準備。

[註]:

(1) 主循環的多線程最佳化:

- a. 主循環可以通過多線程方式進行最佳化，例如將圖像擷取、推理和後處理分配到不同的線程，以提升系統效率。

(2) 模型推理與動態記憶體管理:

- a. 在每次推理後，動態釋放記憶體以防止記憶體洩漏，確保系統穩定運行。

(二) 主程式循環

1. 主程式主循環流程:

(1) 主程式 - 影像緩衝區

影像緩衝區 (framebuf) 是在主程式 `main_task()` 函數的主循環中，用於管理從相機獲取的影像數據的重要環節。Framebuf 的主要功能是暫存來自相機的影像幀，然後進行格式轉換、推理等處理。

a. 實際流程:

- (a) `get_empty_framebuf()`: 獲取一個空的緩衝幀，用於填入新的影像數據。

```
//frame buffer managemnet function
static S_FRAMEBUF *get_empty_framebuf()
{
    int i;

    for (i = 0; i < NUM_FRAMEBUF; i ++){
        if (s_asFramebuf[i].eState == eFRAMEBUF_EMPTY)
            return &s_asFramebuf[i];
    }

    return NULL;
}
```

圖 189、`get_empty_framebuf()`

- (b) `get_full_framebuf()`: 獲取一個已填滿的幀，將其縮放並轉換成

YOLOX_nano 模型的格式。

```
static S_FRAMEBUF *get_full_framebuf()
{
    int i;

    for (i = 0; i < NUM_FRAMEBUF; i++)
    {
        if (s_asFramebuf[i].eState == eFRAMEBUF_FULL)
            return &s_asFramebuf[i];
    }

    return NULL;
}
```

圖 190、get_full_framebuf()

- (c) get_inf_framebuf(): 獲取已完成推理的幀，將偵測結果（如邊界框和標籤）繪製到影像上，並顯示在螢幕上。

```
static S_FRAMEBUF *get_inf_framebuf()
{
    int i;

    for (i = 0; i < NUM_FRAMEBUF; i++)
    {
        if (s_asFramebuf[i].eState == eFRAMEBUF_INF)
            return &s_asFramebuf[i];
    }

    return NULL;
}
```

圖 191、get_inf_framebuf()

b. 相關函數實現：

- (a) get_empty_framebuf(): 使用迴圈檢查緩衝幀是否為空並返回。
- (b) get_full_framebuf(): 使用相似邏輯查找已填充數據的緩衝幀。

(c) `get_inf_framebuf()`: 用於處理完成推理的幀並返回給主程式顯示模組。

(2) 擷取影像

擷取影像的過程中，緩衝區的管理至關重要。緩衝幀的狀態（空、已滿、完成推理）決定了擷取和處理邏輯是否可以正常執行。

```
//frame buffer managemnet function
static S_FRAMEBUF *get_empty_framebuf()
{
    int i;

    for (i = 0; i < NUM_FRAMEBUF; i ++ )
    {
        if (s_asFramebuf[i].eState == eFRAMEBUF_EMPTY)
            return &s_asFramebuf[i];
    }

    return NULL;
}
```

圖 192、避免在捕捉影像時覆蓋現有數據

在擷取影像之前，系統需要調用 `get_empty_framebuf()` 確保獲得一個可用的緩衝幀，用於儲存新擷取的數據。

a. 完整的擷取過程如下：

- (a) 確認是否有資源可供使用，避免在捕捉影像時覆蓋現有數據(圖十)。
- (b) 通過相機模組介面將數據讀取到分配的緩衝區中。
- (c) 如果發生錯誤（如無法讀取影像數據），則進行錯誤處理並重試。

[註]:

a. 設計多緩衝幀管理:

- (a) 多個緩衝幀的引入可以大幅提升系統吞吐量，尤其是在高幀率攝像場景中。

b. 錯誤處理機制:

- (b) 在讀取影像數據時發生錯誤應該有清晰的錯誤日誌和重試邏輯，以提升系

統穩定性。

(3) 執行推論

在執行推論過程中，主要步驟包括：從已填滿的緩衝區中提取數據，將其傳入模型進行推論，並將推論結果儲存或顯示。以下是相關的核心函式與程序碼說明：

a. `get_full_framebuf()`

此函式負責從已填滿的緩衝區中獲取一個數據塊，用於進一步的推論處理。函式通過檢查每個緩衝區的狀態來確認其是否可用。以下為函式的程式碼實現：

```
static S_FRAMEBUF *get_full_framebuf()
{
    int i;

    for (i = 0; i < NUM_FRAMEBUF; i++)
    {
        if (s_asFramebuf[i].eState == eFRAMEBUF_FULL)
            return &s_asFramebuf[i];
    }

    return NULL;
}
```

圖 193、函式宣告

完成緩衝區數據提取後，將其傳入 YOLOX 模型進行推論，具體的模型推論程式碼如下：

```

//trigger inference
inferenceJob->responseQueue = inferenceResponseQueue;
inferenceJob->pPostProc = &postProcess;
inferenceJob->modelCols = inputImgCols;
inferenceJob->modelRows = inputImgRows;
inferenceJob->srcImgWidth = fullFramebuf->frameImage.w;
inferenceJob->srcImgHeight = fullFramebuf->frameImage.h;
inferenceJob->results = &fullFramebuf->results;

xQueueSend(inferenceProcessQueue, &inferenceJob, portMAX_DELAY);
fullFramebuf->eState = eFRAMEBUF_INF;
}

```

圖 194、執行推論程式碼

b. PresentInferenceResult 函式

此函式用於顯示推論結果，包括偵測物件的類別標籤以及其在影像中的位置。該函式的輸入參數為推論結果與標籤，程式將逐一輸出偵測結果。

以下為程式碼：

```

static bool PresentInferenceResult(const std::vector<arm::app::object_detection::DetectionResult> &results,
std::vector<std::string> &labels)
{
    /* If profiling is enabled, and the time is valid. */
    info("Final results:\n");

    for (uint32_t i = 0; i < results.size(); ++i)
    {
        info("%" PRIu32 " " %s(%f) -> %s {x=%d,y=%d,w=%d,h=%d}\n", i,
            labels[results[i].m_cls].c_str(),
            results[i].m_normalisedVal, "Detection box:",
            results[i].m_x0, results[i].m_y0, results[i].m_w, results[i].m_h);
    }

    return true;
}

```

圖 195、PresentInferenceResult 函式

以上函式將推論的每個結果進行格式化並輸出，同時也可將結果回饋至其他系統模組進行後續處理。

(4) 後處理

在本節中，我們進一步詳細探討後處理步驟，尤其是如何從模型輸出的數據中提取有用的信息並最終將其可視化。以下步驟概述了後處理的邏輯和實現：

- a. 縮放檢測框：確保檢測框的大小與原始圖像一致。
- b. 提取檢測結果：從模型輸出中提取邊界框和對應的信心度。
- c. 執行非極大值抑制 (NMS)：移除多餘的重疊框，只保留最有意義的檢測框。
- d. 儲存結果：將最終的檢測結果存入結果結構中。

a. 後處理核心函數說明

在後處理中，以下核心函數執行了各項功能：

(a) DetectorPostprocessing::RunPostProcessing：該函數的作用是進行全面的後處理操作，包括初始化網路結構、提取邊界框數據、執行非極大值抑制以及結果調整。

- 主要參數包括：
 - `imgNetRows/imgNetCols`: 網路輸入的圖像大小
 - `modelOutput0`: 模型的輸出張量
 - `resultsOut`: 用於存放結果的向量
- 在 `DetectorPostprocessing::RunPostProcessing` 中加入計數功能:
 - **計算總數:**
 - 使用變數 `totalBoundingBoxes` 來統計魚苗的 bounding box 數量
 - 在每次向 `resultsOut` 推入結果時，對該變數進行遞增
(`++totalBoundingBoxes`)
 - **輸出結果:**

使用 `std::cout` 輸出總 bounding box 數量
 - **檢查有效性:**

確保計算僅包括信心值超過閾值的框(已經在條件 `if (it.prob[j] > 0)` 中限制

- **調整 NMS 閾值：**

較小的目標物體可能在非極大值抑制（NMS）中被較大的物體抑制，或者未被有效檢測

- **本案例設置物體信心度閾值=0.6 及非極大值抑制閾值=0.25**

(b) DetectorPostprocessing::InsertTopNDetections：該函數實現了檢測結果列表的最佳化插入，根據信心度排序並維護一個固定大小的列表，確保結果列表不會過大。

(c) DetectorPostprocessing::GetNetworkBoxes：該函數負責將模型的原始輸出轉換為可讀的邊界框數據，並篩選出高信心度的框進行後續處理。

(d) 執行邏輯包括：

- 計算不同層級的 stride 值
- 提取框的坐標信息（x, y, w, h）
- 根據 stride 值調整框的大小和位置

(5) 偵測框繪製並顯示

在此步驟中，我們通過 DrawImageDetectionBoxes 函式實現對物件偵測結果的視覺化顯示。該函式負責在圖像上繪製物件偵測框以及顯示標籤，從而讓使用者能夠直觀地了解模型的偵測結果。

a. DrawImageDetectionBoxes 函式(圖十四)：

該函式會針對每個偵測結果（result），在偵測框位置繪製矩形（bounding box）和標籤。

```
static void DrawImageDetectionBoxes(
    const std::vector<arm::app::object_detection::DetectionResult> &results,
    image_t *drawImg,
    std::vector<std::string> &labels)
{
    for (const auto &result : results)
    {
        imlib_draw_rectangle(drawImg, result.m_x0, result.m_y0, result.m_w, result.m_h, COLOR_B5_MAX, 1, false);
        imlib_draw_string(drawImg, result.m_x0, result.m_y0 - 16, labels[result.m_cls].c_str(), COLOR_B5_MAX, 2, 0, 0, false,
                           false, false, false, 0, false, false);
    }
}
```

圖 196、DrawImageDetectionBoxes 函式

○ 函式參數：

- drawImg: 指向圖像的指標，用於繪製框和標籤的圖像。
- results: 包含偵測框資訊的結果列表。
- labels: 偵測框對應的類別標籤。

(a) 程式碼操作：

- a. 使用 `imlib_draw_rectangle` 繪製邊界框（bounding box），顯示物件的位置範圍。
- b. 使用 `imlib_draw_string` 顯示物件類別的標籤，便於觀察和分析。
此步驟的完成確保了偵測結果的視覺化顯示，可以作為後續分析或展示的一部分。

五、 結論與未來展望

(一) 結論：

魚苗計數是水產養殖業中一個關鍵的管理環節，準確的魚苗計數不僅能夠幫助養殖者最佳化資源配置，還能提高養殖效率，確保水產物的健康成長。隨著科技的進步，傳統的人工計數方法逐漸被自動化和智慧化的技術所取代，尤其是基於影像處理和人工智慧（AI）的計數技術，已經在實際應用中展現出顯著的優勢。這些技術不僅提高了計數的精確度和效率，也能夠應對大規模養殖場中魚苗計數的高需

求。

本研究通過引入現代影像處理技術，設計並實現了一套基於深度學習的魚苗計數系統。該系統能夠有效識別和計算魚苗數量，並成功減少了人工計數的誤差，顯著提高了計數速度與準確度。此外，實驗結果表明，與傳統方法相比，基於圖像辨識的魚苗計數技術能夠更好地處理魚苗在水中活動時的變化，克服了许多傳統方法的限制。

(二) 未來展望：

儘管目前魚苗計數技術已經取得了一定的進展，但仍然面臨一些挑戰。首先，計數系統的準確性在不同的水質環境和魚苗品種中仍然有待提高。水中光線條件、濁度以及魚苗的重疊和擁擠等因素，都可能對計數結果造成影響。因此，未來的研究可以集中在提高系統對複雜環境的適應性，例如通過開發更加高效的影像處理算法，來提升在低光或濁水環境下的計數準確性。

其次，隨著水產養殖規模的擴大，對大數據處理和即時監控系統的需求也日益增加。未來的魚苗計數系統可以結合雲端技術和物聯網，實現全程監控和數據分析，為養殖者提供更加精細化的管理決策支持。這樣的系統能夠即時收集各種環境數據、魚苗生長狀況及其他相關信息，並通過數據分析為養殖者提供最優的操作建議，從而進一步提升養殖效益。

最後，隨著深度學習和人工智慧技術的進一步發展，未來的魚苗計數系統將可能更具自我學習和最佳化的能力。透過持續的數據訓練，這些系統能夠自動適應各種不同的養殖環境，並不斷提升其計數準確性和效率。此外，魚苗計數技術的應用領域也有望擴展至其他水產養殖領域，如幼魚、成魚的數量監測，甚至可以應用於水生物種的保護和監測。

總結來說，魚苗計數技術的發展潛力巨大，隨著科技的進步，這一領域有望進一步改變傳統水產養殖的管理模式，為水產業帶來更高的生產效益與可持續發展。

(三) 參考資料

<https://medium.com/ai-for-k12/%E5%88%A9%E7%94%A8-conda-%E5%BB%BA%E7%AB%8B%E5%8F%8A%E7%AE%A1%E7%90%86-python-%E8%99%9B%E6%93%AC%E7%92%B0%E5%A2%83-cc1c89d96fa9>

<https://pytorch.org/get-started/previous-versions/>

<https://hackmd.io/@zxcasd89525/Syw8BypDi>

https://github.com/MaxCYCHEN/yolox-ti-lite_tflite_int8

拾、智慧農業 3 - 魚種辨識

一、 案例介紹 - 魚種辨識

(一) 應用情境



圖 197、實際辨識結果

利用鏡頭模組拍攝水中的魚類，經過開發板運算後，可將魚種辨識結果顯示於新唐 M55M1 開發板上的 LCD panel。本案例使用 13 種魚種來模擬實際情境如圖(自遠雄海洋公園拍攝):

(二) 專案摘要

利用 YOLOX-ti-lite_tflite_int8 模型訓練魚種資料集，經過模型框架轉換：從 Pytorch 到 ONNX 再到 Tensorflow Lite，再由量化技術轉化成 INT8 格式以減少模型的大小、提高推理速度，最佳化模型大小和性能，再由 Vela 編譯器將模型中適合的運算操作分配到 NPU，藉此加速模型推理，最後經由 Keil 編譯並下載韌體至新唐 M55M1 開發板，並於開發板顯示結果。

(三) 數據處理與模型推理流程

1. 資料集準備 -> 2. 模型框架轉換 -> 3. 輕量化技術及 Vela 編譯 -> 4. 燒錄至開發板 -> 5. 經過開發板運算後顯示結果



圖 198、數據處理與模型推理流程

(四) 結果總結

本案例顯示結果經由驗證集測試準確率達 96.5% (250/259)。

二、系統流程與程式模組說明 - 程式模組

(一) 核心檔案

1. main.cpp: 負責初始化系統與呼叫其他模組。
2. BoardInit.cpp/hpp: 用於初始化硬體板卡，如 GPIO、I2C、SPI 等。
3. InferenceTask.cpp/hpp: 負責模組的加載與執行模型推論。
4. DetectorPostProcessing.cpp/hpp: 處理模型輸出結果，將結果轉換為實際的 bounding box；因為模型輸出為座標數據 (例如 x, y, w, h 或直接的 bounding box)，需要在此檔案中完成數據解析、NMS (非極大值抑制)，以及座標轉換等邏輯。
5. mpu_config_M55M1.h: 定義了開發板上的硬體參數或處理器相關設定。

(二) 支援目錄

1. Device、GCC、Keil、IAR: 提供裝置驅動程式、編譯器專案設定檔。
2. Model: 存放機器學習模型與訓練結果檔案。
3. ProfilerCounter: 用於性能分析。
4. Pattern: 包含測試範例或模板數據。

三、 資料集與模型訓練

(一) 資料準備

1. 使用工具 LabelImg 進行標註。
2. 資料來源：遠雄海洋公園現場拍攝，涵蓋 13 種魚類。
3. 資料標註格式：COCO JSON 格式，包含以下數據分布：

Train Set: 1031 images

Valid Set: 259 images

(二) 模型訓練

1. 訓練框架：PyTorch。
2. 模型選擇：YOLOX-ti-lite_tflite_int8，具有高效能與輕量化特性，適合嵌入式設備。
3. 訓練環境：NVIDIA GeForce RTX 4060 GPU，單次訓練約需 60 分鐘。
4. 模型來源與參考鏈接：

GitHub: https://github.com/MaxCYCHEN/yolox-ti-lite_tflite_int8

(三) 模型框架轉換

1. 模型框架從 PyTorch 轉換至 TensorFlow Lite，進一步進行 INT8 量化以最佳化模型大小與運算速度。
2. 量化技術應用：以兼顧準確度與效率的平衡為核心目標，適合嵌入式設備的部署需求。

(四) 環境建置步驟

1. 建立 Python 環境:

- (1) 使用 Anaconda 建立專用環境:

```
conda create --name yolox_nu python=3.10
```

```
conda activate yolox_nu
```

- (2) 通過上述指令在 Anaconda3 上建立即時環境。

2. 升級 pip:

- (1) 使用以下命令升級 pip 與 setuptools:

```
python -m pip install --upgrade pip setuptools
```

3. 安裝 CUDA、PyTorch 和 MMCV:

- (1) 根據電腦的顯卡配置選擇適合的 CUDA 版本，並依此選擇適配的 PyTorch 和 MMCV 版本。

本案例實際使用的版本: CUDA:11.8，PyTorch:2.0，MMCV:2.0.1

- (1) 安裝命令:

```
pip install mmcv==2.0.1 -f
```

<https://download.openmmlab.com/mmcv/dist/cu118/torch2.0/index.html>

4. 安裝其他需求套件:

- (1) 進入已下載的專案目錄，並使用以下命令安裝 requirements.txt 中的依賴:

```
python -m pip install --no-input -r requirements.txt
```

5. 安裝 YOLOX:

- (1) 使用以下命令完成 YOLOX 的安裝:

```
python setup.py develop
```

6. 模型訓練資料目錄結構:

準備好自己的訓練資料，並整理成以下列式結構放在 dataset 目錄中:

Datasets/<your_datasets_name>/

annotations/

train_annotation_json_file

val_annotation_json_file

train2017/ train_img

val2017/ validation_img

7. 訓練參數設置

在 yolox_nano_ti_lite_nu.py 中設置以下參數:

```
class Exp(MyExp):
    def __init__(self):
        super(Exp, self).__init__()
        # ----- model config ----- #
        self.num_classes = 13
        self.depth = 0.33
        self.width = 0.25
        self.input_size = (320, 320)
        self.random_size = (10, 20)
        self.mosaic_scale = (0.5, 1.5)
        self.mosaic_prob = 0.5
        self.enable_mixup = False
        self.exp_name = os.path.split(os.path.realpath(__file__))[1].split(".")[0]
        self.act = "relu"

        # Define yourself dataset path
        self.data_dir = "datasets/fish_coco_final"
        self.train_ann = "train2017.json"
        self.val_ann = "val2017.json"
```

圖 199、訓練參數設置

- (1) self.input_size = (320, 320)
- (2) self.test_size = (320, 320)
- (3) self.num_classes = 13
- (4) self.max_epoch = 150

8. 使用預訓練模型進行訓練

- (1) 使用 exps/default/yolox_nano_ti_lite_nu.py 預設配置文件進行訓練。
- (2) 更新預訓練資料集檔案路徑:

```
self.data_dir = "datasets/your_coco"
```

```
self.train_ann = "your_train.json"
```

```
self.val_ann = "your_val.json"
```

```
# Define yourself dataset path
self.data_dir = "datasets/coco128"
self.train_ann = "train_annotations.coco.json"
self.val_ann = "val_annotations.coco.json"
```

圖 200、訓練路徑

9. 開始訓練:

- (1) 使用以下命令啟動訓練:

```
python tools/train.py -f <MODEL_CONFIG_FILE> -d 1 -b <BATCH_SIZE> --
fp16 -o -c <PRETRAIN_MODEL_PATH>
```

10. 機器學習模型框架轉換

- (1) PyTorch to ONNX:

- a. 使用以下命令將 PyTorch 模型轉換為 ONNX 模型:

```
python tools/export_onnx.py -f <MODEL_CONFIG_FILE> -c
<TRAINED_PYTORCH_MODEL> --output-name <ONNX_MODEL_PATH>
```

11. 輕量化

- (1) Create Calibration Data:

- a. 生成校準數據的命令:

```
python demo/TFLite/generate_calib_data.py --img-size <IMG_SIZE> --n-img
<NUMBER_IMG_FOR_CALI> -o <CALI_DATA_NPY_FILE> --img-dir
<PATH_OF_TRAIN_IMAGE_DIR>
```

- (2) Convert ONNX to TFLite:

- a. 使用以下命令進行模型轉換:

```
onnx2tf -i <ONNX_MODEL_PATH> -oiqt -qcind images
```

<CALI_DATA_NPY_FILE> "[[[[1,1,1]]]]"

12. Vela 編譯

(1) 編譯步驟:

- a. 將輕量化完成的模型放到 vela/generated/ 目錄下。
- b. 修改 vela 目錄下的 variables.bat 文件，設置以下參數:
- c. set MODEL_SRC_FILE=<your tflite model>
- d. set MODEL_OPTIMISE_FILE=<output vela model>
- 執行 gen_modle_cpp 文件，生成編譯結果。
- 結果將出現在以下位置:

vela/generated/yolox_nano_ti_lite_nu_full_integer_quant_vela.tflite.cc

四、C++ 設計、部署與燒錄

(一)程式流程:

1. 軟體功能設計:

- (1) Main.cpp: 負責開發板模塊的調用，主要包含:

鏡頭模組、LCD 顯示模組、偵測框繪製功能

2. 燒錄部分:

- (1) Keil 軟體介紹: 用於嵌入式開發與燒錄模組。
- (2) Project 建立: 建立專案以支援軟硬體協同開發。
- (3) 燒錄模型檔: 使用 keil 燒錄軟體將模型文件寫入目標硬體設備。

3. 程式流程簡介

- (1) 系統啟動與初始化:
 - 呼叫 BoardInit.cpp 初始化硬體資源。
 - 初始化其他模組 (如記憶體分配、周邊啟動)。
- (2) 模型載入與推論任務:

- 從 Model 資料夾載入機器學習模型。
- 呼叫 InferenceTask.cpp 執行模型推論：

- (a) 將輸入資料 (如影像) 傳入模型。
- (b) 接收輸出結果 (如目標檢測結果)。

(3) 推論結果後處理：

- a. 呼叫 DetectorPostProcessing.cpp:
 - (a) 對推論結果執行過濾與整理 (如非極大值抑制、邊界框處理)。
 - (b) 將處理後的結果儲存或傳回主程式。

(4) 輸出結果或回應：

- a. 回傳推論結果至主程式，供後續處理。
- b. 將結果顯示於螢幕或通過網路傳輸。

4. 硬體資源配置與軟體實現 - 主程式 Main.cpp

- (1) 執行開發板初始化：包括時鐘、GPIO 和其他基本硬體模組的啟動。
- (2) 常數定義：使用 USE_CCAP & USE_DISPLAY 確定是否啟用影像感測器 (CCAP) 與 LCD 顯示模組。
- (3) AI 模型推理初始化：使用 Arm®::app::YoloXnanoNu model 建立 YOLO 模型實例，並初始化模型和 Tensor Arena 緩衝區。
- (4) 推理任務創建：使用 FreeRTOS 的 xTaskCreate 函式創建推理任務，處理推理過程所需的影像資料。
- (5) 執行影像處理流程：包括影像擷取、縮放、格式轉換、推理觸發以及結果的後處理，後處理包括偵測物件繪製框架及標籤。

5. 主程式流程

流程包括系統啟動、主循環執行影像處理與推理的多步驟邏輯，當完成所有影像處理時結束程式：

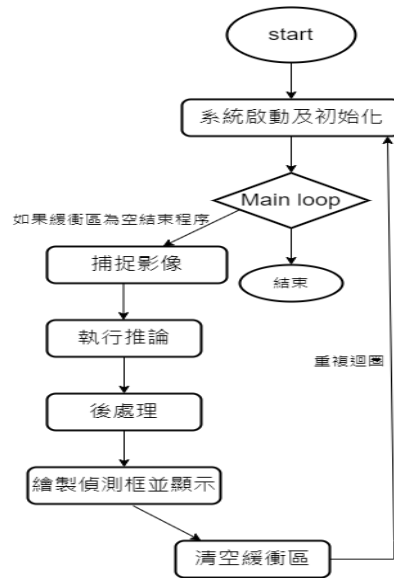


圖 201、主程式流程

- (1) 系統啟動與初始化。
- (2) 進入主循環 (Main loop)，逐步處理:
 - a. 擷取影像: 從攝影機模組獲取資料。
 - b. 執行推論: 將影像輸入至模型並獲得推論結果。
 - c. 後處理: 包括邊界框繪製與結果顯示。
 - d. 清空緩衝區並重複。

6. 系統初始化

BoardInit() 函數: 負責執行硬體相關的初始化操作，確保基礎硬體資源準備完成，並為後續影像處理與推理提供穩定環境。

- (1) 確保整個系統其他模組（如影像處理與神經網路推理）的運行有穩定的 CLK 與可靠的通訊機制。
- (2) 提供基本的輸入/輸出功能。
- (3) 初始化包括 HYPERRAM、UART 和核心時鐘 (CLK):
 - a. 開發板硬體初始化:
 - (a) 初始化系統時鐘 (SYS_Init)，啟用外部 RC 與 HXT 時鐘源。
 - (b) 設置時鐘路徑與更新系統核心頻率。

```
static void SYS_Init(void)
{
    /*-----
    /* Init System Clock
    /*-----

    /* Enable Internal RC 12MHz clock */
    CLK_EnableXtalRC(CLK_SRCCTL_HIRCEN_Msk);

    /* Waiting for Internal RC clock ready */
    CLK_WaitClockReady(CLK_STATUS_HIRCSTB_Msk);

    /* Enable HXT clock */
    CLK_EnableXtalRC(CLK_SRCCTL_HXTEN_Msk);

    /* Waiting for HXT clock ready */
    CLK_WaitClockReady(CLK_STATUS_HXTSTB_Msk);

    /* Switch SCLK clock source to APLL0 and Enable APLL0 180MHz clock */
    CLK_SetBusClock(CLK_SCLKSEL_SCLKSEL_APLL0, CLK_APLLCTL_APLLSRC_HIRC, FREQ_180MHZ);

    /* Update System Core Clock */
    /* User can use SystemCoreClockUpdate() to calculate SystemCoreClock. */
    SystemCoreClockUpdate();
}
```

圖 202、主程式 CLK 初始化

- b. 周邊裝置初始化:

```
int BoardInit(void)
{
    /* Unlock protected registers */
    SYS_UnlockReg();

    SYS_Init();

    /* UART init - will enable valid use of printf (stdout
    | * re-directed at this UART (UART6) */
    InitDebugUart();

    SYS_LockReg();          /* Unlock register lock protect */

    HyperRAM_Init(HYPERRAM_SPIM_PORT);
    /* Enter direct-mapped mode to run new applications */
    SPIM_HYPER_EnterDirectMapMode(HYPERRAM_SPIM_PORT);

    info("%s: complete\n", __FUNCTION__);
}
```

圖 203、開發版周邊硬體初始化

- c. UART 配置: 用於通訊和除錯資訊的輸出。
- d. HYPERRAM 配置: 設定記憶體映射以支持神經網路運行所需的大量資料緩衝區。

7. 主程式-Main Loop

主程式中的主循環負責執行整個系統的核心處理邏輯，包括從影像擷取到結果顯示的所有步驟。

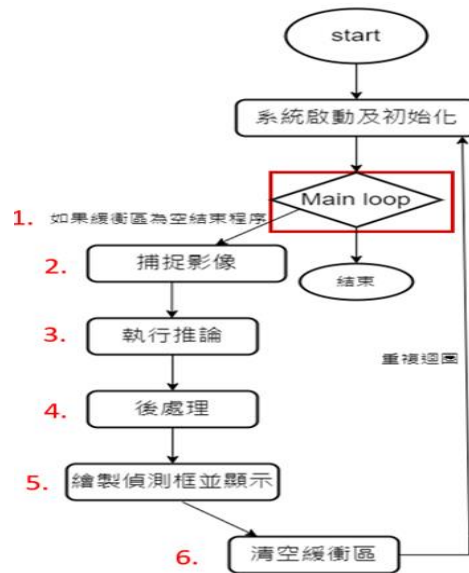


圖 204、主程式-Main Loop

(1) 檢查緩衝區是否為空:

- a. 每次循環開始時，檢查緩衝區是否有可用的影像數據。
- b. 如果緩衝區為空，則退出循環，結束程序。

(2) 擷取圖像:

- b. 使用相機模組或感測器擷取新的圖像，並將其放入緩衝區中。

(3) 執行推理:

- a. 使用預處理後的圖像進行魚種模型推理，檢測目標物件。

(4) 後處理:

- a. 將模型的輸出結果進行處理，例如篩選檢測框、解碼坐標，生成最終的檢測結果。

(5) 繪製檢測結果並顯示:

- a. 將檢測結果（目標物件的邊界框）繪製到圖像上，以便直觀顯示。
- b. 通過顯示模組將包含檢測結果的圖像顯示出來。

(6) 回收緩衝區:

- a. 將當前緩衝區清空，為下一次圖像捕捉做好準備。

[註]:

(1) 主循環的多線程最佳化:

- a. 主循環可以通過多線程方式進行最佳化，例如將圖像擷取、推理和後處理分配到不同的線程，以提升系統效率。

(2) 模型推理與動態記憶體管理:

- a. 在每次推理後，動態釋放記憶體以防止記憶體泄漏，確保系統穩定運行。

(二) 主程式循環:

1. 主程式主循環流程:

(1) 主程式 - 影像緩衝區

影像緩衝區 (framebuf) 是在主程式 `main_task()` 函數的主循環中，用於管理從相機獲取的影像數據的重要環節。Framebuf 的主要功能是暫存來自相機的影像幀，然後進行格式轉換、推理等處理。

a. 實際流程:

- (a) `get_empty_framebuf()`: 獲取一個空的緩衝幀，用於填入新的影像數據。

```
//frame buffer managemnet function
static S_FRAMEBUF *get_empty_framebuf()
{
    int i;

    for (i = 0; i < NUM_FRAMEBUF; i ++){
        if (s_asFramebuf[i].eState == eFRAMEBUF_EMPTY)
            return &s_asFramebuf[i];
    }

    return NULL;
}
```

圖 205、`get_empty_framebuf()`

- (b) `get_full_framebuf()`: 獲取一個已填滿的幀，將其縮放並轉換成

YOLOX_nano 模型的格式。

```
static S_FRAMEBUF *get_full_framebuf()
{
    int i;

    for (i = 0; i < NUM_FRAMEBUF; i++)
    {
        if (s_asFramebuf[i].eState == eFRAMEBUF_FULL)
            return &s_asFramebuf[i];
    }

    return NULL;
}
```

圖 206、get_full_framebuf()

- (c) get_inf_framebuf(): 獲取已完成推理的幀，將偵測結果（如邊界框和標籤）繪製到影像上，並顯示在螢幕上。

```
static S_FRAMEBUF *get_inf_framebuf()
{
    int i;

    for (i = 0; i < NUM_FRAMEBUF; i++)
    {
        if (s_asFramebuf[i].eState == eFRAMEBUF_INF)
            return &s_asFramebuf[i];
    }

    return NULL;
}
```

圖 207、get_inf_framebuf()

b. 相關函數實現：

- (a) get_empty_framebuf(): 使用迴圈檢查緩衝幀是否為空並返回。
- (b) get_full_framebuf(): 使用相似邏輯查找已填充數據的緩衝幀。

(c) `get_inf_framebuf()`: 用於處理完成推理的幀並返回給主程式顯示模組。

(3) 擷取影像

擷取影像的過程中，緩衝區的管理至關重要。緩衝幀的狀態（空、已滿、完成推理）決定了擷取和處理邏輯是否可以正常執行。

```
//frame buffer managemnet function
static S_FRAMEBUF *get_empty_framebuf()
{
    int i;

    for (i = 0; i < NUM_FRAMEBUF; i ++)
    {
        if (s_asFramebuf[i].eState == eFRAMEBUF_EMPTY)
            return &s_asFramebuf[i];
    }

    return NULL;
}
```

圖 208、避免在捕捉影像時覆蓋現有數據

在擷取影像之前，系統需要調用 `get_empty_framebuf()` 確保獲得一個可用的緩衝幀，用於儲存新擷取的數據。

a. 完整的擷取過程如下：

- (a) 確認是否有資源可供使用，避免在捕捉影像時覆蓋現有數據(圖十)。
- (b) 通過相機模組介面將數據讀取到分配的緩衝區中。
- (c) 如果發生錯誤（如無法讀取影像數據），則進行錯誤處理並重試。

[註]:

a. 設計多緩衝幀管理:

- (a) 多個緩衝幀的引入可以大幅提升系統吞吐量，尤其是在高幀率攝像場景中。

b. 錯誤處理機制:

- (a) 在讀取影像數據時發生錯誤應該有清晰的錯誤日誌和重試邏輯，以提升系

統穩定性。

(4) 執行推論

在執行推論過程中，主要步驟包括：從已填滿的緩衝區中提取數據，將其傳入模型進行推論，並將推論結果儲存或顯示。以下是相關的核心函式與程序碼說明：

a. `get_full_framebuf()`

此函式負責從已填滿的緩衝區中獲取一個數據塊，用於進一步的推論處理。函式通過檢查每個緩衝區的狀態來確認其是否可用。以下為函式的程式碼實現：

```
static S_FRAMEBUF *get_full_framebuf()
{
    int i;

    for (i = 0; i < NUM_FRAMEBUF; i++)
    {
        if (s_asFramebuf[i].eState == eFRAMEBUF_FULL)
            return &s_asFramebuf[i];
    }

    return NULL;
}
```

圖 209、函式宣告

完成緩衝區數據提取後，將其傳入 YOLOX 模型進行推論，具體的模型推論程式碼如下：

```

//trigger inference
inferenceJob->responseQueue = inferenceResponseQueue;
inferenceJob->pPostProc = &postProcess;
inferenceJob->modelCols = inputImgCols;
inferenceJob->modelRows = inputImgRows;
inferenceJob->srcImgWidth = fullFramebuf->frameImage.w;
inferenceJob->srcImgHeight = fullFramebuf->frameImage.h;
inferenceJob->results = &fullFramebuf->results;

xQueueSend(inferenceProcessQueue, &inferenceJob, portMAX_DELAY);
fullFramebuf->eState = eFRAMEBUF_INF;
}

```

圖 210、執行推論程式碼

b. PresentInferenceResult 函式

此函式用於顯示推論結果，包括偵測物件的類別標籤以及其在影像中的位置。該函式的輸入參數為推論結果與標籤，程式將逐一輸出偵測結果。

以下為程式碼：

```

static bool PresentInferenceResult(const std::vector<arm::app::object_detection::DetectionResult> &results,
std::vector<std::string> &labels)
{
    /* If profiling is enabled, and the time is valid. */
    info("Final results:\n");

    for (uint32_t i = 0; i < results.size(); ++i)
    {
        info("%" PRIu32 " ) %s(%f) -> %s {x=%d,y=%d,w=%d,h=%d}\n", i,
            labels[results[i].m_cls].c_str(),
            results[i].m_normalisedVal, "Detection box:",
            results[i].m_x0, results[i].m_y0, results[i].m_w, results[i].m_h);
    }

    return true;
}

```

圖 211、PresentInferenceResult 函式

以上函式將推論的每個結果進行格式化並輸出，同時也可將結果回饋至其他系統模組進行後續處理。

(5) 後處理

在本節中，我們進一步詳細探討後處理步驟，尤其是如何從模型輸出的數據中提取有用的信息並最終將其可視化。以下步驟概述了後處理的邏輯和實現：

- a. 縮放檢測框：確保檢測框的大小與原始圖像一致。
- b. 提取檢測結果：從模型輸出中提取邊界框和對應的信心度。
- c. 執行非極大值抑制 (NMS)：移除多餘的重疊框，只保留最有意義的檢測框。
- d. 儲存結果：將最終的檢測結果存入結果結構中。

a. 後處理核心函數說明

在後處理中，以下核心函數執行了各項功能：

(a) DetectorPostprocessing::RunPostProcessing：該函數的作用是進行全面的後處理操作，包括初始化網路結構、提取邊界框數據、執行非極大值抑制以及結果調整。

- 主要參數包括：
 - `imgNetRows/imgNetCols`: 網路輸入的圖像大小
 - `modelOutput0`: 模型的輸出張量
 - `resultsOut`: 用於存放結果的向量

(b) DetectorPostprocessing::InsertTopNDetections：該函數實現了檢測結果列表的最佳化插入，根據信心度排序並維護一個固定大小的列表，確保結果列表不會過大。

(c) DetectorPostprocessing::GetNetworkBoxes：該函數負責將模型的原始輸出轉換為可讀的邊界框數據，並篩選出高信心度的框進行後續處理。

(d) 執行邏輯包括：

- 計算不同層級的 `stride` 值
- 提取框的坐標信息 (`x, y, w, h`)
- 根據 `stride` 值調整框的大小和位置

(6) 偵測框繪製並顯示

在此步驟中，我們通過 DrawImageDetectionBoxes 函式實現對物件偵測結果的視覺化顯示。該函式負責在圖像上繪製物件偵測框以及顯示標籤，從而讓使用者能夠直觀地了解模型的偵測結果。

a. DrawImageDetectionBoxes 函式(圖十四)：

該函式會針對每個偵測結果（result），在偵測框位置繪製矩形（bounding box）和標籤。

```
static void DrawImageDetectionBoxes(  
    const std::vector<arm::app::object_detection::DetectionResult> &results,  
    image_t *drawImg,  
    std::vector<std::string> &labels)  
{  
    for (const auto &result : results)  
    {  
        imlib_draw_rectangle(drawImg, result.m_x0, result.m_y0, result.m_w, result.m_h, COLOR_B5_MAX, 1, false);  
        imlib_draw_string(drawImg, result.m_x0, result.m_y0 - 16, labels[result.m_cls].c_str(), COLOR_B5_MAX, 2, 0, 0, false,  
                           false, false, false, 0, false, false);  
    }  
}
```

圖 212、DrawImageDetectionBoxes 函式

(a) 函式參數：

- drawImg: 指向圖像的指標，用於繪製框和標籤的圖像。
- results: 包含偵測框資訊的結果列表。
- labels: 偵測框對應的類別標籤。

(b) 程式碼操作：

a. 使用 imlib_draw_rectangle 繪製邊界框（bounding box），顯示物件的位置範圍。

b. 使用 imlib_draw_string 顯示物件類別的標籤，便於觀察和分析。

此步驟的完成確保了偵測結果的視覺化顯示，可以作為後續分析或展示的一部分。

五、 結論與未來展望

(一) 結論

本研究通過訓練魚種辨識模型，成功實現了基於 YOLOX-ti-lite_tflite_int8 的高效推理系統。經過多階段的模型轉換與最佳化過程，我們不僅提高了模型的推理速度，還顯著減少了模型的體積，使其更適合在嵌入式設備上運行。

1. 模型訓練與轉換

本研究首先使用 Pytorch 進行魚種辨識模型的訓練，並通過 ONNX 格式將其轉換為 TensorFlow Lite 模型，進一步利用 INT8 量化技術來減少模型大小，並提高推理速度。量化過程顯著降低了模型的儲存需求，同時在不影響辨識精度的情況下提升了推理效率，為部署於資源受限設備上的應用奠定了基礎。

2. NPU 加速與性能最佳化

通過使用 Vela 編譯器對模型進行最佳化，我們將適合的運算操作分配到 NPU（神經網路處理單元），進一步加速了模型的推理速度。這一過程不僅改善了推理的實時性，還提升了系統的整體性能，使得在低功耗硬體上的實時魚種辨識成為可能。

3. 開發板部署與顯示結果

最終，我們將經過最佳化的模型燒錄至新唐 M55M1 開發板，並實現了在開發板上即時顯示魚種辨識結果。這一部署展示了我們所開發的系統在實際應用中的可行性，證明了基於低功耗硬體平台的即時推理技術具有廣泛的應用潛力。

(二) 未來展望

儘管本研究在魚種辨識系統的開發中取得了積極成果，但仍有許多值得探索與改進的方向。未來的研究可以從以下幾個方面進行拓展：

1. 模型精度與大小的平衡

雖然量化技術成功減少了模型的體積並提高了推理速度，但在某些情況下，模型精度可能會受到影響。未來可以探索更多先進的量化技術或混合精度運算方法，以實現模型精度與推理速度之間的最佳平衡。

2. 增強模型的泛化能力

目前的魚種辨識模型在特定資料集上表現優異，但對於新環境下的魚類圖像識別仍有提升空間。未來可以考慮進行更多樣化的數據收集與訓練，並引入增強學習等技術，以提升模型在不同場景下的泛化能力。

3. 即時影像處理與延遲最佳化

雖然本研究的系統已經實現了即時辨識，但仍然可以進一步最佳化模型的推理延遲。隨著硬體性能的提升，未來可以考慮更加複雜的模型架構，並通過軟硬體協同設計來進一步減少推理時間，實現更加流暢的即時影像處理。

4. 更廣泛的應用場景

本研究專注於魚種辨識，但這一技術的應用範圍不僅限於此。未來可以探索將該系統擴展到其他生物物種的識別，甚至應用於工業質量檢測、醫療影像分析等領域，進一步拓寬其實際應用價值。

5. 低功耗設備的進一步開發與應用

隨著 IoT 和邊緣計算技術的發展，低功耗設備將在各種現場應用中扮演越來越重要的角色。未來可以針對更為低功耗的硬體平台進行最佳化，使得魚種辨識系統能夠在更小型、功耗更低的設備上運行，實現更廣泛的部署。

(三) 參考資料

<https://medium.com/ai-for-k12/%E5%88%A9%E7%94%A8-conda-%E5%BB%BA%E7%AB%8B%E5%8F%8A%E7%AE%A1%E7%90%86-python-%E8%99%9B%E6%93%AC%E7%92%B0%E5%A2%83-cc1c89d96fa9>

<https://pytorch.org/get-started/previous-versions/>

<https://hackmd.io/@zxcasd89525/Syw8BypDi>

https://github.com/MaxCYCHEN/yolox-ti-lite_tflite_int8

